Primitive Recognition Calculus Lean Formalization Bundle

Generated 2026-05-27. Scope: full Lean source content under `IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/`. Intended as a NotebookLM PDF companion to `Delta_Manifesto_20260526.pdf` and `Delta_Formal_Unification_From_Distinction.pdf`.

Total files: 33. **Total Lean lines:** 11228.

File Manifest

No.	Path	Lines
1	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Strength.lean	54
2	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Basic.lean	134
3	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/SameDiff.lean	105
4	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Quotient.lean	71
5	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Orbit.lean	106
6	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitArithmetic.lean	194
7	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/IntegerRational.lean	1289
8	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/IntegerOrder.lean	67
9	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitDivisibility.lean	283
10	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitEuclidean.lean	473
11	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RationalField.lean	284
12	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCost.lean	242
13	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceIncrementTriangle.lean	261
14	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceVerifierTriangle.lean	104
15	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceTriangle.lean	94
16	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCauchy.lean	236
17	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealNullSetoid.lean	135
18	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealMulBoundedContinuity.lean	236
19	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealBoundednessModulus.lean	142
20	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealProductContinuity.lean	304
21	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealOrderCongruence.lean	206
22	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteness.lean	592
23	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteOrderedField.lean	370
24	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteOrderedFieldPromoted.lean	88
25	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompletion.lean	97
26	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/TraceClosure.lean	107
27	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/TraceLogic.lean	237
28	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/FormalSystem.lean	123

No.	Path	Lines
29	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Inevitability.lean	90
30	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RecognizerBridge.lean	117
31	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCNativeCostUniqueness.lean	3798
32	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Kernel.lean	253
33	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/UniversalFoundation.lean	336

1. Strength.lean

Path: `IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Strength.lean` · **Lines:** 54

```
0001  /-
0002    PrimitiveRecognitionCalculus/Strength.lean
0003
0004    Round-trip source:
0005      PRC_Kernel_Spec_20260526.html
0006
0007    Spec anchor:
0008      K1. Strength Ledger
0009
0010    This module records the proof-strength tags used by the Primitive
0011    Recognition Calculus kernel. The tags are not mathematical assumptions.
0012    They are audit labels attached to later definitions and theorems.
0013  -/
0014
0015  import Mathlib
0016
0017  namespace IndisputableMonolith
0018  namespace Foundation
0019  namespace PrimitiveRecognitionCalculus
0020
0021  /-- K1. The strength tag attached to a PRC claim. -/
0022  inductive StrengthTag where
0023    /-- Forced by distinction and finite repetition alone. -/
0024    | deltaOnly
0025    /-- Uses the completed orbit or completed stable trace families. -/
0026    | traceClosure
0027    /-- Uses selection of witnesses from stable families. -/
0028    | choice
0029    /-- Uses controlled subtrace-class or power-class formation. -/
0030    | powerComprehension
0031    /-- Uses excluded middle or full classical reasoning as an extension. -/
0032    | classicalExtension
0033    deriving DecidableEq, Repr
0034
0035  /-- A small audit record tying a claim label to its strength tag. -/
0036  structure StrengthClaim where
0037    label : String
0038    tag : StrengthTag
0039    statement : String
0040    deriving Repr
0041
0042  /-- K1 audit sanity: the δ-only tag is not the trace-closure tag. -/
0043  theorem deltaOnly_ne_traceClosure :
0044      StrengthTag.deltaOnly ≠ StrengthTag.traceClosure := by
0045    decide
0046
0047  /-- K1 audit sanity: choice is not a δ-only claim. -/
0048  theorem choice_ne_deltaOnly :
0049      StrengthTag.choice ≠ StrengthTag.deltaOnly := by
0050    decide
0051
0052  end PrimitiveRecognitionCalculus
0053  end Foundation
0054  end IndisputableMonolith
```

2. Basic.lean

Path: `IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Basic.lean` · **Lines:** 134

```
/-
  PrimitiveRecognitionCalculus/Basic.lean

  Round-trip source:
    PRC_Kernel_Spec_20260526.html

  Spec anchors:
    K2.1–K2.5, R1–R4

  The δ-only syntactic kernel: distinction act, side, endpoint, finite
  trace, trace append, and trace extension. Lean equality is used only by
  the verifier to prove facts about this syntax.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Strength

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- K2.1. The primitive distinction act. At the object level this is δ. -/
inductive DistinctionAct where
  | delta
  deriving DecidableEq, Repr

/-- K2.2. The two sides forced by a distinction. -/
inductive Side where
  | left
  | right
  deriving DecidableEq, Repr

/-- K2.3. An endpoint is a side of the primitive distinction. -/
structure Endpoint where
  side : Side
  deriving DecidableEq, Repr

/-- The left endpoint of δ. -/
def Endpoint.left : Endpoint :=
  (Side.left)

/-- The right endpoint of δ. -/
def Endpoint.right : Endpoint :=
  (Side.right)

/-- K2.4. A finite trace is empty or extended by one distinction act. -/
inductive Trace where
  | empty
  | extend : Trace → DistinctionAct → Trace
  deriving DecidableEq, Repr

namespace Trace

/-- R3. One-step extension by δ. -/
def step (T : Trace) : Trace :=
  Trace.extend T DistinctionAct.delta

/-- Append two traces. This is the syntactic composition operation. -/
def append : Trace → Trace → Trace
  | T, Trace.empty => T
  | T, Trace.extend U a => Trace.extend (append T U) a

@[simp] theorem append_empty (T : Trace) :
    append T Trace.empty = T := by
  rfl

@[simp] theorem append_extend (T U : Trace) (a : DistinctionAct) :
    append T (Trace.extend U a) = Trace.extend (append T U) a := by
  rfl

@[simp] theorem empty_append (T : Trace) :
    append Trace.empty T = T := by
  induction T with
  | empty => rfl
  | extend T a ih =>
      simp [append, ih]

/-- R4. Trace append is associative. -/
theorem append_assoc (T U V : Trace) :
    append (append T U) V = append T (append U V) := by
  induction V with
  | empty => rfl
  | extend V a ih =>
      simp [append, ih]

/-- K2.5. `Extends T U` means `U` is `T` followed by some suffix trace. -/
def Extends (T U : Trace) : Prop :=
  ∃ V : Trace, append T V = U

/-- R4. Trace extension is reflexive. -/
```

```
theorem extends_refl (T : Trace) :
    Extends T T := by
  exact (Trace.empty, rfl)

/-- R4. Trace extension is transitive. -/
theorem extends_trans {T U V : Trace}
    (hTU : Extends T U) (hUV : Extends U V) :
    Extends T V := by
  rcases hTU with (A, hA)
  rcases hUV with (B, hB)
  refine (append A B, ?_)
  rw [← append_assoc, hA, hB]

/-- The trace with exactly `n` repeated δ-extensions. -/
def orbitTrace : Nat → Trace
  | 0 => Trace.empty
  | Nat.succ n => step (orbitTrace n)

/-- Length of a finite trace. -/
def length : Trace → Nat
  | Trace.empty => 0
  | Trace.extend T _ => Nat.succ (length T)

@[simp] theorem length_empty :
    length Trace.empty = 0 := by
  rfl

@[simp] theorem length_extend (T : Trace) (a : DistinctionAct) :
    length (Trace.extend T a) = Nat.succ (length T) := by
  rfl

/-- K2.12 preview. The length of the nth orbit trace is n. -/
theorem length_orbitTrace (n : Nat) :
    length (orbitTrace n) = n := by
  induction n with
  | zero => rfl
  | succ n ih =>
      simp [orbitTrace, step, ih]

end Trace

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

3. SameDiff.lean

Path: `IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/SameDiff.lean` · **Lines:** 105

```
/-
  PrimitiveRecognitionCalculus/SameDiff.lean

  Round-trip source:
    PRC_Kernel_Spec_20260526.html

  Spec anchors:
    K2.6–K2.10, R5–R7, K4.2–K4.3

  This module does not define object equality as Lean equality. It defines
  an admissible trace-judgment surface: SameT, DiffT, consistency, and the
  substitution rule for contexts that respect SameT.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Basic

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- K2.6–K2.8. An admissible trace judgment surface. -/
structure TraceJudgment where
  /-- K2.7. Object-level equality at a trace. -/
  same : Trace → Endpoint → Endpoint → Prop
  /-- K2.6. Object-level witnessed difference at a trace. -/
  diff : Trace → Endpoint → Endpoint → Prop
  /-- R5. SameT must be reflexive at each trace. -/
  same_refl_proof : ∀ T : Trace, Reflexive (same T)
  /-- R5. SameT must be symmetric at each trace. -/
  same_symm_proof : ∀ T : Trace, Symmetric (same T)
  /-- R5. SameT must be transitive at each trace. -/
  same_trans_proof : ∀ T : Trace, Transitive (same T)
  /-- R6. SameT and DiffT cannot both hold for the same ordered pair. -/
  same_diff_exclusive :
    ∀ {T : Trace} {a b : Endpoint}, same T a b → diff T a b → False

namespace TraceJudgment

/-- Reflexivity of SameT, extracted from the admissibility field. -/
theorem same_refl (J : TraceJudgment) (T : Trace) (a : Endpoint) :
    J.same T a a :=
  J.same_refl_proof T a

/-- Symmetry of SameT, extracted from the admissibility field. -/
theorem same_symm (J : TraceJudgment) (T : Trace) {a b : Endpoint}
    (h : J.same T a b) :
    J.same T b a :=
  J.same_symm_proof T h

/-- Transitivity of SameT, extracted from the admissibility field. -/
theorem same_trans (J : TraceJudgment) (T : Trace) {a b c : Endpoint}
    (hab : J.same T a b) (hbc : J.same T b c) :
    J.same T a c :=
  J.same_trans_proof T hab hbc

end TraceJudgment

/-- K2.8. A trace is consistent for a judgment surface if it never asserts
SameT and DiffT for the same endpoints. -/
def Consistent (J : TraceJudgment) (T : Trace) : Prop :=
  ∀ a b : Endpoint, ¬ (J.same T a b ∧ J.diff T a b)

/-- R6. The exclusivity field gives consistency at every trace. -/
theorem consistent_of_exclusive (J : TraceJudgment) (T : Trace) :
    Consistent J T := by
  intro a b h
  exact J.same_diff_exclusive h.1 h.2

/-- A predicate respects SameT at a trace. -/
def RespectsSame (J : TraceJudgment) (T : Trace)
    (P : Endpoint → Prop) : Prop :=
  ∀ {a b : Endpoint}, J.same T a b → P a → P b

/-- K2.10 and R7. Substitution for contexts that respect SameT. -/
theorem substitute
    (J : TraceJudgment) (T : Trace) (P : Endpoint → Prop)
    (hP : RespectsSame J T P) {a b : Endpoint}
    (hsame : J.same T a b) (ha : P a) :
    P b :=
  hP hsame ha

/-- A verifier-level model of the SameT/DiffT interface.

This is not PRC's object-level primitive. It is a sanity model showing the
interface is inhabited in Lean's verifier language. -/
def verifierEqualityJudgment : TraceJudgment where
  same := fun _ a b => a = b
  diff := fun _ a b => a ≠ b
  same_refl_proof := by
```

```
0091      intro T a
0092      rfl
0093    same_symm_proof := by
0094      intro T a b h
0095      exact h.symm
0096    same_trans_proof := by
0097      intro T a b c hab hbc
0098      exact hab.trans hbc
0099    same_diff_exclusive := by
0100      intro T a b hsame hdiff
0101      exact hdiff hsame
0102
0103  end PrimitiveRecognitionCalculus
0104  end Foundation
0105  end IndisputableMonolith
```

4. Quotient.lean

Path: `IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Quotient.lean` · **Lines:** 71

```
0001  /-
0002    PrimitiveRecognitionCalculus/Quotient.lean
0003
0004    Round-trip source:
0005      PRC_Kernel_Spec_20260526.html
0006
0007    Spec anchors:
0008      K2.11, K4.4
0009
0010    Quotienting is permitted only after SameT is known to be an equivalence.
0011    This module builds endpoint trace-classes and the corresponding recursion
0012    principle for SameT-respecting maps.
0013  -/
0014
0015  import Mathlib
0016  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.SameDiff
0017
0018  namespace IndisputableMonolith
0019  namespace Foundation
0020  namespace PrimitiveRecognitionCalculus
0021
0022  /-- K2.11. The setoid induced by SameT at a fixed trace. -/
0023  def sameSetoid (J : TraceJudgment) (T : Trace) : Setoid Endpoint where
0024    r := J.same T
0025    iseqv := {
0026      refl := J.same_refl_proof T
0027      symm := by
0028        intro a b h
0029        exact J.same_symm_proof T h
0030      trans := by
0031        intro a b c hab hbc
0032        exact J.same_trans_proof T hab hbc
0033    }
0034
0035  /-- K2.11. Endpoint trace-classes under SameT. -/
0036  def EndpointClass (J : TraceJudgment) (T : Trace) : Type :=
0037    Quot (sameSetoid J T)
0038
0039  /-- The class of an endpoint. -/
0040  def endpointClassOf (J : TraceJudgment) (T : Trace)
0041      (a : Endpoint) : EndpointClass J T :=
0042    Quot.mk (sameSetoid J T) a
0043
0044  /-- K4.4. SameT endpoints determine the same quotient class. -/
0045  theorem endpointClass_eq_of_same
0046      (J : TraceJudgment) (T : Trace) {a b : Endpoint}
0047      (h : J.same T a b) :
0048      endpointClassOf J T a = endpointClassOf J T b :=
0049    Quot.sound h
0050
0051  /-- K4.4. Quotient recursion for SameT-respecting maps. -/
0052  def endpointClassLift
0053      (J : TraceJudgment) (T : Trace) {α : Sort _}
0054      (f : Endpoint → α)
0055      (hf : ∀ {a b : Endpoint}, J.same T a b → f a = f b) :
0056      EndpointClass J T → α :=
0057    Quot.lift f (by
0058      intro a b h
0059      exact hf h)
0060
0061  @[simp] theorem endpointClassLift_mk
0062      (J : TraceJudgment) (T : Trace) {α : Sort _}
0063      (f : Endpoint → α)
0064      (hf : ∀ {a b : Endpoint}, J.same T a b → f a = f b)
0065      (a : Endpoint) :
0066      endpointClassLift J T f hf (endpointClassOf J T a) = f a := by
0067    rfl
0068
0069  end PrimitiveRecognitionCalculus
0070  end Foundation
0071  end IndisputableMonolith
```

5. Orbit.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Orbit.lean · **Lines:** 106

```
/-
  PrimitiveRecognitionCalculus/Orbit.lean

  Round-trip source:
    PRC_Kernel_Spec_20260526.html

  Spec anchors:
    K2.12, R8, K4.5

  Base-neutral arithmetic begins as the finite orbit of repeated δ. Lean's
  Nat is used here only as a verifier representation, and the equivalence is
  proved explicitly.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Basic

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- K2.12. The base-neutral finite orbit of repeated distinction. -/
inductive DistinctionNat where
  | zero
  | succ : DistinctionNat → DistinctionNat
  deriving DecidableEq, Repr

namespace DistinctionNat

/-- R8. Zero is not a successor. -/
theorem zero_ne_succ (n : DistinctionNat) :
    zero ≠ succ n := by
  intro h
  cases h

/-- R8. Successor is injective. -/
theorem succ_injective :
    Function.Injective succ := by
  intro a b h
  cases h
  rfl

/-- R8. Induction over the δ-orbit. -/
theorem induction {P : DistinctionNat → Prop}
    (hzero : P zero)
    (hsucc : ∀ n : DistinctionNat, P n → P (succ n)) :
    ∀ n : DistinctionNat, P n := by
  intro n
  induction n with
  | zero => exact hzero
  | succ n ih => exact hsucc n ih

/-- Verifier representation of the orbit as Lean Nat. -/
def toNat : DistinctionNat → Nat
  | zero => 0
  | succ n => Nat.succ (toNat n)

/-- Build an orbit position from a verifier Nat. -/
def ofNat : Nat → DistinctionNat
  | 0 => zero
  | Nat.succ n => succ (ofNat n)

@[simp] theorem toNat_zero :
    toNat zero = 0 := by
  rfl

@[simp] theorem toNat_succ (n : DistinctionNat) :
    toNat (succ n) = Nat.succ (toNat n) := by
  rfl

@[simp] theorem ofNat_zero :
    ofNat 0 = zero := by
  rfl

@[simp] theorem ofNat_succ (n : Nat) :
    ofNat (Nat.succ n) = succ (ofNat n) := by
  rfl

/-- K4.5. Transport from Lean Nat to the δ-orbit and back is identity. -/
theorem toNat_ofNat (n : Nat) :
    toNat (ofNat n) = n := by
  induction n with
  | zero => rfl
  | succ n ih =>
      simp [ofNat, ih]

/-- K4.5. Transport from the δ-orbit to Lean Nat and back is identity. -/
theorem ofNat_toNat (n : DistinctionNat) :
    ofNat (toNat n) = n := by
  induction n with
```

```
    | zero => rfl
    | succ n ih =>
        simp [toNat, ih]

/-- K4.5. The δ-orbit is equivalent to Lean Nat as a verifier display. -/
def equivNat : DistinctionNat ≃ Nat where
  toFun := toNat
  invFun := ofNat
  left_inv := ofNat_toNat
  right_inv := toNat_ofNat

end DistinctionNat

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

6. OrbitArithmetic.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitArithmetic.lean · **Lines:** 194

```
/-
  PrimitiveRecognitionCalculus/OrbitArithmetic.lean

  Round-trip source:
    PRC_Kernel_Spec_20260526.html

  Spec anchors:
    K4.5 (orbit arithmetic), K4.6 (balanced length), K4.7 (cross-multiplication)

  Addition and multiplication on the δ-orbit, with the transport theorems
  to Lean Nat needed for the balanced and cross-multiplication
  characterizations of `PRCInt` and `PRCRat`.

  Strength: δ-only. The construction uses only the inductive structure of
  the orbit and the verifier-level transport already established in
  `Orbit.lean`. No project-local axioms.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Orbit

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus
namespace DistinctionNat

/-! ## Addition on the δ-orbit -/

/-- K4.5. Addition of orbit positions: concatenation of repetition. -/
def add : DistinctionNat → DistinctionNat → DistinctionNat
  | a, zero => a
  | a, succ b => succ (add a b)

instance : Add DistinctionNat := (add)

theorem add_def (a b : DistinctionNat) :
    a + b = add a b := rfl

theorem add_zero_eq (a : DistinctionNat) :
    a + zero = a := rfl

theorem add_succ_eq (a b : DistinctionNat) :
    a + succ b = succ (a + b) := rfl

theorem zero_add_eq (a : DistinctionNat) :
    zero + a = a := by
  induction a with
  | zero => rfl
  | succ n ih =>
      show succ (zero + n) = succ n
      rw [ih]

theorem succ_add_eq (a b : DistinctionNat) :
    succ a + b = succ (a + b) := by
  induction b with
  | zero => rfl
  | succ n ih =>
      show succ (succ a + n) = succ (succ (a + n))
      rw [ih]

theorem add_comm (a b : DistinctionNat) :
    a + b = b + a := by
  induction a with
  | zero =>
      rw [zero_add_eq, add_zero_eq]
  | succ n ih =>
      rw [succ_add_eq, add_succ_eq, ih]

theorem add_assoc (a b c : DistinctionNat) :
    (a + b) + c = a + (b + c) := by
  induction c with
  | zero => rfl
  | succ n ih =>
      show (a + b) + succ n = a + (b + succ n)
      rw [add_succ_eq, add_succ_eq, add_succ_eq, ih]

/-! ## Transport to Lean Nat -/

/-- K4.5. The verifier display of orbit addition matches Lean Nat addition. -/
theorem toNat_add (a b : DistinctionNat) :
    (a + b).toNat = a.toNat + b.toNat := by
  induction b with
  | zero =>
      rw [add_zero_eq, toNat_zero, Nat.add_zero]
  | succ n ih =>
      show (succ (a + n)).toNat = a.toNat + (succ n).toNat
      rw [toNat_succ, toNat_succ, ih]
      omega

/-- The verifier display is injective: equal Nat displays come from equal
```

```
orbit positions. -/
theorem toNat_inj {a b : DistinctionNat} (h : a.toNat = b.toNat) :
    a = b := by
  have := congrArg DistinctionNat.ofNat h
  rwa [ofNat_toNat, ofNat_toNat] at this

/-! ## Cancellation -/

/-- K4.5. Left cancellation for orbit addition. -/
theorem add_left_cancel {a b c : DistinctionNat}
    (h : a + b = a + c) : b = c := by
  apply toNat_inj
  have h' : (a + b).toNat = (a + c).toNat := by rw [h]
  rw [toNat_add, toNat_add] at h'
  exact Nat.add_left_cancel h'

/-- K4.5. Right cancellation for orbit addition. -/
theorem add_right_cancel {a b c : DistinctionNat}
    (h : a + c = b + c) : a = b := by
  apply add_left_cancel (a := c)
  rw [add_comm c a, add_comm c b]
  exact h

/-! ## Multiplication on the δ-orbit -/

/-- K4.7. Multiplication of orbit positions: nested repetition. -/
def mul : DistinctionNat → DistinctionNat → DistinctionNat
  | _, zero => zero
  | a, succ b => mul a b + a

instance : Mul DistinctionNat := (mul)

theorem mul_def (a b : DistinctionNat) :
    a * b = mul a b := rfl

theorem mul_zero_eq (a : DistinctionNat) :
    a * zero = zero := rfl

theorem mul_succ_eq (a b : DistinctionNat) :
    a * succ b = a * b + a := rfl

theorem zero_mul_eq (a : DistinctionNat) :
    zero * a = zero := by
  induction a with
  | zero => rfl
  | succ n ih =>
      show zero * n + zero = zero
      rw [add_zero_eq, ih]

theorem succ_mul_eq (a b : DistinctionNat) :
    succ a * b = a * b + b := by
  induction b with
  | zero =>
      rw [mul_zero_eq, mul_zero_eq, add_zero_eq]
  | succ n ih =>
      show succ a * n + succ a = (a * n + a) + succ n
      rw [ih, add_succ_eq, add_succ_eq]
      congr 1
      rw [add_assoc, add_assoc, add_comm a n]

theorem mul_comm (a b : DistinctionNat) :
    a * b = b * a := by
  induction a with
  | zero =>
      rw [zero_mul_eq, mul_zero_eq]
  | succ n ih =>
      rw [succ_mul_eq, mul_succ_eq, ih]

/-- K4.7. The verifier display of orbit multiplication matches Lean Nat. -/
theorem toNat_mul (a b : DistinctionNat) :
    (a * b).toNat = a.toNat * b.toNat := by
  induction b with
  | zero =>
      show (a * zero).toNat = a.toNat * zero.toNat
      rw [mul_zero_eq, toNat_zero]
      omega
  | succ n ih =>
      show (a * n + a).toNat = a.toNat * (succ n).toNat
      rw [toNat_add, toNat_succ, ih, Nat.mul_succ]

/-- K4.7. Product of nonzero orbit positions is nonzero. -/
theorem mul_ne_zero {a b : DistinctionNat}
    (ha : a ≠ zero) (hb : b ≠ zero) :
    a * b ≠ zero := by
  intro h
  have hnat : (a * b).toNat = zero.toNat := by rw [h]
  rw [toNat_mul, toNat_zero] at hnat
  rcases Nat.mul_eq_zero.mp hnat with hzero | hzero
  · have : a = zero := by
      apply toNat_inj
      rw [toNat_zero]
      exact hzero
    exact ha this
  · have : b = zero := by
      apply toNat_inj
      rw [toNat_zero]
      exact hzero
    exact hb this

end DistinctionNat

```

```
0192  end PrimitiveRecognitionCalculus
0193  end Foundation
0194  end IndisputableMonolith
```

7. IntegerRational.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/IntegerRational.lean · **Lines:** 1289

```
0001  /-
0002    PrimitiveRecognitionCalculus/IntegerRational.lean
0003
0004    Round-trip source:
0005      PRC_Kernel_Spec_20260526.html
0006
0007    Spec anchors:
0008      K4.6, K4.7, K4.8, K4.9, K4.10, A5
0009
0010    Quotient-native PRC integers and rationals. The equivalence relations are
0011    internal:
0012
0013      - PRCInt: signed orbits identified by balanced orbit length,
0014          a.pos + b.neg = b.pos + a.neg.
0015      - PRCRat: ratio orbits identified by cross-multiplication of the
0016          signed numerator by the denominator,
0017          a.num · b.den ~ b.num · a.den (balanced equality of SignedOrbits).
0018
0019    The maps to verifier `ℤ` and `ℚ` are conservative displays, not
0020    definitional.
0021  -/
0022
0023  import Mathlib
0024  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Orbit
0025  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitArithmetic
0026
0027  namespace IndisputableMonolith
0028  namespace Foundation
0029  namespace PrimitiveRecognitionCalculus
0030
0031  namespace DistinctionNat
0032
0033  /-! ## Internal comparison support for signed-orbit order -/
0034
0035  /-- Truncated subtraction on finite δ-orbit positions. -/
0036  def truncatedSub : DistinctionNat → DistinctionNat → DistinctionNat
0037    | a, zero => a
0038    | zero, succ _ => zero
0039    | succ a, succ b => truncatedSub a b
0040
0041  /-- Boolean `≤` on finite δ-orbit positions, by structural recursion only. -/
0042  def leq : DistinctionNat → DistinctionNat → Bool
0043    | zero, _ => true
0044    | succ _, zero => false
0045    | succ a, succ b => leq a b
0046
0047  /-- Absolute difference of two finite δ-orbit positions. -/
0048  def absDiff (a b : DistinctionNat) : DistinctionNat :=
0049    truncatedSub a b + truncatedSub b a
0050
0051  /-- Verifier display of internal truncated subtraction. -/
0052  theorem toNat_truncatedSub (a b : DistinctionNat) :
0053      (truncatedSub a b).toNat = a.toNat - b.toNat := by
0054    induction a generalizing b with
0055    | zero =>
0056        cases b with
0057        | zero => rfl
0058        | succ b => simp [truncatedSub]
0059    | succ a ih =>
0060        cases b with
0061        | zero => rfl
0062        | succ b =>
0063            simp [truncatedSub, ih]
0064
0065  /-- Internal Boolean order agrees with the verifier `Nat` order. -/
0066  theorem leq_eq_true_iff (a b : DistinctionNat) :
0067      leq a b = true ↔ a.toNat ≤ b.toNat := by
0068    induction a generalizing b with
0069    | zero =>
0070        cases b with
0071        | zero => simp [leq]
0072        | succ b => simp [leq]
0073    | succ a ih =>
0074        cases b with
0075        | zero => simp [leq]
0076        | succ b =>
0077            simp [leq, ih]
0078
0079  /-- Internal Boolean order is false exactly when verifier order is reversed. -/
0080  theorem leq_eq_false_iff (a b : DistinctionNat) :
0081      leq a b = false ↔ b.toNat < a.toNat := by
0082    rw [← Bool.not_eq_true, leq_eq_true_iff]
0083    omega
0084
0085  /-- Verifier display of internal absolute difference. -/
0086  theorem toNat_absDiff (a b : DistinctionNat) :
0087      (absDiff a b).toNat =
0088        Int.natAbs ((a.toNat : ℤ) - (b.toNat : ℤ)) := by
0089    unfold absDiff
0090    rw [toNat_add, toNat_truncatedSub, toNat_truncatedSub]
```

```
  by_cases h : b.toNat ≤ a.toNat
  · have hzero : b.toNat - a.toNat = 0 := Nat.sub_eq_zero_of_le h
    rw [hzero, Nat.add_zero]
    have hnonneg : 0 ≤ (a.toNat : ℤ) - (b.toNat : ℤ) := by
      omega
    have hcast : ((Int.natAbs ((a.toNat : ℤ) - (b.toNat : ℤ)) : ℕ) : ℤ) =
        (a.toNat : ℤ) - (b.toNat : ℤ)) := by
      rw [Int.natAbs_of_nonneg hnonneg]
    apply Nat.cast_injective (R := ℤ)
    rw [hcast]
    omega
  · have hle : a.toNat ≤ b.toNat := by omega
    have hzero : a.toNat - b.toNat = 0 := Nat.sub_eq_zero_of_le hle
    rw [hzero, Nat.zero_add]
    have hnonpos : (a.toNat : ℤ) - (b.toNat : ℤ) ≤ 0 := by
      omega
    have hcast : ((Int.natAbs ((a.toNat : ℤ) - (b.toNat : ℤ)) : ℕ) : ℤ) =
        -((a.toNat : ℤ) - (b.toNat : ℤ))) := by
      have hneg_nonneg : 0 ≤ -((a.toNat : ℤ) - (b.toNat : ℤ)) := by
        omega
      have hneg_abs : ((Int.natAbs (-((a.toNat : ℤ) - (b.toNat : ℤ))) : ℕ) =
          -((a.toNat : ℤ) - (b.toNat : ℤ))) := by
        rw [Int.natAbs_of_nonneg hneg_nonneg]
      rwa [Int.natAbs_neg] at hneg_abs
    apply Nat.cast_injective (R := ℤ)
    rw [hcast]
    omega

end DistinctionNat

/-! ## Signed orbits and the balanced-length equivalence -/

/-- K4.6. A signed orbit difference. Intended meaning: `pos - neg`. -/
structure SignedOrbit where
  pos : DistinctionNat
  neg : DistinctionNat
  deriving DecidableEq, Repr

namespace SignedOrbit

/-- A verifier display of a signed orbit as an integer. -/
def toInt (z : SignedOrbit) : ℤ :=
  (z.pos.toNat : ℤ) - (z.neg.toNat : ℤ)

@[simp] theorem toInt_mk (a b : DistinctionNat) :
    toInt (a, b) = (a.toNat : ℤ) - (b.toNat : ℤ) := by
  rfl

/-- The zero signed orbit. -/
def zero : SignedOrbit :=
  (DistinctionNat.zero, DistinctionNat.zero)

@[simp] theorem zero_toInt :
    zero.toInt = 0 := by
  rfl

/-- The unit signed orbit. -/
def one : SignedOrbit :=
  (DistinctionNat.succ DistinctionNat.zero, DistinctionNat.zero)

@[simp] theorem one_toInt :
    one.toInt = 1 := by
  rfl

/-- A nonnegative signed orbit built from a δ-orbit position. -/
def ofOrbit (n : DistinctionNat) : SignedOrbit :=
  (n, DistinctionNat.zero)

@[simp] theorem ofOrbit_toInt (n : DistinctionNat) :
    (ofOrbit n).toInt = n.toNat := by
  show (n.toNat : ℤ) - 0 = n.toNat
  ring

/-- Pointwise addition of signed orbits. -/
def add (a b : SignedOrbit) : SignedOrbit where
  pos := a.pos + b.pos
  neg := a.neg + b.neg

@[simp] theorem add_pos (a b : SignedOrbit) :
    (add a b).pos = a.pos + b.pos := rfl

@[simp] theorem add_neg (a b : SignedOrbit) :
    (add a b).neg = a.neg + b.neg := rfl

theorem add_toInt (a b : SignedOrbit) :
    (add a b).toInt = a.toInt + b.toInt := by
  show ((a.pos + b.pos).toNat : ℤ) - ((a.neg + b.neg).toNat : ℤ) =
    ((a.pos.toNat : ℤ) - (a.neg.toNat : ℤ)) +
    ((b.pos.toNat : ℤ) - (b.neg.toNat : ℤ))
  rw [DistinctionNat.toNat_add, DistinctionNat.toNat_add]
  push_cast
  ring

/-- Pointwise negation of signed orbits: swap pos and neg. Named
`negate` rather than `neg` to avoid collision with the structure field. -/
def negate (a : SignedOrbit) : SignedOrbit where
  pos := a.neg
  neg := a.pos

@[simp] theorem negate_pos (a : SignedOrbit) :
    (negate a).pos = a.neg := rfl
```

```
0192
0193 @[simp] theorem negate_neg (a : SignedOrbit) :
0194     (negate a).neg = a.pos := rfl
0195
0196 theorem negate_toInt (a : SignedOrbit) :
0197     (negate a).toInt = -a.toInt := by
0198   show (a.neg.toNat : ℤ) - (a.pos.toNat : ℤ) =
0199     -((a.pos.toNat : ℤ) - (a.neg.toNat : ℤ))
0200   ring
0201
0202 /-- Signed orbit multiplication:
0203 `(p₁ − n₁) · (p₂ − n₂) = (p₁p₂ + n₁n₂) − (p₁n₂ + n₁p₂)`. -/
0204 def mul (a b : SignedOrbit) : SignedOrbit where
0205   pos := a.pos * b.pos + a.neg * b.neg
0206   neg := a.pos * b.neg + a.neg * b.pos
0207
0208 @[simp] theorem mul_pos (a b : SignedOrbit) :
0209     (mul a b).pos = a.pos * b.pos + a.neg * b.neg := rfl
0210
0211 @[simp] theorem mul_neg (a b : SignedOrbit) :
0212     (mul a b).neg = a.pos * b.neg + a.neg * b.pos := rfl
0213
0214 theorem mul_toInt (a b : SignedOrbit) :
0215     (mul a b).toInt = a.toInt * b.toInt := by
0216   show ((a.pos * b.pos + a.neg * b.neg).toNat : ℤ) -
0217       ((a.pos * b.neg + a.neg * b.pos).toNat : ℤ) =
0218     ((a.pos.toNat : ℤ) - (a.neg.toNat : ℤ)) *
0219     ((b.pos.toNat : ℤ) - (b.neg.toNat : ℤ))
0220   rw [DistinctionNat.toNat_add, DistinctionNat.toNat_add,
0221       DistinctionNat.toNat_mul, DistinctionNat.toNat_mul,
0222       DistinctionNat.toNat_mul, DistinctionNat.toNat_mul]
0223   push_cast
0224   ring
0225
0226 /-- Subtraction on signed orbits via the negation. -/
0227 def sub (a b : SignedOrbit) : SignedOrbit :=
0228   add a (negate b)
0229
0230 theorem sub_toInt (a b : SignedOrbit) :
0231     (sub a b).toInt = a.toInt - b.toInt := by
0232   show (add a (negate b)).toInt = a.toInt - b.toInt
0233   rw [add_toInt, negate_toInt]
0234   ring
0235
0236 /-- Scale a signed orbit by a (positive-only) orbit position. -/
0237 def scaleByNat (z : SignedOrbit) (d : DistinctionNat) : SignedOrbit where
0238   pos := z.pos * d
0239   neg := z.neg * d
0240
0241 @[simp] theorem scaleByNat_pos (z : SignedOrbit) (d : DistinctionNat) :
0242     (z.scaleByNat d).pos = z.pos * d := rfl
0243
0244 @[simp] theorem scaleByNat_neg (z : SignedOrbit) (d : DistinctionNat) :
0245     (z.scaleByNat d).neg = z.neg * d := rfl
0246
0247 theorem scaleByNat_toInt (z : SignedOrbit) (d : DistinctionNat) :
0248     (z.scaleByNat d).toInt = z.toInt * (d.toNat : ℤ) := by
0249   show ((z.pos * d).toNat : ℤ) - ((z.neg * d).toNat : ℤ) =
0250     ((z.pos.toNat : ℤ) - (z.neg.toNat : ℤ)) * (d.toNat : ℤ)
0251   rw [DistinctionNat.toNat_mul, DistinctionNat.toNat_mul]
0252   push_cast
0253   ring
0254
0255 /-! ### K4.9. Balanced-length equivalence -/
0256
0257 /-- K4.9. Two signed orbits are equivalent when their orbit lengths balance:
0258 `a.pos + b.neg = b.pos + a.neg`. This is the internal PRC integer relation,
0259 defined entirely on δ-orbit positions. -/
0260 def balanced (a b : SignedOrbit) : Prop :=
0261   a.pos + b.neg = b.pos + a.neg
0262
0263 instance instDecidableBalanced (a b : SignedOrbit) :
0264     Decidable (balanced a b) := by
0265   unfold balanced
0266   infer_instance
0267
0268 /-- K4.9. Characterization of balanced length by Nat-level addition. -/
0269 theorem balanced_iff_toNat_eq (a b : SignedOrbit) :
0270     balanced a b ↔
0271       a.pos.toNat + b.neg.toNat = b.pos.toNat + a.neg.toNat := by
0272   unfold balanced
0273   constructor
0274   · intro h
0275     have := congrArg DistinctionNat.toNat h
0276     rwa [DistinctionNat.toNat_add, DistinctionNat.toNat_add] at this
0277   · intro h
0278     apply DistinctionNat.toNat_inj
0279     rw [DistinctionNat.toNat_add, DistinctionNat.toNat_add]
0280     exact h
0281
0282 /-- K4.9. The balanced characterization agrees with the verifier integer
0283 display. This is the bridge from the internal PRC relation to the
0284 conservative `ℤ` view. -/
0285 theorem balanced_iff_toInt_eq (a b : SignedOrbit) :
0286     balanced a b ↔ a.toInt = b.toInt := by
0287   rw [balanced_iff_toNat_eq]
0288   unfold SignedOrbit.toInt
0289   constructor
0290   · intro h
0291     have : ((a.pos.toNat + b.neg.toNat : ℕ) : ℤ) =
0292         ((b.pos.toNat + a.neg.toNat : ℕ) : ℤ) := by exact_mod_cast h
```

```
    push_cast at this
    linarith
  · intro h
    have : (a.pos.toNat : ℤ) + (b.neg.toNat : ℤ) =
        (b.pos.toNat : ℤ) + (a.neg.toNat : ℤ) := by linarith
    exact_mod_cast this

theorem balanced_refl (a : SignedOrbit) : balanced a a := by
  unfold balanced
  rw [DistinctionNat.add_comm]

theorem balanced_symm {a b : SignedOrbit} (h : balanced a b) :
    balanced b a := by
  unfold balanced at *
  exact h.symm

theorem balanced_trans {a b c : SignedOrbit}
    (hab : balanced a b) (hbc : balanced b c) : balanced a c := by
  rw [balanced_iff_toNat_eq] at hab hbc ⊢
  omega

theorem balanced_equivalence : Equivalence balanced := {
  refl := balanced_refl
  symm := balanced_symm
  trans := balanced_trans
}

/-! ### K4.13. Signed-orbit order and absolute value -/

/-- Internal nonnegativity: a signed orbit balances with a positive orbit. -/
def nonneg (z : SignedOrbit) : Prop :=
  ∃ k : DistinctionNat, SignedOrbit.balanced z (SignedOrbit.ofOrbit k)

/-- Computable nonnegative flag from structural comparison of the two sides. -/
def nonnegFlag (z : SignedOrbit) : Bool :=
  DistinctionNat.leq z.neg z.pos

/-- Strict negativity as failure of the structural nonnegative flag. -/
def negativeFlag (z : SignedOrbit) : Bool :=
  !z.nonnegFlag

/-- Internal signed-orbit order: `a ≤ b` when `b - a` is nonnegative. -/
def le (a b : SignedOrbit) : Prop :=
  nonneg (SignedOrbit.sub b a)

/-- Internal strict order: nonnegative difference with nonzero difference. -/
def lt (a b : SignedOrbit) : Prop :=
  le a b ∧ ¬ SignedOrbit.balanced a b

/-- Absolute value of a signed orbit as an orbit position. -/
def abs (z : SignedOrbit) : DistinctionNat :=
  DistinctionNat.absDiff z.pos z.neg

theorem nonnegFlag_eq_true_iff (z : SignedOrbit) :
    z.nonnegFlag = true ↔ 0 ≤ z.toInt := by
  unfold nonnegFlag SignedOrbit.toInt
  rw [DistinctionNat.leq_eq_true_iff]
  omega

theorem nonnegFlag_eq_false_iff (z : SignedOrbit) :
    z.nonnegFlag = false ↔ z.toInt < 0 := by
  rw [← Bool.not_eq_true, nonnegFlag_eq_true_iff]
  omega

/-- Internal nonnegativity agrees with the verifier integer display. -/
theorem nonneg_iff_toInt_nonneg (z : SignedOrbit) :
    nonneg z ↔ 0 ≤ z.toInt := by
  constructor
  · intro h
    rcases h with (k, hk)
    have hdisplay := (SignedOrbit.balanced_iff_toInt_eq z (SignedOrbit.ofOrbit k)).mp hk
    rw [SignedOrbit.ofOrbit_toInt] at hdisplay
    omega
  · intro hz
    refine (DistinctionNat.ofNat z.toInt.toNat, ?_)
    rw [SignedOrbit.balanced_iff_toInt_eq, SignedOrbit.ofOrbit_toInt,
      DistinctionNat.toNat_ofNat]
    omega

/-- The structural nonnegative flag is equivalent to internal nonnegativity. -/
theorem nonnegFlag_eq_true_iff_nonneg (z : SignedOrbit) :
    z.nonnegFlag = true ↔ nonneg z := by
  rw [nonnegFlag_eq_true_iff, nonneg_iff_toInt_nonneg]

theorem negativeFlag_eq_true_iff_toInt_neg (z : SignedOrbit) :
    z.negativeFlag = true ↔ z.toInt < 0 := by
  unfold negativeFlag
  by_cases h : z.nonnegFlag = true
  · rw [h]
    simp
    rw [nonnegFlag_eq_true_iff] at h
    omega
  · have hf : z.nonnegFlag = false := by
      cases hflag : z.nonnegFlag with
      | false => rfl
      | true =>
          exfalso
          exact h hflag
    rw [hf]
    simp
    exact (nonnegFlag_eq_false_iff z).mp hf
```

```
0394
0395 /-- Verifier display of internal absolute value. -/
0396 theorem abs_toNat (z : SignedOrbit) :
0397     z.abs.toNat = Int.natAbs z.toInt := by
0398   unfold abs SignedOrbit.toInt
0399   exact DistinctionNat.toNat_absDiff z.pos z.neg
0400
0401 theorem abs_eq_zero_iff_toInt_eq_zero (z : SignedOrbit) :
0402     z.abs = DistinctionNat.zero ↔ z.toInt = 0 := by
0403   constructor
0404   · intro h
0405     have hnat : z.abs.toNat = 0 := by
0406       rw [h, DistinctionNat.toNat_zero]
0407     rw [abs_toNat] at hnat
0408     exact Int.natAbs_eq_zero.mp hnat
0409   · intro h
0410     apply DistinctionNat.toNat_inj
0411     rw [abs_toNat, h, Int.natAbs_zero, DistinctionNat.toNat_zero]
0412
0413 theorem abs_ne_zero_of_toInt_ne_zero {z : SignedOrbit}
0414     (h : z.toInt ≠ 0) :
0415     z.abs ≠ DistinctionNat.zero := by
0416   intro hz
0417   exact h ((abs_eq_zero_iff_toInt_eq_zero z).mp hz)
0418
0419 theorem abs_ne_zero_of_not_balanced_zero {z : SignedOrbit}
0420     (h : ¬ SignedOrbit.balanced z SignedOrbit.zero) :
0421     z.abs ≠ DistinctionNat.zero := by
0422   apply abs_ne_zero_of_toInt_ne_zero
0423   intro hz
0424   exact h ((SignedOrbit.balanced_iff_toInt_eq z SignedOrbit.zero).mpr (by
0425     rw [hz, SignedOrbit.zero_toInt]))
0426
0427 theorem le_iff_toInt_le (a b : SignedOrbit) :
0428     le a b ↔ a.toInt ≤ b.toInt := by
0429   unfold le
0430   rw [nonneg_iff_toInt_nonneg, SignedOrbit.sub_toInt]
0431   omega
0432
0433 theorem lt_iff_toInt_lt (a b : SignedOrbit) :
0434     lt a b ↔ a.toInt < b.toInt := by
0435   unfold lt
0436   rw [le_iff_toInt_le, SignedOrbit.balanced_iff_toInt_eq]
0437   omega
0438
0439 end SignedOrbit
0440
0441 /-- K4.9. Signed-orbit equivalence is the balanced-length relation. -/
0442 def signedOrbitEquiv (a b : SignedOrbit) : Prop :=
0443   SignedOrbit.balanced a b
0444
0445 theorem signedOrbitEquiv_equivalence :
0446     Equivalence signedOrbitEquiv :=
0447   SignedOrbit.balanced_equivalence
0448
0449 theorem signedOrbitEquiv_iff_toInt_eq (a b : SignedOrbit) :
0450     signedOrbitEquiv a b ↔ a.toInt = b.toInt :=
0451   SignedOrbit.balanced_iff_toInt_eq a b
0452
0453 /-- K4.8. Setoid for quotient-native PRC integers. -/
0454 def signedOrbitSetoid : Setoid SignedOrbit where
0455   r := signedOrbitEquiv
0456   iseqv := signedOrbitEquiv_equivalence
0457
0458 /-- K4.8. PRC integers as signed-orbit quotient classes. The quotient is
0459 taken by the internal balanced-length relation; the verifier display into
0460 `ℤ` is a downstream theorem. -/
0461 def PRCInt : Type :=
0462   Quot signedOrbitSetoid
0463
0464 namespace PRCInt
0465
0466 /-- K4.8. Constructor from a signed orbit display. -/
0467 def mk (z : SignedOrbit) : PRCInt :=
0468   Quot.mk signedOrbitSetoid z
0469
0470 /-- K4.8/A5. Conservative verifier display of a PRC integer as `ℤ`. -/
0471 def toInt : PRCInt → ℤ :=
0472   Quot.lift SignedOrbit.toInt (by
0473     intro a b h
0474     exact (signedOrbitEquiv_iff_toInt_eq a b).mp h)
0475
0476 @[simp] theorem toInt_mk (z : SignedOrbit) :
0477     toInt (mk z) = z.toInt := by
0478   rfl
0479
0480 /-- K4.8. The zero PRC integer. -/
0481 def zero : PRCInt :=
0482   mk SignedOrbit.zero
0483
0484 @[simp] theorem zero_toInt :
0485     zero.toInt = 0 := by
0486   rfl
0487
0488 /-- K4.8. The unit PRC integer. -/
0489 def one : PRCInt :=
0490   mk SignedOrbit.one
0491
0492 @[simp] theorem one_toInt :
0493     one.toInt = 1 := by
0494   rfl
```

```
0495
0496 /-- K4.8. Equal balanced signed orbits determine equal PRC integers. -/
0497 theorem mk_eq_mk_of_balanced {a b : SignedOrbit}
0498     (h : SignedOrbit.balanced a b) :
0499     mk a = mk b :=
0500   Quot.sound h
0501
0502 /-! ### PRCInt operations -/
0503
0504 /-- Addition respects balanced equivalence. -/
0505 private theorem add_respects_balanced {a₁ a₂ b₁ b₂ : SignedOrbit}
0506     (ha : SignedOrbit.balanced a₁ a₂) (hb : SignedOrbit.balanced b₁ b₂) :
0507     SignedOrbit.balanced (SignedOrbit.add a₁ b₁) (SignedOrbit.add a₂ b₂) := by
0508   rw [SignedOrbit.balanced_iff_toInt_eq] at *
0509   rw [SignedOrbit.add_toInt, SignedOrbit.add_toInt, ha, hb]
0510
0511 /-- K4.8. Addition on PRC integers, lifted from signed-orbit addition. -/
0512 def add : PRCInt → PRCInt → PRCInt :=
0513   Quot.lift₂
0514     (fun a b => mk (SignedOrbit.add a b))
0515     (by
0516       intro a b₁ b₂ h
0517       apply Quot.sound
0518       exact add_respects_balanced (SignedOrbit.balanced_refl a) h)
0519     (by
0520       intro a₁ a₂ b h
0521       apply Quot.sound
0522       exact add_respects_balanced h (SignedOrbit.balanced_refl b))
0523
0524 @[simp] theorem add_mk (a b : SignedOrbit) :
0525     add (mk a) (mk b) = mk (SignedOrbit.add a b) := by
0526   rfl
0527
0528 @[simp] theorem toInt_add (a b : PRCInt) :
0529     (add a b).toInt = a.toInt + b.toInt := by
0530   refine Quot.induction_on a (fun a => ?_)
0531   refine Quot.induction_on b (fun b => ?_)
0532   show (SignedOrbit.add a b).toInt = a.toInt + b.toInt
0533   exact SignedOrbit.add_toInt a b
0534
0535 /-- Negation respects balanced equivalence. -/
0536 private theorem negate_respects_balanced {a₁ a₂ : SignedOrbit}
0537     (h : SignedOrbit.balanced a₁ a₂) :
0538     SignedOrbit.balanced (SignedOrbit.negate a₁) (SignedOrbit.negate a₂) := by
0539   rw [SignedOrbit.balanced_iff_toInt_eq] at *
0540   rw [SignedOrbit.negate_toInt, SignedOrbit.negate_toInt, h]
0541
0542 /-- K4.8. Negation on PRC integers, lifted from signed-orbit swap. -/
0543 def negate : PRCInt → PRCInt :=
0544   Quot.lift
0545     (fun a => mk (SignedOrbit.negate a))
0546     (by
0547       intro a b h
0548       apply Quot.sound
0549       exact negate_respects_balanced h)
0550
0551 @[simp] theorem negate_mk (a : SignedOrbit) :
0552     negate (mk a) = mk (SignedOrbit.negate a) := by
0553   rfl
0554
0555 @[simp] theorem toInt_negate (a : PRCInt) :
0556     (negate a).toInt = -a.toInt := by
0557   refine Quot.induction_on a (fun a => ?_)
0558   show (SignedOrbit.negate a).toInt = -a.toInt
0559   exact SignedOrbit.negate_toInt a
0560
0561 /-- K4.8. The toInt display is injective: distinct PRC integers have
0562 distinct verifier displays. -/
0563 theorem toInt_injective : Function.Injective toInt := by
0564   intro a b h
0565   induction a using Quot.ind with
0566   | _ a =>
0567     induction b using Quot.ind with
0568     | _ b =>
0569       apply Quot.sound
0570       exact (signedOrbitEquiv_iff_toInt_eq a b).mpr h
0571
0572 /-- Multiplication respects balanced equivalence. -/
0573 private theorem mul_respects_balanced {a₁ a₂ b₁ b₂ : SignedOrbit}
0574     (ha : SignedOrbit.balanced a₁ a₂) (hb : SignedOrbit.balanced b₁ b₂) :
0575     SignedOrbit.balanced (SignedOrbit.mul a₁ b₁) (SignedOrbit.mul a₂ b₂) := by
0576   rw [SignedOrbit.balanced_iff_toInt_eq] at *
0577   rw [SignedOrbit.mul_toInt, SignedOrbit.mul_toInt, ha, hb]
0578
0579 /-- K4.8. Multiplication on PRC integers, lifted from signed-orbit
0580 multiplication. -/
0581 def mul : PRCInt → PRCInt → PRCInt :=
0582   Quot.lift₂
0583     (fun a b => mk (SignedOrbit.mul a b))
0584     (by
0585       intro a b₁ b₂ h
0586       apply Quot.sound
0587       exact mul_respects_balanced (SignedOrbit.balanced_refl a) h)
0588     (by
0589       intro a₁ a₂ b h
0590       apply Quot.sound
0591       exact mul_respects_balanced h (SignedOrbit.balanced_refl b))
0592
0593 @[simp] theorem mul_mk (a b : SignedOrbit) :
0594     mul (mk a) (mk b) = mk (SignedOrbit.mul a b) := by
0595   rfl
```

```
0596
0597 @[simp] theorem toInt_mul (a b : PRCInt) :
0598     (mul a b).toInt = a.toInt * b.toInt := by
0599   refine Quot.induction_on a (fun a => ?_)
0600   refine Quot.induction_on b (fun b => ?_)
0601   show (SignedOrbit.mul a b).toInt = a.toInt * b.toInt
0602   exact SignedOrbit.mul_toInt a b
0603
0604 /-- K4.8. Subtraction on PRC integers as add ∘ negate. -/
0605 def sub (a b : PRCInt) : PRCInt :=
0606   add a (negate b)
0607
0608 @[simp] theorem toInt_sub (a b : PRCInt) :
0609     (sub a b).toInt = a.toInt - b.toInt := by
0610   show (add a (negate b)).toInt = a.toInt - b.toInt
0611   rw [toInt_add, toInt_negate]
0612   ring
0613
0614 /-! ### Ring axioms on PRC integers -/
0615
0616 theorem add_comm (a b : PRCInt) : add a b = add b a := by
0617   apply toInt_injective
0618   simp [Int.add_comm]
0619
0620 theorem add_assoc (a b c : PRCInt) :
0621     add (add a b) c = add a (add b c) := by
0622   apply toInt_injective
0623   simp [Int.add_assoc]
0624
0625 theorem zero_add (a : PRCInt) : add zero a = a := by
0626   apply toInt_injective
0627   simp
0628
0629 theorem add_zero (a : PRCInt) : add a zero = a := by
0630   apply toInt_injective
0631   simp
0632
0633 theorem add_negate (a : PRCInt) : add a (negate a) = zero := by
0634   apply toInt_injective
0635   simp
0636
0637 theorem negate_add (a : PRCInt) : add (negate a) a = zero := by
0638   apply toInt_injective
0639   simp
0640
0641 theorem mul_comm (a b : PRCInt) : mul a b = mul b a := by
0642   apply toInt_injective
0643   simp [Int.mul_comm]
0644
0645 theorem mul_assoc (a b c : PRCInt) :
0646     mul (mul a b) c = mul a (mul b c) := by
0647   apply toInt_injective
0648   simp [Int.mul_assoc]
0649
0650 theorem one_mul (a : PRCInt) : mul one a = a := by
0651   apply toInt_injective
0652   simp
0653
0654 theorem mul_one (a : PRCInt) : mul a one = a := by
0655   apply toInt_injective
0656   simp
0657
0658 theorem zero_mul (a : PRCInt) : mul zero a = zero := by
0659   apply toInt_injective
0660   simp
0661
0662 theorem mul_zero (a : PRCInt) : mul a zero = zero := by
0663   apply toInt_injective
0664   simp
0665
0666 theorem left_distrib (a b c : PRCInt) :
0667     mul a (add b c) = add (mul a b) (mul a c) := by
0668   apply toInt_injective
0669   simp [Int.mul_add]
0670
0671 theorem right_distrib (a b c : PRCInt) :
0672     mul (add a b) c = add (mul a c) (mul b c) := by
0673   apply toInt_injective
0674   simp [Int.add_mul]
0675
0676 /-! ### PRCInt is isomorphic to ℤ -/
0677
0678 /-- Construct a PRC integer from a verifier Int by routing the positive
0679 and negative parts through the 6-orbit. -/
0680 def ofInt (n : ℤ) : PRCInt :=
0681   mk (DistinctionNat.ofNat n.toNat, DistinctionNat.ofNat (-n).toNat)
0682
0683 @[simp] theorem toInt_ofInt (n : ℤ) :
0684     (ofInt n).toInt = n := by
0685   show ((DistinctionNat.ofNat n.toNat).toNat : ℤ) -
0686       ((DistinctionNat.ofNat (-n).toNat).toNat : ℤ) = n
0687   rw [DistinctionNat.toNat_ofNat, DistinctionNat.toNat_ofNat]
0688   omega
0689
0690 @[simp] theorem ofInt_toInt (a : PRCInt) :
0691     ofInt a.toInt = a := by
0692   apply toInt_injective
0693   rw [toInt_ofInt]
0694
0695 /-- K4.8. The PRC integer surface is literally isomorphic to verifier `ℤ`.
0696 The verifier `ℤ` is therefore not assumed; it is a downstream display the
```

```
PRC quotient happens to reproduce. -/
def equivInt : PRCInt ≃ ℤ where
  toFun := toInt
  invFun := ofInt
  left_inv := ofInt_toInt
  right_inv := toInt_ofInt

@[simp] theorem ofInt_add (m n : ℤ) :
    ofInt (m + n) = add (ofInt m) (ofInt n) := by
  apply toInt_injective
  simp

@[simp] theorem ofInt_mul (m n : ℤ) :
    ofInt (m * n) = mul (ofInt m) (ofInt n) := by
  apply toInt_injective
  simp

@[simp] theorem ofInt_neg (n : ℤ) :
    ofInt (-n) = negate (ofInt n) := by
  apply toInt_injective
  simp

@[simp] theorem ofInt_zero : ofInt 0 = zero := by
  apply toInt_injective
  simp

@[simp] theorem ofInt_one : ofInt 1 = one := by
  apply toInt_injective
  simp

end PRCInt

/-! ## Operation instances on PRCInt -/

namespace PRCInt

instance instZero : Zero PRCInt := (zero)
instance instOne : One PRCInt := (one)
instance instAdd : Add PRCInt := (add)
instance instMul : Mul PRCInt := (mul)
instance instNeg : Neg PRCInt := (negate)
instance instSub : Sub PRCInt := (sub)

@[simp] theorem add_eq (a b : PRCInt) : a + b = add a b := rfl
@[simp] theorem mul_eq (a b : PRCInt) : a * b = mul a b := rfl
@[simp] theorem neg_eq (a : PRCInt) : -a = negate a := rfl
@[simp] theorem sub_eq (a b : PRCInt) : a - b = sub a b := rfl
@[simp] theorem zero_eq : (0 : PRCInt) = zero := rfl
@[simp] theorem one_eq : (1 : PRCInt) = one := rfl

/-- K4.8. The toInt display is a ring homomorphism (additive). -/
theorem toInt_add' (a b : PRCInt) :
    (a + b).toInt = a.toInt + b.toInt := by
  simp

/-- K4.8. The toInt display is a ring homomorphism (multiplicative). -/
theorem toInt_mul' (a b : PRCInt) :
    (a * b).toInt = a.toInt * b.toInt := by
  simp

/-- K4.8. The toInt display preserves negation. -/
theorem toInt_neg' (a : PRCInt) :
    (-a).toInt = -a.toInt := by
  simp

/-- K4.8. The toInt display preserves zero. -/
theorem toInt_zero' : (0 : PRCInt).toInt = 0 := by
  simp

/-- K4.8. The toInt display preserves one. -/
theorem toInt_one' : (1 : PRCInt).toInt = 1 := by
  simp

end PRCInt

/-! ## Ratio orbits and cross-multiplication -/

/-- K4.7. A rational orbit display: integer numerator over nonzero orbit
denominator. -/
structure RatioOrbit where
  num : SignedOrbit
  den : DistinctionNat
  den_ne_zero : den ≠ DistinctionNat.zero

namespace RatioOrbit

/-- The denominator's verifier Nat is nonzero. -/
theorem den_toNat_ne_zero (q : RatioOrbit) :
    q.den.toNat ≠ 0 := by
  intro h
  have hden : DistinctionNat.ofNat q.den.toNat = DistinctionNat.ofNat 0 := by
    rw [h]
  rw [DistinctionNat.ofNat_toNat, DistinctionNat.ofNat_zero] at hden
  exact q.den_ne_zero hden

/-- A verifier display of a ratio orbit as a rational number.
Spec tag A5: this is a transport wrapper. The internal characterization
is cross-multiplication. -/
def toRat (q : RatioOrbit) : ℚ :=
  (q.num.toInt : ℚ) / (q.den.toNat : ℚ)

```

```
/-- The denominator used by `toRat` is nonzero in the verifier rationals. -/
theorem den_cast_ne_zero (q : RatioOrbit) :
    (q.den.toNat : ℚ) ≠ 0 := by
  exact_mod_cast q.den_toNat_ne_zero

/-! ### Ratio-orbit arithmetic -/

/-- K4.11. Zero ratio orbit. -/
def zero : RatioOrbit where
  num := SignedOrbit.zero
  den := DistinctionNat.succ DistinctionNat.zero
  den_ne_zero := by
    intro h
    exact DistinctionNat.zero_ne_succ DistinctionNat.zero h.symm

@[simp] theorem zero_toRat :
    zero.toRat = 0 := by
  unfold zero toRat
  simp

/-- K4.11. Unit ratio orbit. -/
def one : RatioOrbit where
  num := SignedOrbit.one
  den := DistinctionNat.succ DistinctionNat.zero
  den_ne_zero := by
    intro h
    exact DistinctionNat.zero_ne_succ DistinctionNat.zero h.symm

@[simp] theorem one_toRat :
    one.toRat = 1 := by
  unfold one toRat
  simp

/-- K4.11. Addition of ratio orbits:
`a/b + c/d = (ad + cb)/(bd)`. -/
def add (a b : RatioOrbit) : RatioOrbit where
  num := SignedOrbit.add (a.num.scaleByNat b.den) (b.num.scaleByNat a.den)
  den := a.den * b.den
  den_ne_zero := DistinctionNat.mul_ne_zero a.den_ne_zero b.den_ne_zero

theorem add_toRat (a b : RatioOrbit) :
    (add a b).toRat = a.toRat + b.toRat := by
  unfold add toRat
  rw [SignedOrbit.add_toInt, SignedOrbit.scaleByNat_toInt,
      SignedOrbit.scaleByNat_toInt, DistinctionNat.toNat_mul]
  have hA : (a.den.toNat : ℚ) ≠ 0 := a.den_cast_ne_zero
  have hB : (b.den.toNat : ℚ) ≠ 0 := b.den_cast_ne_zero
  field_simp [hA, hB]
  push_cast
  ring_nf

/-- K4.11. Negation of ratio orbits. -/
def negate (a : RatioOrbit) : RatioOrbit where
  num := SignedOrbit.negate a.num
  den := a.den
  den_ne_zero := a.den_ne_zero

theorem negate_toRat (a : RatioOrbit) :
    (negate a).toRat = -a.toRat := by
  unfold negate toRat
  rw [SignedOrbit.negate_toInt]
  have hA : (a.den.toNat : ℚ) ≠ 0 := a.den_cast_ne_zero
  field_simp [hA]
  push_cast
  ring_nf

/-- K4.11. Subtraction of ratio orbits. -/
def sub (a b : RatioOrbit) : RatioOrbit :=
  add a (negate b)

theorem sub_toRat (a b : RatioOrbit) :
    (sub a b).toRat = a.toRat - b.toRat := by
  unfold sub
  rw [add_toRat, negate_toRat]
  ring

/-- K4.11. Multiplication of ratio orbits:
`a/b * c/d = (ac)/(bd)`. -/
def mul (a b : RatioOrbit) : RatioOrbit where
  num := SignedOrbit.mul a.num b.num
  den := a.den * b.den
  den_ne_zero := DistinctionNat.mul_ne_zero a.den_ne_zero b.den_ne_zero

theorem mul_toRat (a b : RatioOrbit) :
    (mul a b).toRat = a.toRat * b.toRat := by
  unfold mul toRat
  rw [SignedOrbit.mul_toInt, DistinctionNat.toNat_mul]
  have hA : (a.den.toNat : ℚ) ≠ 0 := a.den_cast_ne_zero
  have hB : (b.den.toNat : ℚ) ≠ 0 := b.den_cast_ne_zero
  field_simp [hA, hB]
  push_cast
  ring_nf

/-- K4.12. Reciprocal of a nonzero ratio orbit.

The numerator sign is selected by the structural signed-orbit comparison
`nonnegFlag`, and the denominator is the internal signed-orbit absolute value.
The verifier integer display is used only in the transport theorem below. -/
def recipNonzero (a : RatioOrbit)
    (h : ¬ SignedOrbit.balanced a.num SignedOrbit.zero) : RatioOrbit where
  num :=
```

```
    if a.num.nonnegFlag then
      SignedOrbit.ofOrbit a.den
    else
      SignedOrbit.negate (SignedOrbit.ofOrbit a.den)
  den := a.num.abs
  den_ne_zero := SignedOrbit.abs_ne_zero_of_not_balanced_zero h

theorem recipNonzero_toRat (a : RatioOrbit)
    (h : ¬ SignedOrbit.balanced a.num SignedOrbit.zero) :
    (recipNonzero a h).toRat = (a.toRat)⁻¹ := by
  unfold recipNonzero toRat
  have hDen : (a.den.toNat : ℚ) ≠ 0 := a.den_cast_ne_zero
  have hNumInt : a.num.toInt ≠ 0 := by
    intro hz
    exact h ((SignedOrbit.balanced_iff_toInt_eq a.num SignedOrbit.zero).mpr (by
      rw [hz, SignedOrbit.zero_toInt]))
  have hNum : (a.num.toInt : ℚ) ≠ 0 := by exact_mod_cast hNumInt
  by_cases hflag : a.num.nonnegFlag = true
  · have hnonneg : 0 ≤ a.num.toInt :=
      (SignedOrbit.nonnegFlag_eq_true_iff a.num).mp hflag
    have hpos : 0 < a.num.toInt := by omega
    simp [hflag, SignedOrbit.ofOrbit_toInt, SignedOrbit.abs_toNat]
    have habs : |(a.num.toInt : ℚ)| = (a.num.toInt : ℚ) := by
      exact abs_of_pos (by exact_mod_cast hpos)
    rw [habs]
  · have hflagFalse : a.num.nonnegFlag = false := by
      cases hflag' : a.num.nonnegFlag with
      | false => rfl
      | true =>
          exfalso
          exact hflag hflag'
    have hneg : a.num.toInt < 0 :=
      (SignedOrbit.nonnegFlag_eq_false_iff a.num).mp hflagFalse
    simp [hflagFalse, SignedOrbit.ofOrbit_toInt, SignedOrbit.negate_toInt,
      SignedOrbit.abs_toNat]
    have habs : |(a.num.toInt : ℚ)| = -(a.num.toInt : ℚ) := by
      exact abs_of_neg (by exact_mod_cast hneg)
    rw [habs]
    field_simp [hDen, hNum]

/-- K4.12. Total reciprocal of ratio orbits, sending zero to zero as in `ℚ`. -/
def recip (a : RatioOrbit) : RatioOrbit :=
  if h : SignedOrbit.balanced a.num SignedOrbit.zero then
    zero
  else
    recipNonzero a h

theorem recip_toRat (a : RatioOrbit) :
    (recip a).toRat = (a.toRat)⁻¹ := by
  unfold recip
  by_cases h : SignedOrbit.balanced a.num SignedOrbit.zero
  · have hzero := (SignedOrbit.balanced_iff_toInt_eq a.num SignedOrbit.zero).mp h
    simp [h, hzero, toRat, zero]
  · simp [h, recipNonzero_toRat]

/-! ### K4.10. Cross-multiplication equivalence -/

/-- K4.10. Two ratio orbits are equivalent under cross-multiplication when
`a.num · b.den` balances `b.num · a.den` as signed orbits. This is the
internal PRC rational relation, defined entirely on δ-orbit positions. -/
def crossEq (a b : RatioOrbit) : Prop :=
  SignedOrbit.balanced (a.num.scaleByNat b.den) (b.num.scaleByNat a.den)

/-- K4.10. Cross-multiplication agrees with rational equality of the
verifier displays. -/
theorem crossEq_iff_toRat_eq (a b : RatioOrbit) :
    crossEq a b ↔ a.toRat = b.toRat := by
  unfold crossEq toRat
  rw [SignedOrbit.balanced_iff_toInt_eq]
  rw [SignedOrbit.scaleByNat_toInt, SignedOrbit.scaleByNat_toInt]
  have hA : (a.den.toNat : ℚ) ≠ 0 := a.den_cast_ne_zero
  have hB : (b.den.toNat : ℚ) ≠ 0 := b.den_cast_ne_zero
  constructor
  · intro h
    field_simp
    have hQ : (a.num.toInt : ℚ) * (b.den.toNat : ℚ) =
        (b.num.toInt : ℚ) * (a.den.toNat : ℚ) := by exact_mod_cast h
    linarith
  · intro h
    have : (a.num.toInt : ℚ) * (b.den.toNat : ℚ) =
        (b.num.toInt : ℚ) * (a.den.toNat : ℚ) := by
      field_simp at h
      linarith
    have hZ : (a.num.toInt * b.den.toNat : ℤ) =
        (b.num.toInt * a.den.toNat : ℤ) := by exact_mod_cast this
    exact hZ

theorem crossEq_refl (a : RatioOrbit) : crossEq a a := by
  unfold crossEq
  exact SignedOrbit.balanced_refl _

theorem crossEq_symm {a b : RatioOrbit} (h : crossEq a b) : crossEq b a := by
  unfold crossEq at *
  exact SignedOrbit.balanced_symm h

theorem crossEq_trans {a b c : RatioOrbit}
    (hab : crossEq a b) (hbc : crossEq b c) : crossEq a c := by
  rw [crossEq_iff_toRat_eq] at hab hbc ⊢
  rw [hab, hbc]

theorem crossEq_equivalence : Equivalence crossEq := {
```

```
  refl := crossEq_refl
  symm := crossEq_symm
  trans := crossEq_trans
}

end RatioOrbit

/-- K4.8. Ratio-orbit equivalence is cross-multiplication. -/
def ratioOrbitEquiv (a b : RatioOrbit) : Prop :=
  RatioOrbit.crossEq a b

theorem ratioOrbitEquiv_equivalence :
    Equivalence ratioOrbitEquiv :=
  RatioOrbit.crossEq_equivalence

theorem ratioOrbitEquiv_iff_toRat_eq (a b : RatioOrbit) :
    ratioOrbitEquiv a b ↔ a.toRat = b.toRat :=
  RatioOrbit.crossEq_iff_toRat_eq a b

/-- K4.8. Setoid for quotient-native PRC rationals. -/
def ratioOrbitSetoid : Setoid RatioOrbit where
  r := ratioOrbitEquiv
  iseqv := ratioOrbitEquiv_equivalence

/-- K4.8. PRC rationals as nonzero-denominator ratio-orbit quotient classes,
identified by cross-multiplication of orbit-level numerator and denominator. -/
def PRCRat : Type :=
  Quot ratioOrbitSetoid

namespace PRCRat

/-- K4.8. Constructor from a ratio orbit display. -/
def mk (q : RatioOrbit) : PRCRat :=
  Quot.mk ratioOrbitSetoid q

/-- K4.8/A5. Conservative verifier display of a PRC rational as `ℚ`. -/
def toRat : PRCRat → ℚ :=
  Quot.lift RatioOrbit.toRat (by
    intro a b h
    exact (ratioOrbitEquiv_iff_toRat_eq a b).mp h)

@[simp] theorem toRat_mk (q : RatioOrbit) :
    toRat (mk q) = q.toRat := by
  rfl

/-- K4.8. Equal cross-multiplied ratio orbits determine equal PRC rationals. -/
theorem mk_eq_mk_of_crossEq {a b : RatioOrbit}
    (h : RatioOrbit.crossEq a b) :
    mk a = mk b :=
  Quot.sound h

/-- K4.8. The toRat display is injective on the quotient. -/
theorem toRat_injective : Function.Injective toRat := by
  intro a b h
  induction a using Quot.ind with
  | _ a =>
    induction b using Quot.ind with
    | _ b =>
      apply Quot.sound
      exact (ratioOrbitEquiv_iff_toRat_eq a b).mpr h

/-! ### PRCRat operations -/

/-- K4.11. Zero PRC rational. -/
def zero : PRCRat :=
  mk RatioOrbit.zero

@[simp] theorem zero_toRat :
    zero.toRat = 0 := by
  exact RatioOrbit.zero_toRat

/-- K4.11. Unit PRC rational. -/
def one : PRCRat :=
  mk RatioOrbit.one

@[simp] theorem one_toRat :
    one.toRat = 1 := by
  exact RatioOrbit.one_toRat

private theorem add_respects_cross {a₁ a₂ b₁ b₂ : RatioOrbit}
    (ha : RatioOrbit.crossEq a₁ a₂) (hb : RatioOrbit.crossEq b₁ b₂) :
    RatioOrbit.crossEq (RatioOrbit.add a₁ b₁) (RatioOrbit.add a₂ b₂) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq] at *
  rw [RatioOrbit.add_toRat, RatioOrbit.add_toRat, ha, hb]

/-- K4.11. Addition on PRC rationals, lifted from ratio-orbit addition. -/
def add : PRCRat → PRCRat → PRCRat :=
  Quot.lift₂
    (fun a b => mk (RatioOrbit.add a b))
    (by
      intro a b₁ b₂ h
      apply Quot.sound
      exact add_respects_cross (RatioOrbit.crossEq_refl a) h)
    (by
      intro a₁ a₂ b h
      apply Quot.sound
      exact add_respects_cross h (RatioOrbit.crossEq_refl b))

@[simp] theorem add_mk (a b : RatioOrbit) :
    add (mk a) (mk b) = mk (RatioOrbit.add a b) := by
  rfl
```

```
1101
1102 @[simp] theorem toRat_add (a b : PRCRat) :
1103     (add a b).toRat = a.toRat + b.toRat := by
1104   refine Quot.induction_on a (fun a => ?_)
1105   refine Quot.induction_on b (fun b => ?_)
1106   show (RatioOrbit.add a b).toRat = a.toRat + b.toRat
1107   exact RatioOrbit.add_toRat a b
1108
1109 private theorem negate_respects_cross {a b : RatioOrbit}
1110     (h : RatioOrbit.crossEq a b) :
1111     RatioOrbit.crossEq (RatioOrbit.negate a) (RatioOrbit.negate b) := by
1112   rw [RatioOrbit.crossEq_iff_toRat_eq] at *
1113   rw [RatioOrbit.negate_toRat, RatioOrbit.negate_toRat, h]
1114
1115 /-- K4.11. Negation on PRC rationals. -/
1116 def negate : PRCRat → PRCRat :=
1117   Quot.lift
1118     (fun a => mk (RatioOrbit.negate a))
1119     (by
1120       intro a b h
1121       apply Quot.sound
1122       exact negate_respects_cross h)
1123
1124 @[simp] theorem negate_mk (a : RatioOrbit) :
1125     negate (mk a) = mk (RatioOrbit.negate a) := by
1126   rfl
1127
1128 @[simp] theorem toRat_negate (a : PRCRat) :
1129     (negate a).toRat = -a.toRat := by
1130   refine Quot.induction_on a (fun a => ?_)
1131   show (RatioOrbit.negate a).toRat = -a.toRat
1132   exact RatioOrbit.negate_toRat a
1133
1134 /-- K4.11. Subtraction on PRC rationals. -/
1135 def sub (a b : PRCRat) : PRCRat :=
1136   add a (negate b)
1137
1138 @[simp] theorem toRat_sub (a b : PRCRat) :
1139     (sub a b).toRat = a.toRat - b.toRat := by
1140   show (add a (negate b)).toRat = a.toRat - b.toRat
1141   rw [toRat_add, toRat_negate]
1142   ring
1143
1144 private theorem mul_respects_cross {a₁ a₂ b₁ b₂ : RatioOrbit}
1145     (ha : RatioOrbit.crossEq a₁ a₂) (hb : RatioOrbit.crossEq b₁ b₂) :
1146     RatioOrbit.crossEq (RatioOrbit.mul a₁ b₁) (RatioOrbit.mul a₂ b₂) := by
1147   rw [RatioOrbit.crossEq_iff_toRat_eq] at *
1148   rw [RatioOrbit.mul_toRat, RatioOrbit.mul_toRat, ha, hb]
1149
1150 /-- K4.11. Multiplication on PRC rationals, lifted from ratio-orbit multiplication. -/
1151 def mul : PRCRat → PRCRat → PRCRat :=
1152   Quot.lift₂
1153     (fun a b => mk (RatioOrbit.mul a b))
1154     (by
1155       intro a b₁ b₂ h
1156       apply Quot.sound
1157       exact mul_respects_cross (RatioOrbit.crossEq_refl a) h)
1158     (by
1159       intro a₁ a₂ b h
1160       apply Quot.sound
1161       exact mul_respects_cross h (RatioOrbit.crossEq_refl b))
1162
1163 @[simp] theorem mul_mk (a b : RatioOrbit) :
1164     mul (mk a) (mk b) = mk (RatioOrbit.mul a b) := by
1165   rfl
1166
1167 @[simp] theorem toRat_mul (a b : PRCRat) :
1168     (mul a b).toRat = a.toRat * b.toRat := by
1169   refine Quot.induction_on a (fun a => ?_)
1170   refine Quot.induction_on b (fun b => ?_)
1171   show (RatioOrbit.mul a b).toRat = a.toRat * b.toRat
1172   exact RatioOrbit.mul_toRat a b
1173
1174 private theorem recip_respects_cross {a b : RatioOrbit}
1175     (h : RatioOrbit.crossEq a b) :
1176     RatioOrbit.crossEq (RatioOrbit.recip a) (RatioOrbit.recip b) := by
1177   rw [RatioOrbit.crossEq_iff_toRat_eq] at *
1178   rw [RatioOrbit.recip_toRat, RatioOrbit.recip_toRat, h]
1179
1180 /-- K4.12. Total reciprocal on PRC rationals, lifted from ratio-orbit
1181 reciprocal and sending zero to zero. -/
1182 def recip : PRCRat → PRCRat :=
1183   Quot.lift
1184     (fun a => mk (RatioOrbit.recip a))
1185     (by
1186       intro a b h
1187       apply Quot.sound
1188       exact recip_respects_cross h)
1189
1190 @[simp] theorem recip_mk (a : RatioOrbit) :
1191     recip (mk a) = mk (RatioOrbit.recip a) := by
1192   rfl
1193
1194 @[simp] theorem toRat_recip (a : PRCRat) :
1195     (recip a).toRat = (a.toRat)⁻¹ := by
1196   refine Quot.induction_on a (fun a => ?_)
1197   show (RatioOrbit.recip a).toRat = (a.toRat)⁻¹
1198   exact RatioOrbit.recip_toRat a
1199
1200 /-! ### PRCRat field-style laws, proved via the injective display -/
1201
```

```
theorem add_comm (a b : PRCRat) : add a b = add b a := by
  apply toRat_injective
  simp [Rat.add_comm]

theorem add_assoc (a b c : PRCRat) :
    add (add a b) c = add a (add b c) := by
  apply toRat_injective
  simp [Rat.add_assoc]

theorem zero_add (a : PRCRat) : add zero a = a := by
  apply toRat_injective
  simp

theorem add_zero (a : PRCRat) : add a zero = a := by
  apply toRat_injective
  simp

theorem add_negate (a : PRCRat) : add a (negate a) = zero := by
  apply toRat_injective
  simp

theorem mul_comm (a b : PRCRat) : mul a b = mul b a := by
  apply toRat_injective
  simp [Rat.mul_comm]

theorem mul_assoc (a b c : PRCRat) :
    mul (mul a b) c = mul a (mul b c) := by
  apply toRat_injective
  simp [Rat.mul_assoc]

theorem one_mul (a : PRCRat) : mul one a = a := by
  apply toRat_injective
  simp

theorem mul_one (a : PRCRat) : mul a one = a := by
  apply toRat_injective
  simp

theorem left_distrib (a b c : PRCRat) :
    mul a (add b c) = add (mul a b) (mul a c) := by
  apply toRat_injective
  simp [Rat.mul_add]

theorem mul_recip_cancel {a : PRCRat} (h : a.toRat ≠ 0) :
    mul a (recip a) = one := by
  apply toRat_injective
  rw [toRat_mul, toRat_recip, one_toRat]
  field_simp [h]

/-! ### Operation instances on PRCRat -/

instance instZero : Zero PRCRat := (zero)
instance instOne : One PRCRat := (one)
instance instAdd : Add PRCRat := (add)
instance instMul : Mul PRCRat := (mul)
instance instNeg : Neg PRCRat := (negate)
instance instSub : Sub PRCRat := (sub)
instance instInv : Inv PRCRat := (recip)

@[simp] theorem add_eq (a b : PRCRat) : a + b = add a b := rfl
@[simp] theorem mul_eq (a b : PRCRat) : a * b = mul a b := rfl
@[simp] theorem neg_eq (a : PRCRat) : -a = negate a := rfl
@[simp] theorem sub_eq (a b : PRCRat) : a - b = sub a b := rfl
@[simp] theorem inv_eq (a : PRCRat) : a⁻¹ = recip a := rfl
@[simp] theorem zero_eq : (0 : PRCRat) = zero := rfl
@[simp] theorem one_eq : (1 : PRCRat) = one := rfl

theorem toRat_add' (a b : PRCRat) :
    (a + b).toRat = a.toRat + b.toRat := by
  simp

theorem toRat_mul' (a b : PRCRat) :
    (a * b).toRat = a.toRat * b.toRat := by
  simp

theorem toRat_neg' (a : PRCRat) :
    (-a).toRat = -a.toRat := by
  simp

theorem toRat_inv' (a : PRCRat) :
    (a⁻¹).toRat = (a.toRat)⁻¹ := by
  simp

end PRCRat

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

8. IntegerOrder.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/IntegerOrder.lean · **Lines:** 67

```
/-
  PrimitiveRecognitionCalculus/IntegerOrder.lean

  Round-trip sources:
    6/PRC_Kernel_Spec_20260526.html
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchors:
    Build Order step 1: internal signed-orbit order and absolute value.

  This module is the named certificate surface for integer order. The core
  declarations live in `IntegerRational.lean` because `RatioOrbit.recipNonzero`
  needs `SignedOrbit.abs` and `SignedOrbit.nonnegFlag` without creating an
  import cycle.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerRational

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- Step-1 certificate for internal signed-orbit order and absolute value. -/
structure IntegerOrderCertificate : Prop where
  truncated_sub_display :
    ∀ a b : DistinctionNat,
      (DistinctionNat.truncatedSub a b).toNat = a.toNat - b.toNat
  leq_display :
    ∀ a b : DistinctionNat,
      DistinctionNat.leq a b = true ↔ a.toNat ≤ b.toNat
  absdiff_display :
    ∀ a b : DistinctionNat,
      (DistinctionNat.absDiff a b).toNat =
        Int.natAbs ((a.toNat : ℤ) - (b.toNat : ℤ))
  signed_nonneg_display :
    ∀ z : SignedOrbit, SignedOrbit.nonneg z ↔ 0 ≤ z.toInt
  signed_nonneg_flag_display :
    ∀ z : SignedOrbit, z.nonnegFlag = true ↔ 0 ≤ z.toInt
  signed_abs_display :
    ∀ z : SignedOrbit, z.abs.toNat = Int.natAbs z.toInt
  signed_le_display :
    ∀ a b : SignedOrbit, SignedOrbit.le a b ↔ a.toInt ≤ b.toInt
  signed_lt_display :
    ∀ a b : SignedOrbit, SignedOrbit.lt a b ↔ a.toInt < b.toInt
  abs_nonzero_internal :
    ∀ z : SignedOrbit,
      (¬ SignedOrbit.balanced z SignedOrbit.zero) →
        z.abs ≠ DistinctionNat.zero

/-- The internal signed-orbit order surface is closed. -/
theorem integer_order_certificate : IntegerOrderCertificate where
  truncated_sub_display := DistinctionNat.toNat_truncatedSub
  leq_display := DistinctionNat.leq_eq_true_iff
  absdiff_display := DistinctionNat.toNat_absDiff
  signed_nonneg_display := SignedOrbit.nonneg_iff_toInt_nonneg
  signed_nonneg_flag_display := SignedOrbit.nonnegFlag_eq_true_iff
  signed_abs_display := SignedOrbit.abs_toNat
  signed_le_display := SignedOrbit.le_iff_toInt_le
  signed_lt_display := SignedOrbit.lt_iff_toInt_lt
  abs_nonzero_internal := by
    intro z h
    exact SignedOrbit.abs_ne_zero_of_not_balanced_zero h

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

9. OrbitDivisibility.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitDivisibility.lean · **Lines:** 283

```
0001  /-
0002    PrimitiveRecognitionCalculus/OrbitDivisibility.lean
0003
0004    Round-trip sources:
0005      6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006      6/PRC_Structural_Brainstorm_20260527.html
0007
0008    Spec anchors:
0009      Build Order step 2: divisibility, units, factorization, and primality on
0010      orbit positions.
0011
0012    Strength: δ-only for definitions. Nat divisibility and Nat arithmetic appear
0013    only in verifier transport theorems.
0014  -/
0015
0016  import Mathlib
0017  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitArithmetic
0018
0019  namespace IndisputableMonolith
0020  namespace Foundation
0021  namespace PrimitiveRecognitionCalculus
0022  namespace DistinctionNat
0023
0024  /-! ## Native divisibility on finite δ-orbit positions -/
0025
0026  /-- The multiplicative unit orbit position. -/
0027  def one : DistinctionNat :=
0028    succ zero
0029
0030  @[simp] theorem one_toNat :
0031      one.toNat = 1 := by
0032    rfl
0033
0034  theorem one_ne_zero :
0035      one ≠ zero := by
0036    intro h
0037    exact zero_ne_succ zero h.symm
0038
0039  theorem mul_one_eq (a : DistinctionNat) :
0040      a * one = a := by
0041    unfold one
0042    rw [mul_succ_eq, mul_zero_eq, zero_add_eq]
0043
0044  theorem one_mul_eq (a : DistinctionNat) :
0045      one * a = a := by
0046    rw [mul_comm, mul_one_eq]
0047
0048  theorem mul_assoc (a b c : DistinctionNat) :
0049      (a * b) * c = a * (b * c) := by
0050    apply toNat_inj
0051    rw [toNat_mul, toNat_mul, toNat_mul, toNat_mul]
0052    exact Nat.mul_assoc a.toNat b.toNat c.toNat
0053
0054  /-- Native orbit divisibility: `a` divides `b` when multiplying `a` by another
0055  orbit position yields `b`. -/
0056  def divides (a b : DistinctionNat) : Prop :=
0057    ∃ k : DistinctionNat, a * k = b
0058
0059  /-- Native unit predicate. In the finite δ-orbit, the only multiplicative unit
0060  is the one-step orbit. -/
0061  def unit (a : DistinctionNat) : Prop :=
0062    a = one
0063
0064  /-- Native nontrivial factorization. Both factors must be nonzero and non-unit. -/
0065  def nontrivialFactorization (n : DistinctionNat) : Prop :=
0066    ∃ a b : DistinctionNat,
0067      a ≠ zero ∧ b ≠ zero ∧ ¬ unit a ∧ ¬ unit b ∧ a * b = n
0068
0069  /-- Native prime orbit position: nonzero, non-unit, and with no nontrivial
0070  factorization. -/
0071  def primeOrbit (p : DistinctionNat) : Prop :=
0072    p ≠ zero ∧ ¬ unit p ∧ ¬ nontrivialFactorization p
0073
0074  theorem divides_refl (a : DistinctionNat) :
0075      divides a a := by
0076    exact (one, mul_one_eq a)
0077
0078  theorem divides_zero (a : DistinctionNat) :
0079      divides a zero := by
0080    exact (zero, mul_zero_eq a)
0081
0082  theorem one_divides (a : DistinctionNat) :
0083      divides one a := by
0084    exact (a, one_mul_eq a)
0085
0086  theorem divides_trans {a b c : DistinctionNat}
0087      (hab : divides a b) (hbc : divides b c) :
0088      divides a c := by
0089    rcases hab with (m, hm)
0090    rcases hbc with (n, hn)
```

```
  refine (m * n, ?_)
  rw [← mul_assoc, hm, hn]

/-- Divisibility is native, but it displays as Nat divisibility. -/
theorem divides_iff_toNat_dvd (a b : DistinctionNat) :
    divides a b ↔ a.toNat ∣ b.toNat := by
  constructor
  · intro h
    rcases h with ⟨k, hk⟩
    refine ⟨k.toNat, ?_⟩
    have hnat := congrArg DistinctionNat.toNat hk
    rw [toNat_mul] at hnat
    exact hnat.symm
  · intro h
    rcases h with ⟨k, hk⟩
    refine ⟨ofNat k, ?_⟩
    apply toNat_inj
    rw [toNat_mul, toNat_ofNat]
    exact hk.symm

theorem unit_iff_toNat_eq_one (a : DistinctionNat) :
    unit a ↔ a.toNat = 1 := by
  constructor
  · intro h
    unfold unit at h
    rw [h, one_toNat]
  · intro h
    unfold unit
    apply toNat_inj
    rw [h, one_toNat]

private theorem ofNat_ne_zero_of_ne_zero {n : Nat} (h : n ≠ 0) :
    ofNat n ≠ zero := by
  intro hz
  have hnat := congrArg DistinctionNat.toNat hz
  rw [toNat_ofNat, toNat_zero] at hnat
  exact h hnat

private theorem not_unit_ofNat_of_ne_one {n : Nat} (h : n ≠ 1) :
    ¬ unit (ofNat n) := by
  intro hu
  rw [unit_iff_toNat_eq_one] at hu
  rw [toNat_ofNat] at hu
  exact h hu

/-- Native nontrivial factorization displays as ordinary Nat nontrivial
factorization. -/
theorem nontrivialFactorization_iff_toNat (n : DistinctionNat) :
    nontrivialFactorization n ↔
      ∃ a b : Nat,
        a ≠ 0 ∧ b ≠ 0 ∧ a ≠ 1 ∧ b ≠ 1 ∧ a * b = n.toNat := by
  constructor
  · intro h
    rcases h with ⟨a, b, ha0, hb0, ha1, hb1, hmul⟩
    refine ⟨a.toNat, b.toNat, ?_, ?_, ?_, ?_, ?_⟩
    · intro hz
      have : a = zero := by
        apply toNat_inj
        rw [hz, toNat_zero]
      exact ha0 this
    · intro hz
      have : b = zero := by
        apply toNat_inj
        rw [hz, toNat_zero]
      exact hb0 this
    · intro h1
      apply ha1
      rw [unit_iff_toNat_eq_one]
      exact h1
    · intro h1
      apply hb1
      rw [unit_iff_toNat_eq_one]
      exact h1
    · have hnat := congrArg DistinctionNat.toNat hmul
      rw [toNat_mul] at hnat
      exact hnat
  · intro h
    rcases h with ⟨a, b, ha0, hb0, ha1, hb1, hmul⟩
    refine ⟨ofNat a, ofNat b, ?_, ?_, ?_, ?_, ?_⟩
    · exact ofNat_ne_zero_of_ne_zero ha0
    · exact ofNat_ne_zero_of_ne_zero hb0
    · exact not_unit_ofNat_of_ne_one ha1
    · exact not_unit_ofNat_of_ne_one hb1
    · apply toNat_inj
      rw [toNat_mul, toNat_ofNat, toNat_ofNat, hmul]

/-- Native prime-orbit predicate displays as the Nat no-nontrivial-factor
predicate, without defining primality by importing Nat prime theory. -/
theorem primeOrbit_iff_toNat_no_nontrivial_factor (p : DistinctionNat) :
    primeOrbit p ↔
      p.toNat ≠ 0 ∧ p.toNat ≠ 1 ∧
        ¬ ∃ a b : Nat,
          a ≠ 0 ∧ b ≠ 0 ∧ a ≠ 1 ∧ b ≠ 1 ∧ a * b = p.toNat := by
  unfold primeOrbit
  rw [unit_iff_toNat_eq_one, nontrivialFactorization_iff_toNat]
  constructor
  · intro h
    rcases h with ⟨hp0, hp1, hfac⟩
    refine ⟨?_, hp1, hfac⟩
    intro hz
    have : p = zero := by
```

```
      apply toNat_inj
      rw [hz, toNat_zero]
    exact hp0 this
  · intro h
    rcases h with (hp0, hp1, hfac)
    refine (?_, hp1, hfac)
    intro hz
    exact hp0 (by rw [hz, toNat_zero])

/-- If an orbit is prime, every native factorization has a unit factor. -/
theorem unit_or_unit_of_mul_eq_prime {a b p : DistinctionNat}
    (hp : primeOrbit p) (hmul : a * b = p) :
    unit a ∨ unit b := by
  by_cases ha0 : a = zero
  · exfalso
    rcases hp with (hp0, _, _)
    apply hp0
    rw [← hmul, ha0, zero_mul_eq]
  · by_cases hb0 : b = zero
    · exfalso
      rcases hp with (hp0, _, _)
      apply hp0
      rw [← hmul, hb0, mul_zero_eq]
    · by_cases ha1 : unit a
      · exact Or.inl ha1
      · by_cases hb1 : unit b
        · exact Or.inr hb1
        · exfalso
          rcases hp with (_, _, hnf)
          exact hnf (a, b, ha0, hb0, ha1, hb1, hmul)

/-- Converse native factor theorem: if a nonzero non-unit orbit position has
only unit factors, then it is a prime orbit. -/
theorem primeOrbit_of_unit_or_unit
    {p : DistinctionNat}
    (hp0 : p ≠ zero)
    (hp1 : ¬ unit p)
    (hfac : ∀ a b : DistinctionNat, a * b = p → unit a ∨ unit b) :
    primeOrbit p := by
  refine (hp0, hp1, ?_)
  intro hnon
  rcases hnon with (a, b, _ha0, _hb0, ha1, hb1, hmul)
  rcases hfac a b hmul with ha | hb
  · exact ha1 ha
  · exact hb1 hb

/-- Bundling certificate for the native divisibility surface. -/
structure OrbitDivisibilityCertificate : Prop where
  divides_display :
    ∀ a b : DistinctionNat, divides a b ↔ a.toNat ∣ b.toNat
  divides_reflexive :
    ∀ a : DistinctionNat, divides a a
  divides_transitive :
    ∀ {a b c : DistinctionNat}, divides a b → divides b c → divides a c
  one_divides_all :
    ∀ a : DistinctionNat, divides one a
  unit_display :
    ∀ a : DistinctionNat, unit a ↔ a.toNat = 1
  nontrivial_factorization_display :
    ∀ n : DistinctionNat,
      nontrivialFactorization n ↔
        ∃ a b : Nat,
          a ≠ 0 ∧ b ≠ 0 ∧ a ≠ 1 ∧ b ≠ 1 ∧ a * b = n.toNat
  prime_orbit_display :
    ∀ p : DistinctionNat,
      primeOrbit p ↔
        p.toNat ≠ 0 ∧ p.toNat ≠ 1 ∧
          ¬ ∃ a b : Nat,
            a ≠ 0 ∧ b ≠ 0 ∧ a ≠ 1 ∧ b ≠ 1 ∧ a * b = p.toNat
  prime_factor_property :
    ∀ {a b p : DistinctionNat},
      primeOrbit p → a * b = p → unit a ∨ unit b

/-- The native orbit divisibility surface is closed. -/
theorem orbit_divisibility_certificate : OrbitDivisibilityCertificate where
  divides_display := divides_iff_toNat_dvd
  divides_reflexive := divides_refl
  divides_transitive := by
    intro a b c hab hbc
    exact divides_trans hab hbc
  one_divides_all := one_divides
  unit_display := unit_iff_toNat_eq_one
  nontrivial_factorization_display := nontrivialFactorization_iff_toNat
  prime_orbit_display := primeOrbit_iff_toNat_no_nontrivial_factor
  prime_factor_property := by
    intro a b p hp hmul
    exact unit_or_unit_of_mul_eq_prime hp hmul

end DistinctionNat
end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

10. OrbitEuclidean.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitEuclidean.lean · **Lines:** 473

```
/-
  PrimitiveRecognitionCalculus/OrbitEuclidean.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchors:
    Build Order step 3: Euclidean quotient/remainder, GCD, coprime, and
    rational normalization targets.

  Strength: δ-only for definitions. Nat division, modulo, and gcd appear only
  in verifier transport theorems.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerRational
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitDivisibility

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus
namespace DistinctionNat

/-! ## Object-level quotient and remainder by repeated subtraction -/

/-- Fuelled quotient/remainder by repeated subtraction. The first argument is
an orbit fuel, not verifier `Nat`. -/
def divModFuel : DistinctionNat → DistinctionNat → DistinctionNat → DistinctionNat × DistinctionNat
  | zero, n, _ => (zero, n)
  | succ fuel, n, d =>
      if DistinctionNat.leq d n then
        let qr := divModFuel fuel (DistinctionNat.truncatedSub n d) d
        (succ qr.1, qr.2)
      else
        (zero, n)

/-- Euclidean quotient/remainder. The fuel `n` is enough when `d` is nonzero,
because each successful subtraction lowers the dividend by at least one. -/
def divMod (n d : DistinctionNat) (_hd : d ≠ zero) : DistinctionNat × DistinctionNat :=
  divModFuel n n d

/-- Object-level quotient. -/
def quotient (n d : DistinctionNat) (hd : d ≠ zero) : DistinctionNat :=
  (divMod n d hd).1

/-- Object-level remainder. -/
def remainder (n d : DistinctionNat) (hd : d ≠ zero) : DistinctionNat :=
  (divMod n d hd).2

private theorem divModFuel_toNat_aux (fuel n d : DistinctionNat)
    (hd : d.toNat ≠ 0)
    (hbound : n.toNat ≤ fuel.toNat) :
    let qr := divModFuel fuel n d
    qr.1.toNat = n.toNat / d.toNat ∧
      qr.2.toNat = n.toNat % d.toNat := by
  induction fuel generalizing n with
  | zero =>
      rw [toNat_zero] at hbound
      have hn0 : n.toNat = 0 := by omega
      simp [divModFuel, hn0]
  | succ fuel ih =>
      rw [toNat_succ] at hbound
      unfold divModFuel
      by_cases hleq : DistinctionNat.leq d n = true
      · have hle : d.toNat ≤ n.toNat :=
          (DistinctionNat.leq_eq_true_iff d n).mp hleq
        have hpos : 0 < d.toNat := by omega
        simp [hleq]
        have hbound' : (DistinctionNat.truncatedSub n d).toNat ≤ fuel.toNat := by
          rw [DistinctionNat.toNat_truncatedSub]
          omega
        have ih' := ih (DistinctionNat.truncatedSub n d) hbound'
        rcases ih' with (hq, hr)
        rw [hq, hr]
        constructor
        · rw [DistinctionNat.toNat_truncatedSub, Nat.div_eq_sub_div hpos hle]
        · rw [DistinctionNat.toNat_truncatedSub]
          exact (Nat.mod_eq_sub_mod hle).symm
      · have hlt : n.toNat < d.toNat := by
          have hf := (DistinctionNat.leq_eq_false_iff d n).mp (by
            cases h : DistinctionNat.leq d n with
            | false => rfl
            | true =>
                exfalso
                exact hleq h)
          exact hf
        simp [hleq]
        constructor
        · exact (Nat.div_eq_of_lt hlt).symm
        · exact (Nat.mod_eq_of_lt hlt).symm
```

```
0091
0092 /-- Euclidean quotient/remainder transports to verifier Nat division and
0093 modulus. -/
0094 theorem divMod_toNat (n d : DistinctionNat) (hd : d ≠ zero) :
0095     let qr := divMod n d hd
0096     qr.1.toNat = n.toNat / d.toNat ∧
0097       qr.2.toNat = n.toNat % d.toNat := by
0098   unfold divMod
0099   apply divModFuel_toNat_aux
0100   · intro hzero
0101     have : d = zero := by
0102       apply toNat_inj
0103       rw [hzero, toNat_zero]
0104     exact hd this
0105   · omega
0106
0107 theorem quotient_toNat (n d : DistinctionNat) (hd : d ≠ zero) :
0108     (quotient n d hd).toNat = n.toNat / d.toNat := by
0109   have h := divMod_toNat n d hd
0110   exact h.1
0111
0112 theorem remainder_toNat (n d : DistinctionNat) (hd : d ≠ zero) :
0113     (remainder n d hd).toNat = n.toNat % d.toNat := by
0114   have h := divMod_toNat n d hd
0115   exact h.2
0116
0117 theorem remainder_lt_divisor (n d : DistinctionNat) (hd : d ≠ zero) :
0118     (remainder n d hd).toNat < d.toNat := by
0119   rw [remainder_toNat]
0120   apply Nat.mod_lt
0121   exact Nat.pos_of_ne_zero (by
0122     intro hzero
0123     have : d = zero := by
0124       apply toNat_inj
0125       rw [hzero, toNat_zero]
0126     exact hd this)
0127
0128 /-! ## Object-level GCD by subtractive Euclidean descent -/
0129
0130 /-- Fuelled subtractive Euclidean GCD. -/
0131 def gcdFuel : DistinctionNat → DistinctionNat → DistinctionNat → DistinctionNat
0132   | zero, a, b => a + b
0133   | succ fuel, a, b =>
0134       if a = zero then
0135         b
0136       else if b = zero then
0137         a
0138       else if DistinctionNat.leq b a then
0139         gcdFuel fuel (DistinctionNat.truncatedSub a b) b
0140       else
0141         gcdFuel fuel a (DistinctionNat.truncatedSub b a)
0142
0143 /-- Object-level GCD by subtractive Euclidean descent. -/
0144 def gcd (a b : DistinctionNat) : DistinctionNat :=
0145   gcdFuel (a + b) a b
0146
0147 /-- Object-level coprimality. -/
0148 def coprime (a b : DistinctionNat) : Prop :=
0149   unit (gcd a b)
0150
0151 private theorem gcdFuel_toNat_aux (fuel a b : DistinctionNat)
0152     (hbound : a.toNat + b.toNat ≤ fuel.toNat) :
0153     (gcdFuel fuel a b).toNat = Nat.gcd a.toNat b.toNat := by
0154   induction fuel generalizing a b with
0155   | zero =>
0156       rw [toNat_zero] at hbound
0157       have ha0 : a.toNat = 0 := by omega
0158       have hb0 : b.toNat = 0 := by omega
0159       simp [gcdFuel, ha0, hb0, toNat_add]
0160   | succ fuel ih =>
0161       rw [toNat_succ] at hbound
0162       unfold gcdFuel
0163       by_cases ha : a = zero
0164       · simp [ha, Nat.gcd_zero_left]
0165       · by_cases hb : b = zero
0166         · simp [ha, hb, Nat.gcd_zero_right]
0167         · by_cases hleq : DistinctionNat.leq b a = true
0168           · have hle : b.toNat ≤ a.toNat :=
0169               (DistinctionNat.leq_eq_true_iff b a).mp hleq
0170             have hbpos : 0 < b.toNat := by
0171               have hbne : b.toNat ≠ 0 := by
0172                 intro hzero
0173                 have : b = zero := by
0174                   apply toNat_inj
0175                   rw [hzero, toNat_zero]
0176                 exact hb this
0177               omega
0178             have hbound' :
0179                 (DistinctionNat.truncatedSub a b).toNat + b.toNat ≤ fuel.toNat := by
0180               rw [DistinctionNat.toNat_truncatedSub]
0181               omega
0182             simp [ha, hb, hleq]
0183             rw [ih (DistinctionNat.truncatedSub a b) b hbound']
0184             rw [DistinctionNat.toNat_truncatedSub]
0185             exact Nat.gcd_sub_self_left hle
0186           · have hlt : a.toNat < b.toNat := by
0187               exact (DistinctionNat.leq_eq_false_iff b a).mp (by
0188                 cases h : DistinctionNat.leq b a with
0189                 | false => rfl
0190                 | true =>
0191                     exfalso
```

```
                    exact hleq h)
            have hle : a.toNat ≤ b.toNat := by omega
            have hapos : 0 < a.toNat := by
              have hane : a.toNat ≠ 0 := by
                intro hzero
                have : a = zero := by
                  apply toNat_inj
                  rw [hzero, toNat_zero]
                exact ha this
              omega
            have hbound' :
                a.toNat + (DistinctionNat.truncatedSub b a).toNat ≤ fuel.toNat := by
              rw [DistinctionNat.toNat_truncatedSub]
              omega
            simp [ha, hb, hleq]
            rw [ih a (DistinctionNat.truncatedSub b a) hbound']
            rw [DistinctionNat.toNat_truncatedSub]
            exact Nat.gcd_sub_self_right hle

theorem gcd_toNat (a b : DistinctionNat) :
    (gcd a b).toNat = Nat.gcd a.toNat b.toNat := by
  unfold gcd
  apply gcdFuel_toNat_aux
  rw [toNat_add]

theorem coprime_iff_nat_coprime (a b : DistinctionNat) :
    coprime a b ↔ Nat.Coprime a.toNat b.toNat := by
  simp [coprime, gcd_toNat, unit_iff_toNat_eq_one]

/-- The native GCD divides the left input. -/
theorem gcd_divides_left (a b : DistinctionNat) :
    divides (gcd a b) a := by
  rw [divides_iff_toNat_dvd, gcd_toNat]
  exact Nat.gcd_dvd_left a.toNat b.toNat

/-- The native GCD divides the right input. -/
theorem gcd_divides_right (a b : DistinctionNat) :
    divides (gcd a b) b := by
  rw [divides_iff_toNat_dvd, gcd_toNat]
  exact Nat.gcd_dvd_right a.toNat b.toNat

theorem gcd_ne_zero_of_right_ne_zero (a b : DistinctionNat) (hb : b ≠ zero) :
    gcd a b ≠ zero := by
  intro h
  have hnat : (gcd a b).toNat = 0 := by
    rw [h, toNat_zero]
  rw [gcd_toNat] at hnat
  have hb0 : b.toNat = 0 := (Nat.gcd_eq_zero_iff.mp hnat).2
  apply hb
  apply toNat_inj
  rw [hb0, toNat_zero]

theorem quotient_mul_divisor_toNat_of_divides {n d : DistinctionNat}
    (hd : d ≠ zero) (hdiv : divides d n) :
    (quotient n d hd).toNat * d.toNat = n.toNat := by
  rw [quotient_toNat]
  exact Nat.div_mul_cancel ((divides_iff_toNat_dvd d n).mp hdiv)

theorem quotient_ne_zero_of_divides {n d : DistinctionNat}
    (hd : d ≠ zero) (hdiv : divides d n) (hn : n ≠ zero) :
    quotient n d hd ≠ zero := by
  intro hq
  have hqnat : (quotient n d hd).toNat = 0 := by
    rw [hq, toNat_zero]
  have hmul := quotient_mul_divisor_toNat_of_divides (n := n) (d := d) hd hdiv
  rw [hqnat, Nat.zero_mul] at hmul
  apply hn
  apply toNat_inj
  rw [hmul.symm, toNat_zero]

/-! ## Signed rational normalization by native orbit GCD -/

/-- Quotient a signed orbit by a nonzero orbit position, restoring the sign by
the structural signed-orbit comparison. -/
def signedQuotient (z : SignedOrbit) (d : DistinctionNat) (hd : d ≠ zero) :
    SignedOrbit :=
  let q := quotient z.abs d hd
  if z.nonnegFlag then
    SignedOrbit.ofOrbit q
  else
    SignedOrbit.negate (SignedOrbit.ofOrbit q)

theorem signedQuotient_abs_toNat (z : SignedOrbit)
    (d : DistinctionNat) (hd : d ≠ zero) :
    (signedQuotient z d hd).abs.toNat = z.abs.toNat / d.toNat := by
  unfold signedQuotient
  by_cases hflag : z.nonnegFlag = true
  · have hAbsQ :
        (SignedOrbit.ofOrbit (quotient z.abs d hd)).abs.toNat =
          (quotient z.abs d hd).toNat := by
      simp [SignedOrbit.abs_toNat, SignedOrbit.ofOrbit_toInt]
    simpa [hflag, hAbsQ] using quotient_toNat z.abs d hd
  · have hflagFalse : z.nonnegFlag = false := by
      cases h : z.nonnegFlag with
      | false => rfl
      | true =>
          exfalso
          exact hflag h
    have hAbsQ :
        (SignedOrbit.negate (SignedOrbit.ofOrbit (quotient z.abs d hd))).abs.toNat =
          (quotient z.abs d hd).toNat := by
```

```
      simp [SignedOrbit.abs_toNat, SignedOrbit.ofOrbit_toInt,
        SignedOrbit.negate_toInt]
    simpa [hflagFalse, hAbsQ] using quotient_toNat z.abs d hd

theorem signedQuotient_mul_divisor_toInt_of_divides
    (z : SignedOrbit) (d : DistinctionNat) (hd : d ≠ zero)
    (hdiv : divides d z.abs) :
    (signedQuotient z d hd).toInt * (d.toNat : ℤ) = z.toInt := by
  have hquotNat :
      (quotient z.abs d hd).toNat * d.toNat = z.abs.toNat :=
    quotient_mul_divisor_toNat_of_divides (n := z.abs) (d := d) hd hdiv
  have hquotInt :
      ((quotient z.abs d hd).toNat : ℤ) * (d.toNat : ℤ) =
        (z.abs.toNat : ℤ) := by
    exact_mod_cast hquotNat
  unfold signedQuotient
  by_cases hflag : z.nonnegFlag = true
  · have hnonneg : 0 ≤ z.toInt :=
      (SignedOrbit.nonnegFlag_eq_true_iff z).mp hflag
    have habs : (z.abs.toNat : ℤ) = z.toInt := by
      rw [SignedOrbit.abs_toNat]
      exact Int.ofNat_natAbs_of_nonneg hnonneg
    simp [hflag, SignedOrbit.ofOrbit_toInt]
    rw [hquotInt, habs]
  · have hflagFalse : z.nonnegFlag = false := by
      cases h : z.nonnegFlag with
      | false => rfl
      | true =>
          exfalso
          exact hflag h
    have hneg : z.toInt < 0 :=
      (SignedOrbit.nonnegFlag_eq_false_iff z).mp hflagFalse
    have habs : (z.abs.toNat : ℤ) = -z.toInt := by
      rw [SignedOrbit.abs_toNat]
      exact Int.ofNat_natAbs_of_nonpos (le_of_lt hneg)
    simp [hflagFalse, SignedOrbit.ofOrbit_toInt, SignedOrbit.negate_toInt]
    rw [hquotInt, habs]
    ring

/-- Normalize a ratio orbit by dividing numerator magnitude and denominator by
their native orbit GCD. The signed numerator orientation is restored by
`SignedOrbit.nonnegFlag`. -/
def normalizeRatio (q : RatioOrbit) : RatioOrbit :=
  let g := gcd q.num.abs q.den
  have hg : g ≠ zero := gcd_ne_zero_of_right_ne_zero q.num.abs q.den q.den_ne_zero
  {
    num := signedQuotient q.num g hg
    den := quotient q.den g hg
    den_ne_zero :=
      quotient_ne_zero_of_divides
        (n := q.den) (d := g) hg
        (gcd_divides_right q.num.abs q.den)
        q.den_ne_zero
  }

theorem normalizeRatio_num_mul_gcd_toInt (q : RatioOrbit) :
    (normalizeRatio q).num.toInt *
      ((gcd q.num.abs q.den).toNat : ℤ) = q.num.toInt := by
  unfold normalizeRatio
  exact signedQuotient_mul_divisor_toInt_of_divides
    q.num (gcd q.num.abs q.den)
    (gcd_ne_zero_of_right_ne_zero q.num.abs q.den q.den_ne_zero)
    (gcd_divides_left q.num.abs q.den)

theorem normalizeRatio_den_mul_gcd_toNat (q : RatioOrbit) :
    (normalizeRatio q).den.toNat *
      (gcd q.num.abs q.den).toNat = q.den.toNat := by
  unfold normalizeRatio
  exact quotient_mul_divisor_toNat_of_divides
    (n := q.den) (d := gcd q.num.abs q.den)
    (gcd_ne_zero_of_right_ne_zero q.num.abs q.den q.den_ne_zero)
    (gcd_divides_right q.num.abs q.den)

theorem normalizeRatio_toRat (q : RatioOrbit) :
    (normalizeRatio q).toRat = q.toRat := by
  unfold RatioOrbit.toRat
  have hnumZ := normalizeRatio_num_mul_gcd_toInt q
  have hdenN := normalizeRatio_den_mul_gcd_toNat q
  have hnumQ :
      ((normalizeRatio q).num.toInt : ℚ) *
        ((gcd q.num.abs q.den).toNat : ℚ) =
          (q.num.toInt : ℚ) := by
    exact_mod_cast hnumZ
  have hdenQ :
      ((normalizeRatio q).den.toNat : ℚ) *
        ((gcd q.num.abs q.den).toNat : ℚ) =
          (q.den.toNat : ℚ) := by
    exact_mod_cast hdenN
  have hNormDen : ((normalizeRatio q).den.toNat : ℚ) ≠ 0 :=
    (normalizeRatio q).den_cast_ne_zero
  have hDen : (q.den.toNat : ℚ) ≠ 0 := q.den_cast_ne_zero
  field_simp [hNormDen, hDen]
  calc
    ((normalizeRatio q).num.toInt : ℚ) * (q.den.toNat : ℚ)
        = ((normalizeRatio q).num.toInt : ℚ) *
            (((normalizeRatio q).den.toNat : ℚ) *
              ((gcd q.num.abs q.den).toNat : ℚ)) := by
          rw [hdenQ]
    _ = (((normalizeRatio q).num.toInt : ℚ) *
            ((gcd q.num.abs q.den).toNat : ℚ)) *
            ((normalizeRatio q).den.toNat : ℚ) := by ring
```

```
    _ = (q.num.toInt : ℚ) * ((normalizeRatio q).den.toNat : ℚ) := by
          rw [hnumQ]
    _ = ((normalizeRatio q).den.toNat : ℚ) * (q.num.toInt : ℚ) := by ring

theorem normalizeRatio_crossEq (q : RatioOrbit) :
    RatioOrbit.crossEq q (normalizeRatio q) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq]
  exact (normalizeRatio_toRat q).symm

theorem normalizeRatio_coprime (q : RatioOrbit) :
    coprime (normalizeRatio q).num.abs (normalizeRatio q).den := by
  rw [coprime_iff_nat_coprime]
  unfold normalizeRatio
  rw [signedQuotient_abs_toNat, quotient_toNat]
  have hgpos : 0 < (gcd q.num.abs q.den).toNat := by
    rw [gcd_toNat]
    apply Nat.gcd_pos_of_pos_right
    exact Nat.pos_of_ne_zero (by
      intro hzero
      apply q.den_ne_zero
      apply toNat_inj
      rw [hzero, toNat_zero])
  have hgposNat : 0 < Nat.gcd q.num.abs.toNat q.den.toNat := by
    rw [← gcd_toNat]
    exact hgpos
  rw [gcd_toNat]
  exact Nat.coprime_div_gcd_div_gcd
    (m := q.num.abs.toNat) (n := q.den.toNat) hgposNat

/-- After signed division by orbit GCD, every `RatioOrbit` admits a balanced
equivalent representative whose numerator absolute value is coprime to the
denominator. -/
def RatioNormalizationTarget : Prop :=
  ∀ q : RatioOrbit,
    ∃ q' : RatioOrbit,
      RatioOrbit.crossEq q q' ∧
      coprime q'.num.abs q'.den

theorem ratio_normalization_target : RatioNormalizationTarget := by
  intro q
  exact ⟨normalizeRatio q, normalizeRatio_crossEq q, normalizeRatio_coprime q⟩

/-- Bundling certificate for the Euclidean orbit surface closed in this pass. -/
structure OrbitEuclideanCertificate : Prop where
  divmod_display :
    ∀ (n d : DistinctionNat) (hd : d ≠ zero),
      let qr := divMod n d hd
      qr.1.toNat = n.toNat / d.toNat ∧
        qr.2.toNat = n.toNat % d.toNat
  quotient_display :
    ∀ (n d : DistinctionNat) (hd : d ≠ zero),
      (quotient n d hd).toNat = n.toNat / d.toNat
  remainder_display :
    ∀ (n d : DistinctionNat) (hd : d ≠ zero),
      (remainder n d hd).toNat = n.toNat % d.toNat
  remainder_bound :
    ∀ (n d : DistinctionNat) (hd : d ≠ zero),
      (remainder n d hd).toNat < d.toNat
  gcd_display :
    ∀ a b : DistinctionNat, (gcd a b).toNat = Nat.gcd a.toNat b.toNat
  coprime_display :
    ∀ a b : DistinctionNat, coprime a b ↔ Nat.Coprime a.toNat b.toNat
  ratio_normalization :
    RatioNormalizationTarget

/-- The closed δ-only Euclidean orbit surface, including signed-rational
normalization by native orbit GCD. -/
theorem orbit_euclidean_certificate : OrbitEuclideanCertificate where
  divmod_display := divMod_toNat
  quotient_display := quotient_toNat
  remainder_display := remainder_toNat
  remainder_bound := remainder_lt_divisor
  gcd_display := gcd_toNat
  coprime_display := coprime_iff_nat_coprime
  ratio_normalization := ratio_normalization_target

end DistinctionNat
end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

11. RationalField.lean

Path: `IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RationalField.lean` · **Lines:** 284

```
/-
  PrimitiveRecognitionCalculus/RationalField.lean

  Round-trip source:
    δ/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 4: package the PRC rational quotient as the reusable
    field-like substrate needed by PRC real completion and cost.

  Strength: δ-only for the object operations and positivity predicate.
  Verifier `ℚ` appears only in display theorems and law proofs.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCost

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

namespace RatioOrbit

/-- PRC-native positivity for a ratio orbit: positive signed numerator over a
nonzero orbit denominator. The denominator is an orbit position, so nonzero
means positive in the verifier display but is not part of the object definition. -/
def positive (q : RatioOrbit) : Prop :=
  SignedOrbit.nonneg q.num ∧ ¬ SignedOrbit.balanced q.num SignedOrbit.zero

theorem positive_iff_toRat_pos (q : RatioOrbit) :
    positive q ↔ 0 < q.toRat := by
  unfold positive RatioOrbit.toRat
  have hdenNat : 0 < q.den.toNat := Nat.pos_of_ne_zero q.den_toNat_ne_zero
  have hdenQ : 0 < (q.den.toNat : ℚ) := by exact_mod_cast hdenNat
  constructor
  · intro h
    have hnum_nonneg : 0 ≤ q.num.toInt :=
      (SignedOrbit.nonneg_iff_toInt_nonneg q.num).mp h.1
    have hnum_ne : q.num.toInt ≠ 0 := by
      intro hz
      exact h.2 ((SignedOrbit.balanced_iff_toInt_eq q.num SignedOrbit.zero).mpr (by
        rw [hz, SignedOrbit.zero_toInt]))
    have hnum_pos : 0 < q.num.toInt := by omega
    have hnumQ : 0 < (q.num.toInt : ℚ) := by exact_mod_cast hnum_pos
    positivity
  · intro h
    have hnum_pos : 0 < q.num.toInt := by
      have hden_ne : (q.den.toNat : ℚ) ≠ 0 := q.den_cast_ne_zero
      have hmul : 0 < ((q.num.toInt : ℚ) / (q.den.toNat : ℚ)) * (q.den.toNat : ℚ) :=
        mul_pos h hdenQ
      have hnumQ : 0 < (q.num.toInt : ℚ) := by
        field_simp [hden_ne] at hmul
        exact hmul
      exact_mod_cast hnumQ
    constructor
    · exact (SignedOrbit.nonneg_iff_toInt_nonneg q.num).mpr (by omega)
    · intro hbal
      have hnum_zero : q.num.toInt = 0 := by
        have := (SignedOrbit.balanced_iff_toInt_eq q.num SignedOrbit.zero).mp hbal
        simpa using this
      omega

theorem positive_normalize {q : RatioOrbit}
    (h : positive q) : positive (DistinctionNat.normalizeRatio q) := by
  rw [positive_iff_toRat_pos, DistinctionNat.normalizeRatio_toRat]
  exact (positive_iff_toRat_pos q).mp h

theorem positive_not_zero {q : RatioOrbit}
    (h : positive q) : q.toRat ≠ 0 := by
  exact ne_of_gt ((positive_iff_toRat_pos q).mp h)

end RatioOrbit

namespace PRCRat

/-! ## Division and positive rationals -/

/-- Division on PRC rationals, defined from PRC multiplication and reciprocal. -/
def div (a b : PRCRat) : PRCRat :=
  a * b⁻¹

instance instDiv : Div PRCRat := (div)

@[simp] theorem div_eq (a b : PRCRat) : a / b = div a b := rfl

theorem toRat_div (a b : PRCRat) :
    (a / b).toRat = a.toRat / b.toRat := by
  unfold HDiv.hDiv instHDiv Div.div instDiv div
  rw [toRat_mul', toRat_inv']
  rfl
```

```
/-- A PRC rational is positive if it has a positive ratio-orbit representative. -/
def positive (q : PRCRat) : Prop :=
  ∃ r : RatioOrbit, q = mk r ∧ RatioOrbit.positive r

theorem positive_iff_toRat_pos (q : PRCRat) :
    positive q ↔ 0 < q.toRat := by
  constructor
  · intro h
    rcases h with ⟨r, rfl, hr⟩
    rw [toRat_mk]
    exact (RatioOrbit.positive_iff_toRat_pos r).mp hr
  · refine Quot.induction_on q ?_
    intro r hr
    exact ⟨r, rfl, (RatioOrbit.positive_iff_toRat_pos r).mpr (by simpa using hr)⟩

theorem positive_ne_zero {q : PRCRat}
    (h : positive q) : q.toRat ≠ 0 :=
  ne_of_gt ((positive_iff_toRat_pos q).mp h)

/-! ## Operator-form field laws -/

theorem add_assoc' (a b c : PRCRat) :
    (a + b) + c = a + (b + c) :=
  add_assoc a b c

theorem zero_add' (a : PRCRat) :
    0 + a = a :=
  zero_add a

theorem add_zero' (a : PRCRat) :
    a + 0 = a :=
  add_zero a

theorem add_left_neg' (a : PRCRat) :
    -a + a = 0 := by
  apply toRat_injective
  simp

theorem add_right_neg' (a : PRCRat) :
    a + -a = 0 :=
  add_negate a

theorem mul_assoc' (a b c : PRCRat) :
    (a * b) * c = a * (b * c) :=
  mul_assoc a b c

theorem one_mul' (a : PRCRat) :
    1 * a = a :=
  one_mul a

theorem mul_one' (a : PRCRat) :
    a * 1 = a :=
  mul_one a

theorem zero_mul' (a : PRCRat) :
    0 * a = 0 := by
  apply toRat_injective
  simp

theorem mul_zero' (a : PRCRat) :
    a * 0 = 0 := by
  apply toRat_injective
  simp

theorem right_distrib' (a b c : PRCRat) :
    (a + b) * c = a * c + b * c := by
  apply toRat_injective
  simp [Rat.add_mul]

theorem left_distrib' (a b c : PRCRat) :
    a * (b + c) = a * b + a * c :=
  left_distrib a b c

theorem zero_ne_one : (0 : PRCRat) ≠ 1 := by
  intro h
  have hrat := congrArg toRat h
  simp at hrat

theorem inv_zero : (0 : PRCRat)⁻¹ = 0 := by
  apply toRat_injective
  simp

theorem inv_mul_cancel {a : PRCRat} (h : a.toRat ≠ 0) :
    a⁻¹ * a = 1 := by
  apply toRat_injective
  rw [toRat_mul', toRat_inv']
  simp [h]

theorem div_mul_cancel {a b : PRCRat} (h : b.toRat ≠ 0) :
    (a / b) * b = a := by
  apply toRat_injective
  rw [toRat_mul', toRat_div]
  field_simp [h]

theorem mul_div_cancel {a b : PRCRat} (h : b.toRat ≠ 0) :
    a * b / b = a := by
  apply toRat_injective
  rw [toRat_div, toRat_mul']
  field_simp [h]
```

```
end PRCRat

namespace PRCJCost

/-- PRC J-cost lifted from ratio-orbit representatives to the rational quotient. -/
def onPRCRat : PRCRat → PRCRat :=
  Quot.lift
    (fun q => PRCRat.mk (onRatioOrbit q))
    (by
      intro a b h
      apply PRCRat.toRat_injective
      rw [PRCRat.toRat_mk, PRCRat.toRat_mk, onRatioOrbit_toRat, onRatioOrbit_toRat]
      have hrat : a.toRat = b.toRat := (ratioOrbitEquiv_iff_toRat_eq a b).mp h
      rw [hrat])

@[simp] theorem onPRCRat_mk (q : RatioOrbit) :
    onPRCRat (PRCRat.mk q) = PRCRat.mk (onRatioOrbit q) := by
  rfl

theorem onPRCRat_toRat (q : PRCRat) :
    (onPRCRat q).toRat = (q.toRat + q.toRat⁻¹) / 2 - 1 := by
  refine Quot.induction_on q ?_
  intro r
  change (PRCRat.mk (onRatioOrbit r)).toRat =
    ((PRCRat.mk r).toRat + (PRCRat.mk r).toRat⁻¹) / 2 - 1
  rw [PRCRat.toRat_mk, PRCRat.toRat_mk, onRatioOrbit_toRat]

theorem onPRCRat_normalized_representative (q : RatioOrbit) :
    onPRCRat (PRCRat.mk q) =
      onPRCRat (PRCRat.mk (DistinctionNat.normalizeRatio q)) := by
  apply PRCRat.toRat_injective
  rw [onPRCRat_toRat, onPRCRat_toRat, PRCRat.toRat_mk, PRCRat.toRat_mk,
    DistinctionNat.normalizeRatio_toRat]

end PRCJCost

/-! ## Bundled certificate -/

structure RationalFieldCertificate : Prop where
  add_comm : ∀ a b : PRCRat, a + b = b + a
  add_assoc : ∀ a b c : PRCRat, (a + b) + c = a + (b + c)
  zero_add : ∀ a : PRCRat, 0 + a = a
  add_zero : ∀ a : PRCRat, a + 0 = a
  add_left_neg : ∀ a : PRCRat, -a + a = 0
  mul_comm : ∀ a b : PRCRat, a * b = b * a
  mul_assoc : ∀ a b c : PRCRat, (a * b) * c = a * (b * c)
  one_mul : ∀ a : PRCRat, 1 * a = a
  mul_one : ∀ a : PRCRat, a * 1 = a
  left_distrib : ∀ a b c : PRCRat, a * (b + c) = a * b + a * c
  right_distrib : ∀ a b c : PRCRat, (a + b) * c = a * c + b * c
  zero_ne_one : (0 : PRCRat) ≠ 1
  inv_zero : (0 : PRCRat)⁻¹ = 0
  mul_inv_cancel : ∀ a : PRCRat, a.toRat ≠ 0 → a * a⁻¹ = 1
  inv_mul_cancel : ∀ a : PRCRat, a.toRat ≠ 0 → a⁻¹ * a = 1
  div_display : ∀ a b : PRCRat, (a / b).toRat = a.toRat / b.toRat
  positive_display : ∀ q : PRCRat, PRCRat.positive q ↔ 0 < q.toRat
  ratio_positive_display : ∀ q : RatioOrbit, RatioOrbit.positive q ↔ 0 < q.toRat
  jcost_display :
    ∀ q : PRCRat, (PRCJCost.onPRCRat q).toRat = (q.toRat + q.toRat⁻¹) / 2 - 1
  jcost_normalized_representative :
    ∀ q : RatioOrbit,
      PRCJCost.onPRCRat (PRCRat.mk q) =
        PRCJCost.onPRCRat (PRCRat.mk (DistinctionNat.normalizeRatio q))

theorem rational_field_certificate : RationalFieldCertificate where
  add_comm := PRCRat.add_comm
  add_assoc := PRCRat.add_assoc'
  zero_add := PRCRat.zero_add'
  add_zero := PRCRat.add_zero'
  add_left_neg := PRCRat.add_left_neg'
  mul_comm := PRCRat.mul_comm
  mul_assoc := PRCRat.mul_assoc'
  one_mul := PRCRat.one_mul'
  mul_one := PRCRat.mul_one'
  left_distrib := PRCRat.left_distrib'
  right_distrib := PRCRat.right_distrib'
  zero_ne_one := PRCRat.zero_ne_one
  inv_zero := PRCRat.inv_zero
  mul_inv_cancel := by
    intro a h
    exact PRCRat.mul_recip_cancel h
  inv_mul_cancel := by
    intro a h
    exact PRCRat.inv_mul_cancel h
  div_display := PRCRat.toRat_div
  positive_display := PRCRat.positive_iff_toRat_pos
  ratio_positive_display := RatioOrbit.positive_iff_toRat_pos
  jcost_display := PRCJCost.onPRCRat_toRat
  jcost_normalized_representative := PRCJCost.onPRCRat_normalized_representative

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

12. PRCJCost.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCost.lean · **Lines:** 242

```
/-
  PrimitiveRecognitionCalculus/PRCJCost.lean

  Round-trip source:
    δ/PRC_Universal_Foundation_Execution_Plan_20260526.html
    δ/Logic_Functional_Equation.tex

  Spec anchors:
    Build Order steps 5–7: ratio cost surface, RCL surface, and the strongest
    currently provable bridge to the existing J-cost uniqueness theorem.

  Strength: δ-only for the rational cost object. The bridge to `Cost.Jcost`
  and `law_of_logic_forces_jcost` is a classical-extension transport surface
  because it lives on continuous positive real ratios.
-/

import Mathlib
import IndisputableMonolith.Cost.FunctionalEquation
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitEuclidean

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

namespace PRCJCost

/-! ## A rational PRC cost object -/

/-- The two-step orbit position. -/
def twoOrbit : DistinctionNat :=
  DistinctionNat.succ DistinctionNat.one

@[simp] theorem twoOrbit_toNat :
    twoOrbit.toNat = 2 := by
  rfl

/-- The ratio orbit `2`. -/
def two : RatioOrbit where
  num := SignedOrbit.ofOrbit twoOrbit
  den := DistinctionNat.one
  den_ne_zero := DistinctionNat.one_ne_zero

@[simp] theorem two_toRat :
    two.toRat = 2 := by
  unfold two RatioOrbit.toRat
  simp [twoOrbit_toNat, SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat]

/-- The ratio orbit `1/2`. -/
def half : RatioOrbit where
  num := SignedOrbit.ofOrbit DistinctionNat.one
  den := twoOrbit
  den_ne_zero := by
    intro h
    have hnat := congrArg DistinctionNat.toNat h
    rw [twoOrbit_toNat, DistinctionNat.toNat_zero] at hnat
    norm_num at hnat

@[simp] theorem half_toRat :
    half.toRat = (1 / 2 : ℚ) := by
  unfold half RatioOrbit.toRat
  simp [twoOrbit_toNat, SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat]

/-- PRC's rational J-cost object on a ratio orbit:
`J(q) = ((q + q⁻¹) / 2) − 1`.

This is a ratio-orbit object. It is not the real analytic uniqueness theorem;
that theorem is bridged below. -/
def onRatioOrbit (q : RatioOrbit) : RatioOrbit :=
  RatioOrbit.sub (RatioOrbit.mul (RatioOrbit.add q (RatioOrbit.recip q)) half) RatioOrbit.one

theorem onRatioOrbit_toRat (q : RatioOrbit) :
    (onRatioOrbit q).toRat = (q.toRat + q.toRat⁻¹) / 2 - 1 := by
  unfold onRatioOrbit
  rw [RatioOrbit.sub_toRat, RatioOrbit.mul_toRat, RatioOrbit.add_toRat,
    RatioOrbit.recip_toRat, half_toRat, RatioOrbit.one_toRat]
  ring

/-- The PRC rational cost transports to the existing real `Cost.Jcost`
formula on the verifier display. -/
theorem onRatioOrbit_toReal_jcost (q : RatioOrbit) :
    ((onRatioOrbit q).toRat : ℝ) = Cost.Jcost ((q.toRat : ℚ) : ℝ) := by
  rw [onRatioOrbit_toRat]
  unfold Cost.Jcost
  rw [Rat.cast_sub, Rat.cast_div, Rat.cast_add, Rat.cast_inv]
  norm_num

/-- Reciprocal symmetry of the PRC rational cost. -/
theorem reciprocal_symmetric (q : RatioOrbit) :
    RatioOrbit.crossEq (onRatioOrbit q) (onRatioOrbit (RatioOrbit.recip q)) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq]
```

```
  rw [onRatioOrbit_toRat, onRatioOrbit_toRat, RatioOrbit.recip_toRat]
  by_cases hq : q.toRat = 0
  · simp [hq]
  · field_simp [hq]
    ring

/-- Normalizing a ratio representative by native orbit GCD preserves the PRC
cost. -/
theorem normalized_invariant (q : RatioOrbit) :
    RatioOrbit.crossEq (onRatioOrbit q)
      (onRatioOrbit (DistinctionNat.normalizeRatio q)) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq]
  rw [onRatioOrbit_toRat, onRatioOrbit_toRat, DistinctionNat.normalizeRatio_toRat]

/-- Division of ratio orbits, defined from multiplication and reciprocal. -/
def div (q r : RatioOrbit) : RatioOrbit :=
  RatioOrbit.mul q (RatioOrbit.recip r)

theorem div_toRat (q r : RatioOrbit) :
    (div q r).toRat = q.toRat / r.toRat := by
  unfold div
  rw [RatioOrbit.mul_toRat, RatioOrbit.recip_toRat]
  rfl

private def rclLHS (x y : RatioOrbit) : RatioOrbit :=
  RatioOrbit.add (onRatioOrbit (RatioOrbit.mul x y)) (onRatioOrbit (div x y))

private def rclRHS (x y : RatioOrbit) : RatioOrbit :=
  RatioOrbit.add
    (RatioOrbit.add
      (RatioOrbit.mul two (RatioOrbit.mul (onRatioOrbit x) (onRatioOrbit y)))
      (RatioOrbit.mul two (onRatioOrbit x)))
    (RatioOrbit.mul two (onRatioOrbit y))

/-- Canonical PRC J-cost satisfies the RCL algebraically on nonzero ratio
orbits. This is the rational surface of the composition law, not the
continuous-real uniqueness theorem. -/
theorem canonical_rcl_surface {x y : RatioOrbit}
    (hx : x.toRat ≠ 0) (hy : y.toRat ≠ 0) :
    RatioOrbit.crossEq (rclLHS x y) (rclRHS x y) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq]
  unfold rclLHS rclRHS
  rw [RatioOrbit.add_toRat, RatioOrbit.add_toRat, RatioOrbit.add_toRat,
    RatioOrbit.mul_toRat, RatioOrbit.mul_toRat, RatioOrbit.mul_toRat,
    RatioOrbit.mul_toRat,
    onRatioOrbit_toRat, onRatioOrbit_toRat, onRatioOrbit_toRat,
    onRatioOrbit_toRat, div_toRat]
  simp [two_toRat]
  rw [RatioOrbit.mul_toRat]
  have hxy : x.toRat * y.toRat ≠ 0 := mul_ne_zero hx hy
  field_simp [hx, hy, hxy]
  ring_nf

/-! ## Bridge to the existing continuous positive-real uniqueness theorem -/

/-- Hypotheses for the later PRC-native cost-classification theorem. The
`two_calibrated` field rules out the identically-zero cost on the discrete
rational surface, playing the role of the continuous theorem's unit
log-curvature calibration until the internal real completion exists. -/
structure PRCNativeCostHypotheses (F : RatioOrbit → RatioOrbit) : Prop where
  reciprocal :
    ∀ q, RatioOrbit.crossEq (F q) (F (RatioOrbit.recip q))
  normalized_invariant :
    ∀ q, RatioOrbit.crossEq (F q) (F (DistinctionNat.normalizeRatio q))
  canonical_rcl :
    ∀ {x y : RatioOrbit}, x.toRat ≠ 0 → y.toRat ≠ 0 →
      RatioOrbit.crossEq
        (RatioOrbit.add (F (RatioOrbit.mul x y)) (F (div x y)))
        (RatioOrbit.add
          (RatioOrbit.add
            (RatioOrbit.mul two (RatioOrbit.mul (F x) (F y)))
            (RatioOrbit.mul two (F x)))
          (RatioOrbit.mul two (F y)))
  unit_zero :
    F RatioOrbit.one = RatioOrbit.zero
  two_calibrated :
    RatioOrbit.crossEq (F two) (onRatioOrbit two)

/-- Exact missing native theorem for a later pass: classify every admissible
PRC cost on normalized ratio orbits, then transport to the continuous
positive-real theorem as a corollary rather than using the real theorem as
the premise. -/
def PRCNativeCostUniquenessTarget : Prop :=
  ∀ F : RatioOrbit → RatioOrbit,
    PRCNativeCostHypotheses F →
    ∀ q : RatioOrbit, RatioOrbit.crossEq (F q) (onRatioOrbit q)

/-- The real-domain uniqueness theorem currently used by PRC. The quantified
`AczelSmoothnessPackage` keeps the Aczél regularity commitment explicit. -/
theorem bridge_to_existing_jcost_uniqueness
    (F : ℝ → ℝ)
    (hAczel : Cost.FunctionalEquation.AczelSmoothnessPackage)
    (hRecip : Cost.FunctionalEquation.IsReciprocalCost F)
    (hNorm : Cost.FunctionalEquation.IsNormalized F)
    (hComp : Cost.FunctionalEquation.SatisfiesCompositionLaw F)
    (hCalib : Cost.FunctionalEquation.IsCalibrated F)
    (hCont : ContinuousOn F (Set.Ioi 0)) :
    ∀ x : ℝ, 0 < x → F x = Cost.Jcost x := by
  let _ : Cost.FunctionalEquation.AczelSmoothnessPackage := hAczel
  exact Cost.FunctionalEquation.law_of_logic_forces_jcost
    F hRecip hNorm hComp hCalib hCont
```

```
0192
0193  /-- Certificate for the PRC cost pass. -/
0194  structure PRCJCostCertificate : Prop where
0195    rational_formula :
0196      ∀ q : RatioOrbit,
0197        (onRatioOrbit q).toRat = (q.toRat + q.toRat⁻¹) / 2 - 1
0198    real_jcost_bridge :
0199      ∀ q : RatioOrbit,
0200        ((onRatioOrbit q).toRat : ℝ) = Cost.Jcost ((q.toRat : ℚ) : ℝ)
0201    reciprocal :
0202      ∀ q : RatioOrbit,
0203        RatioOrbit.crossEq (onRatioOrbit q) (onRatioOrbit (RatioOrbit.recip q))
0204    normalization :
0205      ∀ q : RatioOrbit,
0206        RatioOrbit.crossEq (onRatioOrbit q)
0207          (onRatioOrbit (DistinctionNat.normalizeRatio q))
0208    canonical_rcl :
0209      ∀ {x y : RatioOrbit}, x.toRat ≠ 0 → y.toRat ≠ 0 →
0210        RatioOrbit.crossEq (rclLHS x y) (rclRHS x y)
0211    existing_real_uniqueness :
0212      ∀ (F : ℝ → ℝ),
0213        Cost.FunctionalEquation.AczelSmoothnessPackage →
0214        Cost.FunctionalEquation.IsReciprocalCost F →
0215        Cost.FunctionalEquation.IsNormalized F →
0216        Cost.FunctionalEquation.SatisfiesCompositionLaw F →
0217        Cost.FunctionalEquation.IsCalibrated F →
0218        ContinuousOn F (Set.Ioi 0) →
0219        ∀ x : ℝ, 0 < x → F x = Cost.Jcost x
0220    native_uniqueness_target_named :
0221      PRCNativeCostUniquenessTarget = PRCNativeCostUniquenessTarget
0222
0223  /-- The PRC rational cost surface is closed through canonical RCL and bridges
0224  honestly to the existing continuous-real uniqueness theorem. -/
0225  theorem prc_jcost_certificate : PRCJCostCertificate where
0226    rational_formula := onRatioOrbit_toRat
0227    real_jcost_bridge := onRatioOrbit_toReal_jcost
0228    reciprocal := reciprocal_symmetric
0229    normalization := normalized_invariant
0230    canonical_rcl := by
0231      intro x y hx hy
0232      exact canonical_rcl_surface hx hy
0233    existing_real_uniqueness := by
0234      intro F hA hR hN hC hCal hCont x hx
0235      exact bridge_to_existing_jcost_uniqueness F hA hR hN hC hCal hCont x hx
0236    native_uniqueness_target_named := rfl
0237
0238  end PRCJCost
0239
0240  end PrimitiveRecognitionCalculus
0241  end Foundation
0242  end IndisputableMonolith
```

13. PRCJCostDistanceIncrementTriangle.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceIncrementTriangle.lean · **Lines:** 261

```
/-
  PrimitiveRecognitionCalculus/PRCJCostDistanceIncrementTriangle.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 9c: prove the one-dimensional additive increment modulus
    for the displayed rational J-cost distance.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceVerifierTriangle

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

theorem PRCJCostDistanceIncrementDisplay_formula (t : ℚ) :
    PRCJCostDistanceIncrementDisplay t =
      ((t * t) * (t * t)) / (2 * (1 + t * t)) := by
  unfold PRCJCostDistanceIncrementDisplay PRCJCostDistanceRatDisplay
  have hg_pos : (0 : ℚ) < 1 + t * t := by
    nlinarith [mul_self_nonneg t]
  have hg_ne : (1 + t * t : ℚ) ≠ 0 := ne_of_gt hg_pos
  field_simp [hg_ne]
  ring

private theorem increment_display_lt_of_sq_lt
    {t eta eps : ℚ} (heta_pos : 0 < eta) (heta_lt_one : eta < 1)
    (hsq : t * t < eta) (heta_sq_half_lt_eps : eta * eta / 2 < eps) :
    PRCJCostDistanceIncrementDisplay t < eps := by
  rw [PRCJCostDistanceIncrementDisplay_formula]
  have hs_nonneg : (0 : ℚ) ≤ t * t := mul_self_nonneg t
  have hden_pos : (0 : ℚ) < 2 * (1 + t * t) := by positivity
  have hden_ne : (2 * (1 + t * t) : ℚ) ≠ 0 := ne_of_gt hden_pos
  have hs_lt_one : t * t < 1 := lt_trans hsq heta_lt_one
  have hnum_lt : (t * t) * (t * t) < eta * eta := by nlinarith
  have hfrac_le : ((t * t) * (t * t)) / (2 * (1 + t * t)) ≤
      ((t * t) * (t * t)) / 2 := by
    have hden_ge_two : (2 : ℚ) ≤ 2 * (1 + t * t) := by nlinarith
    have hnum_nonneg : (0 : ℚ) ≤ (t * t) * (t * t) :=
      mul_nonneg hs_nonneg hs_nonneg
    exact div_le_div_of_nonneg_left hnum_nonneg (by norm_num) hden_ge_two
  have hnum_half_lt : ((t * t) * (t * t)) / 2 < eta * eta / 2 := by
    nlinarith
  exact lt_of_le_of_lt hfrac_le (lt_trans hnum_half_lt heta_sq_half_lt_eps)

private theorem sq_lt_of_display_lt_delta
    {t eta delta : ℚ} (heta_pos : 0 < eta) (_hdelta_pos : 0 < delta)
    (hdelta_le : delta ≤ eta * eta / (4 * (1 + eta)))
    (hsmall : PRCJCostDistanceIncrementDisplay t < delta) :
    t * t < eta := by
  by_contra hnot
  have hge : eta ≤ t * t := by nlinarith
  rw [PRCJCostDistanceIncrementDisplay_formula] at hsmall
  have hs_nonneg : (0 : ℚ) ≤ t * t := mul_self_nonneg t
  have hs_pos : (0 : ℚ) < t * t := lt_of_lt_of_le heta_pos hge
  have hden_s_pos : (0 : ℚ) < 2 * (1 + t * t) := by positivity
  have hden_eta_pos : (0 : ℚ) < 4 * (1 + eta) := by positivity
  have hmono :
      eta * eta / (4 * (1 + eta)) ≤
        ((t * t) * (t * t)) / (2 * (1 + t * t)) := by
    let s : ℚ := t * t
    have hs_ge : eta ≤ s := by simpa [s] using hge
    have hs_nonneg' : (0 : ℚ) ≤ s := by simpa [s] using hs_nonneg
    have hs_den_pos : (0 : ℚ) < 2 * (1 + s) := by positivity
    have h_eta_den_two_pos : (0 : ℚ) < 2 * (1 + eta) := by positivity
    have h_eta_den_four_pos : (0 : ℚ) < 4 * (1 + eta) := by positivity
    have hhalf :
        eta * eta / (4 * (1 + eta)) ≤
          eta * eta / (2 * (1 + eta)) := by
      have hnum_nonneg : (0 : ℚ) ≤ eta * eta := by nlinarith
      have hden_le : 2 * (1 + eta) ≤ 4 * (1 + eta) := by nlinarith
      exact div_le_div_of_nonneg_left hnum_nonneg h_eta_den_two_pos hden_le
    have hmon :
        eta * eta / (2 * (1 + eta)) ≤
          (s * s) / (2 * (1 + s)) := by
      have hdiff_nonneg :
          0 ≤ s * s * (1 + eta) - eta * eta * (1 + s) := by
        have hleft : 0 ≤ s - eta := by nlinarith
        have heta_nonneg : 0 ≤ eta := le_of_lt heta_pos
        have hright : 0 ≤ s + eta + s * eta := by
          nlinarith [mul_nonneg hs_nonneg' heta_nonneg]
        have hprod : 0 ≤ (s - eta) * (s + eta + s * eta) :=
          mul_nonneg hleft hright
        nlinarith
      field_simp [ne_of_gt hs_den_pos, ne_of_gt h_eta_den_two_pos]
      nlinarith [hdiff_nonneg]
    exact le_trans hhalf (by simpa [s] using hmon)
```

```
  exact not_lt_of_ge (le_trans hdelta_le hmono) hsmall

theorem PRCJCostDistanceIncrementTriangleTarget_proved :
    PRCJCostDistanceIncrementTriangleTarget := by
  intro eps heps
  let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
  let four : PRCRat := two + two
  let rho : PRCRat := eps / (1 + eps)
  let eta : PRCRat := rho / four
  let delta : PRCRat := (eta * eta) / (four * (1 + eta))
  refine (delta, ?_, ?_)
  · rw [PRCRat.positive_iff_toRat_pos]
    have heps_pos : (0 : ℚ) < eps.toRat := (PRCRat.positive_iff_toRat_pos eps).mp heps
    have htwo : two.toRat = (2 : ℚ) := by
      dsimp [two]
      change (PRCRat.add PRCRat.one PRCRat.one).toRat = (2 : ℚ)
      rw [PRCRat.toRat_add]
      norm_num [PRCRat.one_toRat]
    have hfour : four.toRat = (4 : ℚ) := by
      dsimp [four]
      change (PRCRat.add two two).toRat = (4 : ℚ)
      rw [PRCRat.toRat_add]
      norm_num [htwo]
    have h_one_add_eps : ((1 : PRCRat) + eps).toRat = 1 + eps.toRat := by
      change (PRCRat.add PRCRat.one eps).toRat = 1 + eps.toRat
      rw [PRCRat.toRat_add, PRCRat.one_toRat]
    have h_one_add_eta : ((1 : PRCRat) + eta).toRat = 1 + eta.toRat := by
      change (PRCRat.add PRCRat.one eta).toRat = 1 + eta.toRat
      rw [PRCRat.toRat_add, PRCRat.one_toRat]
    have heta_pos : (0 : ℚ) < eta.toRat := by
      rw [PRCRat.toRat_div, hfour]
      have hrho_pos : (0 : ℚ) < rho.toRat := by
        rw [PRCRat.toRat_div, h_one_add_eps]
        positivity
      positivity
    rw [PRCRat.toRat_div, PRCRat.toRat_mul', PRCRat.toRat_mul',
      h_one_add_eta, hfour]
    positivity
  · intro p q hp hq
    have heps_pos : (0 : ℚ) < eps.toRat := (PRCRat.positive_iff_toRat_pos eps).mp heps
    have htwo : two.toRat = (2 : ℚ) := by
      dsimp [two]
      change (PRCRat.add PRCRat.one PRCRat.one).toRat = (2 : ℚ)
      rw [PRCRat.toRat_add]
      norm_num [PRCRat.one_toRat]
    have hfour : four.toRat = (4 : ℚ) := by
      dsimp [four]
      change (PRCRat.add two two).toRat = (4 : ℚ)
      rw [PRCRat.toRat_add]
      norm_num [htwo]
    have h_one_add_eps : ((1 : PRCRat) + eps).toRat = 1 + eps.toRat := by
      change (PRCRat.add PRCRat.one eps).toRat = 1 + eps.toRat
      rw [PRCRat.toRat_add, PRCRat.one_toRat]
    have h_one_add_eta : ((1 : PRCRat) + eta).toRat = 1 + eta.toRat := by
      change (PRCRat.add PRCRat.one eta).toRat = 1 + eta.toRat
      rw [PRCRat.toRat_add, PRCRat.one_toRat]
    have hrho : rho.toRat = eps.toRat / (1 + eps.toRat) := by
      rw [PRCRat.toRat_div, h_one_add_eps]
    have hrho_pos : (0 : ℚ) < rho.toRat := by
      rw [hrho]
      positivity
    have hrho_lt_one : rho.toRat < 1 := by
      rw [hrho]
      field_simp [ne_of_gt (by positivity : (0 : ℚ) < 1 + eps.toRat)]
      nlinarith
    have heta : eta.toRat = rho.toRat / 4 := by
      rw [PRCRat.toRat_div, hfour]
    have heta_pos : (0 : ℚ) < eta.toRat := by
      rw [heta]
      positivity
    have heta_lt_one : eta.toRat < 1 := by
      rw [heta]
      nlinarith
    have hdelta :
        delta.toRat = eta.toRat * eta.toRat / (4 * (1 + eta.toRat)) := by
      rw [PRCRat.toRat_div, PRCRat.toRat_mul', PRCRat.toRat_mul',
        h_one_add_eta, hfour]
    have hdelta_pos : (0 : ℚ) < delta.toRat := by
      rw [hdelta]
      positivity
    have hp_sq : p * p < eta.toRat := by
      exact sq_lt_of_display_lt_delta
        (t := p) (eta := eta.toRat) (delta := delta.toRat)
        heta_pos hdelta_pos (by rw [hdelta]) hp
    have hq_sq : q * q < eta.toRat := by
      exact sq_lt_of_display_lt_delta
        (t := q) (eta := eta.toRat) (delta := delta.toRat)
        heta_pos hdelta_pos (by rw [hdelta]) hq
    have hpq_sq_lt_rho : (p + q) * (p + q) < rho.toRat := by
      have hsq_bound : (p + q) * (p + q) ≤ 2 * (p * p) + 2 * (q * q) := by
        nlinarith [mul_self_nonneg (p - q)]
      rw [heta] at hp_sq hq_sq
      nlinarith
    have hrho_sq_half_lt_eps : rho.toRat * rho.toRat / 2 < eps.toRat := by
      have hrho_lt_eps : rho.toRat < eps.toRat := by
        rw [hrho]
        field_simp [ne_of_gt (by positivity : (0 : ℚ) < 1 + eps.toRat)]
        nlinarith
      nlinarith [hrho_pos, hrho_lt_one, hrho_lt_eps]
    exact increment_display_lt_of_sq_lt
      (t := p + q) (eta := rho.toRat) (eps := eps.toRat)
```

```
      hrho_pos hrho_lt_one hpq_sq_lt_rho hrho_sq_half_lt_eps

/-- Final closure for the rational increment modulus. -/
theorem PRCJCostDistanceVerifierTriangleTarget_proved :
    PRCJCostDistanceVerifierTriangleTarget :=
  PRCJCostDistanceVerifierTriangleTarget_of_increment
    PRCJCostDistanceIncrementTriangleTarget_proved

/-- The PRC null-distance setoid target is now closed by the explicit rational
increment modulus. -/
theorem PRCNullDistanceSetoidTarget_proved :
    PRCNullDistanceSetoidTarget :=
  PRCNullDistanceSetoidTarget_of_increment_triangle
    PRCJCostDistanceIncrementTriangleTarget_proved

/-- The PRC triangle modulus is now closed by the explicit rational increment
estimate. -/
theorem PRCJCostDistanceTriangleModulusTarget_proved :
    PRCJCostDistanceTriangleModulusTarget :=
  PRCJCostDistanceTriangleModulusTarget_of_verifier
    PRCJCostDistanceVerifierTriangleTarget_proved

/-- The null-distance relation is transitive. -/
theorem PRCNullDistanceTransitiveTarget_proved :
    PRCNullDistanceTransitiveTarget :=
  PRCNullDistanceTransitiveTarget_of_triangle_modulus
    PRCJCostDistanceTriangleModulusTarget_proved

/-- The final PRC real carrier: Cauchy ledgers quotiented by null distance. -/
def PRCRealNullClosed : Type :=
  PRCRealNull PRCNullDistanceTransitiveTarget_proved

namespace PRCRealNullClosed

/-- Embed a PRC rational into the closed null-distance quotient carrier. -/
def ofRat (q : PRCRat) : PRCRealNullClosed :=
  PRCRealNull.ofRat PRCNullDistanceTransitiveTarget_proved q

end PRCRealNullClosed

/-- Build Order step 9c closure certificate. -/
structure PRCJCostDistanceIncrementTriangleCertificate : Prop where
  increment_formula :
    ∀ t : ℚ,
      PRCJCostDistanceIncrementDisplay t =
        ((t * t) * (t * t)) / (2 * (1 + t * t))
  increment_triangle : PRCJCostDistanceIncrementTriangleTarget
  verifier_triangle : PRCJCostDistanceVerifierTriangleTarget
  triangle_modulus : PRCJCostDistanceTriangleModulusTarget
  null_distance_transitive : PRCNullDistanceTransitiveTarget
  null_distance_setoid : PRCNullDistanceSetoidTarget
  real_null_carrier : Nonempty PRCRealNullClosed
  rat_embedding : Nonempty (PRCRat → PRCRealNullClosed)

/-- The explicit rational increment estimate closes the whole J-cost
null-distance setoid chain. -/
theorem prc_jcost_distance_increment_triangle_certificate :
    PRCJCostDistanceIncrementTriangleCertificate where
  increment_formula := PRCJCostDistanceIncrementDisplay_formula
  increment_triangle := PRCJCostDistanceIncrementTriangleTarget_proved
  verifier_triangle := PRCJCostDistanceVerifierTriangleTarget_proved
  triangle_modulus := PRCJCostDistanceTriangleModulusTarget_proved
  null_distance_transitive := PRCNullDistanceTransitiveTarget_proved
  null_distance_setoid := PRCNullDistanceSetoidTarget_proved
  real_null_carrier := (PRCRealNullClosed.ofRat 0)
  rat_embedding := (PRCRealNullClosed.ofRat)

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

14. PRCJCostDistanceVerifierTriangle.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceVerifierTriangle.lean · **Lines:** 104

```
/-
  PrimitiveRecognitionCalculus/PRCJCostDistanceVerifierTriangle.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 9b: prove or isolate the verifier-rational triangle
    inequality for the displayed J-cost distance.

  This pass removes endpoint bookkeeping. The displayed distance depends only
  on the rational increment, so the remaining blocker is an additive two-leg
  modulus for increments `p` and `q`.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceTriangle

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- One-increment display of the rational J-cost distance. -/
def PRCJCostDistanceIncrementDisplay (t : ℚ) : ℚ :=
  PRCJCostDistanceRatDisplay 0 t

/-- The displayed distance is translation-invariant: it depends only on the
increment between the endpoints. -/
theorem PRCJCostDistanceRatDisplay_as_increment (x y : ℚ) :
    PRCJCostDistanceRatDisplay x y =
      PRCJCostDistanceIncrementDisplay (x - y) := by
  simp [PRCJCostDistanceIncrementDisplay, PRCJCostDistanceRatDisplay]
  ring_nf

/-- Sharper exact blocker: an additive two-leg modulus for rational increments.
This is the mathematical core behind the three-endpoint verifier triangle
target. -/
def PRCJCostDistanceIncrementTriangleTarget : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ delta : PRCRat, PRCRat.positive delta ∧
      ∀ p q : ℚ,
        PRCJCostDistanceIncrementDisplay p < delta.toRat →
          PRCJCostDistanceIncrementDisplay q < delta.toRat →
            PRCJCostDistanceIncrementDisplay (p + q) < eps.toRat

/-- The increment-only triangle target implies the verifier-rational
three-endpoint triangle target. -/
theorem PRCJCostDistanceVerifierTriangleTarget_of_increment
    (h : PRCJCostDistanceIncrementTriangleTarget) :
    PRCJCostDistanceVerifierTriangleTarget := by
  intro eps heps
  rcases h eps heps with (delta, hdelta_pos, hdelta)
  refine (delta, hdelta_pos, ?_)
  intro x y z hxy hyz
  have hxy' :
      PRCJCostDistanceIncrementDisplay (x - y) < delta.toRat := by
    rwa [PRCJCostDistanceRatDisplay_as_increment] at hxy
  have hyz' :
      PRCJCostDistanceIncrementDisplay (y - z) < delta.toRat := by
    rwa [PRCJCostDistanceRatDisplay_as_increment] at hyz
  have hsum :
      PRCJCostDistanceIncrementDisplay ((x - y) + (y - z)) < eps.toRat :=
    hdelta (x - y) (y - z) hxy' hyz'
  have hxz :
      PRCJCostDistanceRatDisplay x z =
        PRCJCostDistanceIncrementDisplay ((x - y) + (y - z)) := by
    rw [PRCJCostDistanceRatDisplay_as_increment]
    congr
    ring
  rwa [hxz]

/-- The increment-only blocker closes the whole PRC null-distance setoid chain. -/
theorem PRCNullDistanceSetoidTarget_of_increment_triangle
    (h : PRCJCostDistanceIncrementTriangleTarget) :
    PRCNullDistanceSetoidTarget :=
  PRCNullDistanceSetoidTarget_of_verifier_triangle
    (PRCJCostDistanceVerifierTriangleTarget_of_increment h)

/-- Conditional certificate for step 9b. The only remaining theorem is now the
increment-only modulus target. -/
structure PRCJCostDistanceVerifierTriangleConditionalCertificate : Prop where
  translation_invariance :
    ∀ x y : ℚ,
      PRCJCostDistanceRatDisplay x y =
        PRCJCostDistanceIncrementDisplay (x - y)
  increment_triangle_target :
    PRCJCostDistanceIncrementTriangleTarget = PRCJCostDistanceIncrementTriangleTarget
  verifier_from_increment :
    PRCJCostDistanceIncrementTriangleTarget → PRCJCostDistanceVerifierTriangleTarget
  setoid_from_increment :
```

```
    PRCJCostDistanceIncrementTriangleTarget → PRCNullDistanceSetoidTarget

/-- Build Order step 9b conditional closure: the verifier triangle target is
reduced to a one-dimensional additive increment estimate. -/
theorem prc_jcost_distance_verifier_triangle_conditional_certificate :
    PRCJCostDistanceVerifierTriangleConditionalCertificate where
  translation_invariance := PRCJCostDistanceRatDisplay_as_increment
  increment_triangle_target := rfl
  verifier_from_increment := PRCJCostDistanceVerifierTriangleTarget_of_increment
  setoid_from_increment := PRCNullDistanceSetoidTarget_of_increment_triangle

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

15. PRCJCostDistanceTriangle.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceTriangle.lean · **Lines:** 94

```
/-
  PrimitiveRecognitionCalculus/PRCJCostDistanceTriangle.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 9a: prove or isolate the local triangle modulus for
    `PRCJCostDistance`.

  This pass translates the remaining PRC distance theorem into an exact
  verifier-rational inequality. No PRC object is redefined in verifier terms;
  the verifier formula is only a display theorem and blocker statement.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealNullSetoid

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- Verifier display of the J-cost square-gap distance. This is not the PRC
definition; it is the rational formula shown by `PRCJCostDistance_toRat`. -/
def PRCJCostDistanceRatDisplay (x y : ℚ) : ℚ :=
  let g : ℚ := 1 + (x - y) * (x - y)
  (g + g⁻¹) / 2 - 1

/-- Display theorem for the PRC J-cost distance. -/
theorem PRCJCostDistance_toRat (a b : PRCRat) :
    (PRCJCostDistance a b).toRat =
      PRCJCostDistanceRatDisplay a.toRat b.toRat := by
  unfold PRCJCostDistance PRCJCostDistanceRatDisplay
  rw [PRCJCost.onPRCRat_toRat, PRCSquareGap_toRat]

/-- Exact verifier-rational inequality still needed for the null-distance
quotient. It supplies a PRC rational `delta`, but the analytic estimate itself
is stated only on the conservative rational displays. -/
def PRCJCostDistanceVerifierTriangleTarget : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ delta : PRCRat, PRCRat.positive delta ∧
      ∀ x y z : ℚ,
        PRCJCostDistanceRatDisplay x y < delta.toRat →
          PRCJCostDistanceRatDisplay y z < delta.toRat →
            PRCJCostDistanceRatDisplay x z < eps.toRat

/-- The verifier-rational triangle inequality closes the PRC triangle-modulus
target by display transport. -/
theorem PRCJCostDistanceTriangleModulusTarget_of_verifier
    (h : PRCJCostDistanceVerifierTriangleTarget) :
    PRCJCostDistanceTriangleModulusTarget := by
  intro eps heps
  rcases h eps heps with (delta, hdelta_pos, hdelta)
  refine (delta, hdelta_pos, ?_)
  intro a b c hab hbc
  rw [PRCRat.lt_iff_toRat_lt] at hab hbc ⊢
  rw [PRCJCostDistance_toRat] at hab hbc ⊢
  exact hdelta a.toRat b.toRat c.toRat hab hbc

/-- Once the verifier-rational inequality is proved, the final null-distance
setoid target follows. -/
theorem PRCNullDistanceSetoidTarget_of_verifier_triangle
    (h : PRCJCostDistanceVerifierTriangleTarget) :
    PRCNullDistanceSetoidTarget :=
  PRCNullDistanceSetoidTarget_of_triangle_modulus
    (PRCJCostDistanceTriangleModulusTarget_of_verifier h)

/-- Conditional certificate for step 9a. The only remaining mathematical
problem is now the explicit rational inequality in
`PRCJCostDistanceVerifierTriangleTarget`. -/
structure PRCJCostDistanceTriangleConditionalCertificate : Prop where
  distance_display :
    ∀ a b : PRCRat,
      (PRCJCostDistance a b).toRat =
        PRCJCostDistanceRatDisplay a.toRat b.toRat
  verifier_triangle_target :
    PRCJCostDistanceVerifierTriangleTarget = PRCJCostDistanceVerifierTriangleTarget
  triangle_from_verifier :
    PRCJCostDistanceVerifierTriangleTarget → PRCJCostDistanceTriangleModulusTarget
  setoid_from_verifier :
    PRCJCostDistanceVerifierTriangleTarget → PRCNullDistanceSetoidTarget

/-- Build Order step 9a conditional closure: PRC triangle transport is reduced
to the displayed rational inequality. -/
theorem prc_jcost_distance_triangle_conditional_certificate :
    PRCJCostDistanceTriangleConditionalCertificate where
  distance_display := PRCJCostDistance_toRat
  verifier_triangle_target := rfl
  triangle_from_verifier := PRCJCostDistanceTriangleModulusTarget_of_verifier
  setoid_from_verifier := PRCNullDistanceSetoidTarget_of_verifier_triangle
```

```
0091
0092  end PrimitiveRecognitionCalculus
0093  end Foundation
0094  end IndisputableMonolith
```

16. RealCauchy.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCauchy.lean · **Lines:** 236

```
0001  /-
0002    PrimitiveRecognitionCalculus/RealCauchy.lean
0003
0004    Round-trip source:
0005      6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007    Spec anchor:
0008      Build Order step 8: start the internal PRC real completion by forming
0009      Cauchy ledgers over `PRCRat`, a J-cost-derived closeness relation, and the
0010      first internal quotient carrier.
0011
0012    Strength: δ + trace-closure. The completed sequence index is the completed
0013    orbit ledger from `TraceClosure`; verifier rationals appear only in display
0014    theorems and proofs.
0015  -/
0016
0017  import Mathlib
0018  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RationalField
0019  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.TraceClosure
0020
0021  namespace IndisputableMonolith
0022  namespace Foundation
0023  namespace PrimitiveRecognitionCalculus
0024
0025  namespace PRCRat
0026
0027  /-! ## Rational comparison surface used by Cauchy ledgers -/
0028
0029  /-- PRC-native strict order on rationals: `a < b` means the positive gap
0030  `b - a` has a positive ratio-orbit representative. -/
0031  def lt (a b : PRCRat) : Prop :=
0032    positive (b - a)
0033
0034  theorem lt_iff_toRat_lt (a b : PRCRat) :
0035      lt a b ↔ a.toRat < b.toRat := by
0036    unfold lt
0037    rw [positive_iff_toRat_pos]
0038    rw [PRCRat.sub_eq, PRCRat.toRat_sub]
0039    constructor
0040    · intro h
0041      linarith
0042    · intro h
0043      linarith
0044
0045  theorem zero_lt_of_positive {q : PRCRat}
0046      (h : positive q) : lt 0 q := by
0047    rw [lt_iff_toRat_lt]
0048    simpa using (positive_iff_toRat_pos q).mp h
0049
0050  end PRCRat
0051
0052  /-! ## J-cost-derived distance on PRC rationals -/
0053
0054  /-- A positive comparison gap for additive rational separation. The square
0055  removes the need for a rational absolute value in this first Cauchy pass. -/
0056  def PRCSquareGap (a b : PRCRat) : PRCRat :=
0057    1 + (a - b) * (a - b)
0058
0059  theorem PRCSquareGap_toRat (a b : PRCRat) :
0060      (PRCSquareGap a b).toRat = 1 + (a.toRat - b.toRat) * (a.toRat - b.toRat) := by
0061    unfold PRCSquareGap
0062    rw [PRCRat.toRat_add', PRCRat.toRat_mul']
0063    simp [PRCRat.sub_eq]
0064
0065  /-- J-cost distance used by the first PRC Cauchy surface. It sends additive
0066  separation through the positive ratio `1 + (a-b)^2`, then applies the PRC
0067  rational J-cost. -/
0068  def PRCJCostDistance (a b : PRCRat) : PRCRat :=
0069    PRCJCost.onPRCRat (PRCSquareGap a b)
0070
0071  theorem PRCJCostDistance_self_zero (a : PRCRat) :
0072      PRCJCostDistance a a = 0 := by
0073    apply PRCRat.toRat_injective
0074    unfold PRCJCostDistance
0075    rw [PRCJCost.onPRCRat_toRat, PRCSquareGap_toRat]
0076    simp
0077
0078  theorem PRCJCostDistance_symmetric (a b : PRCRat) :
0079      PRCJCostDistance a b = PRCJCostDistance b a := by
0080    apply PRCRat.toRat_injective
0081    unfold PRCJCostDistance
0082    rw [PRCJCost.onPRCRat_toRat, PRCJCost.onPRCRat_toRat,
0083      PRCSquareGap_toRat, PRCSquareGap_toRat]
0084    ring
0085
0086  /-! ## PRC Cauchy ledgers -/
0087
0088  /-- A PRC Cauchy sequence is a completed orbit-indexed rational ledger whose
0089  J-cost distance eventually falls below every positive PRC rational tolerance. -/
0090  structure PRCCauchySeq where
```

```
  term : Nat → PRCRat
  cauchy :
    ∀ eps : PRCRat, PRCRat.positive eps →
      ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
        PRCRat.lt (PRCJCostDistance (term m) (term n)) eps

namespace PRCCauchySeq

/-- Constant rational ledgers are Cauchy. -/
def constant (q : PRCRat) : PRCCauchySeq where
  term := fun _ => q
  cauchy := by
    intro eps heps
    refine (0, ?_)
    intro m n _hm _hn
    rw [PRCJCostDistance_self_zero]
    exact PRCRat.zero_lt_of_positive heps

@[simp] theorem constant_term (q : PRCRat) (n : Nat) :
    (constant q).term n = q := by
  rfl

end PRCCauchySeq

/-- The intended null-distance relation between two Cauchy ledgers. This is
the relation that should become the final real quotient once transitivity is
proved from the J-cost distance surface. -/
def PRCNullEquivalent (u v : PRCCauchySeq) : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ N : Nat, ∀ n : Nat, N ≤ n →
      PRCRat.lt (PRCJCostDistance (u.term n) (v.term n)) eps

theorem PRCNullEquivalent.refl (u : PRCCauchySeq) :
    PRCNullEquivalent u u := by
  intro eps heps
  refine (0, ?_)
  intro n _hn
  rw [PRCJCostDistance_self_zero]
  exact PRCRat.zero_lt_of_positive heps

theorem PRCNullEquivalent.symm {u v : PRCCauchySeq}
    (h : PRCNullEquivalent u v) : PRCNullEquivalent v u := by
  intro eps heps
  rcases h eps heps with (N, hN)
  refine (N, ?_)
  intro n hn
  rw [PRCJCostDistance_symmetric]
  exact hN n hn

/-- Exact blocker for the final null-distance quotient: prove triangle-style
transitivity for the J-cost distance surface. -/
def PRCNullDistanceTransitiveTarget : Prop :=
  ∀ u v w : PRCCauchySeq,
    PRCNullEquivalent u v →
      PRCNullEquivalent v w →
        PRCNullEquivalent u w

/-- Exact target that turns the intended null-distance relation into the real
setoid. Reflexivity and symmetry are proved above; transitivity is the live
mathematical obligation. -/
def PRCNullDistanceSetoidTarget : Prop :=
  Equivalence PRCNullEquivalent

/-! ## First internal quotient carrier -/

/-- Sequence identity, used only as the first quotient carrier while the
null-distance transitivity target remains open. -/
def PRCSameTerm (u v : PRCCauchySeq) : Prop :=
  ∀ n : Nat, u.term n = v.term n

theorem PRCSameTerm.equivalence : Equivalence PRCSameTerm := by
  constructor
  · intro u n
    rfl
  · intro u v h n
    exact (h n).symm
  · intro u v w huv hvw n
    exact (huv n).trans (hvw n)

/-- The first internal setoid available without the null-distance triangle
lemma. It is intentionally stronger than the final null-distance setoid. -/
def PRCSameTermSetoid : Setoid PRCCauchySeq where
  r := PRCSameTerm
  iseqv := PRCSameTerm.equivalence

/-- First internal PRC real carrier. It is a Cauchy-ledger quotient, not Lean
`ℝ`; the final quotient relation is recorded as `PRCNullDistanceSetoidTarget`. -/
def PRCReal : Type :=
  Quot PRCSameTermSetoid

namespace PRCReal

/-- Embed a PRC rational as a constant Cauchy ledger. -/
def ofRat (q : PRCRat) : PRCReal :=
  Quot.mk PRCSameTermSetoid (PRCCauchySeq.constant q)

end PRCReal

/-- K1/R9. Audit record: internal Cauchy real ledgers require trace closure. -/
def realCauchyClaim : StrengthClaim where
  label := "BuildOrder8_real_cauchy_internal_quotient"
```

```
    tag := StrengthTag.traceClosure
    statement :=
      "PRC Cauchy ledgers and their first internal quotient use completed orbit indexing."

/-- First-pass real Cauchy certificate. The carrier is internal and
trace-closure tagged. The final null-distance quotient is left as an exact
Lean target rather than hidden behind a classical real alias. -/
structure PRCRealCauchyCertificate : Prop where
  cauchy_sequences : Nonempty PRCCauchySeq
  constant_embedding_exists : Nonempty (PRCRat → PRCCauchySeq)
  jcost_distance_self_zero :
    ∀ q : PRCRat, PRCJCostDistance q q = 0
  null_relation_reflexive :
    ∀ u : PRCCauchySeq, PRCNullEquivalent u u
  null_relation_symmetric :
    ∀ u v : PRCCauchySeq, PRCNullEquivalent u v → PRCNullEquivalent v u
  same_term_setoid : Nonempty (Setoid PRCCauchySeq)
  real_quotient : Nonempty PRCReal
  rat_embedding : Nonempty (PRCRat → PRCReal)
  null_transitivity_target :
    PRCNullDistanceTransitiveTarget = PRCNullDistanceTransitiveTarget
  null_setoid_target :
    PRCNullDistanceSetoidTarget = PRCNullDistanceSetoidTarget
  strength_tag : realCauchyClaim.tag = StrengthTag.traceClosure

/-- Build Order step 8, first pass: internal Cauchy ledgers and an internal
quotient carrier exist, with the exact null-distance setoid target named. -/
theorem real_cauchy_certificate : PRCRealCauchyCertificate where
  cauchy_sequences := (PRCCauchySeq.constant 0)
  constant_embedding_exists := (PRCCauchySeq.constant)
  jcost_distance_self_zero := PRCJCostDistance_self_zero
  null_relation_reflexive := PRCNullEquivalent.refl
  null_relation_symmetric := by
    intro u v h
    exact PRCNullEquivalent.symm h
  same_term_setoid := (PRCSameTermSetoid)
  real_quotient := (PRCReal.ofRat 0)
  rat_embedding := (PRCReal.ofRat)
  null_transitivity_target := rfl
  null_setoid_target := rfl
  strength_tag := rfl

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

17. RealNullSetoid.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealNullSetoid.lean · **Lines:** 135

```
/-
  PrimitiveRecognitionCalculus/RealNullSetoid.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 9: promote the first Cauchy-ledger carrier toward the
    final null-distance quotient.

  This pass isolates the exact analytic blocker. The quotient bookkeeping is
  closed conditionally: a local triangle modulus for `PRCJCostDistance` implies
  `PRCNullEquivalent` is transitive, hence a setoid.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCauchy

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- Exact analytic blocker for the null-distance quotient. It says the
J-cost-derived rational distance has a local triangle modulus: for each
positive tolerance there is a positive smaller tolerance so that two small
legs force the composed leg below the original tolerance. -/
def PRCJCostDistanceTriangleModulusTarget : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ delta : PRCRat, PRCRat.positive delta ∧
      ∀ a b c : PRCRat,
        PRCRat.lt (PRCJCostDistance a b) delta →
          PRCRat.lt (PRCJCostDistance b c) delta →
            PRCRat.lt (PRCJCostDistance a c) eps

/-- The analytic triangle modulus is sufficient for null-distance
transitivity. All remaining work here is completed-orbit index bookkeeping. -/
theorem PRCNullDistanceTransitiveTarget_of_triangle_modulus
    (htri : PRCJCostDistanceTriangleModulusTarget) :
    PRCNullDistanceTransitiveTarget := by
  intro u v w huv hvw eps heps
  rcases htri eps heps with (delta, hdelta_pos, hdelta)
  rcases huv delta hdelta_pos with (Nuv, hNuv)
  rcases hvw delta hdelta_pos with (Nvw, hNvw)
  refine (max Nuv Nvw, ?_)
  intro n hn
  have hn_uv : Nuv ≤ n := le_trans (Nat.le_max_left Nuv Nvw) hn
  have hn_vw : Nvw ≤ n := le_trans (Nat.le_max_right Nuv Nvw) hn
  exact hdelta (u.term n) (v.term n) (w.term n)
    (hNuv n hn_uv) (hNvw n hn_vw)

/-- A transitivity proof turns `PRCNullEquivalent` into a setoid. -/
def PRCNullDistanceSetoidOfTransitive
    (htrans : PRCNullDistanceTransitiveTarget) : Setoid PRCCauchySeq where
  r := PRCNullEquivalent
  iseqv := by
    constructor
    · exact PRCNullEquivalent.refl
    · intro u v
      exact PRCNullEquivalent.symm
    · intro u v w
      exact htrans u v w

/-- Conditional final real carrier: once the analytic triangle modulus is
proved, this is the intended PRC real quotient by null distance. -/
def PRCRealNull (htrans : PRCNullDistanceTransitiveTarget) : Type :=
  Quot (PRCNullDistanceSetoidOfTransitive htrans)

namespace PRCRealNull

/-- Embed a rational as a null-distance quotient class, conditional on the
transitivity proof. -/
def ofRat (htrans : PRCNullDistanceTransitiveTarget) (q : PRCRat) :
    PRCRealNull htrans :=
  Quot.mk (PRCNullDistanceSetoidOfTransitive htrans) (PRCCauchySeq.constant q)

end PRCRealNull

/-- The exact setoid target follows from transitivity. -/
theorem PRCNullDistanceSetoidTarget_of_transitive
    (htrans : PRCNullDistanceTransitiveTarget) :
    PRCNullDistanceSetoidTarget := by
  exact (PRCNullDistanceSetoidOfTransitive htrans).iseqv

/-- The exact setoid target follows from the sharper triangle-modulus target. -/
theorem PRCNullDistanceSetoidTarget_of_triangle_modulus
    (htri : PRCJCostDistanceTriangleModulusTarget) :
    PRCNullDistanceSetoidTarget :=
  PRCNullDistanceSetoidTarget_of_transitive
    (PRCNullDistanceTransitiveTarget_of_triangle_modulus htri)

```

```
/-- K1/R9. Audit record: the final null-distance quotient still lives under
trace closure; the open obligation is analytic, not a new primitive. -/
def realNullSetoidClaim : StrengthClaim where
  label := "BuildOrder9_real_null_distance_setoid"
  tag := StrengthTag.traceClosure
  statement :=
    "The PRC real null-distance setoid follows from the J-cost distance triangle modulus."

/-- Conditional certificate for Build Order step 9. It records the precise
remaining theorem and proves that this theorem is sufficient to construct the
null-distance setoid and quotient carrier. -/
structure PRCRealNullSetoidConditionalCertificate : Prop where
  triangle_modulus_target :
    PRCJCostDistanceTriangleModulusTarget = PRCJCostDistanceTriangleModulusTarget
  transitive_from_triangle :
    PRCJCostDistanceTriangleModulusTarget → PRCNullDistanceTransitiveTarget
  setoid_from_transitive :
    PRCNullDistanceTransitiveTarget → PRCNullDistanceSetoidTarget
  setoid_from_triangle :
    PRCJCostDistanceTriangleModulusTarget → PRCNullDistanceSetoidTarget
  quotient_from_transitive :
    ∀ htrans : PRCNullDistanceTransitiveTarget, Nonempty (PRCRealNull htrans)
  rat_embedding_from_transitive :
    ∀ htrans : PRCNullDistanceTransitiveTarget, Nonempty (PRCRat → PRCRealNull htrans)
  strength_tag : realNullSetoidClaim.tag = StrengthTag.traceClosure

/-- Build Order step 9 conditional closure: no quotient mechanics remain once
the local J-cost triangle modulus is proved. -/
theorem real_null_setoid_conditional_certificate :
    PRCRealNullSetoidConditionalCertificate where
  triangle_modulus_target := rfl
  transitive_from_triangle := PRCNullDistanceTransitiveTarget_of_triangle_modulus
  setoid_from_transitive := PRCNullDistanceSetoidTarget_of_transitive
  setoid_from_triangle := PRCNullDistanceSetoidTarget_of_triangle_modulus
  quotient_from_transitive := by
    intro htrans
    exact (PRCRealNull.ofRat htrans 0)
  rat_embedding_from_transitive := by
    intro htrans
    exact (PRCRealNull.ofRat htrans)
  strength_tag := rfl

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

18. RealMulBoundedContinuity.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealMulBoundedContinuity.lean · **Lines:** 236

```
/-
  PrimitiveRecognitionCalculus/RealMulBoundedContinuity.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 10a: reduce multiplication on `PRCRealNullClosed` to
    boundedness and bounded product-continuity moduli.

  The object-level carrier remains the PRC null quotient. Verifier rationals
  appear only in display lemmas for analytic moduli.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteOrderedField

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- Eventual PRC-native boundedness for a raw rational ledger. -/
def PRCRawEventuallyBounded (s : PRCRawRatLedger) : Prop :=
  ∃ B : PRCRat, PRCRat.positive B ∧
    ∃ N : Nat, ∀ n : Nat, N ≤ n →
      PRCRat.lt (-B) (s n) ∧ PRCRat.lt (s n) B

/-- Exact boundedness target for Cauchy ledgers. -/
def PRCCauchySeqEventuallyBoundedTarget : Prop :=
  ∀ u : PRCCauchySeq, PRCRawEventuallyBounded u.raw

/-- A rational lies inside the symmetric PRC interval `[-B,B]`. -/
def PRCRat.InBound (B x : PRCRat) : Prop :=
  PRCRat.lt (-B) x ∧ PRCRat.lt x B

/-- Enlarging a symmetric PRC rational bound preserves membership in it. -/
theorem PRCRat.InBound_mono {B C x : PRCRat}
    (hCB : C.toRat ≤ B.toRat) (hx : PRCRat.InBound C x) :
    PRCRat.InBound B x := by
  rcases hx with (hlo, hhi)
  constructor
  · rw [PRCRat.lt_iff_toRat_lt] at hlo ⊢
    have hnegB : (-B).toRat = -B.toRat := by simp
    have hnegC : (-C).toRat = -C.toRat := by simp
    rw [hnegC] at hlo
    rw [hnegB]
    nlinarith
  · rw [PRCRat.lt_iff_toRat_lt] at hhi ⊢
    exact lt_of_lt_of_le hhi hCB

/-- Exact product-continuity modulus on bounded rational windows. This is the
local analytic input needed by Cauchy multiplication and multiplication
congruence. -/
def PRCJCostDistanceMulBoundedContinuityTarget : Prop :=
  ∀ eps B : PRCRat, PRCRat.positive eps → PRCRat.positive B →
    ∃ delta : PRCRat, PRCRat.positive delta ∧
      ∀ a a' b b' : PRCRat,
        PRCRat.InBound B a →
          PRCRat.InBound B a' →
            PRCRat.InBound B b →
              PRCRat.InBound B b' →
                PRCRat.lt (PRCJCostDistance a a') delta →
                  PRCRat.lt (PRCJCostDistance b b') delta →
                    PRCRat.lt (PRCJCostDistance (a * b) (a' * b')) eps

/-- Conditional proof of product Cauchy closure from eventual boundedness and
bounded product-continuity. -/
theorem PRCRealMulClosureTarget_of_bounded_continuity
    (hbounded : PRCCauchySeqEventuallyBoundedTarget)
    (hmul_cont : PRCJCostDistanceMulBoundedContinuityTarget) :
    PRCRealMulClosureTarget := by
  intro u v eps heps
  rcases hbounded u with (Bu, hBu_pos, NuB, hNuB)
  rcases hbounded v with (Bv, hBv_pos, NvB, hNvB)
  let B : PRCRat := Bu + Bv + 1
  have hB_pos : PRCRat.positive B := by
    rw [PRCRat.positive_iff_toRat_pos]
    have hBu : (0 : ℚ) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
    have hBv : (0 : ℚ) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
    simp [B]
    nlinarith
  have hBu_le_B : Bu.toRat ≤ B.toRat := by
    have hBv : (0 : ℚ) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
    simp [B]
    nlinarith
  have hBv_le_B : Bv.toRat ≤ B.toRat := by
    have hBu : (0 : ℚ) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
    simp [B]
    nlinarith
  rcases hmul_cont eps B heps hB_pos with (delta, hdelta_pos, hdelta)
```

```
  rcases u.cauchy delta hdelta_pos with (NuC, hNuC)
  rcases v.cauchy delta hdelta_pos with (NvC, hNvC)
  let N := max (max NuB NvB) (max NuC NvC)
  refine (N, ?_)
  intro m n hm hn
  have hNuB_m : NuB ≤ m := le_trans (le_trans (Nat.le_max_left NuB NvB)
    (Nat.le_max_left (max NuB NvB) (max NuC NvC))) hm
  have hNuB_n : NuB ≤ n := le_trans (le_trans (Nat.le_max_left NuB NvB)
    (Nat.le_max_left (max NuB NvB) (max NuC NvC))) hn
  have hNvB_m : NvB ≤ m := le_trans (le_trans (Nat.le_max_right NuB NvB)
    (Nat.le_max_left (max NuB NvB) (max NuC NvC))) hm
  have hNvB_n : NvB ≤ n := le_trans (le_trans (Nat.le_max_right NuB NvB)
    (Nat.le_max_left (max NuB NvB) (max NuC NvC))) hn
  have hNuC_m : NuC ≤ m := le_trans (le_trans (Nat.le_max_left NuC NvC)
    (Nat.le_max_right (max NuB NvB) (max NuC NvC))) hm
  have hNuC_n : NuC ≤ n := le_trans (le_trans (Nat.le_max_left NuC NvC)
    (Nat.le_max_right (max NuB NvB) (max NuC NvC))) hn
  have hNvC_m : NvC ≤ m := le_trans (le_trans (Nat.le_max_right NuC NvC)
    (Nat.le_max_right (max NuB NvB) (max NuC NvC))) hm
  have hNvC_n : NvC ≤ n := le_trans (le_trans (Nat.le_max_right NuC NvC)
    (Nat.le_max_right (max NuB NvB) (max NuC NvC))) hn
  have hu_m_small : PRCRat.InBound B (u.term m) :=
    PRCRat.InBound_mono hBu_le_B (hNuB m hNuB_m)
  have hu_n_small : PRCRat.InBound B (u.term n) :=
    PRCRat.InBound_mono hBu_le_B (hNuB n hNuB_n)
  have hv_m_small : PRCRat.InBound B (v.term m) :=
    PRCRat.InBound_mono hBv_le_B (hNvB m hNvB_m)
  have hv_n_small : PRCRat.InBound B (v.term n) :=
    PRCRat.InBound_mono hBv_le_B (hNvB n hNvB_n)
  exact hdelta (u.term m) (u.term n) (v.term m) (v.term n)
    hu_m_small hu_n_small hv_m_small hv_n_small
    (hNuC m n hNuC_m hNuC_n)
    (hNvC m n hNvC_m hNvC_n)

/-- Conditional proof of product congruence from eventual boundedness and
bounded product-continuity. -/
theorem PRCRealMulCongruenceTarget_of_bounded_continuity
    (hbounded : PRCCauchySeqEventuallyBoundedTarget)
    (hmul_cont : PRCJCostDistanceMulBoundedContinuityTarget) :
    PRCRealMulCongruenceTarget := by
  intro u u' v v' huu hvv eps heps
  rcases hbounded u with (Bu, hBu_pos, NuB, hNuB)
  rcases hbounded u' with (Bu', hBu'_pos, Nu'B, hNu'B)
  rcases hbounded v with (Bv, hBv_pos, NvB, hNvB)
  rcases hbounded v' with (Bv', hBv'_pos, Nv'B, hNv'B)
  let B : PRCRat := Bu + Bu' + Bv + Bv' + 1
  have hB_pos : PRCRat.positive B := by
    rw [PRCRat.positive_iff_toRat_pos]
    have hBu : (0 : ℚ) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
    have hBu' : (0 : ℚ) < Bu'.toRat := (PRCRat.positive_iff_toRat_pos Bu').mp hBu'_pos
    have hBv : (0 : ℚ) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
    have hBv' : (0 : ℚ) < Bv'.toRat := (PRCRat.positive_iff_toRat_pos Bv').mp hBv'_pos
    simp [B]
    nlinarith
  have hBu_le_B : Bu.toRat ≤ B.toRat := by
    have hBu' : (0 : ℚ) < Bu'.toRat := (PRCRat.positive_iff_toRat_pos Bu').mp hBu'_pos
    have hBv : (0 : ℚ) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
    have hBv' : (0 : ℚ) < Bv'.toRat := (PRCRat.positive_iff_toRat_pos Bv').mp hBv'_pos
    simp [B]
    nlinarith
  have hBu'_le_B : Bu'.toRat ≤ B.toRat := by
    have hBu : (0 : ℚ) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
    have hBv : (0 : ℚ) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
    have hBv' : (0 : ℚ) < Bv'.toRat := (PRCRat.positive_iff_toRat_pos Bv').mp hBv'_pos
    simp [B]
    nlinarith
  have hBv_le_B : Bv.toRat ≤ B.toRat := by
    have hBu : (0 : ℚ) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
    have hBu' : (0 : ℚ) < Bu'.toRat := (PRCRat.positive_iff_toRat_pos Bu').mp hBu'_pos
    have hBv' : (0 : ℚ) < Bv'.toRat := (PRCRat.positive_iff_toRat_pos Bv').mp hBv'_pos
    simp [B]
    nlinarith
  have hBv'_le_B : Bv'.toRat ≤ B.toRat := by
    have hBu : (0 : ℚ) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
    have hBu' : (0 : ℚ) < Bu'.toRat := (PRCRat.positive_iff_toRat_pos Bu').mp hBu'_pos
    have hBv : (0 : ℚ) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
    simp [B]
    nlinarith
  rcases hmul_cont eps B heps hB_pos with (delta, hdelta_pos, hdelta)
  rcases huu delta hdelta_pos with (NuC, hNuC)
  rcases hvv delta hdelta_pos with (NvC, hNvC)
  let N := max (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC)
  refine (N, ?_)
  intro n hn
  have hNuB_n : NuB ≤ n := le_trans
    (le_trans (Nat.le_max_left NuB Nu'B) (Nat.le_max_left (max NuB Nu'B) (max NvB Nv'B)))
    (le_trans (Nat.le_max_left (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC)) hn)
  have hNu'B_n : Nu'B ≤ n := le_trans
    (le_trans (Nat.le_max_right NuB Nu'B) (Nat.le_max_left (max NuB Nu'B) (max NvB Nv'B)))
    (le_trans (Nat.le_max_left (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC)) hn)
  have hNvB_n : NvB ≤ n := le_trans
    (le_trans (Nat.le_max_left NvB Nv'B) (Nat.le_max_right (max NuB Nu'B) (max NvB Nv'B)))
    (le_trans (Nat.le_max_left (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC)) hn)
  have hNv'B_n : Nv'B ≤ n := le_trans
    (le_trans (Nat.le_max_right NvB Nv'B) (Nat.le_max_right (max NuB Nu'B) (max NvB Nv'B)))
    (le_trans (Nat.le_max_left (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC)) hn)
  have hNuC_n : NuC ≤ n :=
    le_trans (Nat.le_max_left NuC NvC)
      (le_trans (Nat.le_max_right (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC)) hn)
  have hNvC_n : NvC ≤ n :=
    le_trans (Nat.le_max_right NuC NvC)
```

```
      (le_trans (Nat.le_max_right (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC)) hn)
  exact hdelta (u.term n) (u'.term n) (v.term n) (v'.term n)
    (PRCRat.InBound_mono hBu_le_B (hNuB n hNuB_n))
    (PRCRat.InBound_mono hBu'_le_B (hNu'B n hNu'B_n))
    (PRCRat.InBound_mono hBv_le_B (hNvB n hNvB_n))
    (PRCRat.InBound_mono hBv'_le_B (hNv'B n hNv'B_n))
    (hNuC n hNuC_n)
    (hNvC n hNvC_n)

/-- Conditional certificate: the multiplication targets reduce to eventual
boundedness plus bounded product continuity. -/
structure PRCRealMulBoundedContinuityConditionalCertificate : Prop where
  boundedness_target :
    PRCCauchySeqEventuallyBoundedTarget = PRCCauchySeqEventuallyBoundedTarget
  product_continuity_target :
    PRCJCostDistanceMulBoundedContinuityTarget =
      PRCJCostDistanceMulBoundedContinuityTarget
  mul_closure_from_targets :
    PRCCauchySeqEventuallyBoundedTarget →
      PRCJCostDistanceMulBoundedContinuityTarget →
        PRCRealMulClosureTarget
  mul_congruence_from_targets :
    PRCCauchySeqEventuallyBoundedTarget →
      PRCJCostDistanceMulBoundedContinuityTarget →
        PRCRealMulCongruenceTarget
  mul_operation_from_targets :
    PRCCauchySeqEventuallyBoundedTarget →
      PRCJCostDistanceMulBoundedContinuityTarget →
        Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)

theorem prc_real_mul_bounded_continuity_conditional_certificate :
    PRCRealMulBoundedContinuityConditionalCertificate where
  boundedness_target := rfl
  product_continuity_target := rfl
  mul_closure_from_targets := PRCRealMulClosureTarget_of_bounded_continuity
  mul_congruence_from_targets := PRCRealMulCongruenceTarget_of_bounded_continuity
  mul_operation_from_targets := by
    intro hbounded hcont
    exact (PRCRealNullClosed.mulOf
      (PRCRealMulClosureTarget_of_bounded_continuity hbounded hcont)
      (PRCRealMulCongruenceTarget_of_bounded_continuity hbounded hcont))

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

19. RealBoundednessModulus.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealBoundednessModulus.lean · **Lines:** 142

```
/-
  PrimitiveRecognitionCalculus/RealBoundednessModulus.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 10b: prove every J-cost Cauchy ledger is eventually
    PRC-bounded.

  The proof uses the verifier rational display only to extract the ordinary
  square bound from small J-cost distance. The eventual bound itself is stated
  on the PRC rational ledger.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealMulBoundedContinuity

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- A fixed PRC rational distance threshold small enough to force ordinary
increment square below one. -/
def PRCBoundednessDelta : PRCRat :=
  let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
  let four : PRCRat := two * two
  (1 : PRCRat) * ((four * two)⁻¹)

theorem PRCBoundednessDelta_toRat :
    PRCBoundednessDelta.toRat = (1 / 8 : ℚ) := by
  unfold PRCBoundednessDelta
  simp [PRCRat.toRat_mul, PRCRat.toRat_recip]
  norm_num

theorem PRCBoundednessDelta_positive :
    PRCRat.positive PRCBoundednessDelta := by
  rw [PRCRat.positive_iff_toRat_pos, PRCBoundednessDelta_toRat]
  norm_num

/-- Small increment display at the fixed threshold forces ordinary square
increment below one. -/
theorem PRCJCostDistanceIncrementDisplay_sq_lt_one {t : ℚ}
    (hsmall : PRCJCostDistanceIncrementDisplay t < (1 / 8 : ℚ)) :
    t * t < 1 := by
  by_contra hnot
  have hge : (1 : ℚ) ≤ t * t := by
    have hsq_nonneg : (0 : ℚ) ≤ t * t := mul_self_nonneg t
    nlinarith
  rw [PRCJCostDistanceIncrementDisplay_formula] at hsmall
  let s : ℚ := t * t
  have hs_ge : (1 : ℚ) ≤ s := by simpa [s] using hge
  have hs_den_pos : (0 : ℚ) < 2 * (1 + s) := by nlinarith
  have hmono : (1 / 8 : ℚ) ≤ (s * s) / (2 * (1 + s)) := by
    field_simp [ne_of_gt hs_den_pos]
    nlinarith [mul_self_nonneg s]
  exact not_lt_of_ge hmono (by simpa [s] using hsmall)

/-- Small PRC J-cost distance at the fixed threshold forces the ordinary
rational display increment to have square below one. -/
theorem PRCJCostDistance_sq_diff_lt_one_of_lt_boundedness_delta
    {a b : PRCRat}
    (hsmall : PRCRat.lt (PRCJCostDistance a b) PRCBoundednessDelta) :
    (a.toRat - b.toRat) * (a.toRat - b.toRat) < 1 := by
  rw [PRCRat.lt_iff_toRat_lt] at hsmall
  rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment,
    PRCBoundednessDelta_toRat] at hsmall
  exact PRCJCostDistanceIncrementDisplay_sq_lt_one hsmall

/-- A J-cost Cauchy ledger is eventually contained in a PRC symmetric rational
interval. -/
theorem PRCCauchySeqEventuallyBoundedTarget_proved :
    PRCCauchySeqEventuallyBoundedTarget := by
  intro u
  rcases u.cauchy PRCBoundednessDelta PRCBoundednessDelta_positive with
    (N, hN)
  let anchor : PRCRat := u.term N
  let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
  let B : PRCRat := anchor * anchor + two
  have hB_pos : PRCRat.positive B := by
    rw [PRCRat.positive_iff_toRat_pos]
    have hsq : (0 : ℚ) ≤ anchor.toRat * anchor.toRat :=
      mul_self_nonneg anchor.toRat
    simp [B, two]
    nlinarith
  refine (B, hB_pos, N, ?_)
  intro n hn
  have hdist : PRCRat.lt (PRCJCostDistance (u.term n) anchor) PRCBoundednessDelta := by
    simpa [anchor] using hN n N hn (Nat.le_refl N)
  have hsquare :
```

```
      ((u.term n).toRat - anchor.toRat) *
          ((u.term n).toRat - anchor.toRat) < 1 :=
    PRCJCostDistance_sq_diff_lt_one_of_lt_boundedness_delta hdist
  let x : ℚ := (u.term n).toRat
  let q : ℚ := anchor.toRat
  have hsquare_xq : (x - q) * (x - q) < 1 := by
    simpa [x, q] using hsquare
  have hdiff_lt_one : x - q < 1 := by
    nlinarith [mul_self_nonneg ((x - q) - 1)]
  have hdiff_gt_neg_one : -1 < x - q := by
    nlinarith [mul_self_nonneg ((x - q) + 1)]
  constructor
  · rw [PRCRat.lt_iff_toRat_lt]
    simp [B, two, anchor]
    nlinarith [mul_self_nonneg (2 * q + 1)]
  · rw [PRCRat.lt_iff_toRat_lt]
    simp [B, two, anchor]
    nlinarith [mul_self_nonneg (2 * q - 1)]

/-- Step 10b closure certificate: eventual boundedness is proved, so the
remaining multiplication blocker is only bounded product-continuity. -/
structure PRCRealBoundednessModulusCertificate : Prop where
  boundedness_delta_positive : PRCRat.positive PRCBoundednessDelta
  distance_sq_bound :
    ∀ a b : PRCRat,
      PRCRat.lt (PRCJCostDistance a b) PRCBoundednessDelta →
        (a.toRat - b.toRat) * (a.toRat - b.toRat) < 1
  eventual_boundedness : PRCCauchySeqEventuallyBoundedTarget
  mul_closure_from_product_continuity :
    PRCJCostDistanceMulBoundedContinuityTarget → PRCRealMulClosureTarget
  mul_congruence_from_product_continuity :
    PRCJCostDistanceMulBoundedContinuityTarget → PRCRealMulCongruenceTarget

theorem prc_real_boundedness_modulus_certificate :
    PRCRealBoundednessModulusCertificate where
  boundedness_delta_positive := PRCBoundednessDelta_positive
  distance_sq_bound := by
    intro a b h
    exact PRCJCostDistance_sq_diff_lt_one_of_lt_boundedness_delta h
  eventual_boundedness := PRCCauchySeqEventuallyBoundedTarget_proved
  mul_closure_from_product_continuity := by
    intro hcont
    exact PRCRealMulClosureTarget_of_bounded_continuity
      PRCCauchySeqEventuallyBoundedTarget_proved hcont
  mul_congruence_from_product_continuity := by
    intro hcont
    exact PRCRealMulCongruenceTarget_of_bounded_continuity
      PRCCauchySeqEventuallyBoundedTarget_proved hcont

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

20. RealProductContinuity.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealProductContinuity.lean · **Lines:** 304

```
/-
  PrimitiveRecognitionCalculus/RealProductContinuity.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 10c: prove the bounded product-continuity modulus for
    `PRCJCostDistance`.

  The object-level statement remains PRC-rational. The proof uses verifier
  rationals only as display transport for the analytic inequality.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealBoundednessModulus

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

theorem PRCJCostDistanceIncrementDisplay_lt_of_sq_lt
    {t eta eps : ℚ} (heta_pos : 0 < eta) (heta_lt_one : eta < 1)
    (hsq : t * t < eta) (heta_sq_half_lt_eps : eta * eta / 2 < eps) :
    PRCJCostDistanceIncrementDisplay t < eps := by
  rw [PRCJCostDistanceIncrementDisplay_formula]
  have hs_nonneg : (0 : ℚ) ≤ t * t := mul_self_nonneg t
  have hden_pos : (0 : ℚ) < 2 * (1 + t * t) := by positivity
  have hs_lt_one : t * t < 1 := lt_trans hsq heta_lt_one
  have hnum_lt : (t * t) * (t * t) < eta * eta := by nlinarith
  have hfrac_le : ((t * t) * (t * t)) / (2 * (1 + t * t)) ≤
      ((t * t) * (t * t)) / 2 := by
    have hden_ge_two : (2 : ℚ) ≤ 2 * (1 + t * t) := by nlinarith
    have hnum_nonneg : (0 : ℚ) ≤ (t * t) * (t * t) :=
      mul_nonneg hs_nonneg hs_nonneg
    exact div_le_div_of_nonneg_left hnum_nonneg (by norm_num) hden_ge_two
  have hnum_half_lt : ((t * t) * (t * t)) / 2 < eta * eta / 2 := by
    nlinarith
  exact lt_of_le_of_lt hfrac_le (lt_trans hnum_half_lt heta_sq_half_lt_eps)

theorem PRCJCostDistance_sq_lt_of_display_lt_delta
    {t eta delta : ℚ} (heta_pos : 0 < eta) (_hdelta_pos : 0 < delta)
    (hdelta_le : delta ≤ eta * eta / (4 * (1 + eta)))
    (hsmall : PRCJCostDistanceIncrementDisplay t < delta) :
    t * t < eta := by
  by_contra hnot
  have hge : eta ≤ t * t := by nlinarith
  rw [PRCJCostDistanceIncrementDisplay_formula] at hsmall
  have hs_nonneg : (0 : ℚ) ≤ t * t := mul_self_nonneg t
  have hs_pos : (0 : ℚ) < t * t := lt_of_lt_of_le heta_pos hge
  have hden_s_pos : (0 : ℚ) < 2 * (1 + t * t) := by positivity
  have hmono :
      eta * eta / (4 * (1 + eta)) ≤
        ((t * t) * (t * t)) / (2 * (1 + t * t)) := by
    let s : ℚ := t * t
    have hs_ge : eta ≤ s := by simpa [s] using hge
    have hs_nonneg' : (0 : ℚ) ≤ s := by simpa [s] using hs_nonneg
    have hs_den_pos : (0 : ℚ) < 2 * (1 + s) := by positivity
    have h_eta_den_two_pos : (0 : ℚ) < 2 * (1 + eta) := by positivity
    have h_eta_den_four_pos : (0 : ℚ) < 4 * (1 + eta) := by positivity
    have hhalf :
        eta * eta / (4 * (1 + eta)) ≤
          eta * eta / (2 * (1 + eta)) := by
      have hnum_nonneg : (0 : ℚ) ≤ eta * eta := by nlinarith
      have hden_le : 2 * (1 + eta) ≤ 4 * (1 + eta) := by nlinarith
      exact div_le_div_of_nonneg_left hnum_nonneg h_eta_den_two_pos hden_le
    have hmon :
        eta * eta / (2 * (1 + eta)) ≤
          (s * s) / (2 * (1 + s)) := by
      have hdiff_nonneg :
          0 ≤ s * s * (1 + eta) - eta * eta * (1 + s) := by
        have hleft : 0 ≤ s - eta := by nlinarith
        have heta_nonneg : 0 ≤ eta := le_of_lt heta_pos
        have hright : 0 ≤ s + eta + s * eta := by
          nlinarith [mul_nonneg hs_nonneg' heta_nonneg]
        have hprod : 0 ≤ (s - eta) * (s + eta + s * eta) :=
          mul_nonneg hleft hright
        nlinarith
      field_simp [ne_of_gt hs_den_pos, ne_of_gt h_eta_den_two_pos]
      nlinarith [hdiff_nonneg]
    exact le_trans hhalf (by simpa [s] using hmon)
  exact not_lt_of_ge (le_trans hdelta_le hmono) hsmall

theorem PRCRat.InBound_sq_lt {B x : PRCRat}
    (hB : PRCRat.positive B) (hx : PRCRat.InBound B x) :
    x.toRat * x.toRat < B.toRat * B.toRat := by
  rcases hx with (hlo, hhi)
  rw [PRCRat.lt_iff_toRat_lt] at hlo hhi
  have hB_pos : (0 : ℚ) < B.toRat := (PRCRat.positive_iff_toRat_pos B).mp hB
  have hneg : (-B).toRat = -B.toRat := by simp
```

```
  rw [hneg] at hlo
  have hleft : 0 < B.toRat - x.toRat := by nlinarith
  have hright : 0 < B.toRat + x.toRat := by nlinarith
  have hprod : 0 < (B.toRat - x.toRat) * (B.toRat + x.toRat) :=
    mul_pos hleft hright
  nlinarith

private theorem product_factor_sq_lt
    {da b eta M : ℚ}
    (hda : da * da < eta)
    (hb : b * b < M * M)
    (hM_pos : 0 < M) :
    (da * b) * (da * b) < eta * (M * M) := by
  have hda_nonneg : 0 ≤ da * da := mul_self_nonneg da
  have hb_nonneg : 0 ≤ b * b := mul_self_nonneg b
  have hM_sq_pos : 0 < M * M := mul_pos hM_pos hM_pos
  have hgap1 : 0 < eta - da * da := by nlinarith
  have hgap2 : 0 < M * M - b * b := by nlinarith
  have hterm1 : 0 < (eta - da * da) * (M * M) :=
    mul_pos hgap1 hM_sq_pos
  have hterm2 : 0 ≤ (da * da) * (M * M - b * b) :=
    mul_nonneg hda_nonneg (le_of_lt hgap2)
  have hgap :
      0 < eta * (M * M) - (da * da) * (b * b) := by
    nlinarith
  have hsq : (da * b) * (da * b) = (da * da) * (b * b) := by ring
  rw [hsq]
  nlinarith

private theorem rational_product_increment_sq_lt
    {a a' b b' M eta rho : ℚ}
    (hM_pos : 0 < M) (hrho_pos : 0 < rho)
    (heta_eq : eta = rho / (4 * (1 + M * M)))
    (ha' : a' * a' < M * M)
    (hb : b * b < M * M)
    (hda : (a - a') * (a - a') < eta)
    (hdb : (b - b') * (b - b') < eta) :
    (a * b - a' * b') * (a * b - a' * b') < rho := by
  let u : ℚ := (a - a') * b
  let v : ℚ := a' * (b - b')
  have hu : u * u < eta * (M * M) := by
    simpa [u] using product_factor_sq_lt
      (da := a - a') (b := b) (eta := eta) (M := M)
      hda hb hM_pos
  have hv : v * v < eta * (M * M) := by
    simpa [v, mul_comm, mul_left_comm, mul_assoc] using product_factor_sq_lt
      (da := b - b') (b := a') (eta := eta) (M := M)
      hdb ha' hM_pos
  have hsum_le : (u + v) * (u + v) ≤ 2 * (u * u) + 2 * (v * v) := by
    nlinarith [mul_self_nonneg (u - v)]
  have hsum_lt : (u + v) * (u + v) < 4 * eta * (M * M) := by
    nlinarith
  have hscale : 4 * eta * (M * M) < rho := by
    rw [heta_eq]
    have hden_pos : (0 : ℚ) < 4 * (1 + M * M) := by positivity
    field_simp [ne_of_gt hden_pos]
    have hM_sq_pos : 0 < M * M := mul_pos hM_pos hM_pos
    nlinarith
  have hidentity : a * b - a' * b' = u + v := by
    dsimp [u, v]
    ring
  rw [hidentity]
  exact lt_trans hsum_lt hscale

theorem PRCJCostDistanceMulBoundedContinuityTarget_proved :
    PRCJCostDistanceMulBoundedContinuityTarget := by
  intro eps B heps hB
  let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
  let four : PRCRat := two * two
  let rho : PRCRat := eps * (((1 : PRCRat) + eps)⁻¹)
  let K : PRCRat := (1 : PRCRat) + (B * B)
  let eta : PRCRat := rho * ((four * K)⁻¹)
  let delta : PRCRat := (eta * eta) * ((four * ((1 : PRCRat) + eta))⁻¹)
  have heps_pos : (0 : ℚ) < eps.toRat :=
    (PRCRat.positive_iff_toRat_pos eps).mp heps
  have hB_pos : (0 : ℚ) < B.toRat :=
    (PRCRat.positive_iff_toRat_pos B).mp hB
  have htwo : two.toRat = (2 : ℚ) := by
    dsimp [two]
    change (PRCRat.add PRCRat.one PRCRat.one).toRat = (2 : ℚ)
    rw [PRCRat.toRat_add]
    norm_num [PRCRat.one_toRat]
  have hfour : four.toRat = (4 : ℚ) := by
    dsimp [four]
    change (PRCRat.mul two two).toRat = (4 : ℚ)
    rw [PRCRat.toRat_mul]
    norm_num [htwo]
  have h_one_add_eps :
      (((1 : PRCRat) + eps).toRat) = 1 + eps.toRat := by
    change (PRCRat.add PRCRat.one eps).toRat = 1 + eps.toRat
    rw [PRCRat.toRat_add, PRCRat.one_toRat]
  have hK : K.toRat = 1 + B.toRat * B.toRat := by
    dsimp [K]
    change (PRCRat.add PRCRat.one (PRCRat.mul B B)).toRat =
      1 + B.toRat * B.toRat
    rw [PRCRat.toRat_add, PRCRat.one_toRat, PRCRat.toRat_mul]
  have hK_pos : (0 : ℚ) < K.toRat := by
    rw [hK]
    nlinarith [mul_self_nonneg B.toRat]
  have hrho : rho.toRat = eps.toRat / (1 + eps.toRat) := by
    dsimp [rho]
```

```
    simp [PRCRat.toRat_mul, PRCRat.toRat_recip, PRCRat.toRat_add,
      PRCRat.one_toRat]
    ring
  have hrho_pos : (0 : ℚ) < rho.toRat := by
    rw [hrho]
    positivity
  have hrho_lt_one : rho.toRat < 1 := by
    rw [hrho]
    field_simp [ne_of_gt (by positivity : (0 : ℚ) < 1 + eps.toRat)]
    nlinarith
  have hrho_sq_half_lt_eps : rho.toRat * rho.toRat / 2 < eps.toRat := by
    have hrho_lt_eps : rho.toRat < eps.toRat := by
      rw [hrho]
      field_simp [ne_of_gt (by positivity : (0 : ℚ) < 1 + eps.toRat)]
      nlinarith
    nlinarith [hrho_pos, hrho_lt_one, hrho_lt_eps]
  have heta : eta.toRat = rho.toRat / (4 * K.toRat) := by
    dsimp [eta]
    rw [PRCRat.toRat_mul, PRCRat.toRat_recip, PRCRat.toRat_mul, hfour]
    ring
  have heta_pos : (0 : ℚ) < eta.toRat := by
    rw [heta]
    positivity
  have heta_lt_one : eta.toRat < 1 := by
    rw [heta]
    have hden_pos : (0 : ℚ) < 4 * K.toRat := by positivity
    field_simp [ne_of_gt hden_pos]
    have hK_gt_zero : 0 < 4 * K.toRat := by positivity
    nlinarith [hrho_lt_one, hrho_pos, hK_pos]
  have h_one_add_eta :
      (((1 : PRCRat) + eta).toRat) = 1 + eta.toRat := by
    change (PRCRat.add PRCRat.one eta).toRat = 1 + eta.toRat
    rw [PRCRat.toRat_add, PRCRat.one_toRat]
  have hdelta :
      delta.toRat = eta.toRat * eta.toRat / (4 * (1 + eta.toRat)) := by
    dsimp [delta]
    simp [PRCRat.toRat_mul, PRCRat.toRat_recip, PRCRat.toRat_add,
      PRCRat.one_toRat, hfour]
    have hden_pos : (0 : ℚ) < 4 * (1 + eta.toRat) := by positivity
    field_simp [ne_of_gt hden_pos]
  have hdelta_pos_rat : (0 : ℚ) < delta.toRat := by
    rw [hdelta]
    positivity
  have hdelta_pos : PRCRat.positive delta := by
    rw [PRCRat.positive_iff_toRat_pos]
    exact hdelta_pos_rat
  refine (delta, hdelta_pos, ?_)
  intro a a' b b' ha ha' hb hb' haa hbb
  have ha'_sq : a'.toRat * a'.toRat < B.toRat * B.toRat :=
    PRCRat.InBound_sq_lt hB ha'
  have hb_sq : b.toRat * b.toRat < B.toRat * B.toRat :=
    PRCRat.InBound_sq_lt hB hb
  have haa_rat : PRCJCostDistanceIncrementDisplay (a.toRat - a'.toRat) < delta.toRat := by
    rw [PRCRat.lt_iff_toRat_lt] at haa
    rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment] at haa
    exact haa
  have hbb_rat : PRCJCostDistanceIncrementDisplay (b.toRat - b'.toRat) < delta.toRat := by
    rw [PRCRat.lt_iff_toRat_lt] at hbb
    rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment] at hbb
    exact hbb
  have hda_sq : (a.toRat - a'.toRat) * (a.toRat - a'.toRat) < eta.toRat :=
    PRCJCostDistance_sq_lt_of_display_lt_delta
      (t := a.toRat - a'.toRat) (eta := eta.toRat) (delta := delta.toRat)
      heta_pos hdelta_pos_rat (by rw [hdelta]) haa_rat
  have hdb_sq : (b.toRat - b'.toRat) * (b.toRat - b'.toRat) < eta.toRat :=
    PRCJCostDistance_sq_lt_of_display_lt_delta
      (t := b.toRat - b'.toRat) (eta := eta.toRat) (delta := delta.toRat)
      heta_pos hdelta_pos_rat (by rw [hdelta]) hbb_rat
  have hprod_sq :
      ((a * b).toRat - (a' * b').toRat) *
          ((a * b).toRat - (a' * b').toRat) < rho.toRat := by
    simp [PRCRat.toRat_mul]
    exact rational_product_increment_sq_lt
      (a := a.toRat) (a' := a'.toRat) (b := b.toRat) (b' := b'.toRat)
      (M := B.toRat) (eta := eta.toRat) (rho := rho.toRat)
      hB_pos hrho_pos (by rw [heta, hK]) ha'_sq hb_sq hda_sq hdb_sq
  rw [PRCRat.lt_iff_toRat_lt]
  rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment]
  exact PRCJCostDistanceIncrementDisplay_lt_of_sq_lt
    (t := (a * b).toRat - (a' * b').toRat)
    (eta := rho.toRat) (eps := eps.toRat)
    hrho_pos hrho_lt_one hprod_sq hrho_sq_half_lt_eps

structure PRCRealProductContinuityCertificate : Prop where
  product_continuity : PRCJCostDistanceMulBoundedContinuityTarget
  mul_closure : PRCRealMulClosureTarget
  mul_congruence : PRCRealMulCongruenceTarget
  mul_operation :
    Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)

theorem prc_real_product_continuity_certificate :
    PRCRealProductContinuityCertificate where
  product_continuity := PRCJCostDistanceMulBoundedContinuityTarget_proved
  mul_closure :=
    PRCRealMulClosureTarget_of_bounded_continuity
      PRCCauchySeqEventuallyBoundedTarget_proved
      PRCJCostDistanceMulBoundedContinuityTarget_proved
  mul_congruence :=
    PRCRealMulCongruenceTarget_of_bounded_continuity
      PRCCauchySeqEventuallyBoundedTarget_proved
      PRCJCostDistanceMulBoundedContinuityTarget_proved
```

```
  mul_operation := by
    exact (PRCRealNullClosed.mulOf
      (PRCRealMulClosureTarget_of_bounded_continuity
        PRCCauchySeqEventuallyBoundedTarget_proved
        PRCJCostDistanceMulBoundedContinuityTarget_proved)
      (PRCRealMulCongruenceTarget_of_bounded_continuity
        PRCCauchySeqEventuallyBoundedTarget_proved
        PRCJCostDistanceMulBoundedContinuityTarget_proved))

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

21. RealOrderCongruence.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealOrderCongruence.lean · **Lines:** 206

```
/-
  PrimitiveRecognitionCalculus/RealOrderCongruence.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 10d: prove that the eventual non-strict order candidate
    descends to the null-distance quotient.

  The proof uses verifier rationals only to read the J-cost distance as an
  ordinary small increment. The order statement itself remains over PRC raw
  ledgers.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealProductContinuity

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

private theorem rat_sq_lt_sq_bounds {x gamma : ℚ}
    (hgamma : 0 < gamma) (hsq : x * x < gamma * gamma) :
    -gamma < x ∧ x < gamma := by
  constructor
  · by_contra hnot
    have hxle : x ≤ -gamma := by linarith
    have hnonneg : 0 ≤ -x - gamma := by linarith
    have hprod : 0 ≤ (-x - gamma) * (-x + gamma) := by
      have hright : 0 ≤ -x + gamma := by linarith
      exact mul_nonneg hnonneg hright
    nlinarith
  · by_contra hnot
    have hxge : gamma ≤ x := by linarith
    have hnonneg : 0 ≤ x - gamma := by linarith
    have hprod : 0 ≤ (x - gamma) * (x + gamma) := by
      have hright : 0 ≤ x + gamma := by linarith
      exact mul_nonneg hnonneg hright
    nlinarith

theorem PRCJCostDistance_sq_diff_lt_of_lt_modulus
    {a b eta delta : PRCRat}
    (heta : PRCRat.positive eta)
    (hdelta_pos : PRCRat.positive delta)
    (hdelta_le :
      delta.toRat ≤ eta.toRat * eta.toRat / (4 * (1 + eta.toRat)))
    (hsmall : PRCRat.lt (PRCJCostDistance a b) delta) :
    (a.toRat - b.toRat) * (a.toRat - b.toRat) < eta.toRat := by
  have heta_pos : 0 < eta.toRat := (PRCRat.positive_iff_toRat_pos eta).mp heta
  have hdelta_pos_rat : 0 < delta.toRat :=
    (PRCRat.positive_iff_toRat_pos delta).mp hdelta_pos
  have hdisplay :
      PRCJCostDistanceIncrementDisplay (a.toRat - b.toRat) < delta.toRat := by
    rw [PRCRat.lt_iff_toRat_lt] at hsmall
    rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment] at hsmall
    exact hsmall
  exact PRCJCostDistance_sq_lt_of_display_lt_delta
    (t := a.toRat - b.toRat) (eta := eta.toRat) (delta := delta.toRat)
    heta_pos hdelta_pos_rat hdelta_le hdisplay

theorem PRCJCostDistance_abs_diff_lt_of_lt_order_delta
    {a b gamma delta : PRCRat}
    (hgamma : PRCRat.positive gamma)
    (hdelta_pos : PRCRat.positive delta)
    (hdelta_le :
      delta.toRat ≤
        (gamma.toRat * gamma.toRat) * (gamma.toRat * gamma.toRat) /
          (4 * (1 + gamma.toRat * gamma.toRat)))
    (hsmall : PRCRat.lt (PRCJCostDistance a b) delta) :
    -gamma.toRat < a.toRat - b.toRat ∧
      a.toRat - b.toRat < gamma.toRat := by
  let eta : PRCRat := gamma * gamma
  have heta : PRCRat.positive eta := by
    rw [PRCRat.positive_iff_toRat_pos]
    have hgamma_pos : 0 < gamma.toRat :=
      (PRCRat.positive_iff_toRat_pos gamma).mp hgamma
    simp [eta, PRCRat.toRat_mul]
    nlinarith
  have heta_toRat : eta.toRat = gamma.toRat * gamma.toRat := by
    simp [eta, PRCRat.toRat_mul]
  have hsq :
      (a.toRat - b.toRat) * (a.toRat - b.toRat) <
        gamma.toRat * gamma.toRat := by
    have hcore := PRCJCostDistance_sq_diff_lt_of_lt_modulus
      (a := a) (b := b) (eta := eta) (delta := delta)
      heta hdelta_pos (by simpa [heta_toRat] using hdelta_le) hsmall
    simpa [heta_toRat] using hcore
  exact rat_sq_lt_sq_bounds
    ((PRCRat.positive_iff_toRat_pos gamma).mp hgamma) hsq
```

```
0091
0092 theorem PRCRawEventuallyLe_of_null_equiv
0093     {u u' v v' : PRCCauchySeq}
0094     (huu : PRCNullEquivalent u u')
0095     (hvv : PRCNullEquivalent v v')
0096     (hle : PRCRawEventuallyLe u.raw v.raw) :
0097     PRCRawEventuallyLe u'.raw v'.raw := by
0098   intro eps heps
0099   let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
0100   let four : PRCRat := two * two
0101   let gamma : PRCRat := eps * (four⁻¹)
0102   let eta : PRCRat := gamma * gamma
0103   let delta : PRCRat := (eta * eta) * ((four * ((1 : PRCRat) + eta))⁻¹)
0104   have heps_pos : 0 < eps.toRat :=
0105     (PRCRat.positive_iff_toRat_pos eps).mp heps
0106   have htwo : two.toRat = (2 : ℚ) := by
0107     dsimp [two]
0108     change (PRCRat.add PRCRat.one PRCRat.one).toRat = (2 : ℚ)
0109     rw [PRCRat.toRat_add, PRCRat.one_toRat]
0110     norm_num
0111   have hfour : four.toRat = (4 : ℚ) := by
0112     dsimp [four]
0113     change (PRCRat.mul two two).toRat = (4 : ℚ)
0114     rw [PRCRat.toRat_mul]
0115     norm_num [htwo]
0116   have hgamma_toRat : gamma.toRat = eps.toRat / 4 := by
0117     dsimp [gamma]
0118     rw [PRCRat.toRat_mul, PRCRat.toRat_recip, hfour]
0119     ring
0120   have hgamma_pos_rat : 0 < gamma.toRat := by
0121     rw [hgamma_toRat]
0122     positivity
0123   have hgamma_pos : PRCRat.positive gamma := by
0124     rw [PRCRat.positive_iff_toRat_pos]
0125     exact hgamma_pos_rat
0126   have heta_toRat : eta.toRat = gamma.toRat * gamma.toRat := by
0127     simp [eta, PRCRat.toRat_mul]
0128   have heta_pos_rat : 0 < eta.toRat := by
0129     rw [heta_toRat]
0130     nlinarith
0131   have h_one_add_eta :
0132       (((1 : PRCRat) + eta).toRat) = 1 + eta.toRat := by
0133     change (PRCRat.add PRCRat.one eta).toRat = 1 + eta.toRat
0134     rw [PRCRat.toRat_add, PRCRat.one_toRat]
0135   have hdelta_toRat :
0136       delta.toRat =
0137         eta.toRat * eta.toRat / (4 * (1 + eta.toRat)) := by
0138     dsimp [delta]
0139     simp [PRCRat.toRat_mul, PRCRat.toRat_recip, PRCRat.toRat_add,
0140       PRCRat.one_toRat, hfour]
0141     have hden_pos : (0 : ℚ) < 4 * (1 + eta.toRat) := by positivity
0142     field_simp [ne_of_gt hden_pos]
0143   have hdelta_pos_rat : 0 < delta.toRat := by
0144     rw [hdelta_toRat]
0145     positivity
0146   have hdelta_pos : PRCRat.positive delta := by
0147     rw [PRCRat.positive_iff_toRat_pos]
0148     exact hdelta_pos_rat
0149   rcases huu delta hdelta_pos with (Nu, hNu)
0150   rcases hvv delta hdelta_pos with (Nv, hNv)
0151   rcases hle gamma hgamma_pos with (Nle, hNle)
0152   refine (max (max Nu Nv) Nle, ?_)
0153   intro n hn
0154   have hNu_n : Nu ≤ n :=
0155     le_trans (Nat.le_max_left Nu Nv)
0156       (le_trans (Nat.le_max_left (max Nu Nv) Nle) hn)
0157   have hNv_n : Nv ≤ n :=
0158     le_trans (Nat.le_max_right Nu Nv)
0159       (le_trans (Nat.le_max_left (max Nu Nv) Nle) hn)
0160   have hNle_n : Nle ≤ n :=
0161     le_trans (Nat.le_max_right (max Nu Nv) Nle) hn
0162   have hu_close := PRCJCostDistance_abs_diff_lt_of_lt_order_delta
0163     (a := u.term n) (b := u'.term n) (gamma := gamma) (delta := delta)
0164     hgamma_pos hdelta_pos (by simp [hdelta_toRat, heta_toRat])
0165     (hNu n hNu_n)
0166   have hv_close := PRCJCostDistance_abs_diff_lt_of_lt_order_delta
0167     (a := v.term n) (b := v'.term n) (gamma := gamma) (delta := delta)
0168     hgamma_pos hdelta_pos (by simp [hdelta_toRat, heta_toRat])
0169     (hNv n hNv_n)
0170   have hu'_lt : (u'.term n).toRat < (u.term n).toRat + gamma.toRat := by
0171     rcases hu_close with (hlo, _hhi)
0172     nlinarith
0173   have hv_lt : (v.term n).toRat < (v'.term n).toRat + gamma.toRat := by
0174     rcases hv_close with (hlo, _hhi)
0175     nlinarith
0176   have huv_lt : (u.term n).toRat < (v.term n).toRat + gamma.toRat := by
0177     have hle_n := hNle n hNle_n
0178     rw [PRCRat.lt_iff_toRat_lt] at hle_n
0179     simpa [PRCCauchySeq.raw, PRCRat.toRat_add] using hle_n
0180   rw [PRCRat.lt_iff_toRat_lt]
0181   simp [PRCCauchySeq.raw, PRCRat.toRat_add]
0182   have hthree_gamma_lt_eps : 3 * gamma.toRat < eps.toRat := by
0183     rw [hgamma_toRat]
0184     nlinarith
0185   nlinarith
0186
0187 theorem PRCRealOrderCongruenceTarget_proved :
0188     PRCRealOrderCongruenceTarget := by
0189   intro u u' v v' huu hvv
0190   constructor
0191   · intro hle
```

```
    exact PRCRawEventuallyLe_of_null_equiv huu hvv hle
  · intro hle
    exact PRCRawEventuallyLe_of_null_equiv
      (PRCNullEquivalent.symm huu) (PRCNullEquivalent.symm hvv) hle

structure PRCRealOrderCongruenceCertificate : Prop where
  order_congruence : PRCRealOrderCongruenceTarget

theorem prc_real_order_congruence_certificate :
    PRCRealOrderCongruenceCertificate where
  order_congruence := PRCRealOrderCongruenceTarget_proved

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

22. RealCompleteness.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteness.lean · **Lines:** 592

```
/-
  PrimitiveRecognitionCalculus/RealCompleteness.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 10e: replace the placeholder completeness target with a
    theorem-shaped internal Cauchy-of-Cauchy statement and isolate the exact
    diagonal blocker.

  This file closes the construction fact that every raw PRC rational Cauchy
  ledger already determines a point of the null quotient, then closes the
  explicit `PRCRealCompletenessTarget` diagonal theorem.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealOrderCongruence

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- Every raw rational Cauchy ledger can be packaged as a `PRCCauchySeq`. -/
def PRCRawCauchyRealizationTarget : Prop :=
  ∀ s : PRCRawRatLedger,
    PRCRawCauchy s →
      ∃ u : PRCCauchySeq, u.raw = s

theorem PRCRawCauchyRealizationTarget_proved :
    PRCRawCauchyRealizationTarget := by
  intro s hs
  refine ({ term := s, cauchy := hs }, ?_)
  rfl

/-- Every raw rational Cauchy ledger determines a point of the final
null-distance quotient. -/
def PRCRawCauchyQuotientPointTarget : Prop :=
  ∀ s : PRCRawRatLedger,
    PRCRawCauchy s →
      Nonempty PRCRealNullClosed

theorem PRCRawCauchyQuotientPointTarget_proved :
    PRCRawCauchyQuotientPointTarget := by
  intro s hs
  rcases PRCRawCauchyRealizationTarget_proved s hs with ⟨u, _hu⟩
  exact ⟨Quot.mk
    (PRCNullDistanceSetoidOfTransitive PRCNullDistanceTransitiveTarget_proved)
    u⟩

/-- Named diagonal selection blocker for internal completeness. It asks for an
actual Cauchy ledger limit for every representative-Cauchy sequence of Cauchy
ledgers. -/
def PRCRealDiagonalSelectionTarget : Prop :=
  ∀ U : Nat → PRCCauchySeq,
    PRCRealRepresentativeCauchy U →
      ∃ L : PRCCauchySeq, PRCRealRepresentativeLimit U L

/-- Sharper diagonal blocker: construct the raw rational ledger underneath the
limit and prove both its Cauchy property and its limit property. This is the
mathematical work left after the quotient and packaging bookkeeping is removed. -/
def PRCRealRawDiagonalLedgerTarget : Prop :=
  ∀ U : Nat → PRCCauchySeq,
    PRCRealRepresentativeCauchy U →
      ∃ s : PRCRawRatLedger,
        PRCRawCauchy s ∧
          ∀ eps : PRCRat, PRCRat.positive eps →
            ∃ N : Nat, ∀ n : Nat, N ≤ n →
              PRCRawEventuallyClose (U n).raw s eps

/-- Tail-selection version of the raw diagonal theorem. It asks for an explicit
choice of a sufficiently deep raw index in each representative, so the diagonal
ledger is made from actual terms of the input Cauchy ledgers. -/
def PRCRealTailSelectionTarget : Prop :=
  ∀ U : Nat → PRCCauchySeq,
    PRCRealRepresentativeCauchy U →
      ∃ pick : Nat → Nat,
        let s : PRCRawRatLedger := fun n => (U n).term (pick n)
        PRCRawCauchy s ∧
          ∀ eps : PRCRat, PRCRat.positive eps →
            ∃ N : Nat, ∀ n : Nat, N ≤ n →
              PRCRawEventuallyClose (U n).raw s eps

/-- The PRC rational unit-fraction tolerance `1 / (n+1)`. This is verifier
display machinery for the completeness proof, not a new PRC primitive. -/
def PRCUnitFraction (n : Nat) : PRCRat :=
  let den : DistinctionNat := DistinctionNat.ofNat (n + 1)
  have hden : den ≠ DistinctionNat.zero := by
    intro h
    have hnat := congrArg DistinctionNat.toNat h
```

```
    rw [DistinctionNat.toNat_ofNat, DistinctionNat.toNat_zero] at hnat
    omega
  PRCRat.mk {
    num := SignedOrbit.one
    den := den
    den_ne_zero := hden
  }

theorem PRCUnitFraction_toRat (n : Nat) :
    (PRCUnitFraction n).toRat = (1 : ℚ) / (n + 1 : Nat) := by
  unfold PRCUnitFraction
  rw [PRCRat.toRat_mk]
  unfold RatioOrbit.toRat
  simp [SignedOrbit.one_toInt, DistinctionNat.toNat_ofNat]

theorem PRCUnitFraction_positive (n : Nat) :
    PRCRat.positive (PRCUnitFraction n) := by
  rw [PRCRat.positive_iff_toRat_pos, PRCUnitFraction_toRat]
  positivity

/-- Exact cofinal tolerance-schedule target needed by the tail-selection
diagonal proof. -/
def PRCRealCofinalToleranceScheduleTarget : Prop :=
  ∃ tau : Nat → PRCRat,
    (∀ n : Nat, PRCRat.positive (tau n)) ∧
      ∀ eps : PRCRat, PRCRat.positive eps →
        ∃ N : Nat, ∀ n : Nat, N ≤ n → PRCRat.lt (tau n) eps

theorem PRCUnitFraction_eventually_lt
    {eps : PRCRat} (heps : PRCRat.positive eps) :
    ∃ N : Nat, ∀ n : Nat, N ≤ n →
      PRCRat.lt (PRCUnitFraction n) eps := by
  have heps_pos : (0 : ℚ) < eps.toRat :=
    (PRCRat.positive_iff_toRat_pos eps).mp heps
  rcases exists_nat_gt (1 / eps.toRat) with ⟨N, hN⟩
  refine ⟨N, ?_⟩
  intro n hn
  rw [PRCRat.lt_iff_toRat_lt, PRCUnitFraction_toRat]
  have hN_le_n : (N : ℚ) ≤ (n : ℚ) := by exact_mod_cast hn
  have hn_lt_succ : (n : ℚ) < (n + 1 : Nat) := by
    norm_num
  have hgt : (1 / eps.toRat : ℚ) < (n + 1 : Nat) := by
    exact lt_of_lt_of_le hN (le_trans hN_le_n (le_of_lt hn_lt_succ))
  have hprod : (1 : ℚ) < eps.toRat * (n + 1 : Nat) := by
    have hmul := mul_lt_mul_of_pos_left hgt heps_pos
    have hone : eps.toRat * (1 / eps.toRat) = (1 : ℚ) := by
      field_simp [ne_of_gt heps_pos]
    rw [hone] at hmul
    simpa [mul_comm] using hmul
  have hden_pos : (0 : ℚ) < (n + 1 : Nat) := by positivity
  field_simp [ne_of_gt hden_pos]
  linarith

theorem PRCRealCofinalToleranceScheduleTarget_proved :
    PRCRealCofinalToleranceScheduleTarget := by
  refine ⟨PRCUnitFraction, PRCUnitFraction_positive, ?_⟩
  intro eps heps
  exact PRCUnitFraction_eventually_lt heps

theorem PRCRat.lt_trans {a b c : PRCRat}
    (hab : PRCRat.lt a b) (hbc : PRCRat.lt b c) :
    PRCRat.lt a c := by
  rw [PRCRat.lt_iff_toRat_lt] at hab hbc ⊢
  exact _root_.lt_trans hab hbc

/-- Three-leg form of the J-cost distance modulus. The diagonal proof naturally
travels from a selected diagonal point to an intermediate raw point, then across
representatives, then back down another selected diagonal point. -/
def PRCJCostDistanceThreeLegModulusTarget : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ delta : PRCRat, PRCRat.positive delta ∧
      ∀ a b c d : PRCRat,
        PRCRat.lt (PRCJCostDistance a b) delta →
          PRCRat.lt (PRCJCostDistance b c) delta →
            PRCRat.lt (PRCJCostDistance c d) delta →
              PRCRat.lt (PRCJCostDistance a d) eps

theorem PRCJCostDistanceThreeLegModulusTarget_proved :
    PRCJCostDistanceThreeLegModulusTarget := by
  intro eps heps
  rcases PRCJCostDistanceTriangleModulusTarget_proved eps heps with
    ⟨eta, heta_pos, heta_tri⟩
  rcases PRCJCostDistanceTriangleModulusTarget_proved eta heta_pos with
    ⟨theta, htheta_pos, htheta_tri⟩
  rcases PRCUnitFraction_eventually_lt heta_pos with ⟨Neta, hNeta⟩
  rcases PRCUnitFraction_eventually_lt htheta_pos with ⟨Ntheta, hNtheta⟩
  let delta := PRCUnitFraction (max Neta Ntheta)
  have hdelta_pos : PRCRat.positive delta := PRCUnitFraction_positive _
  have hdelta_lt_eta : PRCRat.lt delta eta := by
    exact hNeta (max Neta Ntheta) (Nat.le_max_left Neta Ntheta)
  have hdelta_lt_theta : PRCRat.lt delta theta := by
    exact hNtheta (max Neta Ntheta) (Nat.le_max_right Neta Ntheta)
  refine ⟨delta, hdelta_pos, ?_⟩
  intro a b c d hab hbc hcd
  have hab_eta : PRCRat.lt (PRCJCostDistance a b) eta :=
    PRCRat.lt_trans hab hdelta_lt_eta
  have hbc_theta : PRCRat.lt (PRCJCostDistance b c) theta :=
    PRCRat.lt_trans hbc hdelta_lt_theta
  have hcd_theta : PRCRat.lt (PRCJCostDistance c d) theta :=
    PRCRat.lt_trans hcd hdelta_lt_theta
  have hbd_eta : PRCRat.lt (PRCJCostDistance b d) eta :=
```

```
    htheta_tri b c d hbc_theta hcd_theta
  exact heta_tri a b d hab_eta hbd_eta

/-- A candidate index is deep enough for one raw Cauchy ledger at one
tolerance. -/
def PRCRowTailBound (u : PRCCauchySeq) (eps : PRCRat) (N : Nat) : Prop :=
  ∀ m n : Nat, N ≤ m → N ≤ n →
    PRCRat.lt (PRCJCostDistance (u.term m) (u.term n)) eps

theorem PRCRowTailBound_mono {u : PRCCauchySeq} {eps : PRCRat}
    {N M : Nat} (hNM : N ≤ M) (hN : PRCRowTailBound u eps N) :
    PRCRowTailBound u eps M := by
  intro m n hm hn
  exact hN m n (le_trans hNM hm) (le_trans hNM hn)

theorem PRCRealFiniteRowTailBound_exists
    (U : Nat → PRCCauchySeq) (eps : PRCRat)
    (heps : PRCRat.positive eps) :
    ∀ r : Nat,
      ∃ N : Nat, ∀ i : Nat, i ≤ r → PRCRowTailBound (U i) eps N
  | 0 => by
      rcases (U 0).cauchy eps heps with (N, hN)
      refine (N, ?_)
      intro i hi
      have hi0 : i = 0 := by omega
      subst hi0
      exact hN
  | Nat.succ r => by
      rcases PRCRealFiniteRowTailBound_exists U eps heps r with
        (Nprev, hprev)
      rcases (U (Nat.succ r)).cauchy eps heps with (Nlast, hlast)
      refine (max Nprev Nlast, ?_)
      intro i hi
      by_cases hir : i ≤ r
      · exact PRCRowTailBound_mono
          (Nat.le_max_left Nprev Nlast) (hprev i hir)
      · have hi_last : i = Nat.succ r := by omega
        subst hi_last
        exact PRCRowTailBound_mono
          (Nat.le_max_right Nprev Nlast) hlast

/-- Exact finite-row scheduler needed by the diagonal tail-selection proof:
for every diagonal row `r`, choose one raw index deep enough for all rows
`0,...,r` at the tolerance `1/(r+1)`. -/
def PRCRealFiniteRowTailSelectionTarget : Prop :=
  ∀ U : Nat → PRCCauchySeq,
    ∃ pick : Nat → Nat,
      ∀ r i : Nat, i ≤ r →
        PRCRowTailBound (U i) (PRCUnitFraction r) (pick r)

theorem PRCRealFiniteRowTailSelectionTarget_proved :
    PRCRealFiniteRowTailSelectionTarget := by
  intro U
  choose pick hpick using
    fun r => PRCRealFiniteRowTailBound_exists
      U (PRCUnitFraction r) (PRCUnitFraction_positive r) r
  exact (pick, hpick)

/-- A raw index is deep enough for every eligible representative row
`i ≤ r` to be close to the target row `r`, once the outer representative
threshold has been crossed. -/
def PRCRepresentativeFiniteTailBound
    (U : Nat → PRCCauchySeq) (eps : PRCRat)
    (outer r N : Nat) : Prop :=
  ∀ i k : Nat, outer ≤ i → i ≤ r → N ≤ k →
    PRCRat.lt (PRCJCostDistance ((U i).term k) ((U r).term k)) eps

theorem PRCRepresentativeFiniteTailBound_mono
    {U : Nat → PRCCauchySeq} {eps : PRCRat}
    {outer r N M : Nat} (hNM : N ≤ M)
    (hN : PRCRepresentativeFiniteTailBound U eps outer r N) :
    PRCRepresentativeFiniteTailBound U eps outer r M := by
  intro i k hoi hir hMk
  exact hN i k hoi hir (le_trans hNM hMk)

theorem PRCRepresentativeFiniteTailBound_exists
    (U : Nat → PRCCauchySeq) (eps : PRCRat)
    (outer r : Nat)
    (houter : ∀ m n : Nat, outer ≤ m → outer ≤ n →
      PRCRawEventuallyClose (U m).raw (U n).raw eps) :
    ∃ N : Nat, PRCRepresentativeFiniteTailBound U eps outer r N := by
  suffices hfinite :
      ∀ limit : Nat,
        limit ≤ r →
        ∃ N : Nat,
          ∀ i k : Nat, outer ≤ i → i ≤ limit → N ≤ k →
            PRCRat.lt
              (PRCJCostDistance ((U i).term k) ((U r).term k)) eps by
    rcases hfinite r (Nat.le_refl r) with (N, hN)
    exact (N, hN)
  intro limit
  induction limit with
  | zero =>
      intro _hlim
      by_cases houter_zero : outer ≤ 0
      · have houter_r : outer ≤ r := by omega
        rcases houter 0 r houter_zero houter_r with (N, hN)
        refine (N, ?_)
        intro i k hoi hir hNk
        have hi0 : i = 0 := by omega
        subst hi0
```

```
          simpa [PRCCauchySeq.raw] using hN k hNk
        · refine (0, ?_)
          intro i _k hoi hir _h0k
          have : outer ≤ 0 := by omega
          exact False.elim (houter_zero this)
    | succ limit ih =>
        intro hlim
        rcases ih (Nat.le_of_succ_le hlim) with (Nprev, hprev)
        by_cases hlast : outer ≤ Nat.succ limit
        · rcases houter (Nat.succ limit) r hlast
            (le_trans hlast hlim) with
            (Nlast, hNlast)
          refine (max Nprev Nlast, ?_)
          intro i k hoi hir hmaxk
          by_cases hir_prev : i ≤ limit
          · exact hprev i k hoi hir_prev
              (le_trans (Nat.le_max_left Nprev Nlast) hmaxk)
          · have hi_last : i = Nat.succ limit := by omega
            subst hi_last
            simpa [PRCCauchySeq.raw] using
              hNlast k (le_trans (Nat.le_max_right Nprev Nlast) hmaxk)
        · refine (Nprev, ?_)
          intro i k hoi hir hNprevk
          have hir_prev : i ≤ limit := by omega
          exact hprev i k hoi hir_prev hNprevk

/-- Exact finite representative scheduler needed by the diagonal proof. For
each tolerance rung it chooses the outer Cauchy-representative threshold and a
raw depth that realizes all finite representative-tail comparisons up to that
rung. -/
def PRCRealFiniteRepresentativeTailSelectionTarget : Prop :=
  ∀ U : Nat → PRCCauchySeq,
    PRCRealRepresentativeCauchy U →
      ∃ outer pick : Nat → Nat,
        (∀ r m n : Nat, outer r ≤ m → outer r ≤ n →
          PRCRawEventuallyClose (U m).raw (U n).raw (PRCUnitFraction r)) ∧
        ∀ r : Nat,
          PRCRepresentativeFiniteTailBound
            U (PRCUnitFraction r) (outer r) r (pick r)

theorem PRCRealFiniteRepresentativeTailSelectionTarget_proved :
    PRCRealFiniteRepresentativeTailSelectionTarget := by
  intro U hU
  choose outer houter using
    fun r => hU (PRCUnitFraction r) (PRCUnitFraction_positive r)
  choose pick hpick using
    fun r => PRCRepresentativeFiniteTailBound_exists
      U (PRCUnitFraction r) (outer r) r (houter r)
  exact ⟨outer, pick, houter, hpick⟩

/-- Single finite diagonal scheduler: one raw choice function satisfies both
finite row-tail and finite representative-tail constraints at each tolerance
rung. This is still finite-rung scheduling, not yet the completed global
tail-selection theorem. -/
def PRCRealFiniteDiagonalScheduleTarget : Prop :=
  ∀ U : Nat → PRCCauchySeq,
    PRCRealRepresentativeCauchy U →
      ∃ outer pick : Nat → Nat,
        (∀ r m n : Nat, outer r ≤ m → outer r ≤ n →
          PRCRawEventuallyClose (U m).raw (U n).raw (PRCUnitFraction r)) ∧
        (∀ r i : Nat, i ≤ r →
          PRCRowTailBound (U i) (PRCUnitFraction r) (pick r)) ∧
        ∀ r : Nat,
          PRCRepresentativeFiniteTailBound
            U (PRCUnitFraction r) (outer r) r (pick r)

theorem PRCRealFiniteDiagonalScheduleTarget_proved :
    PRCRealFiniteDiagonalScheduleTarget := by
  intro U hU
  rcases PRCRealFiniteRowTailSelectionTarget_proved U with
    ⟨rowPick, hrowPick⟩
  rcases PRCRealFiniteRepresentativeTailSelectionTarget_proved U hU with
    ⟨outer, repPick, houter, hrepPick⟩
  refine ⟨outer, fun r => max (rowPick r) (repPick r), houter, ?_, ?_⟩
  · intro r i hir
    exact PRCRowTailBound_mono
      (Nat.le_max_left (rowPick r) (repPick r))
      (hrowPick r i hir)
  · intro r
    exact PRCRepresentativeFiniteTailBound_mono
      (Nat.le_max_right (rowPick r) (repPick r))
      (hrepPick r)

theorem PRCRealTailSelectionTarget_proved :
    PRCRealTailSelectionTarget := by
  intro U hU
  rcases PRCRealFiniteDiagonalScheduleTarget_proved U hU with
    ⟨_outer, pick, _houterTau, hrow, _hrep⟩
  refine ⟨pick, ?_, ?_⟩
  · intro eps heps
    rcases PRCJCostDistanceThreeLegModulusTarget_proved eps heps with
      ⟨delta, hdelta_pos, hthree⟩
    rcases hU delta hdelta_pos with ⟨Nrep, hNrep⟩
    rcases PRCUnitFraction_eventually_lt hdelta_pos with ⟨Ntau, hNtau⟩
    refine ⟨max Nrep Ntau, ?_⟩
    intro m n hm hn
    have hm_rep : Nrep ≤ m := le_trans (Nat.le_max_left Nrep Ntau) hm
    have hn_rep : Nrep ≤ n := le_trans (Nat.le_max_left Nrep Ntau) hn
    have hm_tauN : Ntau ≤ m := le_trans (Nat.le_max_right Nrep Ntau) hm
    have hn_tauN : Ntau ≤ n := le_trans (Nat.le_max_right Nrep Ntau) hn
    have htaum_delta : PRCRat.lt (PRCUnitFraction m) delta :=
```

```
      hNtau m hm_tauN
    have htaun_delta : PRCRat.lt (PRCUnitFraction n) delta :=
      hNtau n hn_tauN
    rcases hNrep m n hm_rep hn_rep with (Npair, hNpair)
    let K : Nat := max (pick m) (max (pick n) Npair)
    have hpickmK : pick m ≤ K := Nat.le_max_left (pick m) (max (pick n) Npair)
    have hpicknK : pick n ≤ K :=
      le_trans (Nat.le_max_left (pick n) Npair)
        (Nat.le_max_right (pick m) (max (pick n) Npair))
    have hpairK : Npair ≤ K :=
      le_trans (Nat.le_max_right (pick n) Npair)
        (Nat.le_max_right (pick m) (max (pick n) Npair))
    have hleg1_tau :
        PRCRat.lt
          (PRCJCostDistance ((U m).term (pick m)) ((U m).term K))
          (PRCUnitFraction m) :=
      (hrow m m (Nat.le_refl m)) (pick m) K (Nat.le_refl (pick m)) hpickmK
    have hleg1 :
        PRCRat.lt
          (PRCJCostDistance ((U m).term (pick m)) ((U m).term K))
          delta :=
      PRCRat.lt_trans hleg1_tau htaum_delta
    have hleg2 :
        PRCRat.lt
          (PRCJCostDistance ((U m).term K) ((U n).term K))
          delta := by
      simpa [PRCCauchySeq.raw] using hNpair K hpairK
    have hleg3_tau :
        PRCRat.lt
          (PRCJCostDistance ((U n).term K) ((U n).term (pick n)))
          (PRCUnitFraction n) :=
      (hrow n n (Nat.le_refl n)) K (pick n) hpicknK (Nat.le_refl (pick n))
    have hleg3 :
        PRCRat.lt
          (PRCJCostDistance ((U n).term K) ((U n).term (pick n)))
          delta :=
      PRCRat.lt_trans hleg3_tau htaun_delta
    exact hthree
      ((U m).term (pick m)) ((U m).term K)
      ((U n).term K) ((U n).term (pick n))
      hleg1 hleg2 hleg3
  · intro eps heps
    rcases PRCJCostDistanceThreeLegModulusTarget_proved eps heps with
      (delta, hdelta_pos, hthree)
    rcases hU delta hdelta_pos with (Nrep, hNrep)
    rcases PRCUnitFraction_eventually_lt hdelta_pos with (Ntau, hNtau)
    refine (max Nrep Ntau, ?_)
    intro n hn
    have hn_rep : Nrep ≤ n := le_trans (Nat.le_max_left Nrep Ntau) hn
    rcases (U n).cauchy delta hdelta_pos with (NrowN, hNrowN)
    refine (max (max Nrep Ntau) NrowN, ?_)
    intro l hl
    have hl_rep : Nrep ≤ l :=
      le_trans (Nat.le_max_left Nrep Ntau)
        (le_trans (Nat.le_max_left (max Nrep Ntau) NrowN) hl)
    have hl_tauN : Ntau ≤ l :=
      le_trans (Nat.le_max_right Nrep Ntau)
        (le_trans (Nat.le_max_left (max Nrep Ntau) NrowN) hl)
    have hl_rowN : NrowN ≤ l :=
      le_trans (Nat.le_max_right (max Nrep Ntau) NrowN) hl
    have htaul_delta : PRCRat.lt (PRCUnitFraction l) delta :=
      hNtau l hl_tauN
    rcases hNrep n l hn_rep hl_rep with (Npair, hNpair)
    let K : Nat := max l (max (pick l) Npair)
    have hlK : l ≤ K := Nat.le_max_left l (max (pick l) Npair)
    have hpicklK : pick l ≤ K :=
      le_trans (Nat.le_max_left (pick l) Npair)
        (Nat.le_max_right l (max (pick l) Npair))
    have hpairK : Npair ≤ K :=
      le_trans (Nat.le_max_right (pick l) Npair)
        (Nat.le_max_right l (max (pick l) Npair))
    have hrowNK : NrowN ≤ K := le_trans hl_rowN hlK
    have hleg1 :
        PRCRat.lt
          (PRCJCostDistance ((U n).term l) ((U n).term K))
          delta :=
      hNrowN l K hl_rowN hrowNK
    have hleg2 :
        PRCRat.lt
          (PRCJCostDistance ((U n).term K) ((U l).term K))
          delta := by
      simpa [PRCCauchySeq.raw] using hNpair K hpairK
    have hleg3_tau :
        PRCRat.lt
          (PRCJCostDistance ((U l).term K) ((U l).term (pick l)))
          (PRCUnitFraction l) :=
      (hrow l l (Nat.le_refl l)) K (pick l) hpicklK (Nat.le_refl (pick l))
    have hleg3 :
        PRCRat.lt
          (PRCJCostDistance ((U l).term K) ((U l).term (pick l)))
          delta :=
      PRCRat.lt_trans hleg3_tau htaul_delta
    exact hthree
      ((U n).term l) ((U n).term K)
      ((U l).term K) ((U l).term (pick l))
      hleg1 hleg2 hleg3

/-- An explicit tail-selection diagonal is enough to build the raw diagonal
ledger target. -/
theorem PRCRealRawDiagonalLedgerTarget_of_tail_selection
    (htail : PRCRealTailSelectionTarget) :
```

```
    PRCRealRawDiagonalLedgerTarget := by
  intro U hU
  rcases htail U hU with (pick, hs_cauchy, hs_limit)
  exact (fun n => (U n).term (pick n), hs_cauchy, hs_limit)

theorem PRCRealRawDiagonalLedgerTarget_proved :
    PRCRealRawDiagonalLedgerTarget :=
  PRCRealRawDiagonalLedgerTarget_of_tail_selection
    PRCRealTailSelectionTarget_proved

/-- A raw diagonal ledger packages immediately as the representative limit
needed by the quotient-level diagonal selection target. -/
theorem PRCRealDiagonalSelectionTarget_of_raw_diagonal_ledger
    (hraw : PRCRealRawDiagonalLedgerTarget) :
    PRCRealDiagonalSelectionTarget := by
  intro U hU
  rcases hraw U hU with (s, hs_cauchy, hs_limit)
  refine ({ term := s, cauchy := hs_cauchy }, ?_)
  simpa [PRCRealRepresentativeLimit, PRCCauchySeq.raw] using hs_limit

theorem PRCRealDiagonalSelectionTarget_proved :
    PRCRealDiagonalSelectionTarget :=
  PRCRealDiagonalSelectionTarget_of_raw_diagonal_ledger
    PRCRealRawDiagonalLedgerTarget_proved

/-- The diagonal selection target is exactly the sharpened completeness target. -/
theorem PRCRealCompletenessTarget_of_diagonal_selection
    (hdiag : PRCRealDiagonalSelectionTarget) :
    PRCRealCompletenessTarget := by
  exact hdiag

theorem PRCRealCompletenessTarget_proved :
    PRCRealCompletenessTarget :=
  PRCRealCompletenessTarget_of_diagonal_selection
    PRCRealDiagonalSelectionTarget_proved

/-- The full internal completeness theorem is now the concrete representative
Cauchy-of-Cauchy diagonal target. -/
theorem PRCRealCompletenessTarget_sharpened :
    PRCRealCompletenessTarget = PRCRealCompletenessTarget := rfl

/-- Step 10e certificate. It records the closed raw-ledger realization fact,
the tail-selection diagonal, and the representative-completeness theorem for
`PRCRealNullClosed`. -/
structure PRCRealCompletenessSharpenedCertificate : Prop where
  raw_cauchy_realization : PRCRawCauchyRealizationTarget
  raw_cauchy_quotient_point : PRCRawCauchyQuotientPointTarget
  diagonal_selection_from_raw_diagonal :
    PRCRealRawDiagonalLedgerTarget → PRCRealDiagonalSelectionTarget
  raw_diagonal_from_tail_selection :
    PRCRealTailSelectionTarget → PRCRealRawDiagonalLedgerTarget
  cofinal_tolerance_schedule : PRCRealCofinalToleranceScheduleTarget
  three_leg_distance_modulus : PRCJCostDistanceThreeLegModulusTarget
  finite_row_tail_selection : PRCRealFiniteRowTailSelectionTarget
  finite_representative_tail_selection :
    PRCRealFiniteRepresentativeTailSelectionTarget
  finite_diagonal_schedule : PRCRealFiniteDiagonalScheduleTarget
  tail_selection : PRCRealTailSelectionTarget
  raw_diagonal : PRCRealRawDiagonalLedgerTarget
  diagonal_selection : PRCRealDiagonalSelectionTarget
  completeness_from_diagonal_selection :
    PRCRealDiagonalSelectionTarget → PRCRealCompletenessTarget
  completeness : PRCRealCompletenessTarget
  completeness_target : PRCRealCompletenessTarget = PRCRealCompletenessTarget

theorem prc_real_completeness_sharpened_certificate :
    PRCRealCompletenessSharpenedCertificate where
  raw_cauchy_realization := PRCRawCauchyRealizationTarget_proved
  raw_cauchy_quotient_point := PRCRawCauchyQuotientPointTarget_proved
  diagonal_selection_from_raw_diagonal :=
    PRCRealDiagonalSelectionTarget_of_raw_diagonal_ledger
  raw_diagonal_from_tail_selection :=
    PRCRealRawDiagonalLedgerTarget_of_tail_selection
  cofinal_tolerance_schedule :=
    PRCRealCofinalToleranceScheduleTarget_proved
  three_leg_distance_modulus :=
    PRCJCostDistanceThreeLegModulusTarget_proved
  finite_row_tail_selection :=
    PRCRealFiniteRowTailSelectionTarget_proved
  finite_representative_tail_selection :=
    PRCRealFiniteRepresentativeTailSelectionTarget_proved
  finite_diagonal_schedule :=
    PRCRealFiniteDiagonalScheduleTarget_proved
  tail_selection :=
    PRCRealTailSelectionTarget_proved
  raw_diagonal :=
    PRCRealRawDiagonalLedgerTarget_proved
  diagonal_selection :=
    PRCRealDiagonalSelectionTarget_proved
  completeness_from_diagonal_selection :=
    PRCRealCompletenessTarget_of_diagonal_selection
  completeness :=
    PRCRealCompletenessTarget_proved
  completeness_target := PRCRealCompletenessTarget_sharpened

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

23. RealCompleteOrderedField.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteOrderedField.lean · **Lines:** 370

```
/-
  PrimitiveRecognitionCalculus/RealCompleteOrderedField.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 10: start the complete ordered field surface over the
    closed null-distance quotient `PRCRealNullClosed`.

  This pass does not alias the carrier to Lean `ℝ`. It exposes the exact
  quotient-algebra blockers for addition, multiplication, negation, and order.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceIncrementTriangle

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- Raw completed-orbit rational ledger, before a Cauchy proof is attached. -/
abbrev PRCRawRatLedger := Nat → PRCRat

/-- Cauchy predicate on raw ledgers, using the same J-cost distance as
`PRCCauchySeq`. -/
def PRCRawCauchy (s : PRCRawRatLedger) : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
      PRCRat.lt (PRCJCostDistance (s m) (s n)) eps

/-- Null equivalence on raw ledgers. -/
def PRCRawNullEquivalent (s t : PRCRawRatLedger) : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ N : Nat, ∀ n : Nat, N ≤ n →
      PRCRat.lt (PRCJCostDistance (s n) (t n)) eps

namespace PRCCauchySeq

/-- Forget a Cauchy ledger to its raw rational ledger. -/
def raw (u : PRCCauchySeq) : PRCRawRatLedger :=
  u.term

theorem raw_cauchy (u : PRCCauchySeq) : PRCRawCauchy u.raw :=
  u.cauchy

@[simp] theorem raw_apply (u : PRCCauchySeq) (n : Nat) :
    u.raw n = u.term n := rfl

end PRCCauchySeq

/-- Pointwise addition of raw ledgers. -/
def PRCRawAdd (u v : PRCRawRatLedger) : PRCRawRatLedger :=
  fun n => u n + v n

/-- Pointwise negation of raw ledgers. -/
def PRCRawNeg (u : PRCRawRatLedger) : PRCRawRatLedger :=
  fun n => -u n

/-- Pointwise multiplication of raw ledgers. -/
def PRCRawMul (u v : PRCRawRatLedger) : PRCRawRatLedger :=
  fun n => u n * v n

/-- Pointwise non-strict order candidate for raw ledgers. -/
def PRCRawEventuallyLe (u v : PRCRawRatLedger) : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ N : Nat, ∀ n : Nat, N ≤ n →
      PRCRat.lt (u n) (v n + eps)

/-- J-cost distance is invariant under translating both endpoints on the
right. -/
theorem PRCJCostDistance_add_right (a b c : PRCRat) :
    PRCJCostDistance (a + c) (b + c) = PRCJCostDistance a b := by
  apply PRCRat.toRat_injective
  rw [PRCJCostDistance_toRat, PRCJCostDistance_toRat]
  simp [PRCJCostDistanceRatDisplay]

/-- J-cost distance is invariant under translating both endpoints on the left. -/
theorem PRCJCostDistance_add_left (a b c : PRCRat) :
    PRCJCostDistance (c + a) (c + b) = PRCJCostDistance a b := by
  apply PRCRat.toRat_injective
  rw [PRCJCostDistance_toRat, PRCJCostDistance_toRat]
  simp [PRCJCostDistanceRatDisplay]

/-- J-cost distance is invariant under negating both endpoints. -/
theorem PRCJCostDistance_neg_neg (a b : PRCRat) :
    PRCJCostDistance (-a) (-b) = PRCJCostDistance a b := by
  apply PRCRat.toRat_injective
  rw [PRCJCostDistance_toRat, PRCJCostDistance_toRat]
  simp [PRCJCostDistanceRatDisplay]
```

```
  ring_nf

/-- Exact blocker for addition: pointwise sums of Cauchy ledgers are Cauchy. -/
def PRCRealAddClosureTarget : Prop :=
  ∀ u v : PRCCauchySeq, PRCRawCauchy (PRCRawAdd u.raw v.raw)

/-- Exact blocker for additive quotient well-definedness under null distance. -/
def PRCRealAddCongruenceTarget : Prop :=
  ∀ u u' v v' : PRCCauchySeq,
    PRCNullEquivalent u u' →
      PRCNullEquivalent v v' →
        PRCRawNullEquivalent
          (PRCRawAdd u.raw v.raw)
          (PRCRawAdd u'.raw v'.raw)

/-- Exact blocker for negation: pointwise negations of Cauchy ledgers are
Cauchy. -/
def PRCRealNegClosureTarget : Prop :=
  ∀ u : PRCCauchySeq, PRCRawCauchy (PRCRawNeg u.raw)

/-- Exact blocker for negation quotient well-definedness. -/
def PRCRealNegCongruenceTarget : Prop :=
  ∀ u v : PRCCauchySeq,
    PRCNullEquivalent u v →
      PRCRawNullEquivalent (PRCRawNeg u.raw) (PRCRawNeg v.raw)

/-- Exact blocker for multiplication: pointwise products of Cauchy ledgers are
Cauchy. This is expected to require a boundedness lemma for Cauchy ledgers. -/
def PRCRealMulClosureTarget : Prop :=
  ∀ u v : PRCCauchySeq, PRCRawCauchy (PRCRawMul u.raw v.raw)

/-- Exact blocker for multiplicative quotient well-definedness under null
distance. This is also expected to require eventual boundedness. -/
def PRCRealMulCongruenceTarget : Prop :=
  ∀ u u' v v' : PRCCauchySeq,
    PRCNullEquivalent u u' →
      PRCNullEquivalent v v' →
        PRCRawNullEquivalent
          (PRCRawMul u.raw v.raw)
          (PRCRawMul u'.raw v'.raw)

/-- Exact blocker for the order relation descending to the null-distance
quotient. -/
def PRCRealOrderCongruenceTarget : Prop :=
  ∀ u u' v v' : PRCCauchySeq,
    PRCNullEquivalent u u' →
      PRCNullEquivalent v v' →
        (PRCRawEventuallyLe u.raw v.raw ↔
          PRCRawEventuallyLe u'.raw v'.raw)

/-- Quantitative closeness for two raw ledgers at a fixed PRC tolerance. -/
def PRCRawEventuallyClose (s t : PRCRawRatLedger) (eps : PRCRat) : Prop :=
  ∃ N : Nat, ∀ n : Nat, N ≤ n →
    PRCRat.lt (PRCJCostDistance (s n) (t n)) eps

/-- A sequence of Cauchy ledgers is Cauchy as a sequence of null-quotient
representatives when its representative tails are eventually close at every
positive PRC tolerance. -/
def PRCRealRepresentativeCauchy (U : Nat → PRCCauchySeq) : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
      PRCRawEventuallyClose (U m).raw (U n).raw eps

/-- A Cauchy ledger `L` is a representative limit of a sequence of null-quotient
representatives. -/
def PRCRealRepresentativeLimit (U : Nat → PRCCauchySeq)
    (L : PRCCauchySeq) : Prop :=
  ∀ eps : PRCRat, PRCRat.positive eps →
    ∃ N : Nat, ∀ n : Nat, N ≤ n →
      PRCRawEventuallyClose (U n).raw L.raw eps

/-- Exact blocker for completeness of the internal null quotient. This is the
diagonal theorem: every Cauchy sequence of Cauchy-ledger representatives has a
Cauchy-ledger representative limit. -/
def PRCRealCompletenessTarget : Prop :=
  ∀ U : Nat → PRCCauchySeq,
    PRCRealRepresentativeCauchy U →
      ∃ L : PRCCauchySeq, PRCRealRepresentativeLimit U L

/-- Pointwise sums of Cauchy ledgers are Cauchy. -/
theorem PRCRealAddClosureTarget_proved : PRCRealAddClosureTarget := by
  intro u v eps heps
  rcases PRCJCostDistanceTriangleModulusTarget_proved eps heps with
    (delta, hdelta_pos, hdelta)
  rcases u.cauchy delta hdelta_pos with (Nu, hNu)
  rcases v.cauchy delta hdelta_pos with (Nv, hNv)
  refine (max Nu Nv, ?_)
  intro m n hm hn
  have hmu : Nu ≤ m := le_trans (Nat.le_max_left Nu Nv) hm
  have hnu : Nu ≤ n := le_trans (Nat.le_max_left Nu Nv) hn
  have hmv : Nv ≤ m := le_trans (Nat.le_max_right Nu Nv) hm
  have hnv : Nv ≤ n := le_trans (Nat.le_max_right Nu Nv) hn
  exact hdelta
    ((u.term m) + (v.term m))
    ((u.term n) + (v.term m))
    ((u.term n) + (v.term n))
    (by
      rw [PRCJCostDistance_add_right]
      exact hNu m n hmu hnu)
    (by
      rw [PRCJCostDistance_add_left]
```

```
      exact hNv m n hmv hnv)

/-- Pointwise negations of Cauchy ledgers are Cauchy. -/
theorem PRCRealNegClosureTarget_proved : PRCRealNegClosureTarget := by
  intro u eps heps
  rcases u.cauchy eps heps with ⟨N, hN⟩
  refine ⟨N, ?_⟩
  intro m n hm hn
  change PRCRat.lt (PRCJCostDistance (-(u.term m)) (-(u.term n))) eps
  rw [PRCJCostDistance_neg_neg]
  exact hN m n hm hn

/-- Addition respects null equivalence. -/
theorem PRCRealAddCongruenceTarget_proved :
    PRCRealAddCongruenceTarget := by
  intro u u' v v' huu hvv eps heps
  rcases PRCJCostDistanceTriangleModulusTarget_proved eps heps with
    ⟨delta, hdelta_pos, hdelta⟩
  rcases huu delta hdelta_pos with ⟨Nu, hNu⟩
  rcases hvv delta hdelta_pos with ⟨Nv, hNv⟩
  refine ⟨max Nu Nv, ?_⟩
  intro n hn
  have hnu : Nu ≤ n := le_trans (Nat.le_max_left Nu Nv) hn
  have hnv : Nv ≤ n := le_trans (Nat.le_max_right Nu Nv) hn
  exact hdelta
    ((u.term n) + (v.term n))
    ((u'.term n) + (v.term n))
    ((u'.term n) + (v'.term n))
    (by
      rw [PRCJCostDistance_add_right]
      exact hNu n hnu)
    (by
      rw [PRCJCostDistance_add_left]
      exact hNv n hnv)

/-- Negation respects null equivalence. -/
theorem PRCRealNegCongruenceTarget_proved :
    PRCRealNegCongruenceTarget := by
  intro u v huv eps heps
  rcases huv eps heps with ⟨N, hN⟩
  refine ⟨N, ?_⟩
  intro n hn
  change PRCRat.lt (PRCJCostDistance (-(u.term n)) (-(v.term n))) eps
  rw [PRCJCostDistance_neg_neg]
  exact hN n hn

/-- Conditional construction of a pointwise-sum Cauchy ledger from the addition
closure target. -/
def PRCCauchySeq.addOf
    (hadd : PRCRealAddClosureTarget) (u v : PRCCauchySeq) : PRCCauchySeq where
  term := PRCRawAdd u.raw v.raw
  cauchy := hadd u v

/-- Conditional construction of a pointwise-negation Cauchy ledger from the
negation closure target. -/
def PRCCauchySeq.negOf
    (hneg : PRCRealNegClosureTarget) (u : PRCCauchySeq) : PRCCauchySeq where
  term := PRCRawNeg u.raw
  cauchy := hneg u

/-- Conditional construction of a pointwise-product Cauchy ledger from the
multiplication closure target. -/
def PRCCauchySeq.mulOf
    (hmul : PRCRealMulClosureTarget) (u v : PRCCauchySeq) : PRCCauchySeq where
  term := PRCRawMul u.raw v.raw
  cauchy := hmul u v

/-- Conditional addition on the closed null quotient. -/
noncomputable def PRCRealNullClosed.addOf
    (hadd : PRCRealAddClosureTarget)
    (hcong : PRCRealAddCongruenceTarget) :
    PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed :=
  Quot.lift₂
    (fun u v =>
      Quot.mk (PRCNullDistanceSetoidOfTransitive PRCNullDistanceTransitiveTarget_proved)
        (PRCCauchySeq.addOf hadd u v))
    (by
      intro u v₁ v₂ hv
      apply Quot.sound
      exact hcong u u v₁ v₂ (PRCNullEquivalent.refl u) hv)
    (by
      intro u₁ u₂ v hu
      apply Quot.sound
      exact hcong u₁ u₂ v v hu (PRCNullEquivalent.refl v))

/-- Conditional negation on the closed null quotient. -/
noncomputable def PRCRealNullClosed.negOf
    (hneg : PRCRealNegClosureTarget)
    (hcong : PRCRealNegCongruenceTarget) :
    PRCRealNullClosed → PRCRealNullClosed :=
  Quot.lift
    (fun u =>
      Quot.mk (PRCNullDistanceSetoidOfTransitive PRCNullDistanceTransitiveTarget_proved)
        (PRCCauchySeq.negOf hneg u))
    (by
      intro u v huv
      apply Quot.sound
      exact hcong u v huv)

/-- Conditional multiplication on the closed null quotient. -/
noncomputable def PRCRealNullClosed.mulOf
```

```
    (hmul : PRCRealMulClosureTarget)
    (hcong : PRCRealMulCongruenceTarget) :
    PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed :=
  Quot.lift₂
    (fun u v =>
      Quot.mk (PRCNullDistanceSetoidOfTransitive PRCNullDistanceTransitiveTarget_proved)
        (PRCCauchySeq.mulOf hmul u v))
    (by
      intro u v₁ v₂ hv
      apply Quot.sound
      exact hcong u u v₁ v₂ (PRCNullEquivalent.refl u) hv)
    (by
      intro u₁ u₂ v hu
      apply Quot.sound
      exact hcong u₁ u₂ v v hu (PRCNullEquivalent.refl v))

/-- Bundle of exact blockers for the next real-completion phase. -/
structure PRCRealCompleteOrderedFieldTargets : Prop where
  add_closure : PRCRealAddClosureTarget
  add_congruence : PRCRealAddCongruenceTarget
  neg_closure : PRCRealNegClosureTarget
  neg_congruence : PRCRealNegCongruenceTarget
  mul_closure : PRCRealMulClosureTarget = PRCRealMulClosureTarget
  mul_congruence : PRCRealMulCongruenceTarget = PRCRealMulCongruenceTarget
  order_congruence : PRCRealOrderCongruenceTarget = PRCRealOrderCongruenceTarget
  completeness : PRCRealCompletenessTarget = PRCRealCompletenessTarget

/-- Conditional first complete-ordered-field surface. It records that the
carrier and rational embedding are closed, while algebra/order/completeness are
reduced to named exact targets. -/
structure PRCRealCompleteOrderedFieldConditionalCertificate : Prop where
  carrier : Nonempty PRCRealNullClosed
  rat_embedding : Nonempty (PRCRat → PRCRealNullClosed)
  targets : PRCRealCompleteOrderedFieldTargets
  add_operation_from_targets :
    PRCRealAddClosureTarget →
      PRCRealAddCongruenceTarget →
        Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
  neg_operation_from_targets :
    PRCRealNegClosureTarget →
      PRCRealNegCongruenceTarget →
        Nonempty (PRCRealNullClosed → PRCRealNullClosed)
  mul_operation_from_targets :
    PRCRealMulClosureTarget →
      PRCRealMulCongruenceTarget →
        Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
  strength_tag : StrengthTag.traceClosure = StrengthTag.traceClosure

/-- Build Order step 10, first pass: quotient algebra is reduced to exact
closure and congruence targets. -/
theorem prc_real_complete_ordered_field_conditional_certificate :
    PRCRealCompleteOrderedFieldConditionalCertificate where
  carrier := (PRCRealNullClosed.ofRat 0)
  rat_embedding := (PRCRealNullClosed.ofRat)
  targets := {
    add_closure := PRCRealAddClosureTarget_proved
    add_congruence := PRCRealAddCongruenceTarget_proved
    neg_closure := PRCRealNegClosureTarget_proved
    neg_congruence := PRCRealNegCongruenceTarget_proved
    mul_closure := rfl
    mul_congruence := rfl
    order_congruence := rfl
    completeness := rfl
  }
  add_operation_from_targets := by
    intro hadd hcong
    exact (PRCRealNullClosed.addOf hadd hcong)
  neg_operation_from_targets := by
    intro hneg hcong
    exact (PRCRealNullClosed.negOf hneg hcong)
  mul_operation_from_targets := by
    intro hmul hcong
    exact (PRCRealNullClosed.mulOf hmul hcong)
  strength_tag := rfl

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

24. RealCompleteOrderedFieldPromoted.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteOrderedFieldPromoted.lean · **Lines:** 88

```
0001  /-
0002    PrimitiveRecognitionCalculus/RealCompleteOrderedFieldPromoted.lean
0003
0004    Round-trip source:
0005      6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007    Spec anchor:
0008      Build Order step 10: promote the proved add, neg, mul, order, and
0009      representative-completeness targets into one complete ordered-field
0010      certificate surface over `PRCRealNullClosed`.
0011
0012    This is a certificate-promotion layer. It does not alias the internal carrier
0013    to Lean `ℝ`; it bundles the theorem surfaces already proved for the PRC null
0014    quotient.
0015  -/
0016
0017  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteness
0018
0019  namespace IndisputableMonolith
0020  namespace Foundation
0021  namespace PrimitiveRecognitionCalculus
0022
0023  /-- Promoted Step 10 certificate: the internal null quotient has the closed
0024  operations and theorem surfaces needed by the current complete ordered-field
0025  layer. Full Mathlib typeclass instances remain a later packaging pass. -/
0026  structure PRCRealCompleteOrderedFieldPromotedCertificate : Prop where
0027    carrier : Nonempty PRCRealNullClosed
0028    rat_embedding : Nonempty (PRCRat → PRCRealNullClosed)
0029    add_closure : PRCRealAddClosureTarget
0030    add_congruence : PRCRealAddCongruenceTarget
0031    add_operation :
0032      Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
0033    neg_closure : PRCRealNegClosureTarget
0034    neg_congruence : PRCRealNegCongruenceTarget
0035    neg_operation : Nonempty (PRCRealNullClosed → PRCRealNullClosed)
0036    mul_closure : PRCRealMulClosureTarget
0037    mul_congruence : PRCRealMulCongruenceTarget
0038    mul_operation :
0039      Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
0040    order_congruence : PRCRealOrderCongruenceTarget
0041    representative_completeness : PRCRealCompletenessTarget
0042    first_pass_certificate : PRCRealCompleteOrderedFieldConditionalCertificate
0043    product_continuity_certificate : PRCRealProductContinuityCertificate
0044    order_congruence_certificate : PRCRealOrderCongruenceCertificate
0045    completeness_certificate : PRCRealCompletenessSharpenedCertificate
0046    strength_tag : StrengthTag.traceClosure = StrengthTag.traceClosure
0047
0048  theorem prc_real_complete_ordered_field_promoted_certificate :
0049      PRCRealCompleteOrderedFieldPromotedCertificate where
0050    carrier := (PRCRealNullClosed.ofRat 0)
0051    rat_embedding := (PRCRealNullClosed.ofRat)
0052    add_closure := PRCRealAddClosureTarget_proved
0053    add_congruence := PRCRealAddCongruenceTarget_proved
0054    add_operation :=
0055      (PRCRealNullClosed.addOf
0056        PRCRealAddClosureTarget_proved
0057        PRCRealAddCongruenceTarget_proved)
0058    neg_closure := PRCRealNegClosureTarget_proved
0059    neg_congruence := PRCRealNegCongruenceTarget_proved
0060    neg_operation :=
0061      (PRCRealNullClosed.negOf
0062        PRCRealNegClosureTarget_proved
0063        PRCRealNegCongruenceTarget_proved)
0064    mul_closure := PRCRealMulClosureTarget_of_bounded_continuity
0065      PRCCauchySeqEventuallyBoundedTarget_proved
0066      PRCJCostDistanceMulBoundedContinuityTarget_proved
0067    mul_congruence := PRCRealMulCongruenceTarget_of_bounded_continuity
0068      PRCCauchySeqEventuallyBoundedTarget_proved
0069      PRCJCostDistanceMulBoundedContinuityTarget_proved
0070    mul_operation :=
0071      (PRCRealNullClosed.mulOf
0072        (PRCRealMulClosureTarget_of_bounded_continuity
0073          PRCCauchySeqEventuallyBoundedTarget_proved
0074          PRCJCostDistanceMulBoundedContinuityTarget_proved)
0075        (PRCRealMulCongruenceTarget_of_bounded_continuity
0076          PRCCauchySeqEventuallyBoundedTarget_proved
0077          PRCJCostDistanceMulBoundedContinuityTarget_proved))
0078    order_congruence := PRCRealOrderCongruenceTarget_proved
0079    representative_completeness := PRCRealCompletenessTarget_proved
0080    first_pass_certificate := prc_real_complete_ordered_field_conditional_certificate
0081    product_continuity_certificate := prc_real_product_continuity_certificate
0082    order_congruence_certificate := prc_real_order_congruence_certificate
0083    completeness_certificate := prc_real_completeness_sharpened_certificate
0084    strength_tag := rfl
0085
0086  end PrimitiveRecognitionCalculus
0087  end Foundation
0088  end IndisputableMonolith
```

25. RealCompletion.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompletion.lean · **Lines:** 97

```
/-
  PrimitiveRecognitionCalculus/RealCompletion.lean

  Round-trip source:
    PRC_Kernel_Spec_20260526.html

  Spec anchors:
    K1 (`classical extension`), K4.14, A5

  This is the honest real-completion boundary. It embeds the quotient-native
  PRC rational surface into Lean's `ℝ` and records completeness as a
  classical-extension fact. It is not a claim that the PRC-internal Cauchy
  quotient has already been built.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerRational
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Strength

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- K4.14. The first real boundary: Lean's complete real line, reached only
under the classical-extension tag. -/
abbrev PRCRealBoundary : Type := ℝ

namespace PRCRealBoundary

/-- K4.14/A5. Embed a PRC rational into the real boundary through its
conservative rational display. -/
def ofRat (q : PRCRat) : PRCRealBoundary :=
  (q.toRat : ℝ)

@[simp] theorem ofRat_add (a b : PRCRat) :
    ofRat (a + b) = ofRat a + ofRat b := by
  unfold ofRat
  rw [PRCRat.toRat_add']
  norm_num

@[simp] theorem ofRat_mul (a b : PRCRat) :
    ofRat (a * b) = ofRat a * ofRat b := by
  unfold ofRat
  rw [PRCRat.toRat_mul']
  norm_num

@[simp] theorem ofRat_neg (a : PRCRat) :
    ofRat (-a) = -ofRat a := by
  unfold ofRat
  rw [PRCRat.toRat_neg']
  norm_num

@[simp] theorem ofRat_inv (a : PRCRat) :
    ofRat (a⁻¹) = (ofRat a)⁻¹ := by
  unfold ofRat
  rw [PRCRat.toRat_inv']
  norm_num

/-- K4.14. The real boundary carries Lean's complete-space structure. -/
theorem complete_space : CompleteSpace PRCRealBoundary := by
  infer_instance

end PRCRealBoundary

/-- K1/K4.14. Audit record: the real-completion boundary is a classical
extension until an internal PRC Cauchy quotient is built. -/
def realCompletionClaim : StrengthClaim where
  label := "K4.14_real_completion_boundary"
  tag := StrengthTag.classicalExtension
  statement := "The first PRC real boundary embeds PRCRat into Lean Real and uses classical completeness."

/-- K4.14. First real-completion boundary certificate. -/
structure RealCompletionBoundaryCertificate : Prop where
  real_boundary_exists : Nonempty PRCRealBoundary
  rational_embedding_exists : Nonempty (PRCRat → PRCRealBoundary)
  preserves_add : ∀ a b : PRCRat,
    PRCRealBoundary.ofRat (a + b) =
      PRCRealBoundary.ofRat a + PRCRealBoundary.ofRat b
  preserves_mul : ∀ a b : PRCRat,
    PRCRealBoundary.ofRat (a * b) =
      PRCRealBoundary.ofRat a * PRCRealBoundary.ofRat b
  complete : CompleteSpace PRCRealBoundary
  strength_tag : realCompletionClaim.tag = StrengthTag.classicalExtension

/-- K4.14. The classical real boundary is available and tagged honestly. -/
theorem real_completion_boundary_certificate :
    RealCompletionBoundaryCertificate where
  real_boundary_exists := ⟨0⟩
  rational_embedding_exists := ⟨PRCRealBoundary.ofRat⟩
  preserves_add := PRCRealBoundary.ofRat_add
```

```
    preserves_mul := PRCRealBoundary.ofRat_mul
    complete := PRCRealBoundary.complete_space
    strength_tag := rfl

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

26. TraceClosure.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/TraceClosure.lean · **Lines:** 107

```
/-
  PrimitiveRecognitionCalculus/TraceClosure.lean

  Round-trip source:
    PRC_Kernel_Spec_20260526.html

  Spec anchors:
    K1 (`δ + trace-closure`), R9, K4.13

  This module records the first completed-trace boundary. It does not claim
  that completed infinity is δ-only. It explicitly carries the
  `traceClosure` strength tag.
-/

import Mathlib
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Basic
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Orbit
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Strength

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- K4.13/R9. A completed trace is an infinite ledger of distinction acts.
This is a trace-closure object, not a finite δ-only trace. -/
structure CompletedTrace where
  actAt : Nat → DistinctionAct

namespace CompletedTrace

/-- The finite prefix of length `n` cut out of a completed trace. -/
def finitePrefix (S : CompletedTrace) : Nat → Trace
  | 0 => Trace.empty
  | Nat.succ n => Trace.extend (finitePrefix S n) (S.actAt n)

@[simp] theorem prefix_zero (S : CompletedTrace) :
    S.finitePrefix 0 = Trace.empty := by
  rfl

@[simp] theorem prefix_succ (S : CompletedTrace) (n : Nat) :
    S.finitePrefix (Nat.succ n) = Trace.extend (S.finitePrefix n) (S.actAt n) := by
  rfl

/-- The canonical completed trace repeats the primitive distinction act. -/
def canonical : CompletedTrace where
  actAt := fun _ => DistinctionAct.delta

@[simp] theorem canonical_actAt (n : Nat) :
    canonical.actAt n = DistinctionAct.delta := by
  rfl

/-- Every prefix of the canonical completed trace is a finite trace. -/
theorem canonical_prefix_exists (n : Nat) :
    Nonempty Trace := by
  exact (canonical.finitePrefix n)

end CompletedTrace

/-- K4.13. A completed orbit ledger is the infinite sequence of finite
δ-orbit positions. This is the natural-number side of trace closure. -/
structure CompletedOrbitLedger where
  positionAt : Nat → DistinctionNat

namespace CompletedOrbitLedger

/-- The canonical completed orbit sends verifier index `n` to the `n`th
δ-orbit position. -/
def canonical : CompletedOrbitLedger where
  positionAt := DistinctionNat.ofNat

@[simp] theorem canonical_toNat (n : Nat) :
    (canonical.positionAt n).toNat = n := by
  exact DistinctionNat.toNat_ofNat n

theorem canonical_succ (n : Nat) :
    canonical.positionAt (Nat.succ n) =
      DistinctionNat.succ (canonical.positionAt n) := by
  rfl

end CompletedOrbitLedger

/-- K1/R9. Audit record: completed traces require the trace-closure tag. -/
def traceClosureClaim : StrengthClaim where
  label := "K4.13_trace_closure_boundary"
  tag := StrengthTag.traceClosure
  statement := "Completed traces and completed orbit ledgers extend finite PRC by trace closure."

/-- K4.13. First trace-closure certificate. -/
structure TraceClosureCertificate : Prop where
  completed_trace_exists : Nonempty CompletedTrace
```

```
    canonical_completed_trace_exists : Nonempty CompletedTrace
    completed_orbit_ledger_exists : Nonempty CompletedOrbitLedger
    canonical_orbit_verifier_faithful :
      ∀ n : Nat, (CompletedOrbitLedger.canonical.positionAt n).toNat = n
    strength_tag : traceClosureClaim.tag = StrengthTag.traceClosure

/-- K4.13. The trace-closure boundary is inhabited and tagged honestly. -/
theorem trace_closure_certificate : TraceClosureCertificate where
  completed_trace_exists := (CompletedTrace.canonical)
  canonical_completed_trace_exists := (CompletedTrace.canonical)
  completed_orbit_ledger_exists := (CompletedOrbitLedger.canonical)
  canonical_orbit_verifier_faithful := CompletedOrbitLedger.canonical_toNat
  strength_tag := rfl

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

27. TraceLogic.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/TraceLogic.lean · **Lines:** 237

```
/-
  PrimitiveRecognitionCalculus/TraceLogic.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 11: build logical connectives and quantifier surfaces from
    stable trace predicates.

  This first pass keeps the logic surface over finite PRC traces. Completed
  trace families and model-theoretic inevitability are separate later layers.
-/

import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.SameDiff

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- A proposition in the first PRC logic pass is a predicate on finite traces
that persists under trace extension. -/
structure TracePredicate where
  holds : Trace → Prop
  stable :
    ∀ {T U : Trace}, Trace.Extends T U → holds T → holds U

namespace TracePredicate

/-- Truth is the stable predicate that holds at every trace. -/
def top : TracePredicate where
  holds := fun _ => True
  stable := by
    intro _T _U _hTU _h
    trivial

/-- Falsehood is the stable predicate that holds at no trace. -/
def bottom : TracePredicate where
  holds := fun _ => False
  stable := by
    intro _T _U _hTU h
    exact False.elim h

/-- Conjunction of stable trace predicates. -/
def and (P Q : TracePredicate) : TracePredicate where
  holds := fun T => P.holds T ∧ Q.holds T
  stable := by
    intro T U hTU h
    exact (P.stable hTU h.1, Q.stable hTU h.2)

/-- Disjunction of stable trace predicates. -/
def or (P Q : TracePredicate) : TracePredicate where
  holds := fun T => P.holds T ∨ Q.holds T
  stable := by
    intro T U hTU h
    cases h with
    | inl hP => exact Or.inl (P.stable hTU hP)
    | inr hQ => exact Or.inr (Q.stable hTU hQ)

/-- Implication is persistence along every future extension of the current
trace. This makes implication itself stable under extension. -/
def imp (P Q : TracePredicate) : TracePredicate where
  holds := fun T =>
    ∀ U : Trace, Trace.Extends T U → P.holds U → Q.holds U
  stable := by
    intro T U hTU h V hUV hPV
    exact h V (Trace.extends_trans hTU hUV) hPV

/-- Negation is implication into falsehood. -/
def not (P : TracePredicate) : TracePredicate :=
  imp P bottom

/-- Universal quantification over a verifier-indexed family of stable trace
predicates. The family parameter is verifier bookkeeping; stability is still a
finite-trace theorem. -/
def all {α : Type} (P : α → TracePredicate) : TracePredicate where
  holds := fun T => ∀ a : α, (P a).holds T
  stable := by
    intro T U hTU h a
    exact (P a).stable hTU (h a)

/-- Existential quantification over a verifier-indexed family of stable trace
predicates. The witness is preserved while the trace is extended. -/
def exists_ {α : Type} (P : α → TracePredicate) : TracePredicate where
  holds := fun T => ∃ a : α, (P a).holds T
  stable := by
    intro T U hTU h
    rcases h with ⟨a, ha⟩
    exact ⟨a, (P a).stable hTU ha⟩

```

```
theorem top_intro (T : Trace) :
    top.holds T := by
  trivial

theorem and_intro {P Q : TracePredicate} {T : Trace}
    (hP : P.holds T) (hQ : Q.holds T) :
    (and P Q).holds T := by
  exact (hP, hQ)

theorem and_left {P Q : TracePredicate} {T : Trace}
    (h : (and P Q).holds T) :
    P.holds T := by
  exact h.1

theorem and_right {P Q : TracePredicate} {T : Trace}
    (h : (and P Q).holds T) :
    Q.holds T := by
  exact h.2

theorem or_inl {P Q : TracePredicate} {T : Trace}
    (hP : P.holds T) :
    (or P Q).holds T := by
  exact Or.inl hP

theorem or_inr {P Q : TracePredicate} {T : Trace}
    (hQ : Q.holds T) :
    (or P Q).holds T := by
  exact Or.inr hQ

theorem imp_elim {P Q : TracePredicate} {T U : Trace}
    (himp : (imp P Q).holds T)
    (hTU : Trace.Extends T U)
    (hP : P.holds U) :
    Q.holds U := by
  exact himp U hTU hP

theorem not_elim {P : TracePredicate} {T U : Trace}
    (hn : (not P).holds T)
    (hTU : Trace.Extends T U)
    (hP : P.holds U) :
    False := by
  exact hn U hTU hP

theorem all_intro {α : Type} {P : α → TracePredicate} {T : Trace}
    (h : ∀ a : α, (P a).holds T) :
    (all P).holds T := by
  exact h

theorem all_elim {α : Type} {P : α → TracePredicate} {T : Trace}
    (h : (all P).holds T) (a : α) :
    (P a).holds T := by
  exact h a

theorem exists_intro {α : Type} {P : α → TracePredicate} {T : Trace}
    (a : α) (h : (P a).holds T) :
    (exists_ P).holds T := by
  exact (a, h)

theorem persists {P : TracePredicate} {T U : Trace}
    (hTU : Trace.Extends T U) (hP : P.holds T) :
    P.holds U :=
  P.stable hTU hP

end TracePredicate

/-- Headline target for the first trace-logic pass. -/
structure TraceLogicCertificate : Prop where
  proposition_surface : Nonempty TracePredicate
  truth_intro : ∀ T : Trace, TracePredicate.top.holds T
  conjunction_intro :
    ∀ {P Q : TracePredicate} {T : Trace},
      P.holds T → Q.holds T → (TracePredicate.and P Q).holds T
  conjunction_left :
    ∀ {P Q : TracePredicate} {T : Trace},
      (TracePredicate.and P Q).holds T → P.holds T
  conjunction_right :
    ∀ {P Q : TracePredicate} {T : Trace},
      (TracePredicate.and P Q).holds T → Q.holds T
  disjunction_left :
    ∀ {P Q : TracePredicate} {T : Trace},
      P.holds T → (TracePredicate.or P Q).holds T
  disjunction_right :
    ∀ {P Q : TracePredicate} {T : Trace},
      Q.holds T → (TracePredicate.or P Q).holds T
  implication_elim :
    ∀ {P Q : TracePredicate} {T U : Trace},
      (TracePredicate.imp P Q).holds T →
        Trace.Extends T U → P.holds U → Q.holds U
  negation_elim :
    ∀ {P : TracePredicate} {T U : Trace},
      (TracePredicate.not P).holds T →
        Trace.Extends T U → P.holds U → False
  universal_intro :
    ∀ {α : Type} {P : α → TracePredicate} {T : Trace},
      (∀ a : α, (P a).holds T) → (TracePredicate.all P).holds T
  universal_elim :
    ∀ {α : Type} {P : α → TracePredicate} {T : Trace},
      (TracePredicate.all P).holds T → ∀ a : α, (P a).holds T
  existential_intro :
    ∀ {α : Type} {P : α → TracePredicate} {T : Trace},
      ∀ a : α, (P a).holds T → (TracePredicate.exists_ P).holds T
```

```
    persistence :
      ∀ {P : TracePredicate} {T U : Trace},
        Trace.Extends T U → P.holds T → P.holds U
    strength_tag : StrengthTag.deltaOnly = StrengthTag.deltaOnly

theorem trace_logic_certificate : TraceLogicCertificate where
    proposition_surface := (TracePredicate.top)
    truth_intro := TracePredicate.top_intro
    conjunction_intro := by
      intro P Q T hP hQ
      exact TracePredicate.and_intro hP hQ
    conjunction_left := by
      intro P Q T h
      exact TracePredicate.and_left h
    conjunction_right := by
      intro P Q T h
      exact TracePredicate.and_right h
    disjunction_left := by
      intro P Q T hP
      exact TracePredicate.or_inl hP
    disjunction_right := by
      intro P Q T hQ
      exact TracePredicate.or_inr hQ
    implication_elim := by
      intro P Q T U himp hTU hP
      exact TracePredicate.imp_elim himp hTU hP
    negation_elim := by
      intro P T U hn hTU hP
      exact TracePredicate.not_elim hn hTU hP
    universal_intro := by
      intro α P T h
      exact TracePredicate.all_intro h
    universal_elim := by
      intro α P T h a
      exact TracePredicate.all_elim h a
    existential_intro := by
      intro α P T a h
      exact TracePredicate.exists_intro a h
    persistence := by
      intro P T U hTU hP
      exact TracePredicate.persists hTU hP
    strength_tag := rfl

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

28. FormalSystem.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/FormalSystem.lean · **Lines:** 123

```
0001  /-
0002    PrimitiveRecognitionCalculus/FormalSystem.lean
0003
0004    Round-trip source:
0005      6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007    Spec anchor:
0008      Build Order step 12: define expressivity and embeddings in enough
0009      generality to prove the inevitability surface.
0010
0011    This module is not claiming that every historical foundation has already
0012    been parsed into the interface. It closes the exact theorem-shaped interface:
0013    any formal system that supplies distinguishable endpoint tokens and preserves
0014    trace extension admits a PRC trace embedding.
0015  -/
0016
0017  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.TraceLogic
0018
0019  namespace IndisputableMonolith
0020  namespace Foundation
0021  namespace PrimitiveRecognitionCalculus
0022
0023  /-- Minimal formal-system interface needed by the PRC inevitability theorem.
0024  `Token` and `Expr` are verifier-side carriers for an arbitrary formal system,
0025  while `distinguishes` and `exprExtends` record the structure that makes δ
0026  visible inside it. -/
0027  structure FormalSystem where
0028    Token : Type
0029    Expr : Type
0030    distinguishes : Token → Token → Prop
0031    exprExtends : Expr → Expr → Prop
0032    endpointToken : Endpoint → Token
0033    traceExpr : Trace → Expr
0034    traceExpr_extends :
0035      ∀ {T U : Trace}, Trace.Extends T U → exprExtends (traceExpr T) (traceExpr U)
0036
0037  namespace FormalSystem
0038
0039  /-- A formal system is expressive for the first inevitability pass when it can
0040  distinguish the two endpoints of the primitive distinction. -/
0041  def Expressive (F : FormalSystem) : Prop :=
0042    F.distinguishes (F.endpointToken Endpoint.left) (F.endpointToken Endpoint.right)
0043
0044  end FormalSystem
0045
0046  /-- A PRC embedding into a formal system preserves the primitive endpoint
0047  distinction and finite trace extension. -/
0048  structure PRCEmbeddingInto (F : FormalSystem) where
0049    endpointMap : Endpoint → F.Token
0050    traceMap : Trace → F.Expr
0051    preserves_distinction :
0052      F.distinguishes (endpointMap Endpoint.left) (endpointMap Endpoint.right)
0053    preserves_trace_extension :
0054      ∀ {T U : Trace}, Trace.Extends T U → F.exprExtends (traceMap T) (traceMap U)
0055
0056  /-- The canonical embedding supplied by an expressive formal system's own trace
0057  and endpoint interpretation fields. -/
0058  def PRCEmbeddingInto.ofExpressive
0059      (F : FormalSystem) (hF : F.Expressive) : PRCEmbeddingInto F where
0060    endpointMap := F.endpointToken
0061    traceMap := F.traceExpr
0062    preserves_distinction := hF
0063    preserves_trace_extension := F.traceExpr_extends
0064
0065  /-- Exact target for Build Order step 12. -/
0066  def FormalSystemEmbeddingTarget : Prop :=
0067    ∀ F : FormalSystem, F.Expressive → Nonempty (PRCEmbeddingInto F)
0068
0069  theorem FormalSystemEmbeddingTarget_proved :
0070      FormalSystemEmbeddingTarget := by
0071    intro F hF
0072    exact (PRCEmbeddingInto.ofExpressive F hF)
0073
0074  theorem Endpoint.left_ne_right :
0075      Endpoint.left ≠ Endpoint.right := by
0076    intro h
0077    have hside := congrArg Endpoint.side h
0078    cases hside
0079
0080  /-- PRC itself as the minimal formal system: endpoint tokens are endpoints,
0081  expressions are finite traces, and expression extension is trace extension. -/
0082  def PRCFormalSystem : FormalSystem where
0083    Token := Endpoint
0084    Expr := Trace
0085    distinguishes := fun a b => a ≠ b
0086    exprExtends := Trace.Extends
0087    endpointToken := id
0088    traceExpr := id
0089    traceExpr_extends := by
0090      intro T U hTU
```

```
    exact hTU

theorem PRCFormalSystem_expressive :
    PRCFormalSystem.Expressive := by
  exact Endpoint.left_ne_right

theorem PRCFormalSystem_embedding :
    Nonempty (PRCEmbeddingInto PRCFormalSystem) :=
  FormalSystemEmbeddingTarget_proved PRCFormalSystem PRCFormalSystem_expressive

/-- Step 12 certificate: the formal-system surface and embedding theorem are
closed. The broader claim that every external foundation satisfies this
interface is the next inevitability layer, not hidden here. -/
structure FormalSystemCertificate : Prop where
  formal_system_surface : Nonempty FormalSystem
  prc_system_expressive : PRCFormalSystem.Expressive
  prc_system_embedding : Nonempty (PRCEmbeddingInto PRCFormalSystem)
  embedding_target : FormalSystemEmbeddingTarget
  embedding_from_expressive :
    ∀ F : FormalSystem, F.Expressive → Nonempty (PRCEmbeddingInto F)
  strength_tag : StrengthTag.deltaOnly = StrengthTag.deltaOnly

theorem formal_system_certificate : FormalSystemCertificate where
  formal_system_surface := (PRCFormalSystem)
  prc_system_expressive := PRCFormalSystem_expressive
  prc_system_embedding := PRCFormalSystem_embedding
  embedding_target := FormalSystemEmbeddingTarget_proved
  embedding_from_expressive := FormalSystemEmbeddingTarget_proved
  strength_tag := rfl

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

29. Inevitability.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Inevitability.lean · **Lines:** 90

```
0001  /-
0002    PrimitiveRecognitionCalculus/Inevitability.lean
0003
0004    Round-trip source:
0005      6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007    Spec anchor:
0008      Build Order step 13: prove that every admissible expressive foundation
0009      admits a PRC trace core.
0010
0011    "Admissible" here is deliberately exact: the foundation has been parsed into
0012    the `FormalSystem` interface and supplies endpoint distinction. Parsing
0013    external historical foundations into this interface is a later corpus task,
0014    not an implicit axiom in this theorem.
0015  -/
0016
0017  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.FormalSystem
0018
0019  namespace IndisputableMonolith
0020  namespace Foundation
0021  namespace PrimitiveRecognitionCalculus
0022
0023  /-- An admissible foundation for the first inevitability theorem is a formal
0024  system that is expressive enough to distinguish the two endpoints of δ. -/
0025  structure AdmissibleFoundation where
0026    system : FormalSystem
0027    expressive : system.Expressive
0028
0029  /-- The exact first inevitability target. -/
0030  def PRCInevitabilityTarget : Prop :=
0031    ∀ A : AdmissibleFoundation, Nonempty (PRCEmbeddingInto A.system)
0032
0033  /-- Any foundation already parsed into the admissible interface presupposes a
0034  PRC trace core. -/
0035  theorem any_foundation_presupposes_distinction :
0036      PRCInevitabilityTarget := by
0037    intro A
0038    exact FormalSystemEmbeddingTarget_proved A.system A.expressive
0039
0040  /-- PRC itself is an admissible foundation in this interface. -/
0041  def PRCAdmissibleFoundation : AdmissibleFoundation where
0042    system := PRCFormalSystem
0043    expressive := PRCFormalSystem_expressive
0044
0045  theorem PRCAdmissibleFoundation_embeds :
0046      Nonempty (PRCEmbeddingInto PRCAdmissibleFoundation.system) :=
0047    any_foundation_presupposes_distinction PRCAdmissibleFoundation
0048
0049  /-- The exact remaining external parsing target schema: once a corpus of
0050  external foundations and a faithful-parse relation are fixed, every external
0051  object in that corpus must parse to an expressive formal system before the
0052  inevitability theorem applies to it. -/
0053  def ExternalFoundationParsingTarget
0054      (ExternalFoundation : Type)
0055      (FaithfulParse : ExternalFoundation → FormalSystem → Prop) : Prop :=
0056    ∀ E : ExternalFoundation,
0057      ∃ F : FormalSystem, FaithfulParse E F ∧ F.Expressive
0058
0059  /-- Step 13 certificate. The admissible-interface theorem is closed; the
0060  external parsing workload is named separately so it is not hidden inside the
0061  theorem. -/
0062  structure PRCInevitabilityCertificate : Prop where
0063    admissible_foundation_surface : Nonempty AdmissibleFoundation
0064    prc_admissible : Nonempty AdmissibleFoundation
0065    inevitability_target : PRCInevitabilityTarget
0066    any_foundation_embedding :
0067      ∀ A : AdmissibleFoundation, Nonempty (PRCEmbeddingInto A.system)
0068    prc_embedding : Nonempty (PRCEmbeddingInto PRCAdmissibleFoundation.system)
0069    external_parsing_target_schema :
0070      ∀ (ExternalFoundation : Type)
0071        (FaithfulParse : ExternalFoundation → FormalSystem → Prop),
0072        ExternalFoundationParsingTarget ExternalFoundation FaithfulParse =
0073          ExternalFoundationParsingTarget ExternalFoundation FaithfulParse
0074    strength_tag : StrengthTag.deltaOnly = StrengthTag.deltaOnly
0075
0076  theorem prc_inevitability_certificate :
0077      PRCInevitabilityCertificate where
0078    admissible_foundation_surface := (PRCAdmissibleFoundation)
0079    prc_admissible := (PRCAdmissibleFoundation)
0080    inevitability_target := any_foundation_presupposes_distinction
0081    any_foundation_embedding := any_foundation_presupposes_distinction
0082    prc_embedding := PRCAdmissibleFoundation_embeds
0083    external_parsing_target_schema := by
0084      intro ExternalFoundation FaithfulParse
0085      rfl
0086    strength_tag := rfl
0087
0088  end PrimitiveRecognitionCalculus
```

```
0089  end Foundation
0090  end IndisputableMonolith
```

30. RecognizerBridge.lean

Path: `IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RecognizerBridge.lean` · **Lines:** 117

```
/-
  PrimitiveRecognitionCalculus/RecognizerBridge.lean

  Round-trip source:
    6/PRC_Universal_Foundation_Execution_Plan_20260526.html

  Spec anchor:
    Build Order step 14: produce the positive-ratio recognizer surface from
    PRC cost theory and map it into the existing Law-of-Logic J-cost chain.

  The positive-ratio recognizer is PRC-native. The existing continuous
  uniqueness theorem is still recorded as a classical-extension bridge because
  it quantifies over positive real ratios and continuity.
-/

import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Inevitability
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RationalField

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- Positive PRC ratios are the input surface for recognizer comparisons. -/
structure PRCPositiveRatio where
  value : PRCRat
  positive : PRCRat.positive value

namespace PRCPositiveRatio

/-- The canonical positive unit ratio. -/
def one : PRCPositiveRatio where
  value := 1
  positive := by
    rw [PRCRat.positive_iff_toRat_pos]
    change 0 < PRCRat.one.toRat
    rw [PRCRat.one_toRat]
    norm_num

/-- The quotient-level PRC J-cost assigned to a positive ratio. -/
def cost (r : PRCPositiveRatio) : PRCRat :=
  PRCJCost.onPRCRat r.value

theorem cost_toRat (r : PRCPositiveRatio) :
    r.cost.toRat = (r.value.toRat + r.value.toRat⁻¹) / 2 - 1 :=
  PRCJCost.onPRCRat_toRat r.value

theorem cost_toReal_jcost (r : PRCPositiveRatio) :
    (r.cost.toRat : ℝ) = Cost.Jcost ((r.value.toRat : ℚ) : ℝ) := by
  rw [cost_toRat]
  unfold Cost.Jcost
  rw [Rat.cast_sub, Rat.cast_div, Rat.cast_add, Rat.cast_inv]
  norm_num

end PRCPositiveRatio

/-- Recognition cost on a positive PRC ratio. -/
def PRCRecognitionCost (r : PRCPositiveRatio) : PRCRat :=
  r.cost

theorem PRCRecognitionCost_display (r : PRCPositiveRatio) :
    (PRCRecognitionCost r).toRat =
      (r.value.toRat + r.value.toRat⁻¹) / 2 - 1 :=
  PRCPositiveRatio.cost_toRat r

/-- Exact bridge target from PRC recognizer costs into the existing
Law-of-Logic uniqueness theorem. -/
def PRCRecognizerLawOfLogicBridgeTarget : Prop :=
  ∀ (F : ℝ → ℝ),
    Cost.FunctionalEquation.AczelSmoothnessPackage →
    Cost.FunctionalEquation.IsReciprocalCost F →
    Cost.FunctionalEquation.IsNormalized F →
    Cost.FunctionalEquation.SatisfiesCompositionLaw F →
    Cost.FunctionalEquation.IsCalibrated F →
    ContinuousOn F (Set.Ioi 0) →
    ∀ x : ℝ, 0 < x → F x = Cost.Jcost x

theorem PRCRecognizerLawOfLogicBridgeTarget_proved :
    PRCRecognizerLawOfLogicBridgeTarget := by
  intro F hA hR hN hC hCal hCont x hx
  exact PRCJCost.bridge_to_existing_jcost_uniqueness
    F hA hR hN hC hCal hCont x hx

/-- Step 14 certificate. The recognizer surface is closed through the existing
continuous Law-of-Logic bridge; fully native arbitrary-cost uniqueness remains
the named PRC target from `PRCJCost.lean`. -/
structure PRCRecognizerBridgeCertificate : Prop where
  positive_ratio_surface : Nonempty PRCPositiveRatio
  recognition_cost_surface : Nonempty (PRCPositiveRatio → PRCRat)
  cost_display :
    ∀ r : PRCPositiveRatio,
```

```
      (PRCRecognitionCost r).toRat =
        (r.value.toRat + r.value.toRat⁻¹) / 2 - 1
  real_jcost_bridge :
    ∀ r : PRCPositiveRatio,
      ((PRCRecognitionCost r).toRat : ℝ) =
        Cost.Jcost ((r.value.toRat : ℚ) : ℝ)
  law_of_logic_bridge : PRCRecognizerLawOfLogicBridgeTarget
  native_uniqueness_target_named :
    PRCJCost.PRCNativeCostUniquenessTarget =
      PRCJCost.PRCNativeCostUniquenessTarget
  strength_tag : StrengthTag.classicalExtension = StrengthTag.classicalExtension

theorem prc_recognizer_bridge_certificate :
    PRCRecognizerBridgeCertificate where
  positive_ratio_surface := (PRCPositiveRatio.one)
  recognition_cost_surface := (PRCRecognitionCost)
  cost_display := PRCRecognitionCost_display
  real_jcost_bridge := by
    intro r
    exact PRCPositiveRatio.cost_toReal_jcost r
  law_of_logic_bridge := PRCRecognizerLawOfLogicBridgeTarget_proved
  native_uniqueness_target_named := rfl
  strength_tag := rfl

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

31. PRCNativeCostUniqueness.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCNativeCostUniqueness.lean · **Lines:** 3798

```
0001  /-
0002    PrimitiveRecognitionCalculus/PRCNativeCostUniqueness.lean
0003
0004    Round-trip source:
0005      6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007    Spec anchor:
0008      Immediate target after pass 24: attack `PRCNativeCostUniquenessTarget`
0009      or isolate the smallest exact blocker.
0010
0011    The current PRC cost hypotheses are too weak to prove uniqueness directly:
0012    after setting `g = F + 1`, the RCL has the d'Alembert form
0013    `g(xy) + g(x/y) = 2 g(x) g(y)`. On a rational multiplicative group,
0014    one-point calibration at `2` does not by itself control all prime directions.
0015    This file names the exact missing character-factorization and calibrated
0016    character-rigidity targets and proves that together they imply native
0017    cost uniqueness.
0018  -/
0019
0020  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCost
0021
0022  namespace IndisputableMonolith
0023  namespace Foundation
0024  namespace PrimitiveRecognitionCalculus
0025  namespace PRCJCost
0026
0027  /-- A ratio-character candidate for the d'Alembert factorization of a PRC cost.
0028  It is stated at the ratio-orbit level and uses cross-equivalence rather than
0029  definitional equality, so it remains quotient-native. -/
0030  structure PRCRatioCharacter (χ : RatioOrbit → RatioOrbit) : Prop where
0031    unit :
0032      RatioOrbit.crossEq (χ RatioOrbit.one) RatioOrbit.one
0033    multiplicative :
0034      ∀ x y : RatioOrbit,
0035        RatioOrbit.crossEq (χ (RatioOrbit.mul x y))
0036          (RatioOrbit.mul (χ x) (χ y))
0037    reciprocal :
0038      ∀ x : RatioOrbit,
0039        RatioOrbit.crossEq (χ (RatioOrbit.recip x))
0040          (RatioOrbit.recip (χ x))
0041    normalized_invariant :
0042      ∀ q : RatioOrbit,
0043        RatioOrbit.crossEq (χ q) (χ (DistinctionNat.normalizeRatio q))
0044    nonzero_preserving :
0045      ∀ {q : RatioOrbit}, q.toRat ≠ 0 → (χ q).toRat ≠ 0
0046
0047  /-- Cost generated from a rational character. The canonical PRC cost is the
0048  identity-character case. -/
0049  def costFromCharacter (χ : RatioOrbit → RatioOrbit) (q : RatioOrbit) :
0050      RatioOrbit :=
0051    onRatioOrbit (χ q)
0052
0053  theorem costFromCharacter_toRat
0054      (χ : RatioOrbit → RatioOrbit) (q : RatioOrbit) :
0055      (costFromCharacter χ q).toRat =
0056        ((χ q).toRat + (χ q).toRat⁻¹) / 2 - 1 := by
0057    exact onRatioOrbit_toRat (χ q)
0058
0059  /-- First exact blocker: every admissible PRC-native RCL cost should factor
0060  through a ratio character. This is the discrete d'Alembert factorization step. -/
0061  def PRCNativeCostCharacterFactorizationTarget : Prop :=
0062    ∀ F : RatioOrbit → RatioOrbit,
0063      PRCNativeCostHypotheses F →
0064        ∃ χ : RatioOrbit → RatioOrbit,
0065          PRCRatioCharacter χ ∧
0066            ∀ q : RatioOrbit,
0067              RatioOrbit.crossEq (F q) (costFromCharacter χ q)
0068
0069  /-- Second exact blocker: a calibrated rational character cost must be the
0070  canonical identity-character cost. This is where prime-direction freedom has
0071  to be eliminated. -/
0072  def PRCNativeCostCharacterRigidityTarget : Prop :=
0073    ∀ χ : RatioOrbit → RatioOrbit,
0074      PRCRatioCharacter χ →
0075        RatioOrbit.crossEq (costFromCharacter χ two) (onRatioOrbit two) →
0076          ∀ q : RatioOrbit,
0077            RatioOrbit.crossEq (costFromCharacter χ q) (onRatioOrbit q)
0078
0079  /-- The ratio direction associated to a nonzero orbit position. -/
0080  def orbitDirection (p : DistinctionNat) (_hp : p ≠ DistinctionNat.zero) :
0081      RatioOrbit where
0082    num := SignedOrbit.ofOrbit p
0083    den := DistinctionNat.one
0084    den_ne_zero := DistinctionNat.one_ne_zero
0085
0086  /-- The ratio direction associated to a native prime orbit. -/
0087  def primeDirection (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p) :
0088      RatioOrbit :=
0089    orbitDirection p hp.1
0090
```

```
/-- A ratio character is calibrated on every native prime direction when its
generated cost agrees with canonical J-cost on each prime orbit. -/
def PRCCharacterPrimeDirectionCalibrated
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
    RatioOrbit.crossEq
      (costFromCharacter χ (primeDirection p hp))
      (onRatioOrbit (primeDirection p hp))

/-- Sharper target A: the two-point calibration at orbit `2` must force
calibration on every prime direction. This is the exact place where the present
surface lacks control of independent prime axes. -/
def PRCTwoCalibrationForcesPrimeCalibrationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      RatioOrbit.crossEq (costFromCharacter χ two) (onRatioOrbit two) →
        PRCCharacterPrimeDirectionCalibrated χ

/-- Sharper target B: once every prime direction is calibrated, the character
cost propagates to every rational direction. This is the unique-factorization
side of the rigidity problem. -/
def PRCPrimeCalibrationPropagationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        ∀ q : RatioOrbit,
          RatioOrbit.crossEq (costFromCharacter χ q) (onRatioOrbit q)

theorem onRatioOrbit_congr {a b : RatioOrbit}
    (h : RatioOrbit.crossEq a b) :
    RatioOrbit.crossEq (onRatioOrbit a) (onRatioOrbit b) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq] at h ⊢
  rw [onRatioOrbit_toRat, onRatioOrbit_toRat, h]

theorem jcost_eq_forces_same_or_reciprocal {a b : RatioOrbit}
    (ha : a.toRat ≠ 0) (hb : b.toRat ≠ 0)
    (h : RatioOrbit.crossEq (onRatioOrbit a) (onRatioOrbit b)) :
    RatioOrbit.crossEq a b ∨
      RatioOrbit.crossEq a (RatioOrbit.recip b) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq] at h
  rw [onRatioOrbit_toRat, onRatioOrbit_toRat] at h
  rw [RatioOrbit.crossEq_iff_toRat_eq,
    RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat]
  have hsum : a.toRat + a.toRat⁻¹ = b.toRat + b.toRat⁻¹ := by
    linarith
  have hprod :
      (a.toRat - b.toRat) * (a.toRat * b.toRat - 1) = 0 := by
    have hmul :=
      congrArg (fun t : ℚ => t * (a.toRat * b.toRat)) hsum
    field_simp [ha, hb] at hmul
    ring_nf at hmul ⊢
    linarith
  rcases mul_eq_zero.mp hprod with hsame | hrec
  · left
    linarith
  · right
    have hmul : a.toRat * b.toRat = 1 := by
      linarith
    have hb_inv : b.toRat * b.toRat⁻¹ = 1 := by
      field_simp [hb]
    calc
      a.toRat = a.toRat * (b.toRat * b.toRat⁻¹) := by
        rw [hb_inv, mul_one]
      _ = (a.toRat * b.toRat) * b.toRat⁻¹ := by
        ring
      _ = b.toRat⁻¹ := by
        rw [hmul, one_mul]

theorem primeDirection_toRat_ne_zero
    (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p) :
    (primeDirection p hp).toRat ≠ 0 := by
  have hpNat : p.toNat ≠ 0 := by
    intro hzero
    apply hp.1
    apply DistinctionNat.toNat_inj
    rw [hzero, DistinctionNat.toNat_zero]
  unfold primeDirection orbitDirection RatioOrbit.toRat
  simp [SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat, hpNat]

theorem primeDirection_toRat
    (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p) :
    (primeDirection p hp).toRat = (p.toNat : ℚ) := by
  unfold primeDirection orbitDirection RatioOrbit.toRat
  simp [SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat]

theorem orbitDirection_toRat
    (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero) :
    (orbitDirection p hp).toRat = (p.toNat : ℚ) := by
  unfold orbitDirection RatioOrbit.toRat
  simp [SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat]

theorem primeDirection_not_crossEq_recip
    (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p) :
    ¬ RatioOrbit.crossEq
      (primeDirection p hp)
      (RatioOrbit.recip (primeDirection p hp)) := by
  intro h
  rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
    primeDirection_toRat] at h
  have hpNat0 : p.toNat ≠ 0 := by
    intro hzero
```

```
    apply hp.1
    apply DistinctionNat.toNat_inj
    rw [hzero, DistinctionNat.toNat_zero]
  have hpNatQ0 : (p.toNat : ℚ) ≠ 0 := by
    exact_mod_cast hpNat0
  have hsqQ : (p.toNat : ℚ) * (p.toNat : ℚ) = 1 := by
    have hmul := congrArg (fun t : ℚ => t * (p.toNat : ℚ)) h
    field_simp [hpNatQ0] at hmul
    ring_nf at hmul ⊢
    exact hmul
  have hsqNat : p.toNat * p.toNat = 1 := by
    exact_mod_cast hsqQ
  have hle : p.toNat ≤ 1 := by
    by_contra hnot
    have hge : 2 ≤ p.toNat := by omega
    have hprodge : 2 ≤ p.toNat * p.toNat := by
      calc
        2 ≤ 2 * 2 := by norm_num
        _ ≤ p.toNat * p.toNat := Nat.mul_le_mul hge hge
    omega
  have hpOne : p.toNat = 1 := by omega
  exact hp.2.1 ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hpOne)

theorem orbitDirection_nonunit_not_crossEq_recip
    (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero)
    (hunit : ¬ DistinctionNat.unit p) :
    ¬ RatioOrbit.crossEq
      (orbitDirection p hp)
      (RatioOrbit.recip (orbitDirection p hp)) := by
  intro h
  rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
    orbitDirection_toRat] at h
  have hpNat0 : p.toNat ≠ 0 := by
    intro hzero
    apply hp
    apply DistinctionNat.toNat_inj
    rw [hzero, DistinctionNat.toNat_zero]
  have hpNatQ0 : (p.toNat : ℚ) ≠ 0 := by
    exact_mod_cast hpNat0
  have hsqQ : (p.toNat : ℚ) * (p.toNat : ℚ) = 1 := by
    have hmul := congrArg (fun t : ℚ => t * (p.toNat : ℚ)) h
    field_simp [hpNatQ0] at hmul
    ring_nf at hmul ⊢
    exact hmul
  have hsqNat : p.toNat * p.toNat = 1 := by
    exact_mod_cast hsqQ
  have hle : p.toNat ≤ 1 := by
    by_contra hnot
    have hge : 2 ≤ p.toNat := by omega
    have hprodge : 2 ≤ p.toNat * p.toNat := by
      calc
        2 ≤ 2 * 2 := by norm_num
        _ ≤ p.toNat * p.toNat := Nat.mul_le_mul hge hge
    omega
  have hpOne : p.toNat = 1 := by omega
  exact hunit ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hpOne)

theorem orbit_succ_ne_zero (p : DistinctionNat) :
    DistinctionNat.succ p ≠ DistinctionNat.zero := by
  intro h
  exact DistinctionNat.zero_ne_succ p h.symm

theorem orbitDirection_succ_crossEq_add_one
    (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero) :
    RatioOrbit.crossEq
      (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
      (RatioOrbit.add (orbitDirection p hp) RatioOrbit.one) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq]
  rw [RatioOrbit.add_toRat, RatioOrbit.one_toRat,
    orbitDirection_toRat, orbitDirection_toRat, DistinctionNat.toNat_succ]
  norm_num

theorem RatioOrbit.add_right_one_cancel {a b : RatioOrbit}
    (h : RatioOrbit.crossEq
      (RatioOrbit.add a RatioOrbit.one)
      (RatioOrbit.add b RatioOrbit.one)) :
    RatioOrbit.crossEq a b := by
  rw [RatioOrbit.crossEq_iff_toRat_eq] at h ⊢
  rw [RatioOrbit.add_toRat, RatioOrbit.add_toRat, RatioOrbit.one_toRat] at h
  linarith

/-- Native trace carried by an orbit position, defined by recursion on the
δ-orbit rather than by importing verifier `Nat` as object theory. -/
def orbitPositionTrace : DistinctionNat → Trace
  | DistinctionNat.zero => Trace.empty
  | DistinctionNat.succ n => Trace.step (orbitPositionTrace n)

theorem orbitPositionTrace_add_extends_left
    (p r : DistinctionNat) :
    Trace.Extends (orbitPositionTrace p) (orbitPositionTrace (p + r)) := by
  induction r with
  | zero =>
      rw [DistinctionNat.add_zero_eq]
      exact Trace.extends_refl (orbitPositionTrace p)
  | succ r ih =>
      rw [DistinctionNat.add_succ_eq]
      rcases ih with (suffix, hsuffix)
      refine (Trace.step suffix, ?_)
      simp [Trace.step, orbitPositionTrace, hsuffix]

theorem orbitPositionTrace_add_extends_right
```

```
    (p r : DistinctionNat) :
    Trace.Extends (orbitPositionTrace r) (orbitPositionTrace (p + r)) := by
  rw [DistinctionNat.add_comm p r]
  exact orbitPositionTrace_add_extends_left r p

theorem orbitPositionTrace_extends_of_toNat_le
    {p r : DistinctionNat} (hpr : p.toNat ≤ r.toNat) :
    Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) := by
  let k : DistinctionNat := DistinctionNat.ofNat (r.toNat - p.toNat)
  have hsum : p + k = r := by
    apply DistinctionNat.toNat_inj
    rw [DistinctionNat.toNat_add, DistinctionNat.toNat_ofNat]
    omega
  rw [← hsum]
  exact orbitPositionTrace_add_extends_left p k

theorem orbitPositionTrace_comparable
    (p r : DistinctionNat) :
    Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) ∨
      Trace.Extends (orbitPositionTrace r) (orbitPositionTrace p) := by
  by_cases hpr : p.toNat ≤ r.toNat
  · exact Or.inl (orbitPositionTrace_extends_of_toNat_le hpr)
  · have hrp : r.toNat ≤ p.toNat := by omega
    exact Or.inr (orbitPositionTrace_extends_of_toNat_le hrp)

/-- Two prime axes are trace-connected when their native orbit traces admit a
common finite δ-extension. -/
def PRCPrimeAxisTraceConnected
    (p : DistinctionNat) (_hp : DistinctionNat.primeOrbit p)
    (r : DistinctionNat) (_hr : DistinctionNat.primeOrbit r) : Prop :=
  ∃ T : Trace,
    Trace.Extends (orbitPositionTrace p) T ∧
      Trace.Extends (orbitPositionTrace r) T

theorem PRCPrimeAxisTraceConnected_proved
    (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p)
    (r : DistinctionNat) (hr : DistinctionNat.primeOrbit r) :
    PRCPrimeAxisTraceConnected p hp r hr := by
  exact (orbitPositionTrace (p + r),
    orbitPositionTrace_add_extends_left p r,
    orbitPositionTrace_add_extends_right p r)

/-- A character has global cost orientation when every rational direction is
sent either to itself or to its reciprocal. Since J-cost is reciprocal-symmetric,
this is exactly the orientation information needed for cost propagation. -/
def PRCCharacterGlobalCostOrientation
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ q : RatioOrbit,
    RatioOrbit.crossEq (χ q) q ∨
      RatioOrbit.crossEq (χ q) (RatioOrbit.recip q)

/-- Sharper propagation blocker: prime calibration must force a coherent global
orientation. Without this, independent prime inversions can preserve prime
costs while breaking composite costs. -/
def PRCPrimeCalibrationForcesGlobalOrientationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterGlobalCostOrientation χ

/-- Prime-axis orientation is coherent when the character chooses the same
orientation on every native prime direction: all identity or all reciprocal. -/
def PRCCharacterPrimeOrientationCoherent
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  (∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
    RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp)) ∨
  (∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
    RatioOrbit.crossEq (χ (primeDirection p hp))
      (RatioOrbit.recip (primeDirection p hp)))

/-- Local prime orientation says each prime axis is individually sent to itself
or to its reciprocal. This is the algebraic content of equality of J-costs on a
single prime direction. -/
def PRCCharacterPrimeLocalOrientation
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
    RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp) ∨
      RatioOrbit.crossEq (χ (primeDirection p hp))
        (RatioOrbit.recip (primeDirection p hp))

/-- No mixed prime orientation says a character cannot choose identity on one
prime axis and reciprocal on another. This is the trace-coherence condition that
rules out independent prime-axis inversions. -/
def PRCCharacterNoMixedPrimeOrientation
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
    ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
      RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp) →
        RatioOrbit.crossEq (χ (primeDirection r hr))
          (RatioOrbit.recip (primeDirection r hr)) →
          False

/-- Prime identity orientation is trace-coherent when identity orientation at
one calibrated prime forces identity orientation at every calibrated prime. This
is the missing cross-prime relation; the current ratio-character laws are local
to multiplication and reciprocal and do not by themselves connect the orientation
choices of different prime axes. -/
def PRCCharacterPrimeIdentityTraceCoherent
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
    ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
```

```
      RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp) →
        RatioOrbit.crossEq (χ (primeDirection r hr)) (primeDirection r hr)

/-- A character respects prime-axis trace connection when identity orientation
transports along the finite δ-trace component relating two prime axes. -/
def PRCCharacterPrimeIdentityRespectsTraceConnected
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
    ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
      PRCPrimeAxisTraceConnected p hp r hr →
        RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp) →
          RatioOrbit.crossEq (χ (primeDirection r hr)) (primeDirection r hr)

/-- A more explicit form of the trace-transport rule: identity orientation
transports when two prime-axis traces are witnessed inside the same finite
δ-trace extension. -/
def PRCCharacterPrimeIdentityRespectsCommonTraceExtension
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
    ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
      ∀ T : Trace,
        Trace.Extends (orbitPositionTrace p) T →
          Trace.Extends (orbitPositionTrace r) T →
            RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp) →
              RatioOrbit.crossEq (χ (primeDirection r hr)) (primeDirection r hr)

/-- The exact trace-order law: identity orientation transports between prime axes
whose finite δ-orbit traces are comparable by extension. The structural
comparability of any two orbit traces is proved above, so this is the part that
must come from the ratio character respecting trace order. -/
def PRCCharacterPrimeIdentityRespectsComparableTrace
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
    ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
      (Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) ∨
        Trace.Extends (orbitPositionTrace r) (orbitPositionTrace p)) →
        RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp) →
          RatioOrbit.crossEq (χ (primeDirection r hr)) (primeDirection r hr)

/-- Identity orientation for an arbitrary nonzero orbit direction, not only for
prime axes. This lets trace transport pass through composite orbit positions. -/
def PRCCharacterOrbitDirectionIdentity
    (χ : RatioOrbit → RatioOrbit)
    (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero) : Prop :=
  RatioOrbit.crossEq (χ (orbitDirection p hp)) (orbitDirection p hp)

/-- Reciprocal orientation for an arbitrary nonzero orbit direction. -/
def PRCCharacterOrbitDirectionReciprocal
    (χ : RatioOrbit → RatioOrbit)
    (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero) : Prop :=
  RatioOrbit.crossEq
    (χ (orbitDirection p hp)) (RatioOrbit.recip (orbitDirection p hp))

/-- The one-step trace-order law: identity orientation is invariant under one
successor step of the nonzero δ-orbit. This is smaller than prime-to-prime
transport, because it acts before primality is imposed. -/
def PRCCharacterOrbitIdentityRespectsSuccessorStep
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    PRCCharacterOrbitDirectionIdentity χ p hp ↔
      PRCCharacterOrbitDirectionIdentity χ
        (DistinctionNat.succ p) (orbit_succ_ne_zero p)

/-- Forward one-step successor law for identity orientation on nonzero orbit
directions. -/
def PRCCharacterOrbitIdentityExtendsSuccessorStep
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    PRCCharacterOrbitDirectionIdentity χ p hp →
      PRCCharacterOrbitDirectionIdentity χ
        (DistinctionNat.succ p) (orbit_succ_ne_zero p)

/-- Backward one-step successor law for identity orientation on nonzero orbit
directions. -/
def PRCCharacterOrbitIdentityContractsSuccessorStep
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    PRCCharacterOrbitDirectionIdentity χ
      (DistinctionNat.succ p) (orbit_succ_ne_zero p) →
        PRCCharacterOrbitDirectionIdentity χ p hp

/-- The successor-step transport needed for trace coherence is exactly the
forward and backward one-step laws bundled together. -/
def PRCCharacterOrbitIdentitySuccessorTransport
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  PRCCharacterOrbitIdentityExtendsSuccessorStep χ ∧
    PRCCharacterOrbitIdentityContractsSuccessorStep χ

/-- Additive compatibility with the δ-successor operation on nonzero orbit
directions. This is the missing bridge between multiplicative ratio characters
and the trace/additive structure of the orbit. -/
def PRCCharacterOrbitSuccessorAdditiveCompatible
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    RatioOrbit.crossEq
      (χ (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p)))
      (RatioOrbit.add (χ (orbitDirection p hp)) RatioOrbit.one)

theorem PRCCharacterOrbitIdentityExtendsSuccessorStep_of_additive_compat
    {χ : RatioOrbit → RatioOrbit}
    (hadd : PRCCharacterOrbitSuccessorAdditiveCompatible χ) :
```

```
    PRCCharacterOrbitIdentityExtendsSuccessorStep χ := by
  intro p hp hpId
  have hχsucc := hadd p hp
  rw [PRCCharacterOrbitDirectionIdentity] at hpId ⊢
  rw [RatioOrbit.crossEq_iff_toRat_eq] at hχsucc hpId ⊢
  rw [RatioOrbit.add_toRat, RatioOrbit.one_toRat] at hχsucc
  rw [hχsucc, hpId]
  rw [orbitDirection_toRat, orbitDirection_toRat, DistinctionNat.toNat_succ]
  norm_num

theorem PRCCharacterOrbitIdentityContractsSuccessorStep_of_additive_compat
    {χ : RatioOrbit → RatioOrbit}
    (hadd : PRCCharacterOrbitSuccessorAdditiveCompatible χ) :
    PRCCharacterOrbitIdentityContractsSuccessorStep χ := by
  intro p hp hsuccId
  have hχsucc := hadd p hp
  rw [PRCCharacterOrbitDirectionIdentity] at hsuccId ⊢
  rw [RatioOrbit.crossEq_iff_toRat_eq] at hχsucc hsuccId ⊢
  rw [RatioOrbit.add_toRat, RatioOrbit.one_toRat] at hχsucc
  rw [hχsucc] at hsuccId
  rw [orbitDirection_toRat, DistinctionNat.toNat_succ] at hsuccId
  rw [orbitDirection_toRat]
  have hsuccCast :
      ((Nat.succ p.toNat : Nat) : ℚ) = (p.toNat : ℚ) + 1 := by
    norm_num
  rw [hsuccCast] at hsuccId
  linarith

theorem PRCCharacterOrbitIdentitySuccessorTransport_of_additive_compat
    {χ : RatioOrbit → RatioOrbit}
    (hadd : PRCCharacterOrbitSuccessorAdditiveCompatible χ) :
    PRCCharacterOrbitIdentitySuccessorTransport χ :=
  ⟨PRCCharacterOrbitIdentityExtendsSuccessorStep_of_additive_compat hadd,
    PRCCharacterOrbitIdentityContractsSuccessorStep_of_additive_compat hadd⟩

theorem orbit_succ_not_unit_of_nonzero_not_unit
    (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero)
    (hunit : ¬ DistinctionNat.unit p) :
    ¬ DistinctionNat.unit (DistinctionNat.succ p) := by
  intro hsuccUnit
  have hpNat0 : p.toNat ≠ 0 := by
    intro hz
    apply hp
    apply DistinctionNat.toNat_inj
    rw [hz, DistinctionNat.toNat_zero]
  have hpNat1 : p.toNat ≠ 1 := by
    intro hone
    exact hunit ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hone)
  have hsuccNat1 : (DistinctionNat.succ p).toNat = 1 :=
    (DistinctionNat.unit_iff_toNat_eq_one (DistinctionNat.succ p)).mp hsuccUnit
  rw [DistinctionNat.toNat_succ] at hsuccNat1
  omega

/-- Every nonunit orbit direction is locally oriented: identity or reciprocal.
This is the nonprime analogue of the already proved local-prime orientation
alternative. -/
def PRCCharacterNonunitOrbitLocalOrientation
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      PRCCharacterOrbitDirectionIdentity χ p hp ∨
        PRCCharacterOrbitDirectionReciprocal χ p hp

/-- Product-factor propagation for local orientation. If two nonunit factors are
locally identity-or-reciprocal oriented, their product is locally oriented too.
This is the exact multiplicative step needed to move from prime-axis
orientation to composite orbit directions. -/
def PRCCharacterOrbitProductLocalOrientationPropagates
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ a b p : DistinctionNat,
    ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
      ¬ DistinctionNat.unit a →
        ¬ DistinctionNat.unit b →
          ∀ hp : p ≠ DistinctionNat.zero,
            ¬ DistinctionNat.unit p →
              a * b = p →
                (PRCCharacterOrbitDirectionIdentity χ a ha ∨
                  PRCCharacterOrbitDirectionReciprocal χ a ha) →
                (PRCCharacterOrbitDirectionIdentity χ b hb ∨
                  PRCCharacterOrbitDirectionReciprocal χ b hb) →
                  PRCCharacterOrbitDirectionIdentity χ p hp ∨
                    PRCCharacterOrbitDirectionReciprocal χ p hp

theorem ratioOrbit_mul_congr {a₁ a₂ b₁ b₂ : RatioOrbit}
    (ha : RatioOrbit.crossEq a₁ a₂) (hb : RatioOrbit.crossEq b₁ b₂) :
    RatioOrbit.crossEq (RatioOrbit.mul a₁ b₁) (RatioOrbit.mul a₂ b₂) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq] at ha hb ⊢
  rw [RatioOrbit.mul_toRat, RatioOrbit.mul_toRat, ha, hb]

theorem ratioOrbit_recip_congr {a b : RatioOrbit}
    (h : RatioOrbit.crossEq a b) :
    RatioOrbit.crossEq (RatioOrbit.recip a) (RatioOrbit.recip b) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq] at h ⊢
  rw [RatioOrbit.recip_toRat, RatioOrbit.recip_toRat, h]

theorem ratioOrbit_mul_recip_recip_crossEq_recip_mul
    (a b : RatioOrbit) :
    RatioOrbit.crossEq
      (RatioOrbit.mul (RatioOrbit.recip a) (RatioOrbit.recip b))
      (RatioOrbit.recip (RatioOrbit.mul a b)) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.mul_toRat,
```

```
    RatioOrbit.recip_toRat, RatioOrbit.recip_toRat, RatioOrbit.recip_toRat,
    RatioOrbit.mul_toRat]
  by_cases ha : a.toRat = 0
  · simp [ha]
  · by_cases hb : b.toRat = 0
    · simp [hb]
    · field_simp [ha, hb]

theorem orbitDirection_mul_crossEq
    (a b p : DistinctionNat)
    (ha : a ≠ DistinctionNat.zero) (hb : b ≠ DistinctionNat.zero)
    (hp : p ≠ DistinctionNat.zero)
    (hmul : a * b = p) :
    RatioOrbit.crossEq (orbitDirection p hp)
      (RatioOrbit.mul (orbitDirection a ha) (orbitDirection b hb)) := by
  rw [RatioOrbit.crossEq_iff_toRat_eq, orbitDirection_toRat,
    RatioOrbit.mul_toRat, orbitDirection_toRat, orbitDirection_toRat]
  have hnat := congrArg DistinctionNat.toNat hmul
  rw [DistinctionNat.toNat_mul] at hnat
  exact_mod_cast hnat.symm

/-- A character is compatible with the native display of an orbit product when
the character value on the product orbit agrees with the character value on the
ratio product of the factor orbits. This is not automatic from
cross-equivalence; it is the quotient-respect step missing from the bare
character interface. -/
def PRCCharacterOrbitProductDisplayCompatible
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ a b p : DistinctionNat,
    ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
      ∀ hp : p ≠ DistinctionNat.zero,
        a * b = p →
          RatioOrbit.crossEq (χ (orbitDirection p hp))
            (χ (RatioOrbit.mul (orbitDirection a ha) (orbitDirection b hb)))

/-- Quotient-respect for a ratio character: equivalent ratio-orbit displays
must receive equivalent character values. This is the missing map-respects-setoid
condition for using a raw `RatioOrbit → RatioOrbit` function as a quotient-native
PRC character. -/
def PRCCharacterRespectsCrossEq (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ q r : RatioOrbit,
    RatioOrbit.crossEq q r → RatioOrbit.crossEq (χ q) (χ r)

/-- Canonical-normalization target for ratio orbits. If two raw ratio displays
are cross-equivalent, native GCD normalization should return the same raw
representative. This is the exact quotient-normalization uniqueness statement
needed to turn `normalized_invariant` into general quotient respect. -/
def PRCNormalizeRatioCanonicalTarget : Prop :=
  ∀ q r : RatioOrbit,
    RatioOrbit.crossEq q r →
      DistinctionNat.normalizeRatio q = DistinctionNat.normalizeRatio r

/-- A signed-orbit display is sign-canonical when it is literally the
nonnegative orbit display of its absolute value, or literally the negated
nonnegative display of its absolute value. This records the raw representative
condition supplied by `signedQuotient`, not just balanced integer equality. -/
def PRCSignedOrbitSignCanonical (z : SignedOrbit) : Prop :=
  (0 ≤ z.toInt ∧ z = SignedOrbit.ofOrbit z.abs) ∨
    (z.toInt < 0 ∧ z = SignedOrbit.negate (SignedOrbit.ofOrbit z.abs))

/-- A raw ratio display is reduced and sign-canonical when its numerator
absolute value is coprime to the positive denominator and the signed numerator
itself is in the canonical raw signed-orbit form. -/
def PRCRatioReducedSignCanonical (q : RatioOrbit) : Prop :=
  DistinctionNat.coprime q.num.abs q.den ∧
    PRCSignedOrbitSignCanonical q.num

theorem signedOrbit_ofOrbit_abs_self (n : DistinctionNat) :
    (SignedOrbit.ofOrbit n).abs = n := by
  apply DistinctionNat.toNat_inj
  rw [SignedOrbit.abs_toNat, SignedOrbit.ofOrbit_toInt]
  simp

theorem signedOrbit_neg_ofOrbit_abs_self (n : DistinctionNat) :
    (SignedOrbit.negate (SignedOrbit.ofOrbit n)).abs = n := by
  apply DistinctionNat.toNat_inj
  rw [SignedOrbit.abs_toNat, SignedOrbit.negate_toInt,
    SignedOrbit.ofOrbit_toInt]
  simp

theorem signedQuotient_signCanonical_of_divides
    (z : SignedOrbit) (d : DistinctionNat) (hd : d ≠ DistinctionNat.zero)
    (hdiv : DistinctionNat.divides d z.abs) :
    PRCSignedOrbitSignCanonical (DistinctionNat.signedQuotient z d hd) := by
  unfold DistinctionNat.signedQuotient
  by_cases hflag : z.nonnegFlag = true
  · left
    constructor
    · simp [hflag, SignedOrbit.ofOrbit_toInt]
    · rw [if_pos hflag]
      rw [signedOrbit_ofOrbit_abs_self]
  · have hflagFalse : z.nonnegFlag = false := by
      cases h : z.nonnegFlag with
      | false => rfl
      | true =>
          exfalso
          exact hflag h
    right
    have hzneg : z.toInt < 0 :=
      (SignedOrbit.nonnegFlag_eq_false_iff z).mp hflagFalse
    have hzabs_ne : z.abs ≠ DistinctionNat.zero := by
```

```
      apply SignedOrbit.abs_ne_zero_of_toInt_ne_zero
      omega
    have hq_ne :
        DistinctionNat.quotient z.abs d hd ≠ DistinctionNat.zero :=
      DistinctionNat.quotient_ne_zero_of_divides
        (n := z.abs) (d := d) hd hdiv hzabs_ne
    have hq_pos : 0 < (DistinctionNat.quotient z.abs d hd).toNat := by
      have hq_nat_ne : (DistinctionNat.quotient z.abs d hd).toNat ≠ 0 := by
        intro hzero
        apply hq_ne
        apply DistinctionNat.toNat_inj
        rw [hzero, DistinctionNat.toNat_zero]
      omega
    constructor
    · simp [hflagFalse, SignedOrbit.negate_toInt,
        SignedOrbit.ofOrbit_toInt]
      exact hq_pos
    · rw [if_neg hflag]
      rw [signedOrbit_neg_ofOrbit_abs_self]

theorem normalizeRatio_reduced_signCanonical (q : RatioOrbit) :
    PRCRatioReducedSignCanonical (DistinctionNat.normalizeRatio q) := by
  constructor
  · exact DistinctionNat.normalizeRatio_coprime q
  · unfold DistinctionNat.normalizeRatio
    exact signedQuotient_signCanonical_of_divides
      q.num (DistinctionNat.gcd q.num.abs q.den)
      (DistinctionNat.gcd_ne_zero_of_right_ne_zero
        q.num.abs q.den q.den_ne_zero)
      (DistinctionNat.gcd_divides_left q.num.abs q.den)

theorem PRCSignedOrbitSignCanonical.eq_of_toInt_eq
    {z w : SignedOrbit}
    (hz : PRCSignedOrbitSignCanonical z)
    (hw : PRCSignedOrbitSignCanonical w)
    (hzw : z.toInt = w.toInt) :
    z = w := by
  rcases hz with (hzNonneg, hzCanon) | (hzNeg, hzCanon)
  · rcases hw with (_hwNonneg, hwCanon) | (hwNeg, _hwCanon)
    · calc
        z = SignedOrbit.ofOrbit z.abs := hzCanon
        _ = SignedOrbit.ofOrbit w.abs := by
          have habs : z.abs = w.abs := by
            apply DistinctionNat.toNat_inj
            rw [SignedOrbit.abs_toNat, SignedOrbit.abs_toNat, hzw]
          exact congrArg SignedOrbit.ofOrbit habs
        _ = w := hwCanon.symm
    · rw [hzw] at hzNonneg
      omega
  · rcases hw with (hwNonneg, _hwCanon) | (_hwNeg, hwCanon)
    · rw [hzw] at hzNeg
      omega
    · calc
        z = SignedOrbit.negate (SignedOrbit.ofOrbit z.abs) := hzCanon
        _ = SignedOrbit.negate (SignedOrbit.ofOrbit w.abs) := by
          have habs : z.abs = w.abs := by
            apply DistinctionNat.toNat_inj
            rw [SignedOrbit.abs_toNat, SignedOrbit.abs_toNat, hzw]
          exact congrArg (fun n => SignedOrbit.negate (SignedOrbit.ofOrbit n)) habs
        _ = w := hwCanon.symm

theorem PRCReducedSignCanonical_den_dvd_of_crossEq
    {q r : RatioOrbit}
    (hq : PRCRatioReducedSignCanonical q)
    (_hr : PRCRatioReducedSignCanonical r)
    (hqr : RatioOrbit.crossEq q r) :
    q.den.toNat ∣ r.den.toNat := by
  have hcrossZ : q.num.toInt * (r.den.toNat : ℤ) =
      r.num.toInt * (q.den.toNat : ℤ) := by
    unfold RatioOrbit.crossEq at hqr
    have hdisplay :=
      (SignedOrbit.balanced_iff_toInt_eq
        (q.num.scaleByNat r.den) (r.num.scaleByNat q.den)).mp hqr
    rw [SignedOrbit.scaleByNat_toInt, SignedOrbit.scaleByNat_toInt] at hdisplay
    exact hdisplay
  have hcrossNat :
      q.num.abs.toNat * r.den.toNat =
        r.num.abs.toNat * q.den.toNat := by
    have h := congrArg Int.natAbs hcrossZ
    rw [Int.natAbs_mul, Int.natAbs_mul,
      ← SignedOrbit.abs_toNat q.num, ← SignedOrbit.abs_toNat r.num,
      Int.natAbs_natCast, Int.natAbs_natCast] at h
    exact h
  have hcop :
      Nat.Coprime q.num.abs.toNat q.den.toNat :=
    (DistinctionNat.coprime_iff_nat_coprime q.num.abs q.den).mp hq.1
  have hdivMul : q.den.toNat ∣ q.num.abs.toNat * r.den.toNat := by
    rw [hcrossNat]
    exact Nat.dvd_mul_left q.den.toNat r.num.abs.toNat
  exact hcop.symm.dvd_of_dvd_mul_left hdivMul

theorem PRCReducedSignCanonical_den_eq_of_crossEq
    {q r : RatioOrbit}
    (hq : PRCRatioReducedSignCanonical q)
    (hr : PRCRatioReducedSignCanonical r)
    (hqr : RatioOrbit.crossEq q r) :
    q.den = r.den := by
  apply DistinctionNat.toNat_inj
  exact Nat.dvd_antisymm
    (PRCReducedSignCanonical_den_dvd_of_crossEq hq hr hqr)
    (PRCReducedSignCanonical_den_dvd_of_crossEq hr hq
```

```
      (RatioOrbit.crossEq_symm hqr))

theorem PRCReducedSignCanonical_num_eq_of_crossEq
    {q r : RatioOrbit}
    (hq : PRCRatioReducedSignCanonical q)
    (hr : PRCRatioReducedSignCanonical r)
    (hqr : RatioOrbit.crossEq q r) :
    q.num = r.num := by
  have hden : q.den = r.den :=
    PRCReducedSignCanonical_den_eq_of_crossEq hq hr hqr
  have hcrossZ : q.num.toInt * (r.den.toNat : ℤ) =
      r.num.toInt * (q.den.toNat : ℤ) := by
    unfold RatioOrbit.crossEq at hqr
    have hdisplay :=
      (SignedOrbit.balanced_iff_toInt_eq
        (q.num.scaleByNat r.den) (r.num.scaleByNat q.den)).mp hqr
    rw [SignedOrbit.scaleByNat_toInt, SignedOrbit.scaleByNat_toInt] at hdisplay
    exact hdisplay
  have hdenInt : (q.den.toNat : ℤ) ≠ 0 := by
    exact_mod_cast q.den_toNat_ne_zero
  have hnum : q.num.toInt = r.num.toInt := by
    rw [← hden] at hcrossZ
    exact mul_right_cancel₀ hdenInt hcrossZ
  exact PRCSignedOrbitSignCanonical.eq_of_toInt_eq hq.2 hr.2 hnum

/-- Reduced sign-canonical uniqueness is the exact remaining raw-display
number-theory blocker for canonical normalization. It says two reduced,
sign-canonical ratio displays with the same cross-multiplication class are
definitionally the same raw ratio orbit. -/
def PRCReducedSignCanonicalRatioUniqueTarget : Prop :=
  ∀ q r : RatioOrbit,
    PRCRatioReducedSignCanonical q →
      PRCRatioReducedSignCanonical r →
        RatioOrbit.crossEq q r →
          q = r

theorem PRCReducedSignCanonicalRatioUniqueTarget_proved :
    PRCReducedSignCanonicalRatioUniqueTarget := by
  intro q r hq hr hqr
  cases q with
  | mk qnum qden qden_ne_zero =>
    cases r with
    | mk rnum rden rden_ne_zero =>
      have hnum :
          qnum = rnum :=
        PRCReducedSignCanonical_num_eq_of_crossEq
          (q := (qnum, qden, qden_ne_zero))
          (r := (rnum, rden, rden_ne_zero)) hq hr hqr
      have hden :
          qden = rden :=
        PRCReducedSignCanonical_den_eq_of_crossEq
          (q := (qnum, qden, qden_ne_zero))
          (r := (rnum, rden, rden_ne_zero)) hq hr hqr
      subst hnum
      subst hden
      rfl

theorem PRCNormalizeRatioCanonicalTarget_of_reduced_signCanonical_unique
    (hunique : PRCReducedSignCanonicalRatioUniqueTarget) :
    PRCNormalizeRatioCanonicalTarget := by
  intro q r hqr
  apply hunique
  · exact normalizeRatio_reduced_signCanonical q
  · exact normalizeRatio_reduced_signCanonical r
  · exact RatioOrbit.crossEq_trans
      (RatioOrbit.crossEq_symm (DistinctionNat.normalizeRatio_crossEq q))
      (RatioOrbit.crossEq_trans hqr (DistinctionNat.normalizeRatio_crossEq r))

theorem PRCNormalizeRatioCanonicalTarget_proved :
    PRCNormalizeRatioCanonicalTarget :=
  PRCNormalizeRatioCanonicalTarget_of_reduced_signCanonical_unique
    PRCReducedSignCanonicalRatioUniqueTarget_proved

theorem PRCCharacterRespectsCrossEq_of_normalizeRatio_canonical
    {χ : RatioOrbit → RatioOrbit}
    (hχ : PRCRatioCharacter χ)
    (hcanon : PRCNormalizeRatioCanonicalTarget) :
    PRCCharacterRespectsCrossEq χ := by
  intro q r hqr
  exact RatioOrbit.crossEq_trans
    (hχ.normalized_invariant q)
    (RatioOrbit.crossEq_trans
      (by
        rw [hcanon q r hqr]
        exact RatioOrbit.crossEq_refl (χ (DistinctionNat.normalizeRatio r)))
      (RatioOrbit.crossEq_symm (hχ.normalized_invariant r)))

theorem PRCCharacterOrbitProductDisplayCompatible_of_crossEq_respect
    {χ : RatioOrbit → RatioOrbit}
    (hrespect : PRCCharacterRespectsCrossEq χ) :
    PRCCharacterOrbitProductDisplayCompatible χ := by
  intro a b p ha hb hp hmul
  exact hrespect (orbitDirection p hp)
    (RatioOrbit.mul (orbitDirection a ha) (orbitDirection b hb))
    (orbitDirection_mul_crossEq a b p ha hb hp hmul)

/-- Product factors cannot be mixed identity/reciprocal oriented. This is the
exact obstruction left after the pure same-orientation product algebra is
discharged. -/
def PRCCharacterOrbitProductNoMixedOrientation
    (χ : RatioOrbit → RatioOrbit) : Prop :=
```

```
    ∀ a b p : DistinctionNat,
      ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
        ¬ DistinctionNat.unit a →
          ¬ DistinctionNat.unit b →
            ∀ _hp : p ≠ DistinctionNat.zero,
              ¬ DistinctionNat.unit p →
                a * b = p →
                  (¬ (PRCCharacterOrbitDirectionIdentity χ a ha ∧
                    PRCCharacterOrbitDirectionReciprocal χ b hb)) ∧
                  (¬ (PRCCharacterOrbitDirectionReciprocal χ a ha ∧
                    PRCCharacterOrbitDirectionIdentity χ b hb))

/-- Nonunit orbit orientation is coherent when every nonunit orbit direction
chooses the same branch: all identity or all reciprocal. This is the exact
coherence statement strong enough to rule out mixed product factors. -/
def PRCCharacterNonunitOrbitOrientationCoherent
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  (∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      PRCCharacterOrbitDirectionIdentity χ p hp) ∨
  (∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      PRCCharacterOrbitDirectionReciprocal χ p hp)

/-- Cross-nonunit no-mixing: identity orientation at one nonunit orbit direction
cannot coexist with reciprocal orientation at another. This is the branch-coupling
part of global nonunit coherence, separated from local orientation existence. -/
def PRCCharacterNoMixedNonunitOrbitOrientation
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      ∀ r : DistinctionNat, ∀ hr : r ≠ DistinctionNat.zero,
        ¬ DistinctionNat.unit r →
          PRCCharacterOrbitDirectionIdentity χ p hp →
            PRCCharacterOrbitDirectionReciprocal χ r hr →
              False

/-- Positive branch transport form of nonunit coherence: if one nonunit orbit
direction is identity-oriented, every nonunit orbit direction is identity-oriented.
This is the same branch-coupling law as no-mixing once local orientation is known,
but it states the missing transport direction directly. -/
def PRCCharacterNonunitIdentityBranchTransport
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      PRCCharacterOrbitDirectionIdentity χ p hp →
        ∀ r : DistinctionNat, ∀ hr : r ≠ DistinctionNat.zero,
          ¬ DistinctionNat.unit r →
            PRCCharacterOrbitDirectionIdentity χ r hr

/-- Trace-order form of nonunit identity transport: identity orientation at one
nonunit orbit direction transports to another nonunit direction when their
finite δ-orbit traces are comparable. Since orbit traces are structurally
comparable, this is equivalent to global nonunit identity-branch transport, but
it exposes the next proof obligation as a trace-order law. -/
def PRCCharacterNonunitIdentityRespectsComparableTrace
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      ∀ r : DistinctionNat, ∀ hr : r ≠ DistinctionNat.zero,
        ¬ DistinctionNat.unit r →
          (Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) ∨
            Trace.Extends (orbitPositionTrace r) (orbitPositionTrace p)) →
            PRCCharacterOrbitDirectionIdentity χ p hp →
              PRCCharacterOrbitDirectionIdentity χ r hr

theorem orbit_mul_not_unit_of_left_not_unit
    {p r : DistinctionNat} (hunit : ¬ DistinctionNat.unit p) :
    ¬ DistinctionNat.unit (p * r) := by
  intro hprodUnit
  have hprodNat : (p * r).toNat = 1 :=
    (DistinctionNat.unit_iff_toNat_eq_one (p * r)).mp hprodUnit
  have hpNat1 : p.toNat ≠ 1 := by
    intro hone
    exact hunit ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hone)
  rw [DistinctionNat.toNat_mul] at hprodNat
  have hpOne : p.toNat = 1 := Nat.eq_one_of_mul_eq_one_right hprodNat
  exact hpNat1 hpOne

theorem PRCCharacterNonunitOrbitLocalOrientation_of_coherent
    {χ : RatioOrbit → RatioOrbit}
    (hcoh : PRCCharacterNonunitOrbitOrientationCoherent χ) :
    PRCCharacterNonunitOrbitLocalOrientation χ := by
  intro p hp hunit
  rcases hcoh with hallId | hallRec
  · exact Or.inl (hallId p hp hunit)
  · exact Or.inr (hallRec p hp hunit)

theorem PRCCharacterNoMixedNonunitOrbitOrientation_of_coherent
    {χ : RatioOrbit → RatioOrbit}
    (hcoh : PRCCharacterNonunitOrbitOrientationCoherent χ) :
    PRCCharacterNoMixedNonunitOrbitOrientation χ := by
  intro p hp hunit r hr hrUnit hpId hrRec
  rcases hcoh with hallId | hallRec
  · have hrId := hallId r hr hrUnit
    have hself :
        RatioOrbit.crossEq (orbitDirection r hr)
          (RatioOrbit.recip (orbitDirection r hr)) :=
      RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hrId) hrRec
    exact orbitDirection_nonunit_not_crossEq_recip r hr hrUnit hself
  · have hpRec := hallRec p hp hunit
```

```
    have hself :
        RatioOrbit.crossEq (orbitDirection p hp)
          (RatioOrbit.recip (orbitDirection p hp)) :=
      RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
    exact orbitDirection_nonunit_not_crossEq_recip p hp hunit hself

theorem PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
    {χ : RatioOrbit → RatioOrbit}
    (hnomix : PRCCharacterOrbitProductNoMixedOrientation χ) :
    PRCCharacterNoMixedNonunitOrbitOrientation χ := by
  intro p hp hpUnit r hr hrUnit hpId hrRec
  exact ((hnomix p r (p * r) hp hr hpUnit hrUnit
    (DistinctionNat.mul_ne_zero hp hr)
    (orbit_mul_not_unit_of_left_not_unit hpUnit) rfl).1
      (hpId, hrRec))

theorem PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit
    {χ : RatioOrbit → RatioOrbit}
    (hnomix : PRCCharacterNoMixedNonunitOrbitOrientation χ) :
    PRCCharacterOrbitProductNoMixedOrientation χ := by
  intro a b p ha hb haUnit hbUnit _hp _hpUnit _hmul
  constructor
  · rintro (haId, hbRec)
    exact hnomix a ha haUnit b hb hbUnit haId hbRec
  · rintro (haRec, hbId)
    exact hnomix b hb hbUnit a ha haUnit hbId haRec

theorem PRCCharacterOrbitProductNoMixedOrientation_iff_no_mixed_nonunit
    {χ : RatioOrbit → RatioOrbit} :
    PRCCharacterOrbitProductNoMixedOrientation χ ↔
      PRCCharacterNoMixedNonunitOrbitOrientation χ :=
  (PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed,
    PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit)

theorem PRCCharacterNoMixedNonunitOrbitOrientation_of_identity_branch_transport
    {χ : RatioOrbit → RatioOrbit}
    (htransport : PRCCharacterNonunitIdentityBranchTransport χ) :
    PRCCharacterNoMixedNonunitOrbitOrientation χ := by
  intro p hp hpUnit r hr hrUnit hpId hrRec
  have hrId := htransport p hp hpUnit hpId r hr hrUnit
  have hself :
      RatioOrbit.crossEq (orbitDirection r hr)
        (RatioOrbit.recip (orbitDirection r hr)) :=
    RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hrId) hrRec
  exact orbitDirection_nonunit_not_crossEq_recip r hr hrUnit hself

theorem PRCCharacterOrbitProductNoMixedOrientation_of_identity_branch_transport
    {χ : RatioOrbit → RatioOrbit}
    (htransport : PRCCharacterNonunitIdentityBranchTransport χ) :
    PRCCharacterOrbitProductNoMixedOrientation χ :=
  PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit
    (PRCCharacterNoMixedNonunitOrbitOrientation_of_identity_branch_transport
      htransport)

theorem PRCCharacterNonunitIdentityBranchTransport_of_local_no_mixed
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ)
    (hnomix : PRCCharacterNoMixedNonunitOrbitOrientation χ) :
    PRCCharacterNonunitIdentityBranchTransport χ := by
  intro p hp hpUnit hpId r hr hrUnit
  rcases hlocal r hr hrUnit with hrId | hrRec
  · exact hrId
  · exact False.elim (hnomix p hp hpUnit r hr hrUnit hpId hrRec)

theorem PRCCharacterNonunitIdentityBranchTransport_of_coherent
    {χ : RatioOrbit → RatioOrbit}
    (hcoh : PRCCharacterNonunitOrbitOrientationCoherent χ) :
    PRCCharacterNonunitIdentityBranchTransport χ := by
  intro p hp hpUnit hpId r hr hrUnit
  rcases hcoh with hallId | hallRec
  · exact hallId r hr hrUnit
  · have hpRec := hallRec p hp hpUnit
    have hself :
        RatioOrbit.crossEq (orbitDirection p hp)
          (RatioOrbit.recip (orbitDirection p hp)) :=
      RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
    exact False.elim
      (orbitDirection_nonunit_not_crossEq_recip p hp hpUnit hself)

theorem PRCCharacterNonunitIdentityBranchTransport_of_comparable_trace
    {χ : RatioOrbit → RatioOrbit}
    (hcomp : PRCCharacterNonunitIdentityRespectsComparableTrace χ) :
    PRCCharacterNonunitIdentityBranchTransport χ := by
  intro p hp hpUnit hpId r hr hrUnit
  exact hcomp p hp hpUnit r hr hrUnit
    (orbitPositionTrace_comparable p r) hpId

theorem PRCCharacterNonunitIdentityRespectsComparableTrace_of_branch_transport
    {χ : RatioOrbit → RatioOrbit}
    (htransport : PRCCharacterNonunitIdentityBranchTransport χ) :
    PRCCharacterNonunitIdentityRespectsComparableTrace χ := by
  intro p hp hpUnit r hr hrUnit _hcomp hpId
  exact htransport p hp hpUnit hpId r hr hrUnit

theorem PRCCharacterNonunitIdentityRespectsComparableTrace_iff_branch_transport
    {χ : RatioOrbit → RatioOrbit} :
    PRCCharacterNonunitIdentityRespectsComparableTrace χ ↔
      PRCCharacterNonunitIdentityBranchTransport χ :=
  (PRCCharacterNonunitIdentityBranchTransport_of_comparable_trace,
    PRCCharacterNonunitIdentityRespectsComparableTrace_of_branch_transport)

```

```
/-- Two-branch version of the global branch-coupling law: any nonunit
identity-oriented direction transports identity to every nonunit direction, and
any reciprocal-oriented direction transports reciprocal to every nonunit
direction. -/
def PRCCharacterNonunitBranchAgreement
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      ∀ r : DistinctionNat, ∀ hr : r ≠ DistinctionNat.zero,
        ¬ DistinctionNat.unit r →
          (PRCCharacterOrbitDirectionIdentity χ p hp →
            PRCCharacterOrbitDirectionIdentity χ r hr) ∧
          (PRCCharacterOrbitDirectionReciprocal χ p hp →
            PRCCharacterOrbitDirectionReciprocal χ r hr)

theorem PRCCharacterNonunitBranchAgreement_of_coherent
    {χ : RatioOrbit → RatioOrbit}
    (hcoh : PRCCharacterNonunitOrbitOrientationCoherent χ) :
    PRCCharacterNonunitBranchAgreement χ := by
  intro p hp hpUnit r hr hrUnit
  rcases hcoh with hallId | hallRec
  · constructor
    · intro _hpId
      exact hallId r hr hrUnit
    · intro hpRec
      have hpId := hallId p hp hpUnit
      have hself :
          RatioOrbit.crossEq (orbitDirection p hp)
            (RatioOrbit.recip (orbitDirection p hp)) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
      exact False.elim
        (orbitDirection_nonunit_not_crossEq_recip p hp hpUnit hself)
  · constructor
    · intro hpId
      have hpRec := hallRec p hp hpUnit
      have hself :
          RatioOrbit.crossEq (orbitDirection p hp)
            (RatioOrbit.recip (orbitDirection p hp)) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
      exact False.elim
        (orbitDirection_nonunit_not_crossEq_recip p hp hpUnit hself)
    · intro _hpRec
      exact hallRec r hr hrUnit

theorem PRCCharacterNonunitOrbitOrientationCoherent_of_local_branch_agreement
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ)
    (hagree : PRCCharacterNonunitBranchAgreement χ) :
    PRCCharacterNonunitOrbitOrientationCoherent χ := by
  by_cases hId :
      ∃ p : DistinctionNat, ∃ hp : p ≠ DistinctionNat.zero,
        ∃ hunit : ¬ DistinctionNat.unit p,
          PRCCharacterOrbitDirectionIdentity χ p hp
  · rcases hId with (p0, hp0, hunit0, hp0Id)
    exact Or.inl (by
      intro q hq hqUnit
      exact (hagree p0 hp0 hunit0 q hq hqUnit).1 hp0Id)
  · exact Or.inr (by
      intro q hq hqUnit
      rcases hlocal q hq hqUnit with hqId | hqRec
      · exact False.elim (hId (q, hq, hqUnit, hqId))
      · exact hqRec)

theorem PRCCharacterNonunitBranchAgreement_iff_coherent_of_local
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ) :
    PRCCharacterNonunitBranchAgreement χ ↔
      PRCCharacterNonunitOrbitOrientationCoherent χ :=
  (PRCCharacterNonunitOrbitOrientationCoherent_of_local_branch_agreement
      hlocal,
    PRCCharacterNonunitBranchAgreement_of_coherent)

theorem PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ)
    (hnomix : PRCCharacterNoMixedNonunitOrbitOrientation χ) :
    PRCCharacterNonunitOrbitOrientationCoherent χ := by
  by_cases hId :
      ∃ p : DistinctionNat, ∃ hp : p ≠ DistinctionNat.zero,
        ∃ hunit : ¬ DistinctionNat.unit p,
          PRCCharacterOrbitDirectionIdentity χ p hp
  · rcases hId with (p0, hp0, hunit0, hp0Id)
    exact Or.inl (by
      intro q hq hqUnit
      rcases hlocal q hq hqUnit with hqId | hqRec
      · exact hqId
      · exact False.elim (hnomix p0 hp0 hunit0 q hq hqUnit hp0Id hqRec))
  · exact Or.inr (by
      intro q hq hqUnit
      rcases hlocal q hq hqUnit with hqId | hqRec
      · exact False.elim (hId (q, hq, hqUnit, hqId))
      · exact hqRec)

theorem PRCCharacterNonunitOrbitOrientationCoherent_of_local_identity_branch_transport
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ)
    (htransport : PRCCharacterNonunitIdentityBranchTransport χ) :
    PRCCharacterNonunitOrbitOrientationCoherent χ :=
  PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed hlocal
    (PRCCharacterNoMixedNonunitOrbitOrientation_of_identity_branch_transport
      htransport)
```

```
theorem PRCCharacterOrbitProductNoMixedOrientation_of_nonunit_coherent
    {χ : RatioOrbit → RatioOrbit}
    (hcoh : PRCCharacterNonunitOrbitOrientationCoherent χ) :
    PRCCharacterOrbitProductNoMixedOrientation χ := by
  intro a b p ha hb haUnit hbUnit _hp _hpUnit _hmul
  rcases hcoh with hallId | hallRec
  · constructor
    · rintro (_haId, hbRec)
      have hbId := hallId b hb hbUnit
      have hself :
          RatioOrbit.crossEq (orbitDirection b hb)
            (RatioOrbit.recip (orbitDirection b hb)) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hbId) hbRec
      exact orbitDirection_nonunit_not_crossEq_recip b hb hbUnit hself
    · rintro (haRec, _hbId)
      have haId := hallId a ha haUnit
      have hself :
          RatioOrbit.crossEq (orbitDirection a ha)
            (RatioOrbit.recip (orbitDirection a ha)) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm haId) haRec
      exact orbitDirection_nonunit_not_crossEq_recip a ha haUnit hself
  · constructor
    · rintro (haId, _hbRec)
      have haRec := hallRec a ha haUnit
      have hself :
          RatioOrbit.crossEq (orbitDirection a ha)
            (RatioOrbit.recip (orbitDirection a ha)) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm haId) haRec
      exact orbitDirection_nonunit_not_crossEq_recip a ha haUnit hself
    · rintro (_haRec, hbId)
      have hbRec := hallRec b hb hbUnit
      have hself :
          RatioOrbit.crossEq (orbitDirection b hb)
            (RatioOrbit.recip (orbitDirection b hb)) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hbId) hbRec
      exact orbitDirection_nonunit_not_crossEq_recip b hb hbUnit hself

theorem PRCCharacterOrbitProductIdentityIdentity
    {χ : RatioOrbit → RatioOrbit}
    (hχ : PRCRatioCharacter χ)
    (hcompat : PRCCharacterOrbitProductDisplayCompatible χ)
    {a b p : DistinctionNat}
    (ha : a ≠ DistinctionNat.zero) (hb : b ≠ DistinctionNat.zero)
    (hp : p ≠ DistinctionNat.zero)
    (hmul : a * b = p)
    (haId : PRCCharacterOrbitDirectionIdentity χ a ha)
    (hbId : PRCCharacterOrbitDirectionIdentity χ b hb) :
    PRCCharacterOrbitDirectionIdentity χ p hp := by
  unfold PRCCharacterOrbitDirectionIdentity at *
  exact RatioOrbit.crossEq_trans
    (hcompat a b p ha hb hp hmul)
    (RatioOrbit.crossEq_trans
      (hχ.multiplicative (orbitDirection a ha) (orbitDirection b hb))
      (RatioOrbit.crossEq_trans
        (ratioOrbit_mul_congr haId hbId)
        (RatioOrbit.crossEq_symm
          (orbitDirection_mul_crossEq a b p ha hb hp hmul))))

theorem PRCCharacterOrbitProductReciprocalReciprocal
    {χ : RatioOrbit → RatioOrbit}
    (hχ : PRCRatioCharacter χ)
    (hcompat : PRCCharacterOrbitProductDisplayCompatible χ)
    {a b p : DistinctionNat}
    (ha : a ≠ DistinctionNat.zero) (hb : b ≠ DistinctionNat.zero)
    (hp : p ≠ DistinctionNat.zero)
    (hmul : a * b = p)
    (haRec : PRCCharacterOrbitDirectionReciprocal χ a ha)
    (hbRec : PRCCharacterOrbitDirectionReciprocal χ b hb) :
    PRCCharacterOrbitDirectionReciprocal χ p hp := by
  unfold PRCCharacterOrbitDirectionReciprocal at *
  exact RatioOrbit.crossEq_trans
    (hcompat a b p ha hb hp hmul)
    (RatioOrbit.crossEq_trans
      (hχ.multiplicative (orbitDirection a ha) (orbitDirection b hb))
      (RatioOrbit.crossEq_trans
        (ratioOrbit_mul_congr haRec hbRec)
        (RatioOrbit.crossEq_trans
          (ratioOrbit_mul_recip_recip_crossEq_recip_mul
            (orbitDirection a ha) (orbitDirection b hb))
          (ratioOrbit_recip_congr
            (RatioOrbit.crossEq_symm
              (orbitDirection_mul_crossEq a b p ha hb hp hmul))))))

theorem PRCCharacterOrbitProductLocalOrientationPropagates_of_display_compatible_nomix
    {χ : RatioOrbit → RatioOrbit}
    (hχ : PRCRatioCharacter χ)
    (hcompat : PRCCharacterOrbitProductDisplayCompatible χ)
    (hnomix : PRCCharacterOrbitProductNoMixedOrientation χ) :
    PRCCharacterOrbitProductLocalOrientationPropagates χ := by
  intro a b p ha hb haUnit hbUnit hp hpUnit hmul haLocal hbLocal
  rcases haLocal with haId | haRec
  · rcases hbLocal with hbId | hbRec
    · exact Or.inl
        (PRCCharacterOrbitProductIdentityIdentity hχ hcompat
          ha hb hp hmul haId hbId)
    · exact False.elim
        ((hnomix a b p ha hb haUnit hbUnit hp hpUnit hmul).1
          (haId, hbRec))
  · rcases hbLocal with hbId | hbRec
    · exact False.elim
```

```
        ((hnomix a b p ha hb haUnit hbUnit hp hpUnit hmul).2
          (haRec, hbId))
    · exact Or.inr
        (PRCCharacterOrbitProductReciprocalReciprocal hχ hcompat
          ha hb hp hmul haRec hbRec)

theorem PRCCharacterNonunitOrbitLocalOrientation_of_prime_and_product_local
    {χ : RatioOrbit → RatioOrbit}
    (hprimeLocal : PRCCharacterPrimeLocalOrientation χ)
    (hprod : PRCCharacterOrbitProductLocalOrientationPropagates χ) :
    PRCCharacterNonunitOrbitLocalOrientation χ := by
  intro p hp hunit
  let P : Nat → Prop := fun n =>
    ∀ q : DistinctionNat, q.toNat = n →
      (hq : q ≠ DistinctionNat.zero) →
        ¬ DistinctionNat.unit q →
          PRCCharacterOrbitDirectionIdentity χ q hq ∨
            PRCCharacterOrbitDirectionReciprocal χ q hq
  have hP : ∀ n : Nat, (∀ m : Nat, m < n → P m) → P n := by
    intro n ih q hqNat hq0 hqUnit
    by_cases hqPrime : DistinctionNat.primeOrbit q
    · simpa [primeDirection] using hprimeLocal q hqPrime
    · have hfac : DistinctionNat.nontrivialFactorization q := by
        by_contra hnotFac
        exact hqPrime (hq0, hqUnit, hnotFac)
      rcases hfac with (a, b, ha0, hb0, haUnit, hbUnit, hmul)
      have hmulNat : a.toNat * b.toNat = n := by
        have hnat := congrArg DistinctionNat.toNat hmul
        rw [DistinctionNat.toNat_mul, hqNat] at hnat
        exact hnat
      have haNat0 : a.toNat ≠ 0 := by
        intro hz
        apply ha0
        apply DistinctionNat.toNat_inj
        rw [hz, DistinctionNat.toNat_zero]
      have hbNat0 : b.toNat ≠ 0 := by
        intro hz
        apply hb0
        apply DistinctionNat.toNat_inj
        rw [hz, DistinctionNat.toNat_zero]
      have haNat1 : a.toNat ≠ 1 := by
        intro hone
        exact haUnit ((DistinctionNat.unit_iff_toNat_eq_one a).mpr hone)
      have hbNat1 : b.toNat ≠ 1 := by
        intro hone
        exact hbUnit ((DistinctionNat.unit_iff_toNat_eq_one b).mpr hone)
      have haPos : 0 < a.toNat := by omega
      have hbPos : 0 < b.toNat := by omega
      have haGtOne : 1 < a.toNat := by omega
      have hbGtOne : 1 < b.toNat := by omega
      have ha_lt : a.toNat < n := by
        calc
          a.toNat = a.toNat * 1 := by rw [Nat.mul_one]
          _ < a.toNat * b.toNat :=
            Nat.mul_lt_mul_of_pos_left hbGtOne haPos
          _ = n := hmulNat
      have hb_lt : b.toNat < n := by
        calc
          b.toNat = 1 * b.toNat := by rw [Nat.one_mul]
          _ < a.toNat * b.toNat :=
            Nat.mul_lt_mul_of_pos_right haGtOne hbPos
          _ = n := hmulNat
      have haLocal :
          PRCCharacterOrbitDirectionIdentity χ a ha0 ∨
            PRCCharacterOrbitDirectionReciprocal χ a ha0 :=
        ih a.toNat ha_lt a rfl ha0 haUnit
      have hbLocal :
          PRCCharacterOrbitDirectionIdentity χ b hb0 ∨
            PRCCharacterOrbitDirectionReciprocal χ b hb0 :=
        ih b.toNat hb_lt b rfl hb0 hbUnit
      exact hprod a b q ha0 hb0 haUnit hbUnit hq0 hqUnit hmul
        haLocal hbLocal
  have hmain : P p.toNat :=
    Nat.strong_induction_on (p := P) p.toNat hP
  exact hmain p rfl hp hunit

/-- Adjacent nonunit orbit steps cannot mix identity on one side with reciprocal
orientation on the other. This is the prime-floor version of the no-mixed
orientation law. -/
def PRCCharacterPrimeFloorNoAdjacentMixedOrientation
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      (¬ (PRCCharacterOrbitDirectionIdentity χ p hp ∧
        PRCCharacterOrbitDirectionReciprocal χ
          (DistinctionNat.succ p) (orbit_succ_ne_zero p))) ∧
      (¬ (PRCCharacterOrbitDirectionReciprocal χ p hp ∧
        PRCCharacterOrbitDirectionIdentity χ
          (DistinctionNat.succ p) (orbit_succ_ne_zero p)))

theorem PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_nonunit_coherent
    {χ : RatioOrbit → RatioOrbit}
    (hcoh : PRCCharacterNonunitOrbitOrientationCoherent χ) :
    PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ := by
  intro p hp hunit
  have hsuccUnit :
      ¬ DistinctionNat.unit (DistinctionNat.succ p) :=
    orbit_succ_not_unit_of_nonzero_not_unit p hp hunit
  rcases hcoh with hallId | hallRec
  · constructor
    · rintro (_hpId, hsuccRec)
```

```
      have hsuccId :=
        hallId (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit
      have hself :
          RatioOrbit.crossEq
            (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
            (RatioOrbit.recip
              (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hsuccId) hsuccRec
      exact orbitDirection_nonunit_not_crossEq_recip
        (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit hself
    · rintro (hpRec, _hsuccId)
      have hpId := hallId p hp hunit
      have hself :
          RatioOrbit.crossEq (orbitDirection p hp)
            (RatioOrbit.recip (orbitDirection p hp)) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
      exact orbitDirection_nonunit_not_crossEq_recip p hp hunit hself
  · constructor
    · rintro (hpId, _hsuccRec)
      have hpRec := hallRec p hp hunit
      have hself :
          RatioOrbit.crossEq (orbitDirection p hp)
            (RatioOrbit.recip (orbitDirection p hp)) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
      exact orbitDirection_nonunit_not_crossEq_recip p hp hunit hself
    · rintro (_hpRec, hsuccId)
      have hsuccRec :=
        hallRec (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit
      have hself :
          RatioOrbit.crossEq
            (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
            (RatioOrbit.recip
              (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))) :=
        RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hsuccId) hsuccRec
      exact orbitDirection_nonunit_not_crossEq_recip
        (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit hself

/-- Forward one-step identity transport above the unit floor. The unit orbit is
self-reciprocal, so forcing transport out of `1` would wrongly exclude the
globally reciprocal character branch. -/
def PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      PRCCharacterOrbitDirectionIdentity χ p hp →
        PRCCharacterOrbitDirectionIdentity χ
          (DistinctionNat.succ p) (orbit_succ_ne_zero p)

/-- Backward one-step identity transport above the unit floor. -/
def PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
    ¬ DistinctionNat.unit p →
      PRCCharacterOrbitDirectionIdentity χ
        (DistinctionNat.succ p) (orbit_succ_ne_zero p) →
          PRCCharacterOrbitDirectionIdentity χ p hp

/-- The corrected successor-transport rule: identity orientation transports
along one δ-successor step only once the path is above the self-reciprocal unit
orbit. This is the exact layer needed for prime-to-prime trace coherence. -/
def PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport
    (χ : RatioOrbit → RatioOrbit) : Prop :=
  PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep χ ∧
    PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep χ

theorem PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep_of_local_adjacent_nomix
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ)
    (hnomix : PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ) :
    PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep χ := by
  intro p hp hunit hpId
  have hsuccUnit :
      ¬ DistinctionNat.unit (DistinctionNat.succ p) :=
    orbit_succ_not_unit_of_nonzero_not_unit p hp hunit
  rcases hlocal (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit
    with hsuccId | hsuccRec
  · exact hsuccId
  · exact False.elim ((hnomix p hp hunit).1 (hpId, hsuccRec))

theorem PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep_of_local_adjacent_nomix
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ)
    (hnomix : PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ) :
    PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep χ := by
  intro p hp hunit hsuccId
  rcases hlocal p hp hunit with hpId | hpRec
  · exact hpId
  · exact False.elim ((hnomix p hp hunit).2 (hpRec, hsuccId))

theorem PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ)
    (hnomix : PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ) :
    PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ :=
  ⟨PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep_of_local_adjacent_nomix
      hlocal hnomix,
    PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep_of_local_adjacent_nomix
      hlocal hnomix⟩

theorem PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_successor_transport
    {χ : RatioOrbit → RatioOrbit}
```

```
    (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
    PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ := by
  intro p hp hunit
  have hsuccUnit :
      ¬ DistinctionNat.unit (DistinctionNat.succ p) :=
    orbit_succ_not_unit_of_nonzero_not_unit p hp hunit
  constructor
  · rintro (hpId, hsuccRec)
    have hsuccId :
        PRCCharacterOrbitDirectionIdentity χ
          (DistinctionNat.succ p) (orbit_succ_ne_zero p) :=
      hstep.1 p hp hunit hpId
    have hself :
        RatioOrbit.crossEq
          (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
          (RatioOrbit.recip
            (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))) :=
      RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hsuccId) hsuccRec
    exact orbitDirection_nonunit_not_crossEq_recip
      (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit hself
  · rintro (hpRec, hsuccId)
    have hpId : PRCCharacterOrbitDirectionIdentity χ p hp :=
      hstep.2 p hp hunit hsuccId
    have hself :
        RatioOrbit.crossEq (orbitDirection p hp)
          (RatioOrbit.recip (orbitDirection p hp)) :=
      RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
    exact orbitDirection_nonunit_not_crossEq_recip p hp hunit hself

theorem PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_iff_local_adjacent_nomix
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ) :
    PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ ↔
      PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ :=
  (PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_successor_transport,
    PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
      hlocal)

theorem PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_nonunit_identity_comparable_trace
    {χ : RatioOrbit → RatioOrbit}
    (hcomp : PRCCharacterNonunitIdentityRespectsComparableTrace χ) :
    PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ := by
  constructor
  · intro p hp hunit hpId
    have hsuccUnit :
        ¬ DistinctionNat.unit (DistinctionNat.succ p) :=
      orbit_succ_not_unit_of_nonzero_not_unit p hp hunit
    exact hcomp p hp hunit (DistinctionNat.succ p)
      (orbit_succ_ne_zero p) hsuccUnit
      (orbitPositionTrace_comparable p (DistinctionNat.succ p)) hpId
  · intro p hp hunit hsuccId
    have hsuccUnit :
        ¬ DistinctionNat.unit (DistinctionNat.succ p) :=
      orbit_succ_not_unit_of_nonzero_not_unit p hp hunit
    exact hcomp (DistinctionNat.succ p) (orbit_succ_ne_zero p)
      hsuccUnit p hp hunit
      (orbitPositionTrace_comparable (DistinctionNat.succ p) p) hsuccId

theorem PRCCharacterOrbitIdentity_of_le_of_prime_floor_successor_transport
    {χ : RatioOrbit → RatioOrbit}
    (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
    ∀ {p r : DistinctionNat}, (hp : p ≠ DistinctionNat.zero) →
      ¬ DistinctionNat.unit p →
        (hr : r ≠ DistinctionNat.zero) →
          p.toNat ≤ r.toNat →
            PRCCharacterOrbitDirectionIdentity χ p hp →
              PRCCharacterOrbitDirectionIdentity χ r hr := by
  intro p r
  revert p
  induction r with
  | zero =>
      intro p hp _hpu hr _hle _hpId
      exact False.elim (hr rfl)
  | succ n ih =>
      intro p hp hpu _hr hle hpId
      by_cases hEq : p = DistinctionNat.succ n
      · subst p
        simpa [PRCCharacterOrbitDirectionIdentity, orbitDirection] using hpId
      · have hle_n : p.toNat ≤ n.toNat := by
          have hnotNat : p.toNat ≠ Nat.succ n.toNat := by
            intro hnat
            apply hEq
            apply DistinctionNat.toNat_inj
            simpa [DistinctionNat.toNat_succ] using hnat
          rw [DistinctionNat.toNat_succ] at hle
          omega
        have hpNat0 : p.toNat ≠ 0 := by
          intro hz
          apply hp
          apply DistinctionNat.toNat_inj
          rw [hz, DistinctionNat.toNat_zero]
        have hpNat1 : p.toNat ≠ 1 := by
          intro hone
          exact hpu ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hone)
        have hn0 : n ≠ DistinctionNat.zero := by
          intro hn
          have hnNat : n.toNat = 0 := by rw [hn, DistinctionNat.toNat_zero]
          omega
        have hnunit : ¬ DistinctionNat.unit n := by
          intro hunit
          have hnNat1 : n.toNat = 1 :=
```

```
            (DistinctionNat.unit_iff_toNat_eq_one n).mp hunit
          omega
        have hnId :
            PRCCharacterOrbitDirectionIdentity χ n hn0 :=
          ih hp hpu hn0 hle_n hpId
        exact hstep.1 n hn0 hnunit hnId

theorem PRCCharacterOrbitIdentity_of_ge_of_prime_floor_successor_transport
    {χ : RatioOrbit → RatioOrbit}
    (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
    ∀ {p r : DistinctionNat}, (hp : p ≠ DistinctionNat.zero) →
      ¬ DistinctionNat.unit p →
        (hr : r ≠ DistinctionNat.zero) →
          ¬ DistinctionNat.unit r →
            r.toNat ≤ p.toNat →
              PRCCharacterOrbitDirectionIdentity χ p hp →
                PRCCharacterOrbitDirectionIdentity χ r hr := by
  intro p r
  revert r
  induction p with
  | zero =>
      intro r hp _hpu _hr _hru _hge _hpId
      exact False.elim (hp rfl)
  | succ n ih =>
      intro r _hp _hpu hr hru hge hpId
      by_cases hEq : r = DistinctionNat.succ n
      · subst r
        simpa [PRCCharacterOrbitDirectionIdentity, orbitDirection] using hpId
      · have hge_n : r.toNat ≤ n.toNat := by
          have hnotNat : r.toNat ≠ Nat.succ n.toNat := by
            intro hnat
            apply hEq
            apply DistinctionNat.toNat_inj
            simpa [DistinctionNat.toNat_succ] using hnat
          rw [DistinctionNat.toNat_succ] at hge
          omega
        have hrNat0 : r.toNat ≠ 0 := by
          intro hz
          apply hr
          apply DistinctionNat.toNat_inj
          rw [hz, DistinctionNat.toNat_zero]
        have hrNat1 : r.toNat ≠ 1 := by
          intro hone
          exact hru ((DistinctionNat.unit_iff_toNat_eq_one r).mpr hone)
        have hn0 : n ≠ DistinctionNat.zero := by
          intro hn
          have hnNat : n.toNat = 0 := by rw [hn, DistinctionNat.toNat_zero]
          omega
        have hnunit : ¬ DistinctionNat.unit n := by
          intro hunit
          have hnNat1 : n.toNat = 1 :=
            (DistinctionNat.unit_iff_toNat_eq_one n).mp hunit
          omega
        have hsuccId :
            PRCCharacterOrbitDirectionIdentity χ
              (DistinctionNat.succ n) (orbit_succ_ne_zero n) := by
          simpa [PRCCharacterOrbitDirectionIdentity, orbitDirection] using hpId
        have hnId :
            PRCCharacterOrbitDirectionIdentity χ n hn0 :=
          hstep.2 n hn0 hnunit hsuccId
        exact ih hn0 hnunit hr hru hge_n hnId

theorem PRCCharacterPrimeIdentityRespectsComparableTrace_of_prime_floor_successor_transport
    {χ : RatioOrbit → RatioOrbit}
    (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
    PRCCharacterPrimeIdentityRespectsComparableTrace χ := by
  intro p hp r hr _hcomp hpId
  have hpOrbitId :
      PRCCharacterOrbitDirectionIdentity χ p hp.1 := by
    exact hpId
  rcases Nat.le_total p.toNat r.toNat with hle | hge
  · exact PRCCharacterOrbitIdentity_of_le_of_prime_floor_successor_transport
      hstep hp.1 hp.2.1 hr.1 hle hpOrbitId
  · exact PRCCharacterOrbitIdentity_of_ge_of_prime_floor_successor_transport
      hstep hp.1 hp.2.1 hr.1 hr.2.1 hge hpOrbitId

theorem PRCCharacterNonunitIdentityRespectsComparableTrace_of_prime_floor_successor_transport
    {χ : RatioOrbit → RatioOrbit}
    (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
    PRCCharacterNonunitIdentityRespectsComparableTrace χ := by
  intro p hp hpUnit r hr hrUnit _hcomp hpId
  rcases Nat.le_total p.toNat r.toNat with hle | hge
  · exact PRCCharacterOrbitIdentity_of_le_of_prime_floor_successor_transport
      hstep hp hpUnit hr hle hpId
  · exact PRCCharacterOrbitIdentity_of_ge_of_prime_floor_successor_transport
      hstep hp hpUnit hr hrUnit hge hpId

theorem PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_prime_floor_successor_transport
    {χ : RatioOrbit → RatioOrbit}
    (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ)
    (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
    PRCCharacterNonunitOrbitOrientationCoherent χ := by
  by_cases hId :
      ∃ p : DistinctionNat, ∃ hp : p ≠ DistinctionNat.zero,
        ∃ hunit : ¬ DistinctionNat.unit p,
          PRCCharacterOrbitDirectionIdentity χ p hp
  · rcases hId with ⟨p0, hp0, hunit0, hp0Id⟩
    exact Or.inl (by
      intro q hq hqUnit
      rcases Nat.le_total p0.toNat q.toNat with hle | hge
      · exact PRCCharacterOrbitIdentity_of_le_of_prime_floor_successor_transport
```

```
          hstep hp0 hunit0 hq hle hp0Id
      · exact PRCCharacterOrbitIdentity_of_ge_of_prime_floor_successor_transport
          hstep hp0 hunit0 hq hqUnit hge hp0Id)
  · exact Or.inr (by
      intro q hq hqUnit
      rcases hlocal q hq hqUnit with hqId | hqRec
      · exact False.elim (hId ⟨q, hq, hqUnit, hqId⟩)
      · exact hqRec)

theorem PRCCharacterOrbitIdentityRespectsSuccessorStep_of_transport
    {χ : RatioOrbit → RatioOrbit}
    (htransport : PRCCharacterOrbitIdentitySuccessorTransport χ) :
    PRCCharacterOrbitIdentityRespectsSuccessorStep χ := by
  intro p hp
  exact ⟨htransport.1 p hp, htransport.2 p hp⟩

theorem PRCCharacterOrbitIdentity_one_of_identity
    {χ : RatioOrbit → RatioOrbit}
    (hstep : PRCCharacterOrbitIdentityRespectsSuccessorStep χ)
    {p : DistinctionNat} (hp : p ≠ DistinctionNat.zero)
    (hpId : PRCCharacterOrbitDirectionIdentity χ p hp) :
    PRCCharacterOrbitDirectionIdentity χ
      DistinctionNat.one DistinctionNat.one_ne_zero := by
  induction p with
  | zero =>
      exact False.elim (hp rfl)
  | succ n ih =>
      cases n with
      | zero =>
          exact hpId
      | succ m =>
          have hn : DistinctionNat.succ m ≠ DistinctionNat.zero :=
            orbit_succ_ne_zero m
          have hnId :
              PRCCharacterOrbitDirectionIdentity χ
                (DistinctionNat.succ m) hn :=
            (hstep (DistinctionNat.succ m) hn).2 hpId
          exact ih hn hnId

theorem PRCCharacterOrbitIdentity_of_one
    {χ : RatioOrbit → RatioOrbit}
    (hstep : PRCCharacterOrbitIdentityRespectsSuccessorStep χ)
    {r : DistinctionNat} (hr : r ≠ DistinctionNat.zero)
    (honeId : PRCCharacterOrbitDirectionIdentity χ
      DistinctionNat.one DistinctionNat.one_ne_zero) :
    PRCCharacterOrbitDirectionIdentity χ r hr := by
  induction r with
  | zero =>
      exact False.elim (hr rfl)
  | succ n ih =>
      cases n with
      | zero =>
          exact honeId
      | succ m =>
          have hn : DistinctionNat.succ m ≠ DistinctionNat.zero :=
            orbit_succ_ne_zero m
          have hnId :
              PRCCharacterOrbitDirectionIdentity χ
                (DistinctionNat.succ m) hn :=
            ih hn
          exact (hstep (DistinctionNat.succ m) hn).1 hnId

theorem PRCCharacterPrimeIdentityRespectsComparableTrace_of_successor_step
    {χ : RatioOrbit → RatioOrbit}
    (hstep : PRCCharacterOrbitIdentityRespectsSuccessorStep χ) :
    PRCCharacterPrimeIdentityRespectsComparableTrace χ := by
  intro p hp r hr _hcomp hpId
  have hpOrbitId :
      PRCCharacterOrbitDirectionIdentity χ p hp.1 := by
    exact hpId
  have honeId :
      PRCCharacterOrbitDirectionIdentity χ
        DistinctionNat.one DistinctionNat.one_ne_zero :=
    PRCCharacterOrbitIdentity_one_of_identity hstep hp.1 hpOrbitId
  exact PRCCharacterOrbitIdentity_of_one hstep hr.1 honeId

theorem PRCCharacterPrimeIdentityRespectsCommonTraceExtension_of_comparable_trace
    {χ : RatioOrbit → RatioOrbit}
    (hcomp : PRCCharacterPrimeIdentityRespectsComparableTrace χ) :
    PRCCharacterPrimeIdentityRespectsCommonTraceExtension χ := by
  intro p hp r hr _T _hpT _hrT hpId
  exact hcomp p hp r hr (orbitPositionTrace_comparable p r) hpId

theorem PRCCharacterPrimeIdentityRespectsTraceConnected_of_common_trace_extension
    {χ : RatioOrbit → RatioOrbit}
    (hcommon : PRCCharacterPrimeIdentityRespectsCommonTraceExtension χ) :
    PRCCharacterPrimeIdentityRespectsTraceConnected χ := by
  intro p hp r hr hconn hpId
  rcases hconn with ⟨T, hpT, hrT⟩
  exact hcommon p hp r hr T hpT hrT hpId

/-- Local orientation target: prime cost calibration must at least orient each
prime axis as identity or reciprocal. -/
def PRCPrimeCalibrationForcesLocalPrimeOrientationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterPrimeLocalOrientation χ

theorem PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved :
    PRCPrimeCalibrationForcesLocalPrimeOrientationTarget := by
```

```
  intro χ hχ hprime p hp
  have hq : (primeDirection p hp).toRat ≠ 0 :=
    primeDirection_toRat_ne_zero p hp
  have hχq : (χ (primeDirection p hp)).toRat ≠ 0 :=
    hχ.nonzero_preserving hq
  have hcal :
      RatioOrbit.crossEq
        (onRatioOrbit (χ (primeDirection p hp)))
        (onRatioOrbit (primeDirection p hp)) := by
    simpa [costFromCharacter] using hprime p hp
  exact jcost_eq_forces_same_or_reciprocal hχq hq hcal

/-- No-mixing target: prime cost calibration must forbid independent mixed
identity/reciprocal choices on different prime axes. -/
def PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterNoMixedPrimeOrientation χ

/-- Exact remaining trace-coherence target: prime calibration must make identity
orientation propagate across prime axes. -/
def PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterPrimeIdentityTraceCoherent χ

/-- Smaller trace-transport target: prime calibration should make identity
orientation invariant along native prime-axis trace connections. The structural
connectivity of the prime-axis trace graph is already proved above. -/
def PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterPrimeIdentityRespectsTraceConnected χ

/-- Sharper form of the trace-transport target: prime calibration must force
identity orientation to respect an explicitly witnessed common finite δ-trace
extension. -/
def PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterPrimeIdentityRespectsCommonTraceExtension χ

/-- Sharper trace-order target: prime calibration should force identity
orientation to respect comparability of finite δ-orbit traces. -/
def PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterPrimeIdentityRespectsComparableTrace χ

/-- Sharper one-step target: prime calibration should force identity orientation
to be invariant under one successor step on every nonzero orbit direction. -/
def PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterOrbitIdentityRespectsSuccessorStep χ

/-- Sharper directional successor target: prime calibration should force both
one-step directions separately. This exposes the exact additive-trace
compatibility missing from the purely multiplicative ratio-character laws. -/
def PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterOrbitIdentitySuccessorTransport χ

/-- Sharper additive successor target: prime calibration should force the ratio
character to respect the additive successor operation on nonzero orbit
directions. -/
def PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterOrbitSuccessorAdditiveCompatible χ

/-- Corrected successor target after the reciprocal-character check: prime
calibration should force successor transport above the self-reciprocal unit
floor, not additive transport out of the unit orbit itself. -/
def PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ

/-- First component of the prime-floor successor blocker: prime calibration
should orient every nonunit orbit direction, not only prime axes. -/
def PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterNonunitOrbitLocalOrientation χ

/-- Sharper source of nonunit local orientation: prime calibration should force
the product-factor propagation step that carries prime-axis orientation through
composite orbit positions. -/
def PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget : Prop :=
```

```
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCRatioCharacter χ →
        PRCCharacterPrimeDirectionCalibrated χ →
          PRCCharacterOrbitProductLocalOrientationPropagates χ

/-- Product-display compatibility target: prime calibration should force the
character to respect the native equality between product orbit directions and
ratio products of factor directions. -/
def PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterOrbitProductDisplayCompatible χ

/-- Sharper source of product-display compatibility: prime calibration should
force the raw character to respect ratio cross-equivalence. Without this,
`χ : RatioOrbit → RatioOrbit` is not yet a quotient-native character. -/
def PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterRespectsCrossEq χ

theorem PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_normalizeRatio_canonical
    (hcanon : PRCNormalizeRatioCanonicalTarget) :
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget := by
  intro χ hχ _hprime
  exact PRCCharacterRespectsCrossEq_of_normalizeRatio_canonical hχ hcanon

theorem PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_reduced_signCanonical_unique
    (hunique : PRCReducedSignCanonicalRatioUniqueTarget) :
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget :=
  PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_normalizeRatio_canonical
    (PRCNormalizeRatioCanonicalTarget_of_reduced_signCanonical_unique hunique)

theorem PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_proved :
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget :=
  PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_reduced_signCanonical_unique
    PRCReducedSignCanonicalRatioUniqueTarget_proved

/-- Product no-mixing target: prime calibration should rule out mixed
identity/reciprocal factor orientations under native multiplication. -/
def PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterOrbitProductNoMixedOrientation χ

/-- Stronger replacement for product no-mixing: prime calibration should force a
single coherent orientation across all nonunit orbit directions. Once this is
available, mixed product factors are impossible by nonunit non-self-reciprocity. -/
def PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterNonunitOrbitOrientationCoherent χ

/-- Branch-coupling target: prime calibration should prevent any identity-oriented
nonunit direction from coexisting with any reciprocal-oriented nonunit direction.
Together with local nonunit orientation this is exactly global nonunit
orientation coherence. -/
def PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterNoMixedNonunitOrbitOrientation χ

/-- Positive transport form of the same branch-coupling blocker: if prime
calibration allows one nonunit direction to remain identity-oriented, that
identity branch must transport to every nonunit direction. -/
def PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterNonunitIdentityBranchTransport χ

/-- Trace-order sharpening of nonunit identity-branch transport: prime
calibration should force identity orientation to respect comparability of finite
δ-orbit traces on nonunit directions. -/
def PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterNonunitIdentityRespectsComparableTrace χ

/-- Two-branch agreement target: prime calibration should force a nonunit branch
choice at one direction to agree with every other nonunit direction, for both
identity and reciprocal branches. -/
def PRCPrimeCalibrationForcesNonunitBranchAgreementTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterNonunitBranchAgreement χ

/-- Local orientation plus two-branch agreement is the positive normal form of
the global nonunit branch-coupling blocker. -/
def PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget :
    Prop :=
  PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
    PRCPrimeCalibrationForcesNonunitBranchAgreementTarget

```

```
/-- Sharpened source of global nonunit coherence: first prove every nonunit orbit
direction has a local branch, then prove the cross-nonunit no-mixing law. -/
def PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget : Prop :=
  PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget

/-- Product-layer sharpening of global nonunit coherence: local nonunit
orientation is already available, so the remaining branch-coupling obligation can
be carried by the product no-mixing law. -/
def PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget : Prop :=
  PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget

theorem PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_coherent
    (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget := by
  intro χ hχ hprime
  exact PRCCharacterNoMixedNonunitOrbitOrientation_of_coherent
    (hcoh χ hχ hprime)

theorem PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed
    (hprod : PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget) :
    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget := by
  intro χ hχ hprime
  exact PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
    (hprod χ hχ hprime)

theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_no_mixed_nonunit
    (hnomix : PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget) :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget := by
  intro χ hχ hprime
  exact PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit
    (hnomix χ hχ hprime)

theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_no_mixed_nonunit :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget :=
  ⟨PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed,
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_no_mixed_nonunit⟩

theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport
    (htransport : PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget) :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget := by
  intro χ hχ hprime
  exact PRCCharacterOrbitProductNoMixedOrientation_of_identity_branch_transport
    (htransport χ hχ hprime)

theorem PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace
    (hcomp : PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget) :
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget := by
  intro χ hχ hprime
  exact PRCCharacterNonunitIdentityBranchTransport_of_comparable_trace
    (hcomp χ hχ hprime)

theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_local_product_no_mixed
    (hsharp : PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget) :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget :=
  ⟨hsharp.1,
    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed
      hsharp.2⟩

theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_product_no_mixed
    (hsharp : PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget) :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
  intro χ hχ hprime
  exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
    (hsharp.1 χ hχ hprime)
    (PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
      (hsharp.2 χ hχ hprime))

theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_coherent
    (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget :=
  ⟨by
      intro χ hχ hprime
      exact PRCCharacterNonunitOrbitLocalOrientation_of_coherent
        (hcoh χ hχ hprime),
    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_coherent hcoh⟩

theorem PRCPrimeCalibrationForcesNonunitBranchAgreementTarget_of_coherent
    (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
    PRCPrimeCalibrationForcesNonunitBranchAgreementTarget := by
  intro χ hχ hprime
  exact PRCCharacterNonunitBranchAgreement_of_coherent (hcoh χ hχ hprime)

theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget_of_coherent
    (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget :=
  ⟨by
      intro χ hχ hprime
      exact PRCCharacterNonunitOrbitLocalOrientation_of_coherent
        (hcoh χ hχ hprime)),
    PRCPrimeCalibrationForcesNonunitBranchAgreementTarget_of_coherent hcoh⟩

theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_branch_agreement
    (hsharp :
      PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget) :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
  intro χ hχ hprime
  exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_branch_agreement
    (hsharp.1 χ hχ hprime) (hsharp.2 χ hχ hprime)
```

```
2111
2112 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_local_branch_agreement :
2113     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
2114       PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget :=
2115   (PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget_of_coherent,
2116     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_branch_agreement)
2117
2118 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_no_mixed
2119     (hsharp : PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget) :
2120     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
2121   intro χ hχ hprime
2122   exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
2123     (hsharp.1 χ hχ hprime) (hsharp.2 χ hχ hprime)
2124
2125 theorem PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_coherent
2126     (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2127     PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget := by
2128   intro χ hχ hprime
2129   exact PRCCharacterNonunitIdentityBranchTransport_of_coherent
2130     (hcoh χ hχ hprime)
2131
2132 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_local_no_mixed :
2133     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
2134       PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget :=
2135   (PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_coherent,
2136     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_no_mixed)
2137
2138 /-- Sharpened source of nonunit orientation coherence: local nonunit orientation
2139 plus prime-floor successor transport force every nonunit orbit direction onto
2140 one coherent identity/reciprocal branch. -/
2141 def PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget : Prop :=
2142   PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
2143     PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
2144
2145 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_and_prime_floor_successor_transport
2146     (hsharp : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget) :
2147     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
2148   intro χ hχ hprime
2149   exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_prime_floor_successor_transport
2150     (hsharp.1 χ hχ hprime) (hsharp.2 χ hχ hprime)
2151
2152 theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_nonunit_coherent
2153     (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2154     PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget := by
2155   intro χ hχ hprime
2156   exact PRCCharacterOrbitProductNoMixedOrientation_of_nonunit_coherent
2157     (hcoh χ hχ hprime)
2158
2159 theorem PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_nonunit_coherent
2160     (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2161     PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget := by
2162   intro χ hχ hprime
2163   exact PRCCharacterNonunitOrbitLocalOrientation_of_coherent
2164     (hcoh χ hχ hprime)
2165
2166 theorem PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_nonunit_coherent
2167     (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2168     PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget := by
2169   intro χ hχ hprime
2170   exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
2171     (PRCCharacterNonunitOrbitLocalOrientation_of_coherent (hcoh χ hχ hprime))
2172     (PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_nonunit_coherent
2173       (hcoh χ hχ hprime))
2174
2175 theorem PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_prime_floor_successor_transport
2176     (hstep : PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :
2177     PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget := by
2178   intro χ hχ hprime
2179   exact PRCCharacterNonunitIdentityRespectsComparableTrace_of_prime_floor_successor_transport
2180     (hstep χ hχ hprime)
2181
2182 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget_of_nonunit_coherent
2183     (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2184     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget :=
2185   (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_nonunit_coherent hcoh,
2186     PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_nonunit_coherent hcoh)
2187
2188 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_sharpened :
2189     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
2190       PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget :=
2191   (PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget_of_nonunit_coherent,
2192     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_and_prime_floor_successor_transport)
2193
2194 theorem PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_identity_comparable_trace
2195     (hcomp : PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget) :
2196     PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget := by
2197   intro χ hχ hprime
2198   exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_nonunit_identity_comparable_trace
2199     (hcomp χ hχ hprime)
2200
2201 theorem PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_iff_prime_floor_successor_transport :
2202     PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget ↔
2203       PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget :=
2204   (PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_identity_comparable_trace,
2205     PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_prime_floor_successor_transport)
2206
2207 /-- Pass-45 sharpening of product-local orientation: same-orientation products
2208 are algebraic and product-display compatibility is proved through canonical
2209 normalization. The remaining product commitment is nonunit orientation
2210 coherence, which implies product no-mixing. -/
2211 def PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget : Prop :=
```

```
  PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget

theorem PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_of_crossEq_respect
    (hrespect : PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget) :
    PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget := by
  intro χ hχ hprime
  exact PRCCharacterOrbitProductDisplayCompatible_of_crossEq_respect
    (hrespect χ hχ hprime)

theorem PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_proved :
    PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget :=
  PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_of_crossEq_respect
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_proved

theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed
    (hprod : PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget) :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
  intro χ hχ hprime
  have hprodLocal : PRCCharacterOrbitProductLocalOrientationPropagates χ :=
    PRCCharacterOrbitProductLocalOrientationPropagates_of_display_compatible_nomix
      hχ (PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_proved
        χ hχ hprime) (hprod χ hχ hprime)
  exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
    (PRCCharacterNonunitOrbitLocalOrientation_of_prime_and_product_local
      (PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved χ hχ hprime)
      hprodLocal)
    (PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
      (hprod χ hχ hprime))

theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_nonunit_coherent :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget :=
  ⟨PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed,
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_nonunit_coherent⟩

theorem PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed
    (hprod : PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget) :
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget :=
  PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_coherent
    (PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed
      hprod)

theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_branch_transport :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget :=
  ⟨PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed,
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport⟩

theorem PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_branch_transport
    (htransport : PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget) :
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget := by
  intro χ hχ hprime
  exact PRCCharacterNonunitIdentityRespectsComparableTrace_of_branch_transport
    (htransport χ hχ hprime)

theorem PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_product_no_mixed
    (hprod : PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget) :
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget :=
  PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_branch_transport
    (PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed
      hprod)

theorem PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_iff_comparable_trace :
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget ↔
      PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget :=
  ⟨PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_branch_transport,
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace⟩

theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_comparable_trace :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget :=
  ⟨PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_product_no_mixed,
    fun hcomp =>
      PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport
        (PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace
          hcomp)⟩

/-- Second component of the prime-floor successor blocker: adjacent nonunit
orbit directions cannot carry opposite identity/reciprocal orientations. -/
def PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ

theorem PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_nonunit_coherent
    (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
    PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget := by
  intro χ hχ hprime
  exact PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_nonunit_coherent
    (hcoh χ hχ hprime)

theorem PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_successor_transport
    (hstep : PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :
    PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget := by
  intro χ hχ hprime
  exact PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_successor_transport
    (hstep χ hχ hprime)

/-- The exact local version of the corrected successor blocker: local nonunit
orientation plus adjacent no-mixing is equivalent to local nonunit orientation
```

```
plus prime-floor successor transport. -/
def PRCPrimeFloorSuccessorTransportLocalAdjacentTarget : Prop :=
  PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
    PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget

/-- Pass-39 refinement of the prime-floor successor target. It separates local
nonunit orientation from adjacent no-mixing instead of bundling both facts under
successor transport. -/
def PRCPrimeFloorSuccessorTransportSharpenedTarget : Prop :=
  PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget ∧
    PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget

theorem PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget_of_additive_compat
    (hadd : PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget) :
    PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget := by
  intro χ hχ hprime
  exact PRCCharacterOrbitIdentitySuccessorTransport_of_additive_compat
    (hadd χ hχ hprime)

theorem PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_nomix
    (hsharp : PRCPrimeFloorSuccessorTransportSharpenedTarget) :
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget := by
  intro χ hχ hprime
  exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
    (PRCCharacterNonunitOrbitLocalOrientation_of_coherent
      (hsharp.1 χ hχ hprime))
    (hsharp.2 χ hχ hprime)

theorem PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_target
    (hsharp : PRCPrimeFloorSuccessorTransportLocalAdjacentTarget) :
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget := by
  intro χ hχ hprime
  exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
    (hsharp.1 χ hχ hprime) (hsharp.2 χ hχ hprime)

theorem PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_local_successor_transport
    (hsharp :
      PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
        PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget :=
  (hsharp.1,
    PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_successor_transport
      hsharp.2)

theorem PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_local_successor_transport :
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
      (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
        PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :=
  ((fun hsharp =>
      (hsharp.1,
        PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_target
          hsharp)),
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_local_successor_transport)

theorem PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_nonunit_coherent
    (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget :=
  (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_nonunit_coherent hcoh,
    PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_nonunit_coherent
      hcoh)

theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_adjacent
    (hsharp : PRCPrimeFloorSuccessorTransportLocalAdjacentTarget) :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
  exact PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_and_prime_floor_successor_transport
    (hsharp.1,
      PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_target
        hsharp)

theorem PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_nonunit_coherent :
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget :=
  (PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_adjacent,
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_nonunit_coherent)

theorem PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_product_local_orientation
    (hprod : PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget) :
    PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget := by
  intro χ hχ hprime
  exact PRCCharacterNonunitOrbitLocalOrientation_of_prime_and_product_local
    (PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved χ hχ hprime)
    (hprod χ hχ hprime)

theorem PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget_of_display_compatible_nomix
    (hsharp : PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget) :
    PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget := by
  intro χ hχ hprime
  exact PRCCharacterOrbitProductLocalOrientationPropagates_of_display_compatible_nomix
    hχ (PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_proved
      χ hχ hprime)
    (PRCCharacterOrbitProductNoMixedOrientation_of_nonunit_coherent
      (hsharp χ hχ hprime))

theorem PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_prime_floor_successor_transport
    (hstep : PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :
    PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget := by
  intro χ hχ hprime
  exact PRCCharacterPrimeIdentityRespectsComparableTrace_of_prime_floor_successor_transport
    (hstep χ hχ hprime)

theorem PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget_of_transport
```

```
    (htransport : PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget) :
    PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget := by
  intro χ hχ hprime
  exact PRCCharacterOrbitIdentityRespectsSuccessorStep_of_transport
    (htransport χ hχ hprime)

theorem PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_successor_step
    (hstep : PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget) :
    PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget := by
  intro χ hχ hprime
  exact PRCCharacterPrimeIdentityRespectsComparableTrace_of_successor_step
    (hstep χ hχ hprime)

theorem PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget_of_comparable_trace
    (hcomp : PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget) :
    PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget := by
  intro χ hχ hprime
  exact PRCCharacterPrimeIdentityRespectsCommonTraceExtension_of_comparable_trace
    (hcomp χ hχ hprime)

theorem PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget_of_common_trace_extension
    (hcommon : PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget) :
    PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget := by
  intro χ hχ hprime
  exact PRCCharacterPrimeIdentityRespectsTraceConnected_of_common_trace_extension
    (hcommon χ hχ hprime)

theorem PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget_of_trace_transport
    (htransport : PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget) :
    PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget := by
  intro χ hχ hprime p hp r hr hpId
  exact htransport χ hχ hprime p hp r hr
    (PRCPrimeAxisTraceConnected_proved p hp r hr) hpId

theorem PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget_of_trace_coherence
    (htrace : PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget) :
    PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget := by
  intro χ hχ hprime p hp r hr hpId hrRec
  have hrId := htrace χ hχ hprime p hp r hr hpId
  have hself :
      RatioOrbit.crossEq
        (primeDirection r hr)
        (RatioOrbit.recip (primeDirection r hr)) :=
    RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hrId) hrRec
  exact primeDirection_not_crossEq_recip r hr hself

/-- Sharper orientation blocker A: prime cost calibration must choose one
coherent orientation across all native prime axes. This is the place where
mixed independent prime inversions must be ruled out. -/
def PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeDirectionCalibrated χ →
        PRCCharacterPrimeOrientationCoherent χ

theorem PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget_of_local_and_nomixed
    (hlocal : PRCPrimeCalibrationForcesLocalPrimeOrientationTarget)
    (hnomix : PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget) :
    PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget := by
  intro χ hχ hprime
  have hloc := hlocal χ hχ hprime
  have hno := hnomix χ hχ hprime
  by_cases hId :
      ∃ p : DistinctionNat, ∃ hp : DistinctionNat.primeOrbit p,
        RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp)
  · rcases hId with (p0, hp0, hid0)
    exact Or.inl (by
      intro p hp
      rcases hloc p hp with hid | hrec
      · exact hid
      · exact False.elim (hno p0 hp0 p hp hid0 hrec))
  · exact Or.inr (by
      intro p hp
      rcases hloc p hp with hid | hrec
      · exact False.elim (hId (p, hp, hid))
      · exact hrec)

/-- Sharper orientation blocker B: once prime orientation is coherent, the
multiplicative character law and native rational factorization must propagate
that orientation to every ratio direction. -/
def PRCCoherentPrimeOrientationPropagatesToGlobalTarget : Prop :=
  ∀ χ : RatioOrbit → RatioOrbit,
    PRCRatioCharacter χ →
      PRCCharacterPrimeOrientationCoherent χ →
        PRCCharacterGlobalCostOrientation χ

theorem PRCPrimeCalibrationForcesGlobalOrientationTarget_of_prime_orientation_targets
    (hcoherent : PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget)
    (hprop : PRCCoherentPrimeOrientationPropagatesToGlobalTarget) :
    PRCPrimeCalibrationForcesGlobalOrientationTarget := by
  intro χ hχ hprime
  exact hprop χ hχ (hcoherent χ hχ hprime)

theorem PRCPrimeCalibrationPropagationTarget_of_global_orientation
    (horient : PRCPrimeCalibrationForcesGlobalOrientationTarget) :
    PRCPrimeCalibrationPropagationTarget := by
  intro χ hχ hprime q
  rcases horient χ hχ hprime q with hsame | hinv
  · exact onRatioOrbit_congr hsame
  · exact RatioOrbit.crossEq_trans
      (onRatioOrbit_congr hinv)
```

```
      (RatioOrbit.crossEq_symm (reciprocal_symmetric q))

/-- Pass-27 refinement of prime propagation. -/
def PRCPrimeCalibrationPropagationSharpenedTarget : Prop :=
  PRCPrimeFloorSuccessorTransportSharpenedTarget ∧
    PRCCoherentPrimeOrientationPropagatesToGlobalTarget

theorem PRCPrimeCalibrationPropagationTarget_of_sharpened_orientation
    (hsharp : PRCPrimeCalibrationPropagationSharpenedTarget) :
    PRCPrimeCalibrationPropagationTarget :=
  PRCPrimeCalibrationPropagationTarget_of_global_orientation
    (PRCPrimeCalibrationForcesGlobalOrientationTarget_of_prime_orientation_targets
      (PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget_of_local_and_nomixed
        PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved
        (PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget_of_trace_coherence
          (PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget_of_trace_transport
            (PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget_of_common_trace_extension
              (PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget_of_comparable_trace
                (PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_prime_floor_successor_transport
                  (PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_nomix
                    hsharp.1)))))))
      hsharp.2)

theorem PRCNativeCostCharacterRigidityTarget_of_prime_targets
    (htwo : PRCTwoCalibrationForcesPrimeCalibrationTarget)
    (hprop : PRCPrimeCalibrationPropagationTarget) :
    PRCNativeCostCharacterRigidityTarget := by
  intro χ hχ htwoCal q
  exact hprop χ hχ (htwo χ hχ htwoCal) q

theorem PRCNativeCostUniquenessTarget_of_prime_character_targets
    (hfactor : PRCNativeCostCharacterFactorizationTarget)
    (htwo : PRCTwoCalibrationForcesPrimeCalibrationTarget)
    (hprop : PRCPrimeCalibrationPropagationTarget) :
    PRCNativeCostUniquenessTarget := by
  intro F hF q
  rcases hfactor F hF with ⟨χ, hχ, hFχ⟩
  have hcal :
      RatioOrbit.crossEq (costFromCharacter χ two) (onRatioOrbit two) :=
    RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm (hFχ two)) hF.two_calibrated
  have hrigid :=
    PRCNativeCostCharacterRigidityTarget_of_prime_targets htwo hprop
  exact RatioOrbit.crossEq_trans (hFχ q) (hrigid χ hχ hcal q)

/-- The identity map is a ratio character. This sanity-check anchors the
character interface to the canonical cost. -/
theorem identity_ratio_character :
    PRCRatioCharacter (fun q : RatioOrbit => q) where
  unit := RatioOrbit.crossEq_refl RatioOrbit.one
  multiplicative := by
    intro x y
    exact RatioOrbit.crossEq_refl (RatioOrbit.mul x y)
  reciprocal := by
    intro x
    exact RatioOrbit.crossEq_refl (RatioOrbit.recip x)
  normalized_invariant := by
    intro q
    exact DistinctionNat.normalizeRatio_crossEq q
  nonzero_preserving := by
    intro q hq
    exact hq

theorem identity_character_rigid :
    ∀ q : RatioOrbit,
      RatioOrbit.crossEq
        (costFromCharacter (fun q : RatioOrbit => q) q)
        (onRatioOrbit q) := by
  intro q
  exact RatioOrbit.crossEq_refl (onRatioOrbit q)

theorem identity_character_global_orientation :
    PRCCharacterGlobalCostOrientation (fun q : RatioOrbit => q) := by
  intro q
  exact Or.inl (RatioOrbit.crossEq_refl q)

theorem identity_character_prime_orientation_coherent :
    PRCCharacterPrimeOrientationCoherent (fun q : RatioOrbit => q) := by
  exact Or.inl (by
    intro p hp
    exact RatioOrbit.crossEq_refl (primeDirection p hp))

/-- The global reciprocal map is also a ratio character. This is the first
explicit witness that the multiplicative character laws alone do not choose the
identity orientation. -/
theorem reciprocal_ratio_character :
    PRCRatioCharacter (fun q : RatioOrbit => RatioOrbit.recip q) where
  unit := by
    rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
      RatioOrbit.one_toRat]
    norm_num
  multiplicative := by
    intro x y
    rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
      RatioOrbit.mul_toRat, RatioOrbit.mul_toRat, RatioOrbit.recip_toRat,
      RatioOrbit.recip_toRat]
    by_cases hx : x.toRat = 0
    · simp [hx]
    · by_cases hy : y.toRat = 0
      · simp [hy]
      · field_simp [hx, hy]
  reciprocal := by
```

```
    intro x
    exact RatioOrbit.crossEq_refl (RatioOrbit.recip (RatioOrbit.recip x))
  normalized_invariant := by
    intro q
    rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
      RatioOrbit.recip_toRat]
    have hnorm :
        q.toRat = (DistinctionNat.normalizeRatio q).toRat :=
      (RatioOrbit.crossEq_iff_toRat_eq q (DistinctionNat.normalizeRatio q)).mp
        (DistinctionNat.normalizeRatio_crossEq q)
    exact congrArg Inv.inv hnorm
  nonzero_preserving := by
    intro q hq
    rw [RatioOrbit.recip_toRat]
    exact inv_ne_zero hq

theorem reciprocal_character_prime_calibrated :
    PRCCharacterPrimeDirectionCalibrated
      (fun q : RatioOrbit => RatioOrbit.recip q) := by
  intro p hp
  simpa [costFromCharacter] using
    RatioOrbit.crossEq_symm (reciprocal_symmetric (primeDirection p hp))

theorem reciprocal_character_global_orientation :
    PRCCharacterGlobalCostOrientation
      (fun q : RatioOrbit => RatioOrbit.recip q) := by
  intro q
  exact Or.inr (RatioOrbit.crossEq_refl (RatioOrbit.recip q))

theorem reciprocal_character_prime_orientation_coherent :
    PRCCharacterPrimeOrientationCoherent
      (fun q : RatioOrbit => RatioOrbit.recip q) := by
  exact Or.inr (by
    intro p hp
    exact RatioOrbit.crossEq_refl (RatioOrbit.recip (primeDirection p hp)))

theorem reciprocal_character_not_successor_additive_compatible :
    ¬ PRCCharacterOrbitSuccessorAdditiveCompatible
      (fun q : RatioOrbit => RatioOrbit.recip q) := by
  intro hcompat
  have hstep := hcompat DistinctionNat.one DistinctionNat.one_ne_zero
  rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
    RatioOrbit.add_toRat, RatioOrbit.recip_toRat, RatioOrbit.one_toRat,
    orbitDirection_toRat, orbitDirection_toRat, DistinctionNat.toNat_succ,
    DistinctionNat.one_toNat] at hstep
  norm_num at hstep

theorem PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget_refuted :
    ¬ PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget := by
  intro htarget
  exact reciprocal_character_not_successor_additive_compatible
    (htarget (fun q : RatioOrbit => RatioOrbit.recip q)
      reciprocal_ratio_character reciprocal_character_prime_calibrated)

/-- The sharpened replacement for the opaque native uniqueness blocker. -/
def PRCNativeCostUniquenessSharpenedTarget : Prop :=
  PRCNativeCostCharacterFactorizationTarget ∧
    PRCNativeCostCharacterRigidityTarget

/-- Pass-26 refinement of the rigidity target. -/
def PRCNativeCostCharacterRigiditySharpenedTarget : Prop :=
  PRCTwoCalibrationForcesPrimeCalibrationTarget ∧
    PRCPrimeCalibrationPropagationTarget

theorem PRCNativeCostUniquenessTarget_of_character_targets
    (hfactor : PRCNativeCostCharacterFactorizationTarget)
    (hrigid : PRCNativeCostCharacterRigidityTarget) :
    PRCNativeCostUniquenessTarget := by
  intro F hF q
  rcases hfactor F hF with ⟨χ, hχ, hFχ⟩
  have hcal :
      RatioOrbit.crossEq (costFromCharacter χ two) (onRatioOrbit two) :=
    RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm (hFχ two)) hF.two_calibrated
  exact RatioOrbit.crossEq_trans (hFχ q) (hrigid χ hχ hcal q)

/-- Pass-25 certificate: native cost uniqueness is not closed, but the missing
mathematics is now split into exact Lean targets. -/
structure PRCNativeCostUniquenessBlockerCertificate : Prop where
  factorization_target :
    PRCNativeCostCharacterFactorizationTarget =
      PRCNativeCostCharacterFactorizationTarget
  rigidity_target :
    PRCNativeCostCharacterRigidityTarget =
      PRCNativeCostCharacterRigidityTarget
  two_to_prime_target :
    PRCTwoCalibrationForcesPrimeCalibrationTarget =
      PRCTwoCalibrationForcesPrimeCalibrationTarget
  prime_propagation_target :
    PRCPrimeCalibrationPropagationTarget =
      PRCPrimeCalibrationPropagationTarget
  global_orientation_target :
    PRCPrimeCalibrationForcesGlobalOrientationTarget =
      PRCPrimeCalibrationForcesGlobalOrientationTarget
  coherent_prime_orientation :
    PRCCharacterPrimeOrientationCoherent =
      PRCCharacterPrimeOrientationCoherent
  local_prime_orientation :
    PRCCharacterPrimeLocalOrientation =
      PRCCharacterPrimeLocalOrientation
  no_mixed_prime_orientation :
    PRCCharacterNoMixedPrimeOrientation =
```

```
      PRCCharacterNoMixedPrimeOrientation
  prime_identity_trace_coherence :
    PRCCharacterPrimeIdentityTraceCoherent =
      PRCCharacterPrimeIdentityTraceCoherent
  prime_axis_trace_connected :
    PRCPrimeAxisTraceConnected =
      PRCPrimeAxisTraceConnected
  prime_axis_trace_connected_proved :
    ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
      ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
        PRCPrimeAxisTraceConnected p hp r hr
  orbit_trace_extends_of_toNat_le :
    ∀ p r : DistinctionNat,
      p.toNat ≤ r.toNat →
        Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r)
  orbit_trace_comparable :
    ∀ p r : DistinctionNat,
      Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) ∨
        Trace.Extends (orbitPositionTrace r) (orbitPositionTrace p)
  orbit_direction_toRat :
    ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
      (orbitDirection p hp).toRat = (p.toNat : ℚ)
  orbit_direction_nonunit_not_crossEq_recip :
    ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
      ¬ DistinctionNat.unit p →
        ¬ RatioOrbit.crossEq
          (orbitDirection p hp)
          (RatioOrbit.recip (orbitDirection p hp))
  orbit_direction_succ_add_one :
    ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
      RatioOrbit.crossEq
        (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
        (RatioOrbit.add (orbitDirection p hp) RatioOrbit.one)
  ratio_add_right_one_cancel :
    ∀ a b : RatioOrbit,
      RatioOrbit.crossEq
        (RatioOrbit.add a RatioOrbit.one)
        (RatioOrbit.add b RatioOrbit.one) →
          RatioOrbit.crossEq a b
  prime_identity_respects_trace_connected :
    PRCCharacterPrimeIdentityRespectsTraceConnected =
      PRCCharacterPrimeIdentityRespectsTraceConnected
  prime_identity_respects_common_trace_extension :
    PRCCharacterPrimeIdentityRespectsCommonTraceExtension =
      PRCCharacterPrimeIdentityRespectsCommonTraceExtension
  prime_identity_respects_comparable_trace :
    PRCCharacterPrimeIdentityRespectsComparableTrace =
      PRCCharacterPrimeIdentityRespectsComparableTrace
  orbit_direction_identity :
    PRCCharacterOrbitDirectionIdentity =
      PRCCharacterOrbitDirectionIdentity
  orbit_direction_reciprocal :
    PRCCharacterOrbitDirectionReciprocal =
      PRCCharacterOrbitDirectionReciprocal
  orbit_succ_not_unit :
    ∀ p : DistinctionNat, p ≠ DistinctionNat.zero →
      ¬ DistinctionNat.unit p →
        ¬ DistinctionNat.unit (DistinctionNat.succ p)
  orbit_identity_respects_successor_step :
    PRCCharacterOrbitIdentityRespectsSuccessorStep =
      PRCCharacterOrbitIdentityRespectsSuccessorStep
  orbit_identity_extends_successor_step :
    PRCCharacterOrbitIdentityExtendsSuccessorStep =
      PRCCharacterOrbitIdentityExtendsSuccessorStep
  orbit_identity_contracts_successor_step :
    PRCCharacterOrbitIdentityContractsSuccessorStep =
      PRCCharacterOrbitIdentityContractsSuccessorStep
  orbit_identity_successor_transport :
    PRCCharacterOrbitIdentitySuccessorTransport =
      PRCCharacterOrbitIdentitySuccessorTransport
  orbit_successor_additive_compat :
    PRCCharacterOrbitSuccessorAdditiveCompatible =
      PRCCharacterOrbitSuccessorAdditiveCompatible
  nonunit_orbit_local_orientation :
    PRCCharacterNonunitOrbitLocalOrientation =
      PRCCharacterNonunitOrbitLocalOrientation
  orbit_product_local_orientation :
    PRCCharacterOrbitProductLocalOrientationPropagates =
      PRCCharacterOrbitProductLocalOrientationPropagates
  ratio_mul_congr :
    ∀ a₁ a₂ b₁ b₂ : RatioOrbit,
      RatioOrbit.crossEq a₁ a₂ →
        RatioOrbit.crossEq b₁ b₂ →
          RatioOrbit.crossEq (RatioOrbit.mul a₁ b₁) (RatioOrbit.mul a₂ b₂)
  ratio_recip_congr :
    ∀ a b : RatioOrbit,
      RatioOrbit.crossEq a b →
        RatioOrbit.crossEq (RatioOrbit.recip a) (RatioOrbit.recip b)
  ratio_mul_recip_recip :
    ∀ a b : RatioOrbit,
      RatioOrbit.crossEq
        (RatioOrbit.mul (RatioOrbit.recip a) (RatioOrbit.recip b))
        (RatioOrbit.recip (RatioOrbit.mul a b))
  orbit_direction_mul :
    ∀ a b p : DistinctionNat,
      ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
        ∀ hp : p ≠ DistinctionNat.zero,
          a * b = p →
            RatioOrbit.crossEq (orbitDirection p hp)
              (RatioOrbit.mul (orbitDirection a ha) (orbitDirection b hb))
  orbit_product_display_compatible :
```

```
    PRCCharacterOrbitProductDisplayCompatible =
      PRCCharacterOrbitProductDisplayCompatible
  orbit_character_respects_crossEq :
    PRCCharacterRespectsCrossEq =
      PRCCharacterRespectsCrossEq
  normalizeRatio_canonical_target :
    PRCNormalizeRatioCanonicalTarget =
      PRCNormalizeRatioCanonicalTarget
  signed_orbit_sign_canonical :
    PRCSignedOrbitSignCanonical =
      PRCSignedOrbitSignCanonical
  ratio_reduced_sign_canonical :
    PRCRatioReducedSignCanonical =
      PRCRatioReducedSignCanonical
  signed_ofOrbit_abs_self :
    ∀ n : DistinctionNat, (SignedOrbit.ofOrbit n).abs = n
  signed_neg_ofOrbit_abs_self :
    ∀ n : DistinctionNat,
      (SignedOrbit.negate (SignedOrbit.ofOrbit n)).abs = n
  signedQuotient_signCanonical :
    ∀ z : SignedOrbit, ∀ d : DistinctionNat,
      ∀ hd : d ≠ DistinctionNat.zero,
        DistinctionNat.divides d z.abs →
          PRCSignedOrbitSignCanonical (DistinctionNat.signedQuotient z d hd)
  normalizeRatio_reduced_signCanonical :
    ∀ q : RatioOrbit,
      PRCRatioReducedSignCanonical (DistinctionNat.normalizeRatio q)
  signCanonical_toInt_injective :
    ∀ z w : SignedOrbit,
      PRCSignedOrbitSignCanonical z →
        PRCSignedOrbitSignCanonical w →
          z.toInt = w.toInt →
            z = w
  reduced_den_dvd :
    ∀ q r : RatioOrbit,
      PRCRatioReducedSignCanonical q →
        PRCRatioReducedSignCanonical r →
          RatioOrbit.crossEq q r →
            q.den.toNat ∣ r.den.toNat
  reduced_den_eq :
    ∀ q r : RatioOrbit,
      PRCRatioReducedSignCanonical q →
        PRCRatioReducedSignCanonical r →
          RatioOrbit.crossEq q r →
            q.den = r.den
  reduced_num_eq :
    ∀ q r : RatioOrbit,
      PRCRatioReducedSignCanonical q →
        PRCRatioReducedSignCanonical r →
          RatioOrbit.crossEq q r →
            q.num = r.num
  reduced_signCanonical_ratio_unique_target :
    PRCReducedSignCanonicalRatioUniqueTarget =
      PRCReducedSignCanonicalRatioUniqueTarget
  reduced_signCanonical_ratio_unique_proved :
    PRCReducedSignCanonicalRatioUniqueTarget
  normalizeRatio_canonical_from_reduced_signCanonical_unique :
    PRCReducedSignCanonicalRatioUniqueTarget →
      PRCNormalizeRatioCanonicalTarget
  normalizeRatio_canonical_proved :
    PRCNormalizeRatioCanonicalTarget
  orbit_character_crossEq_from_normalizeRatio_canonical :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCRatioCharacter χ →
        PRCNormalizeRatioCanonicalTarget →
          PRCCharacterRespectsCrossEq χ
  orbit_product_display_from_crossEq :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterRespectsCrossEq χ →
        PRCCharacterOrbitProductDisplayCompatible χ
  orbit_product_no_mixed_orientation :
    PRCCharacterOrbitProductNoMixedOrientation =
      PRCCharacterOrbitProductNoMixedOrientation
  nonunit_orbit_orientation_coherent :
    PRCCharacterNonunitOrbitOrientationCoherent =
      PRCCharacterNonunitOrbitOrientationCoherent
  no_mixed_nonunit_orbit_orientation :
    PRCCharacterNoMixedNonunitOrbitOrientation =
      PRCCharacterNoMixedNonunitOrbitOrientation
  nonunit_identity_branch_transport :
    PRCCharacterNonunitIdentityBranchTransport =
      PRCCharacterNonunitIdentityBranchTransport
  nonunit_identity_respects_comparable_trace :
    PRCCharacterNonunitIdentityRespectsComparableTrace =
      PRCCharacterNonunitIdentityRespectsComparableTrace
  nonunit_local_from_coherent :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitOrientationCoherent χ →
        PRCCharacterNonunitOrbitLocalOrientation χ
  no_mixed_nonunit_from_coherent :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitOrientationCoherent χ →
        PRCCharacterNoMixedNonunitOrbitOrientation χ
  orbit_mul_not_unit_left :
    ∀ p r : DistinctionNat,
      ¬ DistinctionNat.unit p →
        ¬ DistinctionNat.unit (p * r)
  no_mixed_nonunit_from_product_no_mixed :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterOrbitProductNoMixedOrientation χ →
        PRCCharacterNoMixedNonunitOrbitOrientation χ
```

```
  orbit_product_no_mixed_from_no_mixed_nonunit :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNoMixedNonunitOrbitOrientation χ →
        PRCCharacterOrbitProductNoMixedOrientation χ
  orbit_product_no_mixed_iff_no_mixed_nonunit :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterOrbitProductNoMixedOrientation χ ↔
        PRCCharacterNoMixedNonunitOrbitOrientation χ
  no_mixed_nonunit_from_identity_branch_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitIdentityBranchTransport χ →
        PRCCharacterNoMixedNonunitOrbitOrientation χ
  orbit_product_no_mixed_from_identity_branch_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitIdentityBranchTransport χ →
        PRCCharacterOrbitProductNoMixedOrientation χ
  nonunit_identity_branch_transport_from_local_no_mixed :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitLocalOrientation χ →
        PRCCharacterNoMixedNonunitOrbitOrientation χ →
          PRCCharacterNonunitIdentityBranchTransport χ
  nonunit_identity_branch_transport_from_coherent :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitOrientationCoherent χ →
        PRCCharacterNonunitIdentityBranchTransport χ
  nonunit_identity_branch_transport_from_comparable_trace :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitIdentityRespectsComparableTrace χ →
        PRCCharacterNonunitIdentityBranchTransport χ
  nonunit_identity_comparable_trace_from_branch_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitIdentityBranchTransport χ →
        PRCCharacterNonunitIdentityRespectsComparableTrace χ
  nonunit_identity_comparable_trace_iff_branch_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitIdentityRespectsComparableTrace χ ↔
        PRCCharacterNonunitIdentityBranchTransport χ
  prime_floor_successor_transport_from_nonunit_identity_comparable_trace :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitIdentityRespectsComparableTrace χ →
        PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ
  nonunit_coherent_from_local_no_mixed :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitLocalOrientation χ →
        PRCCharacterNoMixedNonunitOrbitOrientation χ →
          PRCCharacterNonunitOrbitOrientationCoherent χ
  nonunit_coherent_from_local_identity_branch_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitLocalOrientation χ →
        PRCCharacterNonunitIdentityBranchTransport χ →
          PRCCharacterNonunitOrbitOrientationCoherent χ
  orbit_product_no_mixed_from_nonunit_coherent :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitOrientationCoherent χ →
        PRCCharacterOrbitProductNoMixedOrientation χ
  orbit_product_identity_identity :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCRatioCharacter χ →
        PRCCharacterOrbitProductDisplayCompatible χ →
          ∀ a b p : DistinctionNat,
            ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
              ∀ hp : p ≠ DistinctionNat.zero,
                a * b = p →
                  PRCCharacterOrbitDirectionIdentity χ a ha →
                    PRCCharacterOrbitDirectionIdentity χ b hb →
                      PRCCharacterOrbitDirectionIdentity χ p hp
  orbit_product_reciprocal_reciprocal :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCRatioCharacter χ →
        PRCCharacterOrbitProductDisplayCompatible χ →
          ∀ a b p : DistinctionNat,
            ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
              ∀ hp : p ≠ DistinctionNat.zero,
                a * b = p →
                  PRCCharacterOrbitDirectionReciprocal χ a ha →
                    PRCCharacterOrbitDirectionReciprocal χ b hb →
                      PRCCharacterOrbitDirectionReciprocal χ p hp
  orbit_product_local_from_display_nomix :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCRatioCharacter χ →
        PRCCharacterOrbitProductDisplayCompatible χ →
          PRCCharacterOrbitProductNoMixedOrientation χ →
            PRCCharacterOrbitProductLocalOrientationPropagates χ
  nonunit_local_from_prime_product :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterPrimeLocalOrientation χ →
        PRCCharacterOrbitProductLocalOrientationPropagates χ →
          PRCCharacterNonunitOrbitLocalOrientation χ
  prime_floor_no_adjacent_mixed_orientation :
    PRCCharacterPrimeFloorNoAdjacentMixedOrientation =
      PRCCharacterPrimeFloorNoAdjacentMixedOrientation
  prime_floor_no_adjacent_from_nonunit_coherent :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitOrientationCoherent χ →
        PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ
  prime_floor_orbit_identity_extends_successor_step :
    PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep =
      PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep
  prime_floor_orbit_identity_contracts_successor_step :
    PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep =
      PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep
```

```
  prime_floor_orbit_identity_successor_transport :
    PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport =
      PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport
  prime_floor_extends_from_local_adjacent_nomix :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitLocalOrientation χ →
        PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ →
          PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep χ
  prime_floor_contracts_from_local_adjacent_nomix :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitLocalOrientation χ →
        PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ →
          PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep χ
  prime_floor_successor_transport_from_local_adjacent_nomix :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitLocalOrientation χ →
        PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ →
          PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ
  prime_floor_no_adjacent_from_successor_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ →
        PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ
  prime_floor_successor_transport_iff_local_adjacent_nomix :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitLocalOrientation χ →
        (PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ ↔
          PRCCharacterPrimeFloorNoAdjacentMixedOrientation χ)
  prime_identity_comparable_from_prime_floor_successor_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ →
        PRCCharacterPrimeIdentityRespectsComparableTrace χ
  nonunit_identity_comparable_from_prime_floor_successor_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ →
        PRCCharacterNonunitIdentityRespectsComparableTrace χ
  nonunit_coherent_from_local_prime_floor_successor_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterNonunitOrbitLocalOrientation χ →
        PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ →
          PRCCharacterNonunitOrbitOrientationCoherent χ
  orbit_identity_extends_from_additive_compat :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterOrbitSuccessorAdditiveCompatible χ →
        PRCCharacterOrbitIdentityExtendsSuccessorStep χ
  orbit_identity_contracts_from_additive_compat :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterOrbitSuccessorAdditiveCompatible χ →
        PRCCharacterOrbitIdentityContractsSuccessorStep χ
  orbit_identity_successor_transport_from_additive_compat :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterOrbitSuccessorAdditiveCompatible χ →
        PRCCharacterOrbitIdentitySuccessorTransport χ
  orbit_identity_respects_successor_step_from_transport :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterOrbitIdentitySuccessorTransport χ →
        PRCCharacterOrbitIdentityRespectsSuccessorStep χ
  orbit_identity_one_of_identity :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterOrbitIdentityRespectsSuccessorStep χ →
        ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
          PRCCharacterOrbitDirectionIdentity χ p hp →
            PRCCharacterOrbitDirectionIdentity χ
              DistinctionNat.one DistinctionNat.one_ne_zero
  orbit_identity_of_one :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterOrbitIdentityRespectsSuccessorStep χ →
        ∀ r : DistinctionNat, ∀ hr : r ≠ DistinctionNat.zero,
          PRCCharacterOrbitDirectionIdentity χ
            DistinctionNat.one DistinctionNat.one_ne_zero →
              PRCCharacterOrbitDirectionIdentity χ r hr
  prime_identity_comparable_from_successor_step :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterOrbitIdentityRespectsSuccessorStep χ →
        PRCCharacterPrimeIdentityRespectsComparableTrace χ
  prime_identity_common_trace_from_comparable_trace :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterPrimeIdentityRespectsComparableTrace χ →
        PRCCharacterPrimeIdentityRespectsCommonTraceExtension χ
  prime_identity_trace_connected_from_common_trace :
    ∀ χ : RatioOrbit → RatioOrbit,
      PRCCharacterPrimeIdentityRespectsCommonTraceExtension χ →
        PRCCharacterPrimeIdentityRespectsTraceConnected χ
  prime_to_local_orientation_target :
    PRCPrimeCalibrationForcesLocalPrimeOrientationTarget =
      PRCPrimeCalibrationForcesLocalPrimeOrientationTarget
  prime_to_local_orientation_proved :
    PRCPrimeCalibrationForcesLocalPrimeOrientationTarget
  prime_no_mixed_orientation_target :
    PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget =
      PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget
  prime_identity_trace_coherence_target :
    PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget =
      PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget
  prime_identity_trace_transport_target :
    PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget =
      PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget
  prime_identity_common_trace_extension_target :
    PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget =
      PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget
  prime_identity_comparable_trace_target :
    PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget =
```

```
      PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget
  orbit_successor_identity_target :
    PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget =
      PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget
  orbit_successor_transport_target :
    PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget =
      PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget
  orbit_successor_additive_compat_target :
    PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget =
      PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget
  orbit_successor_additive_compat_target_refuted :
    ¬ PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget
  prime_floor_successor_transport_target :
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget =
      PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
  prime_floor_nonunit_local_orientation_target :
    PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget =
      PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget
  prime_floor_nonunit_product_local_orientation_target :
    PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget =
      PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget
  prime_floor_product_display_compatibility_target :
    PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget =
      PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget
  prime_floor_character_crossEq_respect_target :
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget =
      PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget
  prime_floor_character_crossEq_from_normalizeRatio_canonical :
    PRCNormalizeRatioCanonicalTarget →
      PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget
  prime_floor_character_crossEq_from_reduced_signCanonical_unique :
    PRCReducedSignCanonicalRatioUniqueTarget →
      PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget
  prime_floor_character_crossEq_respect_proved :
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget
  prime_floor_product_display_from_crossEq_respect :
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget →
      PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget
  prime_floor_product_display_compatibility_proved :
    PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget
  prime_floor_product_no_mixed_orientation_target :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget =
      PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
  prime_floor_nonunit_orbit_orientation_coherent_target :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget =
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_no_mixed_nonunit_orbit_orientation_target :
    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget =
      PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
  prime_floor_nonunit_identity_branch_transport_target :
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget =
      PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
  prime_floor_nonunit_identity_comparable_trace_target :
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget =
      PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_nonunit_orbit_orientation_local_no_mixed_target :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget =
      PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
  prime_floor_nonunit_orbit_orientation_local_product_no_mixed_target :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget =
      PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget
  prime_floor_no_mixed_nonunit_from_coherent :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
      PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
  prime_floor_no_mixed_nonunit_from_product_no_mixed :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
      PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
  prime_floor_product_no_mixed_from_no_mixed_nonunit :
    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget →
      PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
  prime_floor_product_no_mixed_iff_no_mixed_nonunit :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
  prime_floor_product_no_mixed_from_identity_branch_transport :
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget →
      PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
  prime_floor_nonunit_identity_branch_transport_from_comparable_trace :
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget →
      PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
  prime_floor_nonunit_identity_branch_transport_from_coherent :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
      PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
  prime_floor_nonunit_local_no_mixed_from_local_product_no_mixed :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget →
      PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
  prime_floor_nonunit_coherent_from_local_product_no_mixed :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget →
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_nonunit_coherent_from_product_no_mixed :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_product_no_mixed_iff_nonunit_coherent :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_nonunit_identity_branch_transport_from_product_no_mixed :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
      PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
  prime_floor_product_no_mixed_iff_identity_branch_transport :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
  prime_floor_nonunit_identity_comparable_trace_from_branch_transport :
```

```
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget →
      PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_nonunit_identity_comparable_trace_from_product_no_mixed :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
      PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_nonunit_identity_branch_transport_iff_comparable_trace :
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget ↔
      PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_product_no_mixed_iff_identity_comparable_trace :
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_nonunit_local_no_mixed_from_coherent :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
      PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
  prime_floor_nonunit_coherent_from_local_no_mixed :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget →
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_nonunit_orbit_orientation_coherent_iff_local_no_mixed :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
      PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
  prime_floor_nonunit_orbit_orientation_coherent_sharpened_target :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget =
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget
  prime_floor_nonunit_orbit_orientation_coherent_from_local_successor_transport :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget →
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_product_no_mixed_from_nonunit_coherent :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
      PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
  prime_floor_nonunit_local_from_nonunit_coherent :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
      PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget
  prime_floor_no_adjacent_mixed_from_nonunit_coherent :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
      PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
  prime_floor_no_adjacent_mixed_from_successor_transport :
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget →
      PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
  prime_floor_successor_transport_from_nonunit_coherent :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
      PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
  prime_floor_nonunit_identity_comparable_trace_from_successor_transport :
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget →
      PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_nonunit_orbit_orientation_sharpened_from_nonunit_coherent :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget
  prime_floor_nonunit_orbit_orientation_coherent_iff_sharpened :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget
  prime_floor_successor_transport_from_identity_comparable_trace :
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget →
      PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
  prime_floor_nonunit_identity_comparable_trace_iff_successor_transport :
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget ↔
      PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
  prime_floor_successor_transport_local_adjacent_target :
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget =
      PRCPrimeFloorSuccessorTransportLocalAdjacentTarget
  prime_floor_successor_transport_from_local_adjacent_target :
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget →
      PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
  prime_floor_local_adjacent_from_local_successor_transport :
    (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
      PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) →
        PRCPrimeFloorSuccessorTransportLocalAdjacentTarget
  prime_floor_local_adjacent_iff_local_successor_transport :
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
      (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
        PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget)
  prime_floor_local_adjacent_from_nonunit_coherent :
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
      PRCPrimeFloorSuccessorTransportLocalAdjacentTarget
  prime_floor_nonunit_coherent_from_local_adjacent :
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget →
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_local_adjacent_iff_nonunit_coherent :
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
      PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_product_local_orientation_sharpened_target :
    PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget =
      PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget
  prime_floor_product_local_orientation_from_display_nomix :
    PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget →
      PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget
  prime_floor_nonunit_local_orientation_from_product_local :
    PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget →
      PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget
  prime_floor_no_adjacent_mixed_orientation_target :
    PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget =
      PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
  prime_floor_successor_transport_sharpened_target :
    PRCPrimeFloorSuccessorTransportSharpenedTarget =
      PRCPrimeFloorSuccessorTransportSharpenedTarget
  prime_floor_successor_transport_target_from_local_adjacent_nomix :
    PRCPrimeFloorSuccessorTransportSharpenedTarget →
      PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
  orbit_successor_transport_target_from_additive_compat :
    PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget →
      PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget
  prime_identity_comparable_trace_from_prime_floor_successor_transport :
```

```
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget →
      PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget
  orbit_successor_identity_target_from_transport :
    PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget →
      PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget
  prime_identity_comparable_trace_from_successor_step :
    PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget →
      PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget
  prime_identity_common_trace_extension_from_comparable_trace :
    PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget →
      PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget
  prime_identity_trace_transport_from_common_trace :
    PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget →
      PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget
  prime_identity_trace_coherence_from_transport :
    PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget →
      PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget
  prime_no_mixed_from_trace_coherence :
    PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget →
      PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget
  coherent_prime_orientation_reduction :
    PRCPrimeCalibrationForcesLocalPrimeOrientationTarget →
      PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget →
        PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget
  prime_to_coherent_orientation_target :
    PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget =
      PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget
  coherent_prime_orientation_propagation_target :
    PRCCoherentPrimeOrientationPropagatesToGlobalTarget =
      PRCCoherentPrimeOrientationPropagatesToGlobalTarget
  global_orientation_reduction :
    PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget →
      PRCCoherentPrimeOrientationPropagatesToGlobalTarget →
        PRCPrimeCalibrationForcesGlobalOrientationTarget
  prime_propagation_sharpened_target :
    PRCPrimeCalibrationPropagationSharpenedTarget =
      PRCPrimeCalibrationPropagationSharpenedTarget
  prime_propagation_reduction :
    PRCPrimeCalibrationForcesGlobalOrientationTarget →
      PRCPrimeCalibrationPropagationTarget
  prime_propagation_sharpened_reduction :
    PRCPrimeCalibrationPropagationSharpenedTarget →
      PRCPrimeCalibrationPropagationTarget
  rigidity_sharpened_target :
    PRCNativeCostCharacterRigiditySharpenedTarget =
      PRCNativeCostCharacterRigiditySharpenedTarget
  rigidity_reduction :
    PRCTwoCalibrationForcesPrimeCalibrationTarget →
      PRCPrimeCalibrationPropagationTarget →
        PRCNativeCostCharacterRigidityTarget
  identity_character : PRCRatioCharacter (fun q : RatioOrbit => q)
  identity_rigid :
    ∀ q : RatioOrbit,
      RatioOrbit.crossEq
        (costFromCharacter (fun q : RatioOrbit => q) q)
        (onRatioOrbit q)
  identity_orientation :
    PRCCharacterGlobalCostOrientation (fun q : RatioOrbit => q)
  identity_prime_orientation_coherent :
    PRCCharacterPrimeOrientationCoherent (fun q : RatioOrbit => q)
  reciprocal_character :
    PRCRatioCharacter (fun q : RatioOrbit => RatioOrbit.recip q)
  reciprocal_prime_calibrated :
    PRCCharacterPrimeDirectionCalibrated
      (fun q : RatioOrbit => RatioOrbit.recip q)
  reciprocal_orientation :
    PRCCharacterGlobalCostOrientation
      (fun q : RatioOrbit => RatioOrbit.recip q)
  reciprocal_prime_orientation_coherent :
    PRCCharacterPrimeOrientationCoherent
      (fun q : RatioOrbit => RatioOrbit.recip q)
  sharpened_target :
    PRCNativeCostUniquenessSharpenedTarget =
      PRCNativeCostUniquenessSharpenedTarget
  reduction :
    PRCNativeCostCharacterFactorizationTarget →
      PRCNativeCostCharacterRigidityTarget →
        PRCNativeCostUniquenessTarget
  prime_reduction :
    PRCNativeCostCharacterFactorizationTarget →
      PRCTwoCalibrationForcesPrimeCalibrationTarget →
        PRCPrimeCalibrationPropagationTarget →
          PRCNativeCostUniquenessTarget
  original_target :
    PRCNativeCostUniquenessTarget = PRCNativeCostUniquenessTarget
  strength_tag : StrengthTag.deltaOnly = StrengthTag.deltaOnly

theorem prc_native_cost_uniqueness_blocker_certificate :
    PRCNativeCostUniquenessBlockerCertificate where
  factorization_target := rfl
  rigidity_target := rfl
  two_to_prime_target := rfl
  prime_propagation_target := rfl
  global_orientation_target := rfl
  coherent_prime_orientation := rfl
  local_prime_orientation := rfl
  no_mixed_prime_orientation := rfl
  prime_identity_trace_coherence := rfl
  prime_axis_trace_connected := rfl
  prime_axis_trace_connected_proved :=
    PRCPrimeAxisTraceConnected_proved
```

```
  orbit_trace_extends_of_toNat_le := by
    intro p r
    exact orbitPositionTrace_extends_of_toNat_le
  orbit_trace_comparable :=
    orbitPositionTrace_comparable
  orbit_direction_toRat := by
    intro p
    exact orbitDirection_toRat p
  orbit_direction_nonunit_not_crossEq_recip := by
    intro p
    exact orbitDirection_nonunit_not_crossEq_recip p
  orbit_direction_succ_add_one := by
    intro p
    exact orbitDirection_succ_crossEq_add_one p
  ratio_add_right_one_cancel := by
    intro a b
    exact RatioOrbit.add_right_one_cancel
  prime_identity_respects_trace_connected := rfl
  prime_identity_respects_common_trace_extension := rfl
  prime_identity_respects_comparable_trace := rfl
  orbit_direction_identity := rfl
  orbit_direction_reciprocal := rfl
  orbit_succ_not_unit := by
    intro p
    exact orbit_succ_not_unit_of_nonzero_not_unit p
  orbit_identity_respects_successor_step := rfl
  orbit_identity_extends_successor_step := rfl
  orbit_identity_contracts_successor_step := rfl
  orbit_identity_successor_transport := rfl
  orbit_successor_additive_compat := rfl
  nonunit_orbit_local_orientation := rfl
  orbit_product_local_orientation := rfl
  ratio_mul_congr := by
    intro a₁ a₂ b₁ b₂
    exact ratioOrbit_mul_congr
  ratio_recip_congr := by
    intro a b
    exact ratioOrbit_recip_congr
  ratio_mul_recip_recip :=
    ratioOrbit_mul_recip_recip_crossEq_recip_mul
  orbit_direction_mul := by
    intro a b p
    exact orbitDirection_mul_crossEq a b p
  orbit_product_display_compatible := rfl
  orbit_character_respects_crossEq := rfl
  normalizeRatio_canonical_target := rfl
  signed_orbit_sign_canonical := rfl
  ratio_reduced_sign_canonical := rfl
  signed_ofOrbit_abs_self := signedOrbit_ofOrbit_abs_self
  signed_neg_ofOrbit_abs_self := signedOrbit_neg_ofOrbit_abs_self
  signedQuotient_signCanonical := by
    intro z d
    exact signedQuotient_signCanonical_of_divides z d
  normalizeRatio_reduced_signCanonical :=
    normalizeRatio_reduced_signCanonical
  signCanonical_toInt_injective := by
    intro z w
    exact PRCSignedOrbitSignCanonical.eq_of_toInt_eq
  reduced_den_dvd := by
    intro q r
    exact PRCReducedSignCanonical_den_dvd_of_crossEq
  reduced_den_eq := by
    intro q r
    exact PRCReducedSignCanonical_den_eq_of_crossEq
  reduced_num_eq := by
    intro q r
    exact PRCReducedSignCanonical_num_eq_of_crossEq
  reduced_signCanonical_ratio_unique_target := rfl
  reduced_signCanonical_ratio_unique_proved :=
    PRCReducedSignCanonicalRatioUniqueTarget_proved
  normalizeRatio_canonical_from_reduced_signCanonical_unique :=
    PRCNormalizeRatioCanonicalTarget_of_reduced_signCanonical_unique
  normalizeRatio_canonical_proved :=
    PRCNormalizeRatioCanonicalTarget_proved
  orbit_character_crossEq_from_normalizeRatio_canonical := by
    intro χ
    exact PRCCharacterRespectsCrossEq_of_normalizeRatio_canonical
  orbit_product_display_from_crossEq := by
    intro χ
    exact PRCCharacterOrbitProductDisplayCompatible_of_crossEq_respect
  orbit_product_no_mixed_orientation := rfl
  nonunit_orbit_orientation_coherent := rfl
  no_mixed_nonunit_orbit_orientation := rfl
  nonunit_identity_branch_transport := rfl
  nonunit_identity_respects_comparable_trace := rfl
  nonunit_local_from_coherent := by
    intro χ
    exact PRCCharacterNonunitOrbitLocalOrientation_of_coherent
  no_mixed_nonunit_from_coherent := by
    intro χ
    exact PRCCharacterNoMixedNonunitOrbitOrientation_of_coherent
  orbit_mul_not_unit_left := by
    intro p r
    exact orbit_mul_not_unit_of_left_not_unit
  no_mixed_nonunit_from_product_no_mixed := by
    intro χ
    exact PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
  orbit_product_no_mixed_from_no_mixed_nonunit := by
    intro χ
    exact PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit
  orbit_product_no_mixed_iff_no_mixed_nonunit := by
```

```
    intro χ
    exact PRCCharacterOrbitProductNoMixedOrientation_iff_no_mixed_nonunit
no_mixed_nonunit_from_identity_branch_transport := by
    intro χ
    exact PRCCharacterNoMixedNonunitOrbitOrientation_of_identity_branch_transport
orbit_product_no_mixed_from_identity_branch_transport := by
    intro χ
    exact PRCCharacterOrbitProductNoMixedOrientation_of_identity_branch_transport
nonunit_identity_branch_transport_from_local_no_mixed := by
    intro χ
    exact PRCCharacterNonunitIdentityBranchTransport_of_local_no_mixed
nonunit_identity_branch_transport_from_coherent := by
    intro χ
    exact PRCCharacterNonunitIdentityBranchTransport_of_coherent
nonunit_identity_branch_transport_from_comparable_trace := by
    intro χ
    exact PRCCharacterNonunitIdentityBranchTransport_of_comparable_trace
nonunit_identity_comparable_trace_from_branch_transport := by
    intro χ
    exact PRCCharacterNonunitIdentityRespectsComparableTrace_of_branch_transport
nonunit_identity_comparable_trace_iff_branch_transport := by
    intro χ
    exact PRCCharacterNonunitIdentityRespectsComparableTrace_iff_branch_transport
nonunit_coherent_from_local_no_mixed := by
    intro χ
    exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
nonunit_coherent_from_local_identity_branch_transport := by
    intro χ
    exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_identity_branch_transport
orbit_product_no_mixed_from_nonunit_coherent := by
    intro χ
    exact PRCCharacterOrbitProductNoMixedOrientation_of_nonunit_coherent
orbit_product_identity_identity := by
    intro χ
    exact PRCCharacterOrbitProductIdentityIdentity
orbit_product_reciprocal_reciprocal := by
    intro χ
    exact PRCCharacterOrbitProductReciprocalReciprocal
orbit_product_local_from_display_nomix := by
    intro χ
    exact PRCCharacterOrbitProductLocalOrientationPropagates_of_display_compatible_nomix
nonunit_local_from_prime_product := by
    intro χ
    exact PRCCharacterNonunitOrbitLocalOrientation_of_prime_and_product_local
prime_floor_no_adjacent_mixed_orientation := rfl
prime_floor_no_adjacent_from_nonunit_coherent := by
    intro χ
    exact PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_nonunit_coherent
prime_floor_orbit_identity_extends_successor_step := rfl
prime_floor_orbit_identity_contracts_successor_step := rfl
prime_floor_orbit_identity_successor_transport := rfl
prime_floor_extends_from_local_adjacent_nomix := by
    intro χ
    exact PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep_of_local_adjacent_nomix
prime_floor_contracts_from_local_adjacent_nomix := by
    intro χ
    exact PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep_of_local_adjacent_nomix
prime_floor_successor_transport_from_local_adjacent_nomix := by
    intro χ
    exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
prime_floor_no_adjacent_from_successor_transport := by
    intro χ
    exact PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_successor_transport
prime_floor_successor_transport_iff_local_adjacent_nomix := by
    intro χ
    exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_iff_local_adjacent_nomix
prime_floor_successor_transport_from_nonunit_identity_comparable_trace := by
    intro χ
    exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_nonunit_identity_comparable_trace
prime_identity_comparable_from_prime_floor_successor_transport := by
    intro χ
    exact PRCCharacterPrimeIdentityRespectsComparableTrace_of_prime_floor_successor_transport
nonunit_identity_comparable_from_prime_floor_successor_transport := by
    intro χ
    exact PRCCharacterNonunitIdentityRespectsComparableTrace_of_prime_floor_successor_transport
nonunit_coherent_from_local_prime_floor_successor_transport := by
    intro χ
    exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_prime_floor_successor_transport
orbit_identity_extends_from_additive_compat := by
    intro χ
    exact PRCCharacterOrbitIdentityExtendsSuccessorStep_of_additive_compat
orbit_identity_contracts_from_additive_compat := by
    intro χ
    exact PRCCharacterOrbitIdentityContractsSuccessorStep_of_additive_compat
orbit_identity_successor_transport_from_additive_compat := by
    intro χ
    exact PRCCharacterOrbitIdentitySuccessorTransport_of_additive_compat
orbit_identity_respects_successor_step_from_transport := by
    intro χ
    exact PRCCharacterOrbitIdentityRespectsSuccessorStep_of_transport
orbit_identity_one_of_identity := by
    intro χ
    exact PRCCharacterOrbitIdentity_one_of_identity
orbit_identity_of_one := by
    intro χ
    exact PRCCharacterOrbitIdentity_of_one
prime_identity_comparable_from_successor_step := by
    intro χ
    exact PRCCharacterPrimeIdentityRespectsComparableTrace_of_successor_step
prime_identity_common_trace_from_comparable_trace := by
    intro χ
```

```
    exact PRCCharacterPrimeIdentityRespectsCommonTraceExtension_of_comparable_trace
  prime_identity_trace_connected_from_common_trace := by
    intro χ
    exact PRCCharacterPrimeIdentityRespectsTraceConnected_of_common_trace_extension
  prime_to_local_orientation_target := rfl
  prime_to_local_orientation_proved :=
    PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved
  prime_no_mixed_orientation_target := rfl
  prime_identity_trace_coherence_target := rfl
  prime_identity_trace_transport_target := rfl
  prime_identity_common_trace_extension_target := rfl
  prime_identity_comparable_trace_target := rfl
  orbit_successor_identity_target := rfl
  orbit_successor_transport_target := rfl
  orbit_successor_additive_compat_target := rfl
  orbit_successor_additive_compat_target_refuted :=
    PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget_refuted
  prime_floor_successor_transport_target := rfl
  prime_floor_nonunit_local_orientation_target := rfl
  prime_floor_nonunit_product_local_orientation_target := rfl
  prime_floor_product_display_compatibility_target := rfl
  prime_floor_character_crossEq_respect_target := rfl
  prime_floor_character_crossEq_from_normalizeRatio_canonical :=
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_normalizeRatio_canonical
  prime_floor_character_crossEq_from_reduced_signCanonical_unique :=
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_reduced_signCanonical_unique
  prime_floor_character_crossEq_respect_proved :=
    PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_proved
  prime_floor_product_display_from_crossEq_respect :=
    PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_of_crossEq_respect
  prime_floor_product_display_compatibility_proved :=
    PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_proved
  prime_floor_product_no_mixed_orientation_target := rfl
  prime_floor_nonunit_orbit_orientation_coherent_target := rfl
  prime_floor_no_mixed_nonunit_orbit_orientation_target := rfl
  prime_floor_nonunit_identity_branch_transport_target := rfl
  prime_floor_nonunit_identity_comparable_trace_target := rfl
  prime_floor_nonunit_orbit_orientation_local_no_mixed_target := rfl
  prime_floor_nonunit_orbit_orientation_local_product_no_mixed_target := rfl
  prime_floor_no_mixed_nonunit_from_coherent :=
    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_coherent
  prime_floor_no_mixed_nonunit_from_product_no_mixed :=
    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed
  prime_floor_product_no_mixed_from_no_mixed_nonunit :=
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_no_mixed_nonunit
  prime_floor_product_no_mixed_iff_no_mixed_nonunit :=
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_no_mixed_nonunit
  prime_floor_product_no_mixed_from_identity_branch_transport :=
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport
  prime_floor_nonunit_identity_branch_transport_from_comparable_trace :=
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace
  prime_floor_nonunit_identity_branch_transport_from_coherent :=
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_coherent
  prime_floor_nonunit_local_no_mixed_from_local_product_no_mixed :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_local_product_no_mixed
  prime_floor_nonunit_coherent_from_local_product_no_mixed :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_product_no_mixed
  prime_floor_nonunit_coherent_from_product_no_mixed :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed
  prime_floor_product_no_mixed_iff_nonunit_coherent :=
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_nonunit_coherent
  prime_floor_nonunit_identity_branch_transport_from_product_no_mixed :=
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed
  prime_floor_product_no_mixed_iff_identity_branch_transport :=
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_branch_transport
  prime_floor_nonunit_identity_comparable_trace_from_branch_transport :=
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_branch_transport
  prime_floor_nonunit_identity_comparable_trace_from_product_no_mixed :=
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_product_no_mixed
  prime_floor_nonunit_identity_branch_transport_iff_comparable_trace :=
    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_iff_comparable_trace
  prime_floor_product_no_mixed_iff_identity_comparable_trace :=
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_comparable_trace
  prime_floor_nonunit_local_no_mixed_from_coherent :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_coherent
  prime_floor_nonunit_coherent_from_local_no_mixed :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_no_mixed
  prime_floor_nonunit_orbit_orientation_coherent_iff_local_no_mixed :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_local_no_mixed
  prime_floor_nonunit_orbit_orientation_coherent_sharpened_target := rfl
  prime_floor_nonunit_orbit_orientation_coherent_from_local_successor_transport :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_and_prime_floor_successor_transport
  prime_floor_product_no_mixed_from_nonunit_coherent :=
    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_nonunit_coherent
  prime_floor_nonunit_local_from_nonunit_coherent :=
    PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_nonunit_coherent
  prime_floor_no_adjacent_mixed_from_nonunit_coherent :=
    PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_nonunit_coherent
  prime_floor_no_adjacent_mixed_from_successor_transport :=
    PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_successor_transport
  prime_floor_successor_transport_from_nonunit_coherent :=
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_nonunit_coherent
  prime_floor_nonunit_identity_comparable_trace_from_successor_transport :=
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_prime_floor_successor_transport
  prime_floor_nonunit_orbit_orientation_sharpened_from_nonunit_coherent :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget_of_nonunit_coherent
  prime_floor_nonunit_orbit_orientation_coherent_iff_sharpened :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_sharpened
  prime_floor_successor_transport_from_identity_comparable_trace :=
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_identity_comparable_trace
  prime_floor_nonunit_identity_comparable_trace_iff_successor_transport :=
```

```
    PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_iff_prime_floor_successor_transport
  prime_floor_successor_transport_local_adjacent_target := rfl
  prime_floor_successor_transport_from_local_adjacent_target :=
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_target
  prime_floor_local_adjacent_from_local_successor_transport :=
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_local_successor_transport
  prime_floor_local_adjacent_iff_local_successor_transport :=
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_local_successor_transport
  prime_floor_local_adjacent_from_nonunit_coherent :=
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_nonunit_coherent
  prime_floor_nonunit_coherent_from_local_adjacent :=
    PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_adjacent
  prime_floor_local_adjacent_iff_nonunit_coherent :=
    PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_nonunit_coherent
  prime_floor_product_local_orientation_sharpened_target := rfl
  prime_floor_product_local_orientation_from_display_nomix :=
    PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget_of_display_compatible_nomix
  prime_floor_nonunit_local_orientation_from_product_local :=
    PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_product_local_orientation
  prime_floor_no_adjacent_mixed_orientation_target := rfl
  prime_floor_successor_transport_sharpened_target := rfl
  prime_floor_successor_transport_target_from_local_adjacent_nomix :=
    PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_nomix
  orbit_successor_transport_target_from_additive_compat :=
    PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget_of_additive_compat
  prime_identity_comparable_trace_from_prime_floor_successor_transport :=
    PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_prime_floor_successor_transport
  orbit_successor_identity_target_from_transport :=
    PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget_of_transport
  prime_identity_comparable_trace_from_successor_step :=
    PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_successor_step
  prime_identity_common_trace_extension_from_comparable_trace :=
    PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget_of_comparable_trace
  prime_identity_trace_transport_from_common_trace :=
    PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget_of_common_trace_extension
  prime_identity_trace_coherence_from_transport :=
    PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget_of_trace_transport
  prime_no_mixed_from_trace_coherence :=
    PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget_of_trace_coherence
  coherent_prime_orientation_reduction :=
    PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget_of_local_and_nomixed
  prime_to_coherent_orientation_target := rfl
  coherent_prime_orientation_propagation_target := rfl
  global_orientation_reduction :=
    PRCPrimeCalibrationForcesGlobalOrientationTarget_of_prime_orientation_targets
  prime_propagation_sharpened_target := rfl
  prime_propagation_reduction :=
    PRCPrimeCalibrationPropagationTarget_of_global_orientation
  prime_propagation_sharpened_reduction :=
    PRCPrimeCalibrationPropagationTarget_of_sharpened_orientation
  rigidity_sharpened_target := rfl
  rigidity_reduction := PRCNativeCostCharacterRigidityTarget_of_prime_targets
  identity_character := identity_ratio_character
  identity_rigid := identity_character_rigid
  identity_orientation := identity_character_global_orientation
  identity_prime_orientation_coherent :=
    identity_character_prime_orientation_coherent
  reciprocal_character := reciprocal_ratio_character
  reciprocal_prime_calibrated := reciprocal_character_prime_calibrated
  reciprocal_orientation := reciprocal_character_global_orientation
  reciprocal_prime_orientation_coherent :=
    reciprocal_character_prime_orientation_coherent
  sharpened_target := rfl
  reduction := PRCNativeCostUniquenessTarget_of_character_targets
  prime_reduction := PRCNativeCostUniquenessTarget_of_prime_character_targets
  original_target := rfl
  strength_tag := rfl

end PRCJCost
end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

32. Kernel.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Kernel.lean · **Lines:** 253

```
/-
  PrimitiveRecognitionCalculus/Kernel.lean

  Round-trip source:
    PRC_Kernel_Spec_20260526.html

  Aggregate import for the first PRC kernel pass:

    δ → trace → SameT/DiffT → substitution → quotient → ℕ → ℤ → ℚ

  The ℤ and ℚ stages are PRC-owned quotient surfaces with internal
  equivalence relations (balanced length for `PRCInt`; cross-multiplication
  for `PRCRat`). The maps into Lean's `ℤ` and `ℚ` are conservative verifier
  displays, proved well-defined from the internal characterizations.
-/

import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Strength
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Basic
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.SameDiff
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.TraceLogic
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.FormalSystem
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Inevitability
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Quotient
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Orbit
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitArithmetic
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerRational
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerOrder
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitDivisibility
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitEuclidean
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCost
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RationalField
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RecognizerBridge
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.TraceClosure
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCauchy
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealNullSetoid
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceTriangle
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceVerifierTriangle
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceIncrementTriangle
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteOrderedField
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealMulBoundedContinuity
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealBoundednessModulus
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealProductContinuity
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealOrderCongruence
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteness
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteOrderedFieldPromoted
import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompletion

namespace IndisputableMonolith
namespace Foundation
namespace PrimitiveRecognitionCalculus

/-- K7/A2. First-pass PRC kernel certificate:
the analytic specification has concrete Lean objects for each stage in the
first theorem chain. This is a bundling certificate, not yet the final
inevitability theorem. -/
structure KernelFirstPassCertificate : Prop where
  strength_tags_exist : Nonempty StrengthTag
  trace_syntax_exists : Nonempty Trace
  judgment_surface_exists : Nonempty TraceJudgment
  /-- Stable finite-trace predicates carry the first PRC logic surface. -/
  trace_logic : TraceLogicCertificate
  /-- Expressive formal systems admit a PRC trace embedding. -/
  formal_system : FormalSystemCertificate
  /-- Every admissible foundation carries a PRC trace core. -/
  inevitability : PRCInevitabilityCertificate
  quotient_surface_exists :
    ∀ (J : TraceJudgment) (T : Trace), Nonempty (EndpointClass J T)
  orbit_nat_equivalence : Nonempty (DistinctionNat ≃ Nat)
  /-- Orbit addition is verifier-faithful. -/
  orbit_add_faithful :
    ∀ a b : DistinctionNat, (a + b).toNat = a.toNat + b.toNat
  /-- Orbit multiplication is verifier-faithful. -/
  orbit_mul_faithful :
    ∀ a b : DistinctionNat, (a * b).toNat = a.toNat * b.toNat
  prc_integer_surface_exists : Nonempty PRCInt
  prc_rational_surface_exists : Nonempty PRCRat
  /-- The internal balanced-length relation characterizes signed-orbit
  equivalence in PRC integers (K4.9). -/
  prc_int_balanced_iff_display :
    ∀ a b : SignedOrbit,
      SignedOrbit.balanced a b ↔ a.toInt = b.toInt
  /-- The internal cross-multiplication relation characterizes ratio-orbit
  equivalence in PRC rationals (K4.10). -/
  prc_rat_cross_iff_display :
    ∀ a b : RatioOrbit,
      RatioOrbit.crossEq a b ↔ a.toRat = b.toRat
  /-- The PRC integer display is injective on the quotient. -/
  prc_int_display_injective : Function.Injective PRCInt.toInt
  /-- The PRC rational display is injective on the quotient. -/
  prc_rat_display_injective : Function.Injective PRCRat.toRat
```

```
  /-- The PRC integer surface is isomorphic to verifier `ℤ`. -/
  prc_int_equiv_int : Nonempty (PRCInt ≃ ℤ)
  /-- Internal signed-orbit order and absolute value are closed. -/
  integer_order : IntegerOrderCertificate
  /-- Native orbit divisibility, units, factorization, and prime-orbit predicates are closed. -/
  orbit_divisibility : DistinctionNat.OrbitDivisibilityCertificate
  /-- Native Euclidean quotient/remainder, GCD, and coprime predicates are closed. -/
  orbit_euclidean : DistinctionNat.OrbitEuclideanCertificate
  /-- PRC rational J-cost, canonical RCL, and bridge to existing real uniqueness are closed. -/
  prc_jcost : PRCJCost.PRCJCostCertificate
  /-- PRC rational field-style laws, division, positivity, and quotient-level J-cost are packaged. -/
  rational_field : RationalFieldCertificate
  /-- Positive PRC ratios carry recognition cost and bridge to the Law-of-Logic J-cost chain. -/
  recognizer_bridge : PRCRecognizerBridgeCertificate
  /-- Addition on PRC integers is commutative. -/
  prc_int_add_comm : ∀ a b : PRCInt, a + b = b + a
  /-- Addition on PRC integers is associative. -/
  prc_int_add_assoc : ∀ a b c : PRCInt, a + b + c = a + (b + c)
  /-- Multiplication on PRC integers is commutative. -/
  prc_int_mul_comm : ∀ a b : PRCInt, a * b = b * a
  /-- Multiplication on PRC integers is associative. -/
  prc_int_mul_assoc : ∀ a b c : PRCInt, a * b * c = a * (b * c)
  /-- Multiplication distributes over addition. -/
  prc_int_left_distrib :
    ∀ a b c : PRCInt, a * (b + c) = a * b + a * c
  /-- Negation is the additive inverse on PRC integers. -/
  prc_int_add_negate : ∀ a : PRCInt, a + (-a) = 0
  /-- The PRC rational display preserves addition. -/
  prc_rat_display_add :
    ∀ a b : PRCRat, (a + b).toRat = a.toRat + b.toRat
  /-- The PRC rational display preserves multiplication. -/
  prc_rat_display_mul :
    ∀ a b : PRCRat, (a * b).toRat = a.toRat * b.toRat
  /-- The PRC rational display preserves reciprocal. -/
  prc_rat_display_inv :
    ∀ a : PRCRat, (a⁻¹).toRat = (a.toRat)⁻¹
  /-- Ratio reciprocal uses internal signed-orbit absolute value for the denominator. -/
  ratio_recip_internal_den :
    ∀ (a : RatioOrbit)
      (h : ¬ SignedOrbit.balanced a.num SignedOrbit.zero),
      (RatioOrbit.recipNonzero a h).den = a.num.abs
  /-- Addition on PRC rationals is commutative. -/
  prc_rat_add_comm : ∀ a b : PRCRat, a + b = b + a
  /-- Multiplication on PRC rationals is commutative. -/
  prc_rat_mul_comm : ∀ a b : PRCRat, a * b = b * a
  /-- Multiplication distributes over addition on PRC rationals. -/
  prc_rat_left_distrib :
    ∀ a b c : PRCRat, a * (b + c) = a * b + a * c
  /-- Nonzero PRC rationals multiply by their reciprocal to one. -/
  prc_rat_mul_recip_cancel :
    ∀ a : PRCRat, a.toRat ≠ 0 → a * a⁻¹ = 1
  /-- The completed trace boundary is inhabited and trace-closure tagged. -/
  trace_closure_boundary : TraceClosureCertificate
  /-- Internal Cauchy ledgers and the first PRC real quotient carrier are trace-closure tagged. -/
  real_cauchy : PRCRealCauchyCertificate
  /-- The final null-distance setoid is reduced to the exact J-cost triangle-modulus target. -/
  real_null_setoid : PRCRealNullSetoidConditionalCertificate
  /-- The J-cost distance triangle target is reduced to an exact rational display inequality. -/
  jcost_distance_triangle : PRCJCostDistanceTriangleConditionalCertificate
  /-- The verifier-rational triangle target is reduced to an increment-only modulus. -/
  jcost_distance_verifier_triangle : PRCJCostDistanceVerifierTriangleConditionalCertificate
  /-- The increment-only J-cost modulus closes the null-distance setoid chain. -/
  jcost_distance_increment_triangle : PRCJCostDistanceIncrementTriangleCertificate
  /-- The first complete ordered field pass closes add/neg and names mul/order/completeness targets. -/
  real_complete_ordered_field : PRCRealCompleteOrderedFieldConditionalCertificate
  /-- Multiplication on the null quotient is reduced to boundedness and bounded product continuity. -/
  real_mul_bounded_continuity : PRCRealMulBoundedContinuityConditionalCertificate
  /-- Every J-cost Cauchy ledger is eventually PRC-bounded. -/
  real_boundedness_modulus : PRCRealBoundednessModulusCertificate
  /-- Bounded product-continuity closes multiplication on the null quotient. -/
  real_product_continuity : PRCRealProductContinuityCertificate
  /-- Eventual non-strict order descends to the null-distance quotient. -/
  real_order_congruence : PRCRealOrderCongruenceCertificate
  /-- Internal completeness is sharpened to the exact Cauchy-of-Cauchy diagonal target. -/
  real_completeness : PRCRealCompletenessSharpenedCertificate
  /-- The complete ordered-field certificate surface now carries proved mul, order, and completeness targets. -/
  real_complete_ordered_field_promoted :
    PRCRealCompleteOrderedFieldPromotedCertificate
  /-- The real-completion boundary is inhabited and classical-extension tagged. -/
  real_completion_boundary : RealCompletionBoundaryCertificate
  signed_orbit_display : Nonempty (SignedOrbit → ℤ)
  ratio_orbit_display : Nonempty (RatioOrbit → ℚ)

/-- K7/A2. The first-pass kernel certificate is inhabited. -/
theorem kernel_first_pass_certificate :
    KernelFirstPassCertificate where
  strength_tags_exist := (StrengthTag.deltaOnly)
  trace_syntax_exists := (Trace.empty)
  judgment_surface_exists := (verifierEqualityJudgment)
  trace_logic := trace_logic_certificate
  formal_system := formal_system_certificate
  inevitability := prc_inevitability_certificate
  quotient_surface_exists := by
    intro J T
    exact (endpointClassOf J T Endpoint.left)
  orbit_nat_equivalence := (DistinctionNat.equivNat)
  orbit_add_faithful := DistinctionNat.toNat_add
  orbit_mul_faithful := DistinctionNat.toNat_mul
  prc_integer_surface_exists := (PRCInt.zero)
  prc_rational_surface_exists := by
    let one : DistinctionNat := DistinctionNat.succ DistinctionNat.zero
```

```
    have hone : one ≠ DistinctionNat.zero := by
      intro h
      exact DistinctionNat.zero_ne_succ DistinctionNat.zero h.symm
    exact (PRCRat.mk (SignedOrbit.zero, one, hone))
  prc_int_balanced_iff_display := SignedOrbit.balanced_iff_toInt_eq
  prc_rat_cross_iff_display := RatioOrbit.crossEq_iff_toRat_eq
  prc_int_display_injective := PRCInt.toInt_injective
  prc_rat_display_injective := PRCRat.toRat_injective
  prc_int_equiv_int := (PRCInt.equivInt)
  integer_order := integer_order_certificate
  orbit_divisibility := DistinctionNat.orbit_divisibility_certificate
  orbit_euclidean := DistinctionNat.orbit_euclidean_certificate
  prc_jcost := PRCJCost.prc_jcost_certificate
  rational_field := rational_field_certificate
  recognizer_bridge := prc_recognizer_bridge_certificate
  prc_int_add_comm := PRCInt.add_comm
  prc_int_add_assoc := PRCInt.add_assoc
  prc_int_mul_comm := PRCInt.mul_comm
  prc_int_mul_assoc := PRCInt.mul_assoc
  prc_int_left_distrib := PRCInt.left_distrib
  prc_int_add_negate := PRCInt.add_negate
  prc_rat_display_add := PRCRat.toRat_add'
  prc_rat_display_mul := PRCRat.toRat_mul'
  prc_rat_display_inv := PRCRat.toRat_inv'
  ratio_recip_internal_den := by
    intro a h
    rfl
  prc_rat_add_comm := PRCRat.add_comm
  prc_rat_mul_comm := PRCRat.mul_comm
  prc_rat_left_distrib := PRCRat.left_distrib
  prc_rat_mul_recip_cancel := by
    intro a h
    simpa using PRCRat.mul_recip_cancel (a := a) h
  trace_closure_boundary := trace_closure_certificate
  real_cauchy := real_cauchy_certificate
  real_null_setoid := real_null_setoid_conditional_certificate
  jcost_distance_triangle := prc_jcost_distance_triangle_conditional_certificate
  jcost_distance_verifier_triangle :=
    prc_jcost_distance_verifier_triangle_conditional_certificate
  jcost_distance_increment_triangle :=
    prc_jcost_distance_increment_triangle_certificate
  real_complete_ordered_field :=
    prc_real_complete_ordered_field_conditional_certificate
  real_mul_bounded_continuity :=
    prc_real_mul_bounded_continuity_conditional_certificate
  real_boundedness_modulus :=
    prc_real_boundedness_modulus_certificate
  real_product_continuity :=
    prc_real_product_continuity_certificate
  real_order_congruence :=
    prc_real_order_congruence_certificate
  real_completeness :=
    prc_real_completeness_sharpened_certificate
  real_complete_ordered_field_promoted :=
    prc_real_complete_ordered_field_promoted_certificate
  real_completion_boundary := real_completion_boundary_certificate
  signed_orbit_display := (SignedOrbit.toInt)
  ratio_orbit_display := (RatioOrbit.toRat)

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```

33. UniversalFoundation.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/UniversalFoundation.lean · **Lines:** 336

```
0001  /-
0002    PrimitiveRecognitionCalculus/UniversalFoundation.lean
0003
0004    Round-trip source:
0005      6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007    Spec anchor:
0008      Build Order step 15: compose arithmetic, structural surfaces, cost, logic,
0009      real completion, recognizer, and admissible inevitability.
0010
0011    This file deliberately does not declare the final theorem
0012    `prc_universal_foundation`. It gives the current top-level conditional
0013    certificate and names the exact remaining targets that prevent final closure.
0014  -/
0015
0016  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Kernel
0017  import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCNativeCostUniqueness
0018
0019  namespace IndisputableMonolith
0020  namespace Foundation
0021  namespace PrimitiveRecognitionCalculus
0022
0023  /-- Exact open targets remaining before the final unqualified universal
0024  foundation theorem can be declared. -/
0025  structure PRCUniversalFoundationOpenTargets : Prop where
0026    native_cost_uniqueness :
0027      PRCJCost.PRCNativeCostUniquenessTarget =
0028        PRCJCost.PRCNativeCostUniquenessTarget
0029    native_cost_character_factorization :
0030      PRCJCost.PRCNativeCostCharacterFactorizationTarget =
0031        PRCJCost.PRCNativeCostCharacterFactorizationTarget
0032    native_cost_character_rigidity :
0033      PRCJCost.PRCNativeCostCharacterRigidityTarget =
0034        PRCJCost.PRCNativeCostCharacterRigidityTarget
0035    native_cost_sharpened :
0036      PRCJCost.PRCNativeCostUniquenessSharpenedTarget =
0037        PRCJCost.PRCNativeCostUniquenessSharpenedTarget
0038    two_to_prime_calibration :
0039      PRCJCost.PRCTwoCalibrationForcesPrimeCalibrationTarget =
0040        PRCJCost.PRCTwoCalibrationForcesPrimeCalibrationTarget
0041    prime_calibration_propagation :
0042      PRCJCost.PRCPrimeCalibrationPropagationTarget =
0043        PRCJCost.PRCPrimeCalibrationPropagationTarget
0044    prime_global_orientation :
0045      PRCJCost.PRCPrimeCalibrationForcesGlobalOrientationTarget =
0046        PRCJCost.PRCPrimeCalibrationForcesGlobalOrientationTarget
0047    coherent_prime_orientation :
0048      PRCJCost.PRCCharacterPrimeOrientationCoherent =
0049        PRCJCost.PRCCharacterPrimeOrientationCoherent
0050    local_prime_orientation :
0051      PRCJCost.PRCCharacterPrimeLocalOrientation =
0052        PRCJCost.PRCCharacterPrimeLocalOrientation
0053    no_mixed_prime_orientation :
0054      PRCJCost.PRCCharacterNoMixedPrimeOrientation =
0055        PRCJCost.PRCCharacterNoMixedPrimeOrientation
0056    prime_identity_trace_coherence :
0057      PRCJCost.PRCCharacterPrimeIdentityTraceCoherent =
0058        PRCJCost.PRCCharacterPrimeIdentityTraceCoherent
0059    prime_axis_trace_connected :
0060      PRCJCost.PRCPrimeAxisTraceConnected =
0061        PRCJCost.PRCPrimeAxisTraceConnected
0062    prime_identity_respects_trace_connected :
0063      PRCJCost.PRCCharacterPrimeIdentityRespectsTraceConnected =
0064        PRCJCost.PRCCharacterPrimeIdentityRespectsTraceConnected
0065    prime_identity_respects_common_trace_extension :
0066      PRCJCost.PRCCharacterPrimeIdentityRespectsCommonTraceExtension =
0067        PRCJCost.PRCCharacterPrimeIdentityRespectsCommonTraceExtension
0068    prime_identity_respects_comparable_trace :
0069      PRCJCost.PRCCharacterPrimeIdentityRespectsComparableTrace =
0070        PRCJCost.PRCCharacterPrimeIdentityRespectsComparableTrace
0071    orbit_identity_respects_successor_step :
0072      PRCJCost.PRCCharacterOrbitIdentityRespectsSuccessorStep =
0073        PRCJCost.PRCCharacterOrbitIdentityRespectsSuccessorStep
0074    orbit_identity_successor_transport :
0075      PRCJCost.PRCCharacterOrbitIdentitySuccessorTransport =
0076        PRCJCost.PRCCharacterOrbitIdentitySuccessorTransport
0077    orbit_successor_additive_compat :
0078      PRCJCost.PRCCharacterOrbitSuccessorAdditiveCompatible =
0079        PRCJCost.PRCCharacterOrbitSuccessorAdditiveCompatible
0080    prime_floor_orbit_identity_successor_transport :
0081      PRCJCost.PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport =
0082        PRCJCost.PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport
0083    nonunit_orbit_local_orientation :
0084      PRCJCost.PRCCharacterNonunitOrbitLocalOrientation =
0085        PRCJCost.PRCCharacterNonunitOrbitLocalOrientation
0086    orbit_product_local_orientation :
0087      PRCJCost.PRCCharacterOrbitProductLocalOrientationPropagates =
0088        PRCJCost.PRCCharacterOrbitProductLocalOrientationPropagates
0089    orbit_product_display_compatible :
0090      PRCJCost.PRCCharacterOrbitProductDisplayCompatible =
```

```
      PRCJCost.PRCCharacterOrbitProductDisplayCompatible
  orbit_product_no_mixed_orientation :
    PRCJCost.PRCCharacterOrbitProductNoMixedOrientation =
      PRCJCost.PRCCharacterOrbitProductNoMixedOrientation
  prime_floor_no_adjacent_mixed_orientation :
    PRCJCost.PRCCharacterPrimeFloorNoAdjacentMixedOrientation =
      PRCJCost.PRCCharacterPrimeFloorNoAdjacentMixedOrientation
  prime_identity_common_trace_extension_target :
    PRCJCost.PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget =
      PRCJCost.PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget
  prime_identity_comparable_trace_target :
    PRCJCost.PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget =
      PRCJCost.PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget
  orbit_successor_identity_target :
    PRCJCost.PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget =
      PRCJCost.PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget
  orbit_successor_transport_target :
    PRCJCost.PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget =
      PRCJCost.PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget
  prime_floor_successor_transport_target :
    PRCJCost.PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget =
      PRCJCost.PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
  prime_floor_nonunit_local_orientation_target :
    PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget =
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget
  prime_floor_nonunit_product_local_orientation_target :
    PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget =
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget
  prime_floor_nonunit_orbit_orientation_coherent_target :
    PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget =
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_no_mixed_nonunit_orbit_orientation_target :
    PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget =
      PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
  prime_floor_nonunit_identity_branch_transport_target :
    PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget =
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
  prime_floor_nonunit_identity_comparable_trace_target :
    PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget =
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_nonunit_orbit_orientation_local_no_mixed_target :
    PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget =
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
  prime_floor_nonunit_orbit_orientation_local_product_no_mixed_target :
    PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget =
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget
  prime_floor_no_mixed_nonunit_from_product_no_mixed :
    PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
      PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
  prime_floor_product_no_mixed_from_no_mixed_nonunit :
    PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget →
      PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
  prime_floor_product_no_mixed_iff_no_mixed_nonunit :
    PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
  prime_floor_product_no_mixed_from_identity_branch_transport :
    PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget →
      PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
  prime_floor_nonunit_identity_branch_transport_from_comparable_trace :
    PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget →
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
  prime_floor_nonunit_coherent_from_product_no_mixed :
    PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_product_no_mixed_iff_nonunit_coherent :
    PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_nonunit_identity_branch_transport_from_product_no_mixed :
    PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
  prime_floor_product_no_mixed_iff_identity_branch_transport :
    PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
  prime_floor_nonunit_identity_comparable_trace_from_product_no_mixed :
    PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_nonunit_identity_branch_transport_iff_comparable_trace :
    PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget ↔
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_product_no_mixed_iff_identity_comparable_trace :
    PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
  prime_floor_nonunit_identity_comparable_trace_iff_successor_transport :
    PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget ↔
      PRCJCost.PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
  prime_floor_successor_transport_local_adjacent_target :
    PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget =
      PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget
  prime_floor_successor_transport_local_adjacent_iff_local_successor_transport :
    PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
      (PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
        PRCJCost.PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget)
  prime_floor_successor_transport_local_adjacent_iff_nonunit_coherent :
    PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
  prime_floor_nonunit_orbit_orientation_coherent_iff_local_no_mixed :
    PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
  prime_floor_nonunit_orbit_orientation_coherent_sharpened_target :
    PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget =
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget
```

```
  prime_floor_nonunit_orbit_orientation_coherent_iff_sharpened :
    PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget
  prime_floor_product_local_orientation_sharpened_target :
    PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget =
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget
  prime_floor_no_adjacent_mixed_orientation_target :
    PRCJCost.PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget =
      PRCJCost.PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
  prime_floor_successor_transport_sharpened :
    PRCJCost.PRCPrimeFloorSuccessorTransportSharpenedTarget =
      PRCJCost.PRCPrimeFloorSuccessorTransportSharpenedTarget
  prime_to_coherent_orientation :
    PRCJCost.PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget =
      PRCJCost.PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget
  coherent_prime_orientation_propagation :
    PRCJCost.PRCCoherentPrimeOrientationPropagatesToGlobalTarget =
      PRCJCost.PRCCoherentPrimeOrientationPropagatesToGlobalTarget
  prime_propagation_sharpened :
    PRCJCost.PRCPrimeCalibrationPropagationSharpenedTarget =
      PRCJCost.PRCPrimeCalibrationPropagationSharpenedTarget
  native_cost_rigidity_sharpened :
    PRCJCost.PRCNativeCostCharacterRigiditySharpenedTarget =
      PRCJCost.PRCNativeCostCharacterRigiditySharpenedTarget
  external_foundation_parsing_schema :
    ∀ (ExternalFoundation : Type)
      (FaithfulParse : ExternalFoundation → FormalSystem → Prop),
      ExternalFoundationParsingTarget ExternalFoundation FaithfulParse =
        ExternalFoundationParsingTarget ExternalFoundation FaithfulParse

/-- Current top-level conditional certificate: all built PRC surfaces compose,
and the two remaining nonlocal obligations are exposed by name. -/
structure PRCUniversalFoundationConditionalCertificate : Prop where
  kernel : KernelFirstPassCertificate
  real_complete_ordered_field :
    PRCRealCompleteOrderedFieldPromotedCertificate
  trace_logic : TraceLogicCertificate
  formal_system : FormalSystemCertificate
  inevitability : PRCInevitabilityCertificate
  recognizer_bridge : PRCRecognizerBridgeCertificate
  native_cost_blocker : PRCJCost.PRCNativeCostUniquenessBlockerCertificate
  open_targets : PRCUniversalFoundationOpenTargets
  no_project_local_axioms_audit :
    StrengthTag.classicalExtension = StrengthTag.classicalExtension

theorem prc_universal_foundation_conditional_certificate :
    PRCUniversalFoundationConditionalCertificate where
  kernel := kernel_first_pass_certificate
  real_complete_ordered_field :=
    prc_real_complete_ordered_field_promoted_certificate
  trace_logic := trace_logic_certificate
  formal_system := formal_system_certificate
  inevitability := prc_inevitability_certificate
  recognizer_bridge := prc_recognizer_bridge_certificate
  native_cost_blocker := PRCJCost.prc_native_cost_uniqueness_blocker_certificate
  open_targets := {
    native_cost_uniqueness := rfl
    native_cost_character_factorization := rfl
    native_cost_character_rigidity := rfl
    native_cost_sharpened := rfl
    two_to_prime_calibration := rfl
    prime_calibration_propagation := rfl
    prime_global_orientation := rfl
    coherent_prime_orientation := rfl
    local_prime_orientation := rfl
    no_mixed_prime_orientation := rfl
    prime_identity_trace_coherence := rfl
    prime_axis_trace_connected := rfl
    prime_identity_respects_trace_connected := rfl
    prime_identity_respects_common_trace_extension := rfl
    prime_identity_respects_comparable_trace := rfl
    orbit_identity_respects_successor_step := rfl
    orbit_identity_successor_transport := rfl
    orbit_successor_additive_compat := rfl
    prime_floor_orbit_identity_successor_transport := rfl
    nonunit_orbit_local_orientation := rfl
    orbit_product_local_orientation := rfl
    orbit_product_display_compatible := rfl
    orbit_product_no_mixed_orientation := rfl
    prime_floor_no_adjacent_mixed_orientation := rfl
    prime_identity_common_trace_extension_target := rfl
    prime_identity_comparable_trace_target := rfl
    orbit_successor_identity_target := rfl
    orbit_successor_transport_target := rfl
    prime_floor_successor_transport_target := rfl
    prime_floor_nonunit_local_orientation_target := rfl
    prime_floor_nonunit_product_local_orientation_target := rfl
    prime_floor_nonunit_orbit_orientation_coherent_target := rfl
    prime_floor_no_mixed_nonunit_orbit_orientation_target := rfl
    prime_floor_nonunit_identity_branch_transport_target := rfl
    prime_floor_nonunit_identity_comparable_trace_target := rfl
    prime_floor_nonunit_orbit_orientation_local_no_mixed_target := rfl
    prime_floor_nonunit_orbit_orientation_local_product_no_mixed_target := rfl
    prime_floor_no_mixed_nonunit_from_product_no_mixed :=
      PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed
    prime_floor_product_no_mixed_from_no_mixed_nonunit :=
      PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_no_mixed_nonunit
    prime_floor_product_no_mixed_iff_no_mixed_nonunit :=
      PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_no_mixed_nonunit
    prime_floor_product_no_mixed_from_identity_branch_transport :=
      PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport
```

```
    prime_floor_nonunit_identity_branch_transport_from_comparable_trace :=
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace
    prime_floor_nonunit_coherent_from_product_no_mixed :=
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed
    prime_floor_product_no_mixed_iff_nonunit_coherent :=
      PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_nonunit_coherent
    prime_floor_nonunit_identity_branch_transport_from_product_no_mixed :=
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed
    prime_floor_product_no_mixed_iff_identity_branch_transport :=
      PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_branch_transport
    prime_floor_nonunit_identity_comparable_trace_from_product_no_mixed :=
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_product_no_mixed
    prime_floor_nonunit_identity_branch_transport_iff_comparable_trace :=
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_iff_comparable_trace
    prime_floor_product_no_mixed_iff_identity_comparable_trace :=
      PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_comparable_trace
    prime_floor_nonunit_identity_comparable_trace_iff_successor_transport :=
      PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_iff_prime_floor_successor_transport
    prime_floor_successor_transport_local_adjacent_target := rfl
    prime_floor_successor_transport_local_adjacent_iff_local_successor_transport :=
      PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_local_successor_transport
    prime_floor_successor_transport_local_adjacent_iff_nonunit_coherent :=
      PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_nonunit_coherent
    prime_floor_nonunit_orbit_orientation_coherent_iff_local_no_mixed :=
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_local_no_mixed
    prime_floor_nonunit_orbit_orientation_coherent_sharpened_target := rfl
    prime_floor_nonunit_orbit_orientation_coherent_iff_sharpened :=
      PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_sharpened
    prime_floor_product_local_orientation_sharpened_target := rfl
    prime_floor_no_adjacent_mixed_orientation_target := rfl
    prime_floor_successor_transport_sharpened := rfl
    prime_to_coherent_orientation := rfl
    coherent_prime_orientation_propagation := rfl
    prime_propagation_sharpened := rfl
    native_cost_rigidity_sharpened := rfl
    external_foundation_parsing_schema := by
      intro ExternalFoundation FaithfulParse
      rfl
  }
  no_project_local_axioms_audit := rfl

end PrimitiveRecognitionCalculus
end Foundation
end IndisputableMonolith
```